



OpenShift Container Platform 4.22

Postinstallation configuration

Day 2 operations for OpenShift Container Platform

OpenShift Container Platform 4.22 Postinstallation configuration

Day 2 operations for OpenShift Container Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides instructions and guidance on post-installation activities for OpenShift Container Platform.

Table of Contents

CHAPTER 1. POSTINSTALLATION CONFIGURATION OVERVIEW	12
1.1. POSTINSTALLATION CONFIGURATION TASKS	12
1.2. ADDITIONAL RESOURCES	13
CHAPTER 2. CONFIGURING A PRIVATE CLUSTER	14
2.1. ABOUT PRIVATE CLUSTERS	14
2.1.1. DNS	14
2.1.2. Ingress Controller	14
2.1.3. API server	14
2.2. CONFIGURING DNS RECORDS TO BE PUBLISHED IN A PRIVATE ZONE	15
2.3. SETTING THE INGRESS CONTROLLER TO PRIVATE	17
2.4. RESTRICTING THE API SERVER TO PRIVATE FOR AN AMAZON WEB SERVICES CLUSTER	17
2.5. RESTRICTING THE API SERVER TO PRIVATE FOR AN MICROSOFT AZURE CLUSTER	20
2.6. CONFIGURING A PRIVATE STORAGE ENDPOINT ON AZURE	20
2.6.1. Limitations for configuring a private storage endpoint on Azure	21
2.6.2. Configuring a private storage endpoint on Azure by enabling the Image Registry Operator to discover VNet and subnet names	21
2.6.3. Configuring a private storage endpoint on Azure with user-provided VNet and subnet names	23
2.6.4. Optional: Disabling redirect when using a private storage endpoint on Azure	25
CHAPTER 3. CONFIGURING MULTI-ARCHITECTURE COMPUTE MACHINES ON AN OPENSIFT CLUSTER	27
3.1. ABOUT CLUSTERS WITH MULTI-ARCHITECTURE COMPUTE MACHINES	27
3.1.1. Configuring your cluster with multi-architecture compute machines	27
3.1.2. Verifying cluster compatibility	29
3.1.3. Additional resources	30
3.2. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON AWS	30
3.2.1. Adding a multi-architecture compute machine set to your AWS cluster	30
3.2.2. Additional resources	33
3.3. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON AZURE	33
3.3.1. Creating a 64-bit ARM boot image using the Azure image gallery	34
3.3.2. Adding a multi-architecture compute machine set to your Azure cluster	36
3.3.3. Additional resources	39
3.4. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON GOOGLE CLOUD	39
3.4.1. Adding a multi-architecture compute machine set to your Google Cloud cluster	39
3.4.2. Additional resources	42
3.5. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON BARE METAL, IBM POWER, OR IBM Z	42
3.5.1. Creating RHCOS machines by using an ISO image	43
3.5.2. Creating RHCOS machines by PXE or iPXE booting	45
3.5.3. Approving the certificate signing requests for your machines	48
3.5.4. Additional resources	51
3.6. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM LINUXONE WITH Z/VM	51
3.6.1. Additional resources	51
3.6.2. Creating RHCOS machines on IBM Z with z/VM	52
3.6.3. Approving the certificate signing requests for your machines	55
3.7. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM LINUXONE IN AN LPAR	57
3.7.1. Creating RHCOS machines on IBM Z in an LPAR	58
3.7.2. Approving the certificate signing requests for your machines	61
3.8. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM	

LINUXONE WITH RHEL KVM	63
3.8.1. Creating RHCOS machines using virt-install	64
3.8.2. Approving the certificate signing requests for your machines	66
3.8.3. Additional resources	69
3.9. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM POWER	69
3.9.1. Creating RHCOS machines by using an ISO image	70
3.9.2. Creating RHCOS machines by PXE or iPXE booting	71
3.9.3. Approving the certificate signing requests for your machines	74
3.10. MANAGING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES	77
3.10.1. Scheduled workloads on clusters with multi-architecture compute machines	78
3.10.1.1. Sample multi-architecture node workload deployments	78
3.10.2. Enabling 64k pages on the Red Hat Enterprise Linux CoreOS (RHCOS) kernel	81
3.10.3. Importing manifest lists in image streams on your multi-architecture compute machines	83
3.11. MANAGING WORKLOADS ON MULTI-ARCHITECTURE CLUSTERS BY USING THE MULTIARCH TUNING OPERATOR	84
3.11.1. Installing the Multiarch Tuning Operator by using the CLI	85
3.11.2. Installing the Multiarch Tuning Operator by using the web console	87
3.11.3. Multiarch Tuning Operator pod labels and architecture support overview	88
3.11.4. The ClusterPodPlacementConfig object	89
3.11.4.1. Creating the ClusterPodPlacementConfig object by using the CLI	91
3.11.4.2. Creating the ClusterPodPlacementConfig object by using the web console	92
3.11.5. Creating the namespace-scoped PodPlacementConfig object	93
3.11.6. Deleting the ClusterPodPlacementConfig object by using the CLI	95
3.11.7. Deleting the ClusterPodPlacementConfig object by using the web console	95
3.11.8. Uninstalling the Multiarch Tuning Operator by using the CLI	96
3.11.9. Uninstalling the Multiarch Tuning Operator by using the web console	98
3.12. MULTIARCH TUNING OPERATOR RELEASE NOTES	98
3.12.1. Additional resources	98
3.12.2. Release notes for the Multiarch Tuning Operator 1.3.3	98
3.12.2.1. Enhancements	99
3.12.3. Release notes for the Multiarch Tuning Operator 1.3.2	99
3.12.3.1. Bug fixes	99
3.12.3.2. Enhancements	99
3.12.4. Release notes for the Multiarch Tuning Operator 1.3.1	99
3.12.4.1. Enhancements	99
3.12.5. Release notes for the Multiarch Tuning Operator 1.3.0	100
3.12.5.1. New features and enhancements	100
3.12.5.2. Bug fixes	100
3.12.5.3. Enhancements	100
3.12.6. Release notes for the Multiarch Tuning Operator 1.2.2	100
3.12.6.1. Enhancements	100
3.12.7. Release notes for the Multiarch Tuning Operator 1.2.1	101
3.12.7.1. Bug fixes	101
3.12.8. Release notes for the Multiarch Tuning Operator 1.2.0	101
3.12.8.1. New features and enhancements	101
3.12.8.2. Bug fixes	101
3.12.9. Release notes for the Multiarch Tuning Operator 1.1.1	102
3.12.9.1. Bug fixes	102
3.12.10. Release notes for the Multiarch Tuning Operator 1.1.0	102
3.12.10.1. New features and enhancements	102
3.12.10.2. Bug fixes	102
3.12.11. Release notes for the Multiarch Tuning Operator 1.0.0	103
3.12.11.1. New features and enhancements	103

CHAPTER 4. POSTINSTALLATION CLUSTER TASKS	104
4.1. AVAILABLE CLUSTER CUSTOMIZATIONS	104
4.1.1. Cluster configuration resources	104
4.1.2. Operator configuration resources	105
4.1.3. Additional configuration resources	105
4.1.4. Informational Resources	106
4.2. ADDING WORKER NODES	106
4.2.1. Adding worker nodes to an on-premise cluster	106
4.2.2. Adding worker nodes to installer-provisioned infrastructure clusters	107
4.2.3. Adding worker nodes to user-provisioned infrastructure clusters	107
4.2.4. Adding worker nodes to clusters managed by the Assisted Installer	107
4.2.5. Adding worker nodes to clusters managed by the multicluster engine for Kubernetes	107
4.3. ADJUST WORKER NODES	108
4.3.1. Understanding the difference between compute machine sets and the machine config pool	108
4.3.2. Scaling a compute machine set manually	108
4.3.3. The compute machine set deletion policy	109
4.3.4. Creating default cluster-wide node selectors	110
4.4. IMPROVING CLUSTER STABILITY IN HIGH LATENCY ENVIRONMENTS USING WORKER LATENCY PROFILES	113
4.4.1. Understanding worker latency profiles	114
4.4.2. Using and changing worker latency profiles	117
4.5. MANAGING CONTROL PLANE MACHINES	119
4.5.1. Adding a control plane node to your cluster	119
4.6. CREATING INFRASTRUCTURE MACHINE SETS FOR PRODUCTION ENVIRONMENTS	125
4.6.1. Creating a compute machine set	126
4.6.2. Creating an infrastructure node	128
4.6.3. Creating a machine config pool for infrastructure machines	129
4.7. ASSIGNING MACHINE SET RESOURCES TO INFRASTRUCTURE NODES	132
4.7.1. Binding infrastructure node workloads using taints and tolerations	132
4.8. MOVING RESOURCES TO INFRASTRUCTURE MACHINE SETS	134
4.8.1. Moving the router	135
4.8.2. Moving the default registry	137
4.8.3. Moving the monitoring solution	138
4.9. THE CLUSTER AUTOSCALER	140
4.9.1. Automatic node removal	140
4.9.2. Limitations	141
4.9.3. Interaction with other scheduling features	141
4.9.4. Cluster autoscaler resource definition	142
4.9.5. Deploying a cluster autoscaler	145
4.10. APPLYING AUTOSCALING TO YOUR CLUSTER	146
4.11. ENABLING TECHNOLOGY PREVIEW FEATURES USING FEATUREGATES	146
4.11.1. Understanding feature gates	146
4.11.2. Enabling feature sets using the web console	150
4.11.3. Enabling feature sets using the CLI	152
4.12. ETCD TASKS	153
4.12.1. etcd encryption	153
4.12.2. Supported encryption types	154
4.12.3. Enabling etcd encryption	154
4.12.4. Disabling etcd encryption	156
4.12.5. Backing up etcd data	158
4.12.6. Data defragmentation for etcd	160
4.12.7. Automatic defragmentation	160
4.12.8. Manually defragmenting etcd data	161

4.12.9. Restoring to an earlier cluster state for more than one node	164
4.12.10. Issues and workarounds for restoring a persistent storage state	166
4.13. POD DISRUPTION BUDGETS	167
4.13.1. Understanding how to use pod disruption budgets to specify the number of pods that must be up	167
4.13.2. Specifying the number of pods that must be up with pod disruption budgets	169
4.13.3. Specifying the eviction policy for unhealthy pods	170
CHAPTER 5. POSTINSTALLATION NODE TASKS	172
5.1. ADDING RHCOS COMPUTE MACHINES TO AN OPENSIFT CONTAINER PLATFORM CLUSTER	172
5.1.1. Prerequisites	172
5.1.2. Creating RHCOS machines by using an ISO image	172
5.1.3. Creating RHCOS machines by PXE or iPXE booting	174
5.1.4. Approving the certificate signing requests for your machines	177
5.1.5. Adding a new RHCOS worker node with a custom /var partition in AWS	179
5.2. DEPLOYING MACHINE HEALTH CHECKS	185
5.2.1. About machine health checks	185
5.2.1.1. Limitations when deploying machine health checks	186
5.2.2. Sample MachineHealthCheck resource	186
5.2.2.1. Short-circuiting machine health check remediation	187
5.2.2.1.1. Setting maxUnhealthy by using an absolute value	188
5.2.2.1.2. Setting maxUnhealthy by using percentages	188
5.2.3. Creating a machine health check resource	188
5.2.4. Scaling a compute machine set manually	189
5.2.5. Understanding the difference between compute machine sets and the machine config pool	190
5.3. RECOMMENDED NODE HOST PRACTICES	190
5.3.1. Creating a KubeletConfig CR to edit kubelet parameters	191
5.3.2. Modifying the number of unavailable worker nodes	196
5.3.3. Control plane node sizing	197
5.3.4. Setting up CPU Manager	199
5.4. HUGE PAGES	204
5.4.1. What huge pages do	204
5.4.2. How huge pages are consumed by apps	205
5.4.2.1. Allocating huge pages of a specific size	206
5.4.2.2. Huge page requirements	206
5.4.3. Configuring huge pages at boot time	206
5.5. UNDERSTANDING DEVICE PLUGINS	208
5.5.1. Example device plugins	208
5.5.2. Methods for deploying a device plugin	209
5.5.3. Understanding the Device Manager	209
5.5.4. Enabling Device Manager	210
5.6. TAINTS AND TOLERATIONS	211
5.6.1. Understanding taints and tolerations	211
5.6.2. Adding taints and tolerations	214
5.6.3. Adding taints and tolerations using a compute machine set	216
5.6.4. Binding a user to a node using taints and tolerations	218
5.6.5. Controlling nodes with special hardware using taints and tolerations	219
5.6.6. Removing taints and tolerations	220
5.7. TOPOLOGY MANAGER	221
5.7.1. Topology Manager policies	221
5.7.2. Setting up Topology Manager	221
5.7.3. Pod interactions with Topology Manager policies	222
5.8. RESOURCE REQUESTS AND OVERCOMMITMENT	223
5.9. CLUSTER-LEVEL OVERCOMMIT USING THE CLUSTER RESOURCE OVERRIDE OPERATOR	224

5.9.1. Installing the Cluster Resource Override Operator using the web console	226
5.9.2. Installing the Cluster Resource Override Operator using the CLI	228
5.9.3. Configuring cluster-level overcommit	231
5.10. NODE-LEVEL OVERCOMMIT	233
5.10.1. Understanding container CPU and memory requests	233
5.10.2. Understanding overcommitment and quality of service classes	233
5.10.2.1. Understanding how to reserve memory across quality of service tiers	234
5.10.3. Understanding swap memory and QoS	234
5.10.4. Understanding nodes overcommitment	235
5.10.5. Disabling or enforcing CPU limits using CPU CFS quotas	236
5.10.6. Reserving resources for system processes	237
5.10.7. Disabling overcommitment for a node	237
5.11. PROJECT-LEVEL LIMITS	237
5.11.1. Disabling overcommitment for a project	238
5.12. FREEING NODE RESOURCES USING GARBAGE COLLECTION	238
5.12.1. Understanding how terminated containers are removed through garbage collection	238
5.12.2. Understanding how images are removed through garbage collection	239
5.12.3. Configuring garbage collection for containers and images	240
5.13. USING THE NODE TUNING OPERATOR	243
5.13.1. Accessing an example Node Tuning Operator specification	244
5.13.2. Custom tuning specification	244
5.13.3. Default profiles set on a cluster	249
5.13.4. Supported TuneD daemon plugins	250
5.14. CONFIGURING THE MAXIMUM NUMBER OF PODS PER NODE	251
5.15. MACHINE SCALING WITH STATIC IP ADDRESSES	253
5.15.1. Scaling machines to use static IP addresses	253
5.15.2. Machine set scaling of machines with configured static IP addresses	254
5.15.3. Using a machine set to scale machines with configured static IP addresses	255
CHAPTER 6. POSTINSTALLATION NETWORK CONFIGURATION	259
6.1. USING THE CLUSTER NETWORK OPERATOR	259
6.2. NETWORK CONFIGURATION TASKS	259
6.2.1. Creating default network policies for a new project	259
6.2.1.1. Modifying the template for new projects	259
6.2.1.2. Adding network policies to the new project template	260
6.3. ADDITIONAL RESOURCES	262
CHAPTER 7. CONFIGURING IMAGE STREAMS AND IMAGE REGISTRIES	263
7.1. CONFIGURING IMAGE STREAMS FOR A DISCONNECTED CLUSTER	263
7.1.1. Cluster Samples Operator assistance for mirroring	263
7.1.2. Using Cluster Samples Operator image streams with alternate or mirrored registries	264
7.1.3. Preparing your cluster to gather support data	265
7.2. CONFIGURING PERIODIC IMPORTING OF CLUSTER SAMPLE OPERATOR IMAGE STREAM TAGS	266
7.3. ADDITIONAL RESOURCES	267
CHAPTER 8. POSTINSTALLATION STORAGE CONFIGURATION	268
8.1. DYNAMIC PROVISIONING	268
8.2. RECOMMENDED CONFIGURABLE STORAGE TECHNOLOGY	268
8.3. DEPLOY RED HAT OPENSIFT DATA FOUNDATION	269
CHAPTER 9. PREPARING FOR USERS	272
9.1. UNDERSTANDING IDENTITY PROVIDER CONFIGURATION	272
9.1.1. Identity providers in OpenShift Container Platform	272
9.1.2. Supported identity providers	272

9.1.3. Identity provider parameters	273
9.1.4. Sample identity provider CR	273
9.2. USING RBAC TO DEFINE AND APPLY PERMISSIONS	274
9.2.1. RBAC overview	274
9.2.1.1. Default cluster roles	275
9.2.1.2. Evaluating authorization	277
9.2.1.3. Cluster role aggregation	278
9.2.2. Projects and namespaces	278
9.2.3. Default projects	279
9.2.4. Viewing cluster roles and bindings	280
9.2.5. Viewing local roles and bindings	287
9.2.6. Adding roles to users	288
9.2.7. Creating a local role	291
9.2.8. Creating a cluster role	291
9.2.9. Local role binding commands	291
9.2.10. Cluster role binding commands	292
9.2.11. Creating a cluster admin	293
9.2.12. Cluster role bindings for unauthenticated groups	293
9.2.13. Adding unauthenticated groups to cluster roles	294
9.3. THE KUBEADMIN USER	295
9.3.1. Removing the kubeadmin user	295
9.4. POPULATING THE SOFTWARE CATALOG FROM MIRRORED OPERATOR CATALOGS	296
9.4.1. Prerequisites	296
9.4.1.1. Creating the ImageContentSourcePolicy object	296
9.4.1.2. Adding a catalog source to a cluster	296
9.5. ABOUT OPERATOR INSTALLATION FROM THE SOFTWARE CATALOG	299
9.5.1. Installing from the software catalog by using the web console	299
9.5.2. Installing from the software catalog by using the CLI	301
CHAPTER 10. CHANGING THE CLOUD PROVIDER CREDENTIALS CONFIGURATION	310
10.1. ROTATING CLOUD PROVIDER SERVICE KEYS WITH THE CLOUD CREDENTIAL OPERATOR UTILITY	310
10.1.1. Rotating AWS OIDC bound service account signer keys	310
10.1.2. Rotating Google Cloud OIDC bound service account signer keys	315
10.1.3. Rotating Azure OIDC bound service account signer keys	319
10.1.4. Rotating IBM Cloud credentials	323
10.2. ROTATING CLOUD PROVIDER CREDENTIALS	323
10.2.1. Rotating cloud provider credentials manually	323
10.3. REMOVING CLOUD PROVIDER CREDENTIALS	326
10.3.1. Removing cloud provider credentials	326
10.4. ENABLING TOKEN-BASED AUTHENTICATION	327
10.4.1. Configuring the Cloud Credential Operator utility	327
10.4.2. Enabling Microsoft Entra Workload ID on an existing cluster	329
10.4.3. Enabling AWS Security Token Service (STS) on an existing cluster	333
10.4.4. Verifying that a cluster uses short-term credentials	337
10.5. ADDITIONAL RESOURCES	338
CHAPTER 11. CONFIGURING ALERT NOTIFICATIONS	339
11.1. SENDING NOTIFICATIONS TO EXTERNAL SYSTEMS	339
11.2. ADDITIONAL RESOURCES	339
CHAPTER 12. CONVERTING A CONNECTED CLUSTER TO A DISCONNECTED CLUSTER	340
12.1. ADDITIONAL RESOURCES	340
CHAPTER 13. DAY 2 OPERATIONS FOR OPENSIFT CONTAINER PLATFORM CLUSTERS	341

13.1. DAY 2 OPERATIONS FOR OPENSIFT CONTAINER PLATFORM CLUSTERS	341
13.2. UPGRADING OPENSIFT CONTAINER PLATFORM CLUSTERS	341
13.2.1. Upgrading an OpenShift Container Platform cluster	341
13.2.1.1. Cluster updates for OpenShift clusters	341
13.2.2. Verifying cluster API versions between update versions	342
13.2.2.1. OpenShift Container Platform API compatibility	342
13.2.2.2. Determining the cluster version update path	342
13.2.2.3. Selecting the target release	343
13.2.2.3.1. Determining what z-stream updates are available	343
13.2.2.3.2. Changing the channel for a Control Plane Only update	344
13.2.2.3.3. Changing the channel for an early EUS to EUS update	345
13.2.2.3.4. Changing the channel for a y-stream update	346
13.2.3. Preparing a bare-metal cluster for platform update	346
13.2.3.1. Disconnected environment considerations	347
13.2.3.2. Ensuring the host firmware is compatible with the update	347
13.2.3.3. Ensuring that layered products are compatible with the update	347
13.2.3.4. Applying MachineConfigPool labels to nodes before the update	348
13.2.3.4.1. Reviewing configured cluster MachineConfigPool roles	349
13.2.3.4.2. Creating MachineConfigPool groups for the cluster	350
13.2.3.5. Preparing the cluster platform for update	352
13.2.4. Configuring application pods before updating your OpenShift Container Platform cluster	354
13.2.4.1. Ensuring that workloads run uninterrupted with pod disruption budgets	354
13.2.4.2. Ensuring that pods do not run on the same cluster node	354
13.2.4.3. Application liveness, readiness, and startup probes	354
13.2.5. Before you update the telco core CNF cluster	355
13.2.5.1. Pausing worker nodes before the update	355
13.2.5.2. Backup the etcd database before you proceed with the update	356
13.2.5.2.1. Backing up etcd data	356
13.2.5.2.2. Creating a single automated etcd backup	358
13.2.5.3. Checking the cluster health	361
13.2.6. Completing the control plane Only cluster update	362
13.2.6.1. Acknowledging the control plane only or y-stream update	362
13.2.6.2. Starting the cluster update	364
13.2.6.3. Monitoring the cluster update	365
13.2.6.4. Updating the OLM Operators	367
13.2.6.4.1. Performing the second y-stream update	368
13.2.6.4.2. Acknowledging the y-stream release update	370
13.2.6.5. Starting the y-stream control plane update	372
13.2.6.6. Monitoring the second part of a <y+1> cluster update	372
13.2.6.7. Updating all the OLM Operators	374
13.2.6.8. Updating the worker nodes	375
13.2.6.9. Verifying the health of the newly updated cluster	377
13.2.7. Completing the y-stream cluster update	379
13.2.7.1. Acknowledging the control plane only or y-stream update	379
13.2.7.2. Starting the cluster update	381
13.2.7.3. Monitoring the cluster update	381
13.2.7.4. Updating the OLM Operators	383
13.2.7.5. Updating the worker nodes	385
13.2.7.6. Verifying the health of the newly updated cluster	387
13.2.8. Completing the z-stream cluster update	389
13.2.8.1. Starting the cluster update	389
13.2.8.2. Updating the worker nodes	390
13.2.8.3. Verifying the health of the newly updated cluster	392

13.3. UPGRADING OPENSIFT CONTAINER PLATFORM CLUSTERS WITH RHACM AND TALM	394
13.3.1. About cluster updates with RHACM and TALM	394
13.3.1.1. Policy-based cluster updates with RHACM and TALM	394
13.3.1.2. Benefits of TALM for large-scale deployments	394
13.3.1.3. GitOps workflows for RHACM update policies	395
13.3.1.4. Cluster update scenarios	395
13.3.1.4.1. Z-stream updates	395
13.3.1.4.2. Y-stream updates	396
13.3.1.4.3. EUS-to-EUS updates	396
13.3.1.4.4. Selecting an update scenario	397
13.3.1.4.5. Application and cluster configuration policy changes	397
13.3.1.5. Additional resources	397
13.3.2. Prepare RHACM policies and TALM for cluster updates	397
13.3.2.1. Preparing RHACM policies and placement rules for cluster updates	398
13.3.2.2. ClusterGroupUpgrade custom resource configuration	400
13.3.2.2.1. Basic ClusterGroupUpgrade CR	401
13.3.2.2.2. Timeout guidelines	402
13.3.2.2.3. Optional configuration fields	402
13.3.2.2.4. Canary cluster testing	402
13.3.2.2.5. Pre-update blocking checks	403
13.3.2.2.6. Post-update label management	403
13.3.2.2.7. Chained ClusterGroupUpgrade CRs for EUS-to-EUS updates	403
13.3.2.2.8. Applying and monitoring a ClusterGroupUpgrade CR	405
13.3.2.2.9. Troubleshooting	406
13.3.2.3. Additional resources	406
13.3.3. Prepare worker node pools before a cluster update with TALM	406
13.3.3.1. Configuring worker node batching by using machine config pools	406
13.3.3.2. Configuring worker node batching by using pod disruption budgets	407
13.3.3.3. Additional resources	409
13.3.4. Perform health checks before a cluster update with TALM	409
13.3.4.1. Performing health checks before cluster updates	409
13.3.4.2. Additional resources	413
13.3.5. Manage worker nodes during a cluster update with TALM	413
13.3.5.1. Pausing and unpausing worker nodes by using TALM	413
13.3.5.2. Pausing and unpausing worker nodes by using machine config pools	417
13.3.5.3. Additional resources	418
13.3.6. Coordinate OLM-managed Operator updates with TALM	418
13.3.6.1. Coordinating OLM Operator updates with cluster updates	418
13.3.6.2. Additional resources	422
13.3.7. Complete a z-stream cluster update with TALM	422
13.3.7.1. Updating clusters with z-stream releases	423
13.3.7.2. Additional resources	426
13.3.8. Complete a y-stream cluster update with TALM	426
13.3.8.1. Updating clusters through y-stream releases	427
13.3.8.2. Updating through sequential y-stream releases	433
13.3.8.3. Additional resources	437
13.3.9. Complete an EUS-to-EUS cluster update with TALM	437
13.3.9.1. Updating with EUS-to-EUS control-plane-only updates	437
13.3.9.2. Additional resources	443
13.3.10. Troubleshoot cluster updates with TALM	443
13.3.10.1. Diagnosing a ClusterGroupUpgrade CR stuck in Preparing state	443
13.3.10.2. Diagnosing a ClusterGroupUpgrade CR that failed on some clusters	444
13.3.10.3. Diagnosing degraded cluster Operators during an update	444

13.3.10.4. Diagnosing an update stuck in Working towards state	445
13.3.10.5. Diagnosing worker nodes stuck in NotReady state	446
13.3.10.6. Diagnosing a stuck worker node update	446
13.3.10.7. Diagnosing etcd issues during control plane updates	447
13.3.10.8. Diagnosing image pull failures during updates	447
13.3.10.9. Diagnosing update timeout exceeded	448
13.3.10.10. Resolving a noncompliant policy hold during an update	449
13.3.10.11. Collecting diagnostic information for Red Hat support	449
13.3.10.12. Additional resources	450
13.4. TROUBLESHOOTING AND MAINTAINING OPENSIFT CONTAINER PLATFORM CLUSTERS	450
13.4.1. Troubleshooting and maintaining OpenShift Container Platform clusters	450
13.4.1.1. Getting Support	450
13.4.1.1.1. About the Red Hat Knowledgebase	451
13.4.1.1.2. Searching the Red Hat Knowledgebase	451
13.4.1.1.3. Submitting a support case	451
13.4.2. General troubleshooting	453
13.4.2.1. Querying your cluster	453
13.4.2.2. Checking pod logs	455
13.4.2.3. Describing a pod	455
13.4.2.4. Reviewing events	456
13.4.2.5. Connecting to a pod	457
13.4.2.6. Debugging a pod	458
13.4.2.7. Running a command on a pod	458
13.4.3. Cluster maintenance	459
13.4.3.1. Checking cluster Operators	459
13.4.3.2. Watching for failed pods	459
13.4.4. Security	460
13.4.4.1. Authentication	460
13.4.5. Certificate maintenance	460
13.4.5.1. Certificates manually managed by the administrator	460
13.4.5.1.1. Managing proxy certificates	461
13.4.5.1.2. User-provisioned API server certificates	461
13.4.5.2. Certificates managed by the cluster	461
13.4.5.2.1. Certificates managed by etcd	462
13.4.5.2.2. Node certificates	463
13.4.5.2.3. Service CA certificates	463
13.4.6. Machine Config Operator	463
13.4.6.1. Purpose of the Machine Config Operator	463
13.4.6.2. Applying several machine config files at the same time	464
13.4.7. Bare-metal node maintenance	464
13.4.7.1. Connecting to a bare-metal node in your cluster	464
13.4.7.2. Moving applications to pods within the cluster	465
13.4.7.3. DIMM memory replacement	465
13.4.7.4. Disk replacement	466
13.4.7.5. Cluster network card replacement	466
13.5. OBSERVABILITY	466
13.5.1. Observability in OpenShift Container Platform clusters	466
13.5.1.1. Monitoring stack components	466
13.5.1.2. Key performance metrics	467
13.5.1.2.1. Example queries in PromQL	468
13.5.1.2.2. Recommendations for storage of metrics	471
13.5.1.3. Monitoring at the far edge network	471
13.5.1.4. Alerting	475

13.5.1.4.1. Viewing default alerts	475
13.5.1.4.2. Alert notifications	475
13.5.1.5. Workload monitoring	476
13.6. SECURITY	477
13.6.1. Security basics	477
13.6.1.1. RBAC overview	478
13.6.1.1.1. Operational RBAC considerations	478
13.6.1.2. Security accounts overview	479
13.6.1.3. Identity provider configuration	479
13.6.1.4. Replacing the kubeadmin user with a cluster-admin user	479
13.6.1.5. Security considerations	480
13.6.1.6. Advancement of pod security in Kubernetes and OpenShift Container Platform	481
13.6.1.7. Bare-metal infrastructure	481
13.6.1.8. Lifecycle management	481
13.6.2. Host security	481
13.6.2.1. Red Hat Enterprise Linux CoreOS (RHCOS)	481
13.6.2.2. Command-line host access	482
13.6.2.3. Linux capabilities	484
13.6.3. Security context constraints	484

CHAPTER 1. POSTINSTALLATION CONFIGURATION OVERVIEW

After installing OpenShift Container Platform, you can configure machines, cluster services, nodes, networking, storage, users, and alert notifications to meet your operational requirements.

After installing OpenShift Container Platform, a cluster administrator can configure and customize the following components:

- Machine
- Bare metal
- Cluster
- Node
- Network
- Storage
- Users
- Alerts and notifications

1.1. POSTINSTALLATION CONFIGURATION TASKS

You can perform the postinstallation configuration tasks to configure your environment to meet your needs.

The following lists details these configurations:

- Configure operating system features: The Machine Config Operator (MCO) manages **MachineConfig** objects. By using the MCO, you can configure nodes and custom resources.
- Configure cluster features. You can modify the following features of an OpenShift Container Platform cluster:
 - Image registry
 - Networking configuration
 - Image build behavior
 - Identity provider
 - The etcd configuration
 - Machine set creation to handle the workloads
 - Cloud provider credential management
- Configuring a private cluster: By default, the installation program provisions OpenShift Container Platform by using a publicly accessible DNS and endpoints. To make your cluster accessible only from within an internal network, configure the following components to make them private:

- DNS
- Ingress Controller
- API server
- Perform node operations: By default, OpenShift Container Platform uses Red Hat Enterprise Linux CoreOS (RHCOS) compute machines. You can perform the following node operations:
 - Add and remove compute machines.
 - Add and remove taints and tolerations.
 - Configure the maximum number of pods per node.
 - Enable Device Manager.
- Configure users: Users can authenticate themselves to the API by using OAuth access tokens. You can configure OAuth to perform the following tasks:
 - Specify an identity provider.
 - Use role-based access control to define and grant permissions to users.
 - Install an Operator from the software catalog.
- Configuring alert notifications: By default, firing alerts are displayed on the Alerting UI of the web console. You can also configure OpenShift Container Platform to send alert notifications to external systems.

1.2. ADDITIONAL RESOURCES

- [Configure operating system features](#)
- [Configure cluster features](#)
- [Configuring a private cluster](#)
- [Perform node operations](#)
- [Configure users](#)
- [Configuring alert notifications](#)

CHAPTER 2. CONFIGURING A PRIVATE CLUSTER

After installing OpenShift Container Platform, you can restrict access to cluster DNS, ingress, API server, and Azure registry storage endpoints to make core cluster services private.

2.1. ABOUT PRIVATE CLUSTERS

You can make a deployed cluster private by restricting DNS, Ingress Controller, and API server access to internal networks.

By default, OpenShift Container Platform is provisioned using publicly-accessible DNS and endpoints. You can set the DNS, Ingress Controller, and API server to private after you deploy your private cluster.



IMPORTANT

If the cluster has any public subnets, load balancer services created by administrators might be publicly accessible. To ensure cluster security, verify that these services are explicitly annotated as private.

2.1.1. DNS

If you install OpenShift Container Platform on installer-provisioned infrastructure, the installation program creates records in a pre-existing public zone and, where possible, creates a private zone for the cluster's own DNS resolution. In both the public zone and the private zone, the installation program or cluster creates DNS entries for ***.apps**, for the **Ingress** object, and **api**, for the API server.

The ***.apps** records in the public and private zone are identical, so when you delete the public zone, the private zone seamlessly provides all DNS resolution for the cluster.

2.1.2. Ingress Controller

Because the default **Ingress** object is created as public, the load balancer is internet-facing and in the public subnets.

The Ingress Operator generates a default certificate for an Ingress Controller to serve as a placeholder until you configure a custom default certificate. Do not use Operator-generated default certificates in production clusters. The Ingress Operator does not rotate its own signing certificate or the default certificates that it generates. Operator-generated default certificates are intended as placeholders for custom default certificates that you configure.

2.1.3. API server

By default, the installation program creates appropriate network load balancers for the API server to use for both internal and external traffic.

On Amazon Web Services (AWS), separate public and private load balancers are created. The load balancers are identical except that an additional port is available on the internal one for use within the cluster. Although the installation program automatically creates or destroys the load balancer based on API server requirements, the cluster does not manage or maintain them. As long as you preserve the cluster's access to the API server, you can manually modify or move the load balancers. For the public load balancer, port 6443 is open and the health check is configured for HTTPS against the **/readyz** path.

On Google Cloud, a single load balancer is created to manage both internal and external API traffic, so you do not need to modify the load balancer.

On Microsoft Azure, both public and private load balancers are created. However, because of limitations in current implementation, you just retain both load balancers in a private cluster.

2.2. CONFIGURING DNS RECORDS TO BE PUBLISHED IN A PRIVATE ZONE

You can remove the public zone from the cluster DNS configuration so that new DNS records are published only to the private zone and remain available to internal clients.

For all OpenShift Container Platform clusters, whether public or private, DNS records are published in a public zone by default.

You can remove the public zone from the cluster DNS configuration to avoid exposing DNS records to the public. You might want to avoid exposing sensitive information, such as internal domain names, internal IP addresses, or the number of clusters at an organization, or you might simply have no need to publish records publicly. If all the clients that should be able to connect to services within the cluster use a private DNS service that has the DNS records from the private zone, then there is no need to have a public DNS record for the cluster.

After you deploy a cluster, you can modify its DNS to use only a private zone by modifying the **DNS** custom resource (CR). Modifying the **DNS** CR in this way means that any DNS records that are subsequently created are not published to public DNS servers, which keeps knowledge of the DNS records isolated to internal users. This can be done when you configure the cluster to be private, or if you never want DNS records to be publicly resolvable.

Alternatively, even in a private cluster, you might keep the public zone for DNS records because it allows clients to resolve DNS names for applications running on that cluster. For example, an organization can have machines that connect to the public internet and then establish VPN connections for certain private IP ranges in order to connect to private IP addresses. The DNS lookups from these machines use the public DNS to determine the private addresses of those services, and then connect to the private addresses over the VPN.

Procedure

1. Review the **DNS** CR for your cluster by running the following command and observing the output:

```
$ oc get dnses.config.openshift.io/cluster -o yaml
```

Example output

```
apiVersion: config.openshift.io/v1
kind: DNS
metadata:
  creationTimestamp: "2019-10-25T18:27:09Z"
  generation: 2
  name: cluster
  resourceVersion: "37966"
  selfLink: /apis/config.openshift.io/v1/dnses/cluster
  uid: 0e714746-f755-11f9-9cb1-02ff55d8f976
spec:
  baseDomain: <base_domain>
  privateZone:
    tags:
```

```
Name: <infrastructure_id>-int
kubernetes.io/cluster/<infrastructure_id>: owned
publicZone:
  id: Z2XXXXXXXXXXA4
status: {}
```

Note that the **spec** section contains both a private and a public zone.

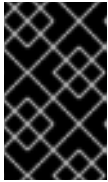
2. Patch the **DNS** CR to remove the public zone by running the following command:

```
$ oc patch dnses.config.openshift.io/cluster --type=merge --patch='{"spec": {"publicZone": null}}'
```

Example output

```
dns.config.openshift.io/cluster patched
```

The Ingress Operator consults the **DNS** CR definition when it creates DNS records for **IngressController** objects. If only private zones are specified, only private records are created.



IMPORTANT

Existing DNS records are not modified when you remove the public zone. You must manually delete previously published public DNS records if you no longer want them to be published publicly.

Verification

- Review the **DNS** CR for your cluster and confirm that the public zone was removed, by running the following command and observing the output:

```
$ oc get dnses.config.openshift.io/cluster -o yaml
```

Example output

```
apiVersion: config.openshift.io/v1
kind: DNS
metadata:
  creationTimestamp: "2019-10-25T18:27:09Z"
  generation: 2
  name: cluster
  resourceVersion: "37966"
  selfLink: /apis/config.openshift.io/v1/dnses/cluster
  uid: 0e714746-f755-11f9-9cb1-02ff55d8f976
spec:
  baseDomain: <base_domain>
  privateZone:
    tags:
      Name: <infrastructure_id>-int
      kubernetes.io/cluster/<infrastructure_id>-wfpg4: owned
status: {}
```

2.3. SETTING THE INGRESS CONTROLLER TO PRIVATE

You can configure the default Ingress Controller to use an internal endpoint so that application routes are published only in the private DNS zone.

After you deploy a cluster, you can modify its Ingress Controller to use only a private zone.

Procedure

1. Modify the default Ingress Controller to use only an internal endpoint:

```
$ oc replace --force --wait --filename - <<EOF
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  namespace: openshift-ingress-operator
  name: default
spec:
  endpointPublishingStrategy:
    type: LoadBalancerService
    loadBalancer:
      scope: Internal
EOF
```

Example output

```
ingresscontroller.operator.openshift.io "default" deleted
ingresscontroller.operator.openshift.io/default replaced
```

The public DNS entry is removed, and the private zone entry is updated.

2.4. RESTRICTING THE API SERVER TO PRIVATE FOR AN AMAZON WEB SERVICES CLUSTER

If the security posture of your organization does not allow clusters to use an open API endpoint, you can restrict the API server to use only internal load balancers. To implement this API server restriction, use the Amazon Web Services (AWS) console and OpenShift CLI (**oc**) to delete the external load balancer components.

IMPORTANT

The OpenShift CLI (**oc**) steps that remove the external load balancers require the Machine API. For clusters that cannot use the Machine API, you must manually remove the external load balancers.

Clusters with the infrastructure platform type **none** cannot use the Machine API. To view the platform type for your cluster, run the following command:

```
$ oc get infrastructure cluster -o jsonpath='{.status.platform}'
```

Prerequisites

- You have installed an OpenShift Container Platform cluster on AWS.
- You have access to the AWS console as a user with administrator privileges.
- You have access to the OpenShift CLI (**oc**) as a user with administrator privileges.

Procedure

1. Log in to the AWS console as a user with administrator privileges.
2. Delete the external load balancer.



NOTE

The API DNS entry in the private zone already points to the internal load balancer, which uses an identical configuration, so you do not need to modify the internal load balancer.

3. Delete the **api.<cluster_name>.<domain_name>** DNS entry in the public zone.
where **<cluster_name>** is the name of the cluster and **<domain_name>** is the base domain for the cluster.
4. To remove the external load balancers, log in to the OpenShift CLI (**oc**) as a user with administrator privileges.
 - If your cluster uses a control plane machine set, remove the external load balancers by editing the **ControlPlaneMachineSet** custom resource (CR).

1. Edit the **ControlPlaneMachineSet** CR by running the following command:

```
$ oc edit controlplanemachineset.machine.openshift.io cluster \
-n openshift-machine-api
```

2. Remove the external load balancers by deleting the corresponding lines in the control plane machine set custom resource (CR).
In the **spec.template.spec.providerSpec.value.loadBalancers** section of the CR, the **name** value for the external load balancer ends in **-ext**. Delete the line with the external load balancer **name** value and the line with the external load balancer **type** value that accompanies it.

```
apiVersion: machine.openshift.io/v1
kind: ControlPlaneMachineSet
metadata:
  name: cluster
  namespace: openshift-machine-api
spec:
  # ...
  template:
    # ...
    spec:
      providerSpec:
        value:
          loadBalancers:
            - name: <cluster_id>-ext
```

```

        type: network
      - name: <cluster_id>-int
        type: network
    # ...

```

3. Save your changes and exit the object specification.

When you save an update to the control plane machine set, the Control Plane Machine Set Operator updates the control plane machines according to your configured update strategy. For more information, see "Updating the control plane configuration".

- If your cluster does not use a control plane machine set, you must delete the external load balancers from each control plane machine.
 - i. List the cluster machines by running the following command:

```
$ oc get machine -n openshift-machine-api
```

Example output

```

NAME                                STATE  TYPE    REGION  ZONE    AGE
<cluster_id>-master-0              running m4.xlarge us-east-1 us-east-1a 17m
<cluster_id>-master-1              running m4.xlarge us-east-1 us-east-1b 17m
<cluster_id>-master-2              running m4.xlarge us-east-1 us-east-1a 17m
<cluster_id>-worker-us-east-1a-<zone_tag> running m4.xlarge us-east-1 us-east-1a 15m
<cluster_id>-worker-us-east-1a-<zone_tag> running m4.xlarge us-east-1 us-east-1a 15m
<cluster_id>-worker-us-east-1b-<zone_tag> running m4.xlarge us-east-1 us-east-1b 15m

```

The control plane machines contain the **master** string in their names.

- ii. Remove the external load balancer from each control plane machine:

- A. Edit a control plane machine object to by running the following command:

```
$ oc edit machines -n openshift-machine-api <control_plane_machine_name>
```

where **<control_plane_machine_name>** is the name of the control plane machine object to modify.

- B. Remove the lines that describe the external load balancer.

In the **spec.providerSpec.value.loadBalancers** section of the CR, the **name** value for the external load balancer ends in **-ext**. Delete the line with the external load balancer **name** value and the the line with the external load balancer **type** value that accompanies it.

```

apiVersion: machine.openshift.io/v1beta1
kind: Machine
metadata:
  name: <control_plane_machine_name>
  namespace: openshift-machine-api
spec:
  providerSpec:
    value:

```

```
loadBalancers:
- name: <cluster_id>-ext
  type: network
- name: <cluster_id>-int
  type: network
# ...
```

C. Save your changes and exit the object specification.

D. Repeat this process for each control plane machine.

Additional resources

- [Updating the control plane configuration](#)

2.5. RESTRICTING THE API SERVER TO PRIVATE FOR AN MICROSOFT AZURE CLUSTER

If the security posture of your organization does not allow clusters to use an open API endpoint, you can restrict the API server to use only internal load balancers. To implement this API server restriction, use the Microsoft Azure console to delete the external load balancer component.

Prerequisites

- You have installed an OpenShift Container Platform cluster on Azure.
- You have access to the Azure console as a user with administrator privileges.

Procedure

1. Log in to the Azure console as a user with administrator privileges.
2. Delete the following resources:
 - The **api-v4** rule for the public load balancer.
 - The **frontendIPConfiguration** parameter that is associated with the **api-v4** rule for the public load balancer.
 - The public IP address that is specified in the **frontendIPConfiguration** parameter.
3. Configure the Ingress Controller endpoint publishing scope to **Internal**. For more information, see "Configuring the Ingress Controller endpoint publishing scope to Internal".
4. Delete the **api.<cluster_name>** DNS entry in the public zone.
where **<cluster_name>** is the name of the cluster.

Additional resources

- [Configuring the Ingress Controller endpoint publishing scope to Internal](#)

2.6. CONFIGURING A PRIVATE STORAGE ENDPOINT ON AZURE

You can configure the Image Registry Operator to use a private Azure storage endpoint so that registry storage is not exposed through a public-facing endpoint.

You can leverage the Image Registry Operator to use private endpoints on Azure, which enables seamless configuration of private storage accounts when OpenShift Container Platform is deployed on private Azure clusters. This allows you to deploy the image registry without exposing public-facing storage endpoints.



IMPORTANT

Do not configure a private storage endpoint on Microsoft Azure Red Hat OpenShift (ARO), because the endpoint can put your Microsoft Azure Red Hat OpenShift cluster in an unrecoverable state.

You can configure the Image Registry Operator to use private storage endpoints on Azure in one of two ways:

- By configuring the Image Registry Operator to discover the VNet and subnet names
- With user-provided Azure Virtual Network (VNet) and subnet names

2.6.1. Limitations for configuring a private storage endpoint on Azure

The following limitations apply when configuring a private storage endpoint on Azure:

- When configuring the Image Registry Operator to use a private storage endpoint, public network access to the storage account is disabled. Consequently, pulling images from the registry outside of OpenShift Container Platform only works by setting **disableRedirect: true** in the registry Operator configuration. With redirect enabled, the registry redirects the client to pull images directly from the storage account, which will no longer work due to disabled public network access. For more information, see "Disabling redirect when using a private storage endpoint on Azure".
- This operation cannot be undone by the Image Registry Operator.

2.6.2. Configuring a private storage endpoint on Azure by enabling the Image Registry Operator to discover VNet and subnet names

You can configure a private Azure storage endpoint by enabling the Image Registry Operator to discover the VNet and subnet, allowing registry storage without public network access.

The following procedure shows you how to set up a private storage endpoint on Azure by configuring the Image Registry Operator to discover VNet and subnet names.

Prerequisites

- You have configured the image registry to run on Azure.
- Your network has been set up using the Installer Provisioned Infrastructure installation method. For users with a custom network setup, see "Configuring a private storage endpoint on Azure with user-provided VNet and subnet names".

Procedure

1. Edit the Image Registry Operator **config** object and set **networkAccess.type** to **Internal**:

```
$ oc edit configs.imageregistry/cluster
```

```
# ...
spec:
  # ...
  storage:
    azure:
      # ...
      networkAccess:
        type: Internal
  # ...
```

- Optional: Enter the following command to confirm that the Operator has completed provisioning. This might take a few minutes.

```
$ oc get configs.imageregistry/cluster -o=jsonpath="{.spec.storage.azure.privateEndpointName}" -w
```

- Optional: If the registry is exposed by a route, and you are configuring your storage account to be private, you must disable redirect if you want pulls external to the cluster to continue to work. Enter the following command to disable redirect on the Image Operator configuration:

```
$ oc patch configs.imageregistry cluster --type=merge -p '{"spec":{"disableRedirect": true}}'
```



NOTE

When redirect is enabled, pulling images from outside of the cluster will not work.

Verification

- Fetch the registry service name by running the following command:

```
$ oc get imagestream -n openshift
```

Example output

```
NAME IMAGE REPOSITORY TAGS UPDATED
cli image-registry.openshift-image-registry.svc:5000/openshift/cli latest 8 hours ago
...
```

- Enter debug mode by running the following command:

```
$ oc debug node/<node_name>
```

- Run the suggested **chroot** command. For example:

```
$ chroot /host
```

- Enter the following command to log in to your container registry:

```
$ podman login --tls-verify=false -u unused -p $(oc whoami -t) image-registry.openshift-image-registry.svc:5000
```

■

Example output

```
Login Succeeded!
```

5. Enter the following command to verify that you can pull an image from the registry:

```
$ podman pull --tls-verify=false image-registry.openshift-image-registry.svc:5000/openshift/tools
```

Example output

```
Trying to pull image-registry.openshift-image-registry.svc:5000/openshift/tools/openshift/tools...
Getting image source signatures
Copying blob 6b245f040973 done
Copying config 22667f5368 done
Writing manifest to image destination
Storing signatures
22667f53682a2920948d19c7133ab1c9c3f745805c14125859d20cede07f11f9
```

2.6.3. Configuring a private storage endpoint on Azure with user-provided VNet and subnet names

You can configure a private Azure storage endpoint for the image registry by specifying user-provided VNet and subnet names, enabling registry storage without public network access.

Use the following procedure to configure a storage account that has public network access disabled and is exposed behind a private storage endpoint on Azure.

Prerequisites

- You have configured the image registry to run on Azure.
- You must know the VNet and subnet names used for your Azure environment.
- If your network was configured in a separate resource group in Azure, you must also know its name.

Procedure

1. Edit the Image Registry Operator **config** object and configure the private endpoint using your VNet and subnet names:

```
$ oc edit configs.imageregistry/cluster
```

```
# ...
spec:
# ...
storage:
  azure:
    # ...
    networkAccess:
```

```

type: Internal
internal:
  subnetName: <subnet_name>
  vnetName: <vnet_name>
  networkResourceGroupName: <network_resource_group_name>
# ...

```

- Optional: Enter the following command to confirm that the Operator has completed provisioning. This might take a few minutes.

```
$ oc get configs.imageregistry/cluster -o=jsonpath="{.spec.storage.azure.privateEndpointName}" -w
```



NOTE

When redirect is enabled, pulling images from outside of the cluster will not work.

Verification

- Fetch the registry service name by running the following command:

```
$ oc get imagestream -n openshift
```

Example output

```

NAME      IMAGE REPOSITORY          TAGS    UPDATED
cli      image-registry.openshift-image-registry.svc:5000/openshift/cli  latest  8 hours ago
...

```

- Enter debug mode by running the following command:

```
$ oc debug node/<node_name>
```

- Run the suggested **chroot** command. For example:

```
$ chroot /host
```

- Enter the following command to log in to your container registry:

```
$ podman login --tls-verify=false -u unused -p $(oc whoami -t) image-registry.openshift-image-registry.svc:5000
```

Example output

```
Login Succeeded!
```

- Enter the following command to verify that you can pull an image from the registry:

```
$ podman pull --tls-verify=false image-registry.openshift-image-registry.svc:5000/openshift/tools
```

Example output

```
Trying to pull image-registry.openshift-image-
registry.svc:5000/openshift/tools/openshift/tools...
Getting image source signatures
Copying blob 6b245f040973 done
Copying config 22667f5368 done
Writing manifest to image destination
Storing signatures
22667f53682a2920948d19c7133ab1c9c3f745805c14125859d20cede07f11f9
```

2.6.4. Optional: Disabling redirect when using a private storage endpoint on Azure

You can disable redirect when using a private Azure storage endpoint so that users outside the cluster can pull images through the image registry route.

By default, redirect is enabled when using the image registry. Redirect allows off-loading of traffic from the registry pods into the object storage, which makes pull faster. When redirect is enabled and the storage account is private, users from outside of the cluster are unable to pull images from the registry.

In some cases, users might want to disable redirect so that users from outside of the cluster can pull images from the registry.

Prerequisites

- You have configured the image registry to run on Azure.
- You have configured a route.

Procedure

- Enter the following command to disable redirect on the image registry configuration:

```
$ oc patch configs.imageregistry cluster --type=merge -p '{"spec":{"disableRedirect": true}}'
```

Verification

1. Fetch the registry service name by running the following command:

```
$ oc get imagestream -n openshift
```

Example output

```
NAME      IMAGE REPOSITORY      TAGS      UPDATED
cli      default-route-openshift-image-registry.<cluster_dns>/cli  latest    8 hours ago
...
```

2. Enter the following command to log in to your container registry:

```
$ podman login --tls-verify=false -u unused -p $(oc whoami -t) default-route-openshift-image-
registry.<cluster_dns>
```

Example output

Login Succeeded!

3. Enter the following command to verify that you can pull an image from the registry:

```
$ podman pull --tls-verify=false default-route-openshift-image-registry.<cluster_dns>
/openshift/tools
```

Example output

```
Trying to pull default-route-openshift-image-registry.<cluster_dns>/openshift/tools...
Getting image source signatures
Copying blob 6b245f040973 done
Copying config 22667f5368 done
Writing manifest to image destination
Storing signatures
22667f53682a2920948d19c7133ab1c9c3f745805c14125859d20cede07f11f9
```

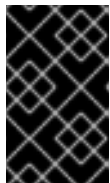
CHAPTER 3. CONFIGURING MULTI-ARCHITECTURE COMPUTE MACHINES ON AN OPENSIFT CLUSTER

3.1. ABOUT CLUSTERS WITH MULTI-ARCHITECTURE COMPUTE MACHINES

An OpenShift Container Platform cluster with multi-architecture compute machines is a cluster that supports compute machines with different architectures.

Configuring multi-architecture compute machines involves some additional considerations:

- When there are nodes with multiple architectures in your cluster, the architecture of the container image that you deploy to a node must be consistent with the architecture of that node. You need to ensure that the pod is assigned to the node with the appropriate architecture and that it matches the container image architecture. For more information on assigning pods to nodes, see "Assigning pods to nodes".
- In installer-provisioned installations, you are restricted to using the infrastructure provided by a single cloud provider. Adding external nodes, regardless of their architecture, to these clusters is not supported.
- Clusters that are installed with the platform type **none** are unable to use some features, such as managing compute machines with the Machine API. This limitation applies even if the compute machines that are attached to the cluster are installed on a platform that would normally support the feature. This parameter cannot be changed after installation.



IMPORTANT

See "Deploying OpenShift 4.x on non-tested platforms using the bare metal install method" before you attempt to install an OpenShift Container Platform cluster in virtualized or cloud environments.

- The Cluster Samples Operator is not supported on clusters with multi-architecture compute machines. Your cluster can be created without this capability. For more information, see "Cluster capabilities".
- For information on migrating your single-architecture cluster to a cluster that supports multi-architecture compute machines, see "Migrating to a cluster with multi-architecture compute machines".

3.1.1. Configuring your cluster with multi-architecture compute machines

To create a cluster with multi-architecture compute machines with different installation options and platforms, see the documentation references.

Table 3.1. Cluster with multi-architecture compute machine installation options

Documentation section	Platform	User-provisioned installation	Installer-provisioned installation	Control Plane	Compute node
"Creating a cluster with multi-architecture compute machines on Azure"	Microsoft Azure	✓	✓	aarch64 or x86_64	aarch64 , x86_64
"Creating a cluster with multi-architecture compute machines on AWS"	Amazon Web Services (AWS)	✓	✓	aarch64 or x86_64	aarch64 , x86_64
"Creating a cluster with multi-architecture compute machines on Google Cloud"	Google Cloud		✓	aarch64 or x86_64	aarch64 , x86_64
"Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z"	Bare metal	✓	✓	aarch64 or x86_64	aarch64 , x86_64
	IBM Power	✓		x86_64 or ppc64le	x86_64 , ppc64le
	IBM Z	✓		x86_64 or s390x	x86_64 , s390x
"Creating a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE with z/VM"	IBM Z® and IBM® LinuxONE	✓		x86_64 , s390x	x86_64 , s390x
"Creating a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE with RHEL KVM"	IBM Z® and IBM® LinuxONE	✓		x86_64 , s390x	x86_64 , s390x
"Creating a cluster with multi-architecture compute machines on IBM Power®"	IBM Power®	✓		x86_64	x86_64 , ppc64le

Additional resources

- [Creating a cluster with multi-architecture compute machines on Azure](#)
- [Creating a cluster with multi-architecture compute machines on AWS](#)

- [Creating a cluster with multi-architecture compute machines on Google Cloud](#)
- [Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z](#)
- [Creating a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE with z/VM](#)
- [Creating a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE with RHEL KVM](#)
- [Creating a cluster with multi-architecture compute machines on IBM Power®](#)

3.1.2. Verifying cluster compatibility

Before you can start adding compute nodes of different architectures to your cluster, you must verify that your cluster is multi-architecture compatible.

Prerequisites

- You installed the OpenShift CLI (**oc**).
- IBM Power only: Ensure that you meet the following prerequisites:
 - When using multiple architectures, hosts for OpenShift Container Platform nodes must share the same storage layer. If they do not have the same storage layer, use a storage provider such as **nfs-provisioner**.
 - You should limit the number of network hops between the compute and control plane as much as possible.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. You can check that your cluster uses the architecture payload by running the following command:

```
$ oc adm release info -o jsonpath="{.metadata.metadata}"
```

Verification

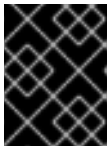
- If you see the following output, your cluster is using the multi-architecture payload:

```
{
  "release.openshift.io/architecture": "multi",
  "url": "https://access.redhat.com/errata/<errata_version>"
}
```

You can then begin adding multi-arch compute nodes to your cluster.

- If you see the following output, your cluster is not using the multi-architecture payload:

```
{  
  "url": "https://access.redhat.com/errata/<errata_version>"  
}
```



IMPORTANT

To migrate your cluster so the cluster supports multi-architecture compute machines, see "Migrating to a cluster with multi-architecture compute machines".

Additional resources

- [Migrating to a cluster with multi-architecture compute machines](#)

3.1.3. Additional resources

- [Assigning pods to nodes](#)
- [Deploying OpenShift 4.x on non-tested platforms using the bare metal install method \(Red Hat Knowledgebase article\)](#)
- [Cluster capabilities](#)
- [Migrating to a cluster with multi-architecture compute machines](#)

3.2. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON AWS

To deploy a cluster on Amazon Web Services (AWS) with multi-architecture compute machines, you must first create a single-architecture installer-provisioned cluster that uses the multi-architecture installer binary.

You can also migrate your current cluster with single-architecture compute machines to a cluster with multi-architecture compute machines. After creating a multi-architecture cluster, you can add nodes with different architectures to the cluster.

3.2.1. Adding a multi-architecture compute machine set to your AWS cluster

After creating a multi-architecture cluster, you can add nodes with different architectures.

You can add multi-architecture compute machines to a multi-architecture cluster in the following ways:

- Adding 64-bit x86 compute machines to a cluster that uses 64-bit ARM control plane machines and already includes 64-bit ARM compute machines. In this case, 64-bit x86 is considered the secondary architecture.
- Adding 64-bit ARM compute machines to a cluster that uses 64-bit x86 control plane machines and already includes 64-bit x86 compute machines. In this case, 64-bit ARM is considered the secondary architecture.



NOTE

Before adding a secondary architecture node to your cluster, it is recommended to install the Multiarch Tuning Operator, and deploy a **ClusterPodPlacementConfig** custom resource. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

Prerequisites

- You installed the OpenShift CLI (**oc**).
- You used the installation program to create a 64-bit ARM or 64-bit x86 single-architecture cluster with the multi-architecture installer binary.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a YAML file and add the configuration to create a compute machine set to control the 64-bit ARM or 64-bit x86 compute nodes in your cluster.

Example **MachineSet** object for an AWS 64-bit ARM or x86 compute node

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
  name: <infrastructure_id>-aws-machine-set-0
  namespace: openshift-machine-api
spec:
  replicas: 1
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: <infrastructure_id>
      machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>-<zone>
  template:
    metadata:
      labels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machine-role: <role>
        machine.openshift.io/cluster-api-machine-type: <role>
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>-<zone>
    spec:
      metadata:
        labels:
          node-role.kubernetes.io/<role>: ""
      providerSpec:
        value:
          ami:
            id: ami-02a574449d4f4d280
          apiVersion: awsproviderconfig.openshift.io/v1beta1
          blockDevices:
            - ebs:
                iops: 0
```

```

        volumeSize: 120
        volumeType: gp2
    credentialsSecret:
        name: aws-cloud-credentials
    deviceIndex: 0
    iamInstanceProfile:
        id: <infrastructure_id>-worker-profile
    instanceType: m6g.xlarge
    kind: AWSMachineProviderConfig
    placement:
        availabilityZone: us-east-1a
        region: <region>
    securityGroups:
        - filters:
            - name: tag:Name
              values:
                - <infrastructure_id>-node
    subnet:
        filters:
            - name: tag:Name
              values:
                - <infrastructure_id>-subnet-private-<zone>
    tags:
        - name: kubernetes.io/cluster/<infrastructure_id>
          value: owned
        - name: <custom_tag_name>
          value: <custom_tag_value>
    userDataSecret:
        name: worker-user-data

```

where:

<infrastructure_id>

Specifies the infrastructure ID that is based on the cluster ID that you set when you provisioned the cluster. If you have the OpenShift CLI (**oc**) installed, you can obtain the infrastructure ID by running the following command:

```
$ oc get -o jsonpath="{.status.infrastructureName}" infrastructure cluster
```

<role>-<zone>

Specifies the infrastructure ID, role node label, and zone.

<role>

Specifies the role node label to add.

ami.id

Specifies a Red Hat Enterprise Linux CoreOS (RHCOS) Amazon Machine Image (AMI) for your AWS region for the nodes. The RHCOS AMI must be compatible with the machine architecture.

```

$ oc get configmap/coreos-bootimages \
  -n openshift-machine-config-operator \
  -o jsonpath='{.data.stream}' | jq \
  -r '.architectures.<arch>.images.aws.regions."<region>".image'

```

instanceType

Specifies a machine type that aligns with the CPU architecture of the chosen AML. For more information, see "Tested instance types for AWS 64-bit ARM".

availabilityZone

Specifies the zone. For example, **us-east-1a**. Ensure that the zone you select has machines with the required architecture.

region

Specifies the region. For example, **us-east-1**. Ensure that the zone you select has machines with the required architecture.

3. Create the compute machine set by running the following command:

```
$ oc create -f <file_name>
```

- Replace **<file_name>** with the name of the YAML file with compute machine set configuration. For example: **aws-arm64-machine-set-0.yaml**, or **aws-amd64-machine-set-0.yaml**.

Verification

1. Verify that the new machines are running by running the following command:

```
$ oc get machineset -n openshift-machine-api
```

The output must include the machine set that you created.

Example output

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
<infrastructure_id>-aws-machine-set-0	2	2	2	2	10m

2. You can check if the nodes are ready and schedulable by running the following command:

```
$ oc get nodes
```

3.2.2. Additional resources

- [Installing a cluster on AWS with customizations](#)
- [Migrating to a cluster with multi-architecture compute machines](#)
- [Tested instance types for AWS 64-bit ARM](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.3. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON AZURE

To deploy a cluster on Microsoft Azure with multi-architecture compute machines, you must first create a single-architecture installer-provisioned cluster that uses the multi-architecture installer binary.

You can also migrate your current cluster with single-architecture compute machines to a cluster with multi-architecture compute machines. After creating a multi-architecture cluster, you can add nodes with different architectures to the cluster.

3.3.1. Creating a 64-bit ARM boot image using the Azure image gallery

You can generate a 64-bit x86 boot image or a 64-bit ARM boot image by using the Azure image gallery.

The procedure example describes how to manually generate a 64-bit ARM boot image. If you want to generate a 64-bit x86 boot image, replace **aarch64** with **x86_64**; Additionally, replace any instances of **rhcos-arm64** with **rhcos-x86_64**.

Prerequisites

- You installed the Azure CLI (**az**).
- You created a single-architecture Azure installer-provisioned cluster with the multi-architecture installer binary.

Procedure

1. Log in to your Azure account by running the following command:

```
$ az login
```

2. Create a storage account and upload the **aarch64** virtual hard drive (VHD) to your storage account. The OpenShift Container Platform installation program creates a resource group, however, the boot image can also be uploaded to a custom named resource group:

```
$ az storage account create -n ${STORAGE_ACCOUNT_NAME} -g  
${RESOURCE_GROUP} -l westus --sku Standard_LRS
```

- The **westus** object is an example region.

3. Create a storage container using the storage account you generated by entering the following command:

```
$ az storage container create -n ${CONTAINER_NAME} --account-name  
${STORAGE_ACCOUNT_NAME}
```

4. You must use the OpenShift Container Platform installation program JSON file to extract the URL and **aarch64** VHD name:

- a. Extract the **URL** field and set it to **RHCOS_VHD_ORIGIN_URL** as the file name by running the following command:

```
$ RHCOS_VHD_ORIGIN_URL=$(oc -n openshift-machine-config-operator get  
configmap/coreos-bootimages -o jsonpath='{.data.stream}' | jq -r  
'architectures.aarch64."rhel-coreos-extensions"."azure-disk".url')
```

- For a 64-bit x86 boot image, replace **architectures.aarch64** with **architectures.x86_64**.

- b. Extract the **aarch64** VHD name and set it to **BLOB_NAME** as the file name by running the following command:

```
$ BLOB_NAME=$(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' | jq -r '.architectures.aarch64."rhel-coreos-extensions"."azure-disk".release')-azure.aarch64.vhd
```

- For a 64-bit x86 boot image, replace **architectures.aarch64** with **architectures.x86_64** and replace **aarch64.vhd** with **x86_64.vhd**.
5. Generate a shared access signature (SAS) token. Use this token to upload the RHCOS VHD to your storage container with the following commands:

```
$ end=`date -u -d "30 minutes" '+%Y-%m-%dT%H:%MZ'`
```

```
$ sas=`az storage container generate-sas -n ${CONTAINER_NAME} --account-name ${STORAGE_ACCOUNT_NAME} --https-only --permissions dlrw --expiry $end -o tsv`
```

6. Copy the RHCOS VHD into the storage container:

```
$ az storage blob copy start --account-name ${STORAGE_ACCOUNT_NAME} --sas-token "$sas" \
--source-uri "${RHCOS_VHD_ORIGIN_URL}" \
--destination-blob "${BLOB_NAME}" --destination-container ${CONTAINER_NAME}
```

You can check the status of the copying process with the following command:

```
$ az storage blob show -c ${CONTAINER_NAME} -n ${BLOB_NAME} --account-name ${STORAGE_ACCOUNT_NAME} | jq .properties.copy
```

Example output

```
{
  "completionTime": null,
  "destinationSnapshot": null,
  "id": "1fd97630-03ca-489a-8c4e-cfe839c9627d",
  "incrementalCopy": null,
  "progress": "17179869696/17179869696",
  "source": "https://rhcos.blob.core.windows.net/imagebucket/rhcos-411.86.202207130959-0-azure.aarch64.vhd",
  "status": "success",
  "statusDescription": null
}
```

If the status parameter displays the **success** object, the copying process is complete.

7. Create an image gallery using the following command:

```
$ az sig create --resource-group ${RESOURCE_GROUP} --gallery-name ${GALLERY_NAME}
```

Use the image gallery to create an image definition. In the following example command, **rhcos-arm64** is the name of the image definition.

```
$ az sig image-definition create --resource-group ${RESOURCE_GROUP} --gallery-name
${GALLERY_NAME} --gallery-image-definition rhcos-arm64 --publisher RedHat --offer arm -
-sku arm64 --os-type linux --architecture Arm64 --hyper-v-generation V2
```

- For a 64-bit x86 boot image, replace **--offer arm --sku arm64** with `--offer x86_64 --sku x86_64``.
8. To get the URL of the VHD and set it to **RHCOS_VHD_URL** as the file name, run the following command:

```
$ RHCOS_VHD_URL=$(az storage blob url --account-name
${STORAGE_ACCOUNT_NAME} -c ${CONTAINER_NAME} -n "${BLOB_NAME}" -o tsv)
```

9. Use the **RHCOS_VHD_URL** file, your storage account, resource group, and image gallery to create an image version. In the following example, **1.0.0** is the image version.

```
$ az sig image-version create --resource-group ${RESOURCE_GROUP} --gallery-name
${GALLERY_NAME} --gallery-image-definition rhcos-arm64 --gallery-image-version 1.0.0 --
os-vhd-storage-account ${STORAGE_ACCOUNT_NAME} --os-vhd-uri
${RHCOS_VHD_URL}
```

10. Optional: Now that your **arm64** boot image is now generated, you can access the ID of your image with the following command:

```
$ az sig image-version show -r $GALLERY_NAME -g $RESOURCE_GROUP -i rhcos-arm64
-e 1.0.0
```

The following example image ID is used in the **resourceID** parameter of the compute machine set:

Example resourceID

```
/resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Compute/galleries/${GALLERY
_NAME}/images/rhcos-arm64/versions/1.0.0
```

3.3.2. Adding a multi-architecture compute machine set to your Azure cluster

After creating a multi-architecture cluster, you can add nodes with different architectures.

You can add multi-architecture compute machines to a multi-architecture cluster in the following ways:

- Adding 64-bit x86 compute machines to a cluster that uses 64-bit ARM control plane machines and already includes 64-bit ARM compute machines. In this case, 64-bit x86 is considered the secondary architecture.
- Adding 64-bit ARM compute machines to a cluster that uses 64-bit x86 control plane machines and already includes 64-bit x86 compute machines. In this case, 64-bit ARM is considered the secondary architecture.

To create a custom compute machine set on Azure, see "Creating a compute machine set on Azure".



NOTE

Before adding a secondary architecture node to your cluster, it is recommended to install the Multiarch Tuning Operator, and deploy a **ClusterPodPlacementConfig** custom resource. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

Prerequisites

- You installed the OpenShift CLI (**oc**).
- You created a 64-bit ARM or 64-bit x86 boot image.
- You used the installation program to create a 64-bit ARM or 64-bit x86 single-architecture cluster with the multi-architecture installer binary.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a YAML file and add the configuration to create a compute machine set to control the 64-bit ARM or 64-bit x86 compute nodes in your cluster.

Example **MachineSet** object for an Azure 64-bit ARM or x86 compute node

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
    machine.openshift.io/cluster-api-machine-role: worker
    machine.openshift.io/cluster-api-machine-type: worker
  name: <infrastructure_id>-machine-set-0
  namespace: openshift-machine-api
spec:
  replicas: 2
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: <infrastructure_id>
      machine.openshift.io/cluster-api-machineset: <infrastructure_id>-machine-set-0
  template:
    metadata:
      labels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machine-role: worker
        machine.openshift.io/cluster-api-machine-type: worker
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-machine-set-0
    spec:
      lifecycleHooks: {}
      metadata: {}
      providerSpec:
        value:
          acceleratedNetworking: true
          apiVersion: machine.openshift.io/v1beta1
          credentialsSecret:
            name: azure-cloud-credentials
```

```

    namespace: openshift-machine-api
  image:
    offer: ""
    publisher: ""
    resourceID:
      /resourceGroups/${RESOURCE_GROUP}/providers/Microsoft.Compute/galleries/${GALLERY_NAME}/images/rhcos-arm64/versions/1.0.0
    sku: ""
    version: ""
  kind: AzureMachineProviderSpec
  location: <region>
  managedIdentity: <infrastructure_id>-identity
  networkResourceGroup: <infrastructure_id>-rg
  osDisk:
    diskSettings: {}
    diskSizeGB: 128
    managedDisk:
      storageAccountType: Premium_LRS
    osType: Linux
  publicIP: false
  publicLoadBalancer: <infrastructure_id>
  resourceGroup: <infrastructure_id>-rg
  subnet: <infrastructure_id>-worker-subnet
  userDataSecret:
    name: worker-user-data
  vmSize: Standard_D4ps_v5
  vnet: <infrastructure_id>-vnet
  zone: "<zone>"

```

where:

image.resourceID

Specifies the boot image, such as **arm64** or **amd64**.

vmSize

Specifies the instance type used in your installation. Some example instance types are **Standard_D4ps_v5** or **D8ps**.

3. Create the compute machine set by running the following command:

```
$ oc create -f <file_name>
```

- Replace **<file_name>** with the name of the YAML file with compute machine set configuration. For example: **arm64-machine-set-0.yaml**, or **amd64-machine-set-0.yaml**.

Verification

1. Verify that the new machines are running by running the following command:

```
$ oc get machineset -n openshift-machine-api
```

The output must include the machine set that you created.

Example output

■

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
<infrastructure_id>-machine-set-0	2	2	2	2	10m

- You can check if the nodes are ready and schedulable by running the following command:

```
$ oc get nodes
```

3.3.3. Additional resources

- [Installing a cluster on Azure with customizations](#)
- [Migrating to a cluster with multi-architecture compute machines](#)
- [Creating a compute machine set on Azure](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.4. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON GOOGLE CLOUD

To deploy a cluster on Google Cloud with multi-architecture compute machines, you must first create a single-architecture installer-provisioned cluster that uses the multi-architecture installer binary.

You can also migrate your current cluster with single-architecture compute machines to a cluster with multi-architecture compute machines. After creating a multi-architecture cluster, you can add nodes with different architectures to the cluster.

3.4.1. Adding a multi-architecture compute machine set to your Google Cloud cluster

After creating a multi-architecture cluster, you can add nodes with different architectures.

You can add multi-architecture compute machines to a multi-architecture cluster in the following ways:

- Adding 64-bit x86 compute machines to a cluster that uses 64-bit ARM control plane machines and already includes 64-bit ARM compute machines. In this case, 64-bit x86 is considered the secondary architecture.
- Adding 64-bit ARM compute machines to a cluster that uses 64-bit x86 control plane machines and already includes 64-bit x86 compute machines. In this case, 64-bit ARM is considered the secondary architecture.



NOTE

Before adding a secondary architecture node to your cluster, it is recommended to install the Multiarch Tuning Operator, and deploy a **ClusterPodPlacementConfig** custom resource. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

Prerequisites

- You installed the OpenShift CLI (**oc**).

- You used the installation program to create a 64-bit ARM or 64-bit x86 single-architecture cluster with the multi-architecture installer binary.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a YAML file and add the configuration to create a compute machine set to control the 64-bit ARM or 64-bit x86 compute nodes in your cluster.

Example **MachineSet** object for a Google Cloud 64-bit ARM or 64-bit x86 compute node

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
  name: <infrastructure_id>-w-a
  namespace: openshift-machine-api
spec:
  replicas: 1
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: <infrastructure_id>
      machine.openshift.io/cluster-api-machineset: <infrastructure_id>-w-a
  template:
    metadata:
      creationTimestamp: null
      labels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machine-role: <role>
        machine.openshift.io/cluster-api-machine-type: <role>
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-w-a
    spec:
      metadata:
        labels:
          node-role.kubernetes.io/<role>: ""
      providerSpec:
        value:
          apiVersion: gcpprovider.openshift.io/v1beta1
          canIPForward: false
          credentialsSecret:
            name: gcp-cloud-credentials
          deletionProtection: false
          disks:
            - autoDelete: true
              boot: true
              image: <path_to_image>
              labels: null
              sizeGb: 128
              type: pd-ssd
          gcpMetadata:
            - key: <custom_metadata_key>
              value: <custom_metadata_value>
          kind: GCPMachineProviderSpec
```

```

machineType: n1-standard-4
metadata:
  creationTimestamp: null
networkInterfaces:
- network: <infrastructure_id>-network
  subnetwork: <infrastructure_id>-worker-subnet
projectId: <project_name>
region: us-central1
serviceAccounts:
- email: <infrastructure_id>-w@<project_name>.iam.gserviceaccount.com
  scopes:
  - https://www.googleapis.com/auth/cloud-platform
tags:
- <infrastructure_id>-worker
userDataSecret:
  name: worker-user-data
zone: us-central1-a

```

where:

<infrastructure_id>

Specifies the infrastructure ID that is based on the cluster ID that you set when you provisioned the cluster. You can obtain the infrastructure ID by running the following command:

```
$ oc get -o jsonpath='{.status.infrastructureName}' infrastructure cluster
```

<role>

Specifies the role node label to add.

<path_to_image>

Specifies the path to the image that is used in current compute machine sets. You need the project and image name for your path to image.

To access the project and image name, run the following command:

```
$ oc get configmap/coreos-bootimages \
-n openshift-machine-config-operator \
-o jsonpath='{.data.stream}' | jq \
-r '.architectures.aarch64.images.gcp'
```

Example output

```

"gcp": {
  "release": "415.92.202309142014-0",
  "project": "rhcos-cloud",
  "name": "rhcos-415-92-202309142014-0-gcp-aarch64"
}

```

Use the **project** and **name** parameters from the output to create the path to image field in your machine set. The path to the image should follow the following format:

```
$ projects/<project>/global/images/<image_name>
```

gcpMetadata

Optional parameter. Specifies custom metadata in the form of a **key:value** pair. For example use cases, see "Setting custom metadata".

machineType

Specifies a machine type that aligns with the CPU architecture of the chosen OS image. For more information, see "Tested instance types for Google Cloud on 64-bit ARM infrastructures".

projectID

Specifies the name of the Google Cloud project that you use for your cluster.

region

Specifies the region. For example, **us-central1**. Ensure that the zone you select has machines with the required architecture.

3. Create the compute machine set by running the following command:

```
$ oc create -f <file_name>
```

- Replace **<file_name>** with the name of the YAML file with compute machine set configuration. For example: **gcp-arm64-machine-set-0.yaml**, or **gcp-amd64-machine-set-0.yaml**.

Verification

1. Verify that the new machines are running by running the following command:

```
$ oc get machineset -n openshift-machine-api
```

The output must include the machine set that you created.

Example output

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
<infrastructure_id>-gcp-machine-set-0	2	2	2	2	10m

2. You can check if the nodes are ready and schedulable by running the following command:

```
$ oc get nodes
```

3.4.2. Additional resources

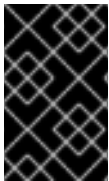
- [Migrating to a cluster with multi-architecture compute machines](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)
- [Tested instance types for Google Cloud on 64-bit ARM infrastructures](#)
- [Setting custom metadata](#)

3.5. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON BARE METAL, IBM POWER, OR IBM Z

You can create a cluster with multi-architecture compute machines on bare metal (**x86_64** or **aarch64**), IBM Power® (**ppc64le**), or IBM Z® (**s390x**). To do this, you must have an existing single-architecture cluster on one of these platforms.

See the following installation procedures for your platform:

- Bare metal with user-provisioned infrastructure: See "Installing a user-provisioned cluster on bare metal". You can then add 64-bit ARM compute machines to your OpenShift Container Platform cluster on bare metal.
- IBM Power®: See "Installing on IBM Power®". You can then add **x86_64** compute machines to your OpenShift Container Platform cluster on IBM Power®.
- IBM Z® and IBM® LinuxONE: See "Installing on IBM Z® and IBM® LinuxONE". You can then add **x86_64** compute machines to your OpenShift Container Platform cluster on IBM Z® and IBM® LinuxONE.



IMPORTANT

The bare metal installer-provisioned infrastructure and the Bare Metal Operator do not support adding secondary architecture nodes during the initial cluster setup. You can add secondary architecture nodes manually only after the initial cluster setup.

Before you can add additional compute nodes to your cluster, you must upgrade your cluster to one that uses the multi-architecture payload. For more information about migrating to the multi-architecture payload, see "Migrating to a cluster with multi-architecture compute machines".

The following procedures explain how to create a RHCOS compute machine by using an ISO image or network PXE booting. This allows you to add additional nodes to your cluster and deploy a cluster with multi-architecture compute machines.



NOTE

Before adding a secondary architecture node to your cluster, you must install the Multiarch Tuning Operator, and deploy a **ClusterPodPlacementConfig** object. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

3.5.1. Creating RHCOS machines by using an ISO image

To scale your OpenShift Container Platform bare metal cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using an ISO image.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You must have the OpenShift CLI (**oc**) installed.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URLs of these files.
3. You can validate that the ignition files are available on the URLs. The following example gets the Ignition config files for the compute node:

```
$ curl -k http://<HTTP_server>/worker.ign
```

4. You can access the ISO image for booting your new machine by running the following command:

```
RHCOS_VHD_ORIGIN_URL=$(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' | jq -r '.architectures.<architecture>.artifacts.metal.formats.iso.disk.location')
```

5. Use the ISO file to install RHCOS on more compute machines. Use the same method that you used when you created machines before you installed the cluster:
 - Burn the ISO image to a disk and boot it directly.
 - Use ISO redirection with a LOM interface.
6. Boot the RHCOS ISO image without specifying any options, or interrupting the live boot sequence. Wait for the installer to boot into a shell prompt in the RHCOS live environment.



NOTE

You can interrupt the RHCOS installation boot process to add kernel arguments. However, for this ISO procedure you must use the **coreos-installer** command as outlined in the following steps, instead of adding kernel arguments.

7. Run the **coreos-installer** command by using **sudo**. The **core** user does not have the root privileges required to perform the installation. Specify the options that meet your installation requirements. At a minimum, you must specify the URL that points to the Ignition config file for the node type, and the device that you are installing to.

```
$ sudo coreos-installer install --ignition-url=http://<HTTP_server>/<node_type>.ign <device> --ignition-hash=sha512-<digest>
```

where:

<digest>

Specifies the Ignition config file SHA512 digest obtained through an HTTP URL to validate the authenticity of the Ignition config file on the cluster node.



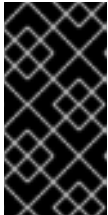
NOTE

If you want to provide your Ignition config files through an HTTPS server that uses TLS, you can add the internal certificate authority (CA) to the system trust store before running **coreos-installer**.

The following example initializes a compute node installation to the **/dev/sda** device. The Ignition config file for the compute node is obtained from an HTTP web server with the IP address 192.168.1.2:

```
$ sudo coreos-installer install --ignition-  
url=http://192.168.1.2:80/installation_directory/worker.ign /dev/sda --ignition-  
hash=sha512-  
a5a2d43879223273c9b60af66b44202a1d1248fc01cf156c46d4a79f552b6bad47bc8cc78ddf  
0116e80c59d2ea9e32ba53bc807afbca581aa059311def2c3e3b
```

8. Monitor the progress of the RHCOS installation on the console of the machine.



IMPORTANT

Ensure that the installation is successful on each node before commencing with the OpenShift Container Platform installation. Observing the installation process can also help to determine the cause of RHCOS installation issues that might arise.

9. Continue to create more compute machines for your cluster.

3.5.2. Creating RHCOS machines by PXE or iPXE booting

To scale your OpenShift Container Platform bare metal cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using PXE or iPXE booting.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You have obtained the URLs of the RHCOS ISO image, compressed metal BIOS, **kernel**, and **initramfs** files that you uploaded to your HTTP server during cluster installation.
- You have access to the PXE booting infrastructure that you used to create the machines for your OpenShift Container Platform cluster during installation. The machines must boot from their local disks after RHCOS is installed on them.
- If you use UEFI, you have access to the **grub.conf** file that you modified during OpenShift Container Platform installation.

Procedure

1. Confirm that your PXE or iPXE installation for the RHCOS images is correct.
 - For PXE:

```
DEFAULT pxeboot  
TIMEOUT 20  
PROMPT 0  
LABEL pxeboot  
  KERNEL http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture>  
  APPEND initrd=http://<HTTP_server>/rhcos-<version>-live-initramfs.
```

```
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img
```

where:

KERNEL

Specifies the location of the live **kernel** file that you uploaded to your HTTP server.

APPEND

Specifies the locations of the RHCOS files that you uploaded to your HTTP server:

initrd

Specifies the location of the live **initramfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file. This parameter supports only HTTP and HTTPS.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file. This parameter supports only HTTP and HTTPS.



NOTE

This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **APPEND** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?".

- For iPXE (**x86_64** + **aarch64**):

```
kernel http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture> initrd=main
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
initrd --name main http://<HTTP_server>/rhcos-<version>-live-initramfs.
<architecture>.img
boot
```

where:

kernel

Specifies the location of the **kernel** file that you uploaded to your HTTP server.

initrd=main

Specifies an argument that is required for booting on UEFI systems.

coreos.live.rootfs_url

Specifies the location of the **rootfs** file that you uploaded to your HTTP server.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file that you uploaded to your HTTP server.

initrd --name main

Specifies the location of the **initramfs** file that you uploaded to your HTTP server.



NOTE

- If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.
- This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **kernel** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?" in the Additional resources section and "Enabling the serial console for PXE and ISO installation" in the "Advanced RHCOS installation configuration" section.



NOTE

To network boot the CoreOS **kernel** on **aarch64** architecture, you need to use a version of iPXE build with the **IMAGE_GZIP** option enabled. For more information, see "IMAGE_GZIP option in iPXE".

- For PXE (with UEFI and GRUB as second stage) on **aarch64**:

```
menuentry 'Install CoreOS' {
    linux rhcos-<version>-live-kernel-<architecture>
    coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
    <architecture>.img coreos.inst.install_dev=/dev/sda
    coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
    initrd rhcos-<version>-live-initramfs.<architecture>.img
}
```

where:

linux

Specifies the location of the live **kernel** file on your TFTP server.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file.

initrd

Specifies the location of the live **initramfs** file on your TFTP server.

**NOTE**

If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.

2. Use the PXE or iPXE infrastructure to create the required compute machines for your cluster.

Additional resources

- [How does one set up a serial terminal and/or console in Red Hat Enterprise Linux? \(Red Hat Knowledgebase article\)](#)
- [IMAGE_GZIP option in iPXE \(iPXE documentation\)](#)

3.5.3. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

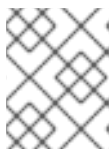
1. Confirm that the cluster recognizes the machines:

```
$ oc get nodes
```

Example output

```
NAME      STATUS   ROLES    AGE   VERSION
master-0  Ready    master   63m   v1.35.4
master-1  Ready    master   63m   v1.35.4
master-2  Ready    master   64m   v1.35.4
```

The output lists all of the machines that you created.

**NOTE**

The preceding output might not include the compute nodes until some CSRs are approved.

2. Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-8b2br	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrap	Pending
csr-8vnps	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrap	Pending
...			

In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

- If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:



NOTE

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.



NOTE

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrap** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```

**NOTE**

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

- Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

```
NAME      AGE   REQUESTOR                                CONDITION
csr-bfd72 5m26s system:node:ip-10-0-50-126.us-east-2.compute.internal
Pending
csr-c57lv 5m26s system:node:ip-10-0-95-157.us-east-2.compute.internal
Pending
...
```

- If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}\n\n{{end}}{{end}}' | xargs oc adm certificate approve
```

- After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes
```

Example output

```
NAME      STATUS  ROLES  AGE  VERSION
master-0  Ready   master  73m  v1.35.4
master-1  Ready   master  73m  v1.35.4
master-2  Ready   master  74m  v1.35.4
worker-0  Ready   worker  11m  v1.35.4
worker-1  Ready   worker  11m  v1.35.4
```

**NOTE**

You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

3.5.4. Additional resources

- [Installing a user provisioned cluster on bare metal](#)
- [Installing a cluster on IBM Power®](#)
- [Installing a cluster on IBM Z® and IBM® LinuxONE](#)
- [Migrating to a cluster with multi-architecture compute machines](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.6. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM LINUXONE WITH Z/VM

To create a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE (**s390x**) with z/VM, you must have an existing single-architecture **x86_64** cluster. You can then add **s390x** compute machines to your OpenShift Container Platform cluster.

Before you can add **s390x** nodes to your cluster, you must upgrade your cluster to one that uses the multi-architecture payload. For more information on migrating to the multi-architecture payload, see "Migrating to a cluster with multi-architecture compute machines".

The following procedures explain how to create a RHCOS compute machine using a z/VM instance. You can add **s390x** nodes to your cluster and deploy a cluster with multi-architecture compute machines.

To create an IBM Z® or IBM® LinuxONE (**s390x**) cluster with multi-architecture compute machines on **x86_64**, follow the instructions for "Installing a cluster on IBM Z® and IBM® LinuxONE". You can then add **x86_64** compute machines as described in "Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z".

**NOTE**

Before adding a secondary architecture node to your cluster, installing the Multiarch Tuning Operator and deploying a **ClusterPodPlacementConfig** object are best practices. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

3.6.1. Additional resources

- [Migrating to a cluster with multi-architecture compute machines](#)
- [Installing a cluster on IBM Z® and IBM® LinuxONE](#)
- [Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.6.2. Creating RHCOS machines on IBM Z with z/VM

You can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines running on IBM Z® with z/VM and attach them to your existing cluster.

Prerequisites

- You have a domain name server (DNS) that can perform hostname and reverse lookup for the nodes.
- You have an HTTP or HTTPS server running on your provisioning machine that is accessible to the machines you create.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URL of this file.
3. Validate that the Ignition file is available on the URL. The following example gets the Ignition config file for the compute node:

```
$ curl -k http://<http_server>/worker.ign
```

4. Download the RHEL live **kernel**, **initramfs**, and **rootfs** files by running the following commands:

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.kernel.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.initramfs.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.rootfs.location')
```

5. Move the downloaded RHEL live **kernel**, **initramfs**, and **rootfs** files to an HTTP or HTTPS server that is accessible from the RHCOS guest you want to add.
6. Create a parameter file for the guest. The following parameters are specific to the virtual machine:
 - Optional: To specify a static IP address, add an **ip=** parameter with the following entries, with each separated by a colon:
 - i. The IP address for the machine.
 - ii. An empty string.

- iii. The gateway.
 - iv. The netmask.
 - v. The machine host and domain name in the form **hostname.domainname**. If you omit this value, RHCOS obtains the hostname through a reverse DNS lookup.
 - vi. The network interface name. If you omit this value, RHCOS applies the IP configuration to all available interfaces.
 - vii. The value **none**.
- For **coreos.inst.ignition_url=**, specify the URL to the **worker.ign** file. Only HTTP and HTTPS protocols are supported.
 - For **coreos.live.rootfs_url=**, specify the matching rootfs artifact for the **kernel** and **initramfs** you are booting. Only HTTP and HTTPS protocols are supported.
 - For installations on DASD-type disks, complete the following tasks:
 - i. For **coreos.inst.install_dev=**, specify **/dev/dasda**.
 - ii. Use **rd.dasd=** to specify the DASD where RHCOS is to be installed.
 - iii. You can adjust further parameters if required.
- The following is an example parameter file, **additional-worker-dasd.parm**:

```

cio_ignore=all,!condev rd.neednet=1 \
console=ttysclp0 \
coreos.inst.install_dev=/dev/dasda \
coreos.inst.ignition_url=http://<http_server>/worker.ign \
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.
<architecture>.img \
ip=<ip>::<gateway>:<netmask>:<hostname>::none nameserver=<dns> \
rd.znet=qeth,0.0.bdf0,0.0.bdf1,0.0.bdf2,layer2=1,portno=0 \
rd.dasd=0.0.3490 \
zfcp.allow_lun_scan=0

```

Write all options in the parameter file as a single line and make sure that you have no newline characters.

- For installations on FCP-type disks, complete the following tasks:
 - i. Use **rd.zfcp=<adapter>,<wwpn>,<lun>** to specify the FCP disk where RHCOS is to be installed. For multipathing, repeat this step for each additional path.



NOTE

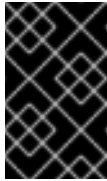
When you install with multiple paths, you must enable multipathing directly after the installation. Enabling multipathing much later can cause problems for your cluster.

- ii. Set the install device as: **coreos.inst.install_dev=/dev/sda**.

**NOTE**

If additional LUNs are configured with NPIV, FCP requires **zfcplib.allow_lun_scan=0**. If you must enable **zfcplib.allow_lun_scan=1** because you use a CSI driver, for example, you must configure your NPIV so that each node cannot access the boot partition of another node.

- iii. You can adjust further parameters if required.

**IMPORTANT**

Additional postinstallation steps are required to fully enable multipathing. For more information, see "Enabling multipathing with kernel arguments on RHCOS" in *Machine configuration*.

The following is an example parameter file, **additional-worker-fcp.parm** for a worker node with multipathing:

```
cio_ignore=all,!condev rd.neednet=1 \
console=ttysclp0 \
coreos.inst.install_dev=/dev/sda \
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.
<architecture>.img \
coreos.inst.ignition_url=http://<http_server>/worker.ign \
ip=<ip>::<gateway>:<netmask>:<hostname>::none nameserver=<dns> \
rd.znet=qeth,0.0.bdf0,0.0.bdf1,0.0.bdf2,layer2=1,portno=0 \
zfcplib.allow_lun_scan=0 \
rd.zfcplib=0.0.1987,0x50050763070bc5e3,0x4008400B00000000 \
rd.zfcplib=0.0.19C7,0x50050763070bc5e3,0x4008400B00000000 \
rd.zfcplib=0.0.1987,0x50050763071bc5e3,0x4008400B00000000 \
rd.zfcplib=0.0.19C7,0x50050763071bc5e3,0x4008400B00000000
```

Write all options in the parameter file as a single line and make sure that you have no newline characters.

7. Transfer the **initramfs**, **kernel**, parameter files, and RHCOS images to z/VM, for example, by using FTP. For details about how to transfer the files with FTP and boot from the virtual reader, see [Bootting the installation on IBM Z® to install RHEL in z/VM](#) .
8. Punch the files to the virtual reader of the z/VM guest virtual machine. See [PUNCH](#) in IBM® Documentation.

TIP

You can use the CP PUNCH command or, if you use Linux, the **vmur** command to transfer files between two z/VM guest virtual machines.

9. Log in to CMS on the bootstrap machine.
10. IPL the bootstrap machine from the reader by running the following command:

```
$ ipl c
```

See [IPL](#) in IBM® Documentation.

3.6.3. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

1. Confirm that the cluster recognizes the machines:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
master-0	Ready	master	63m	v1.35.4
master-1	Ready	master	63m	v1.35.4
master-2	Ready	master	64m	v1.35.4

The output lists all of the machines that you created.



NOTE

The preceding output might not include the compute nodes until some CSRs are approved.

2. Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-8b2br	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrapper	Pending
csr-8vnps	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrapper	Pending
...			

In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

3. If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:

**NOTE**

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.

**NOTE**

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrapper** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```

**NOTE**

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

- Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-bfd72	5m26s	system:node:ip-10-0-50-126.us-east-2.compute.internal	

```
Pending
csr-c57lv 5m26s system:node:ip-10-0-95-157.us-east-2.compute.internal
Pending
...
```

- If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}
{{end}}{{end}}' | xargs oc adm certificate approve
```

- After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes
```

Example output

```
NAME      STATUS  ROLES  AGE  VERSION
master-0  Ready   master  73m  v1.35.4
master-1  Ready   master  73m  v1.35.4
master-2  Ready   master  74m  v1.35.4
worker-0  Ready   worker  11m  v1.35.4
worker-1  Ready   worker  11m  v1.35.4
```



NOTE

You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

3.7. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM LINUXONE IN AN LPAR

To create a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE (**s390x**) in a logical partition (LPAR), you must have an existing single-architecture **x86_64** cluster. You can then add **s390x** compute machines to your OpenShift Container Platform cluster.

Before you can add **s390x** nodes to your cluster, you must upgrade your cluster to one that uses the multi-architecture payload. For more information about migrating to the multi-architecture payload, see "Migrating to a cluster with multi-architecture compute machines".

You can create a RHCOS compute machine by using an LPAR instance. By using an LPAR instance, you can add **s390x** nodes to your cluster and deploy a cluster with multi-architecture compute machines.



NOTE

To create an IBM Z® or IBM® LinuxONE (**s390x**) cluster with multi-architecture compute machines on **x86_64**, follow the instructions for "Installing a cluster on IBM Z® and IBM® LinuxONE". You can then add **x86_64** compute machines as described in "Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z".

Additional resources

- [Migrating to a cluster with multi-architecture compute machines](#)
- [Installing a cluster on IBM Z® and IBM® LinuxONE](#)
- [Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z](#)

3.7.1. Creating RHCOS machines on IBM Z in an LPAR

You can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines running on IBM Z® in a logical partition (LPAR) and attach them to your existing cluster.

Prerequisites

- You have a domain name server (DNS) that can perform hostname and reverse lookup for the nodes.
- You have an HTTP or HTTPS server running on your provisioning machine that is accessible to the machines you create.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URL of this file.
3. Validate that the Ignition file is available on the URL. The following example gets the Ignition config file for the compute node:

```
$ curl -k http://<http_server>/worker.ign
```

4. Download the RHEL live **kernel**, **initramfs**, and **rootfs** files by running the following commands:

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.kernel.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o
jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.initramfs.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o
jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.rootfs.location')
```

5. Move the downloaded RHEL live **kernel**, **initramfs**, and **rootfs** files to an HTTP or HTTPS server that is accessible from the RHCOS guest you want to add.
6. Create a parameter file for the guest. The following parameters are specific to the virtual machine:

- Optional: To specify a static IP address, add an **ip=** parameter with the following entries, with each separated by a colon:
 - i. The IP address for the machine.
 - ii. An empty string.
 - iii. The gateway.
 - iv. The netmask.
 - v. The machine host and domain name in the form **hostname.domainname**. If you omit this value, RHCOS obtains the hostname through a reverse DNS lookup.
 - vi. The network interface name. If you omit this value, RHCOS applies the IP configuration to all available interfaces.
 - vii. The value **none**.
- For **coreos.inst.ignition_url=**, specify the URL to the **worker.ign** file. Only HTTP and HTTPS protocols are supported.
- For **coreos.live.rootfs_url=**, specify the matching rootfs artifact for the **kernel** and **initramfs** you are booting. Only HTTP and HTTPS protocols are supported.
- For installations on DASD-type disks, complete the following tasks:
 - i. For **coreos.inst.install_dev=**, specify **/dev/dasda**.
 - ii. Use **rd.dasd=** to specify the DASD where RHCOS is to be installed.
 - iii. You can adjust further parameters if required.

The following is an example parameter file, **additional-worker-dasd.parm**:

```
cio_ignore=all,!condev rd.neednet=1 \
console=ttySclp0 \
coreos.inst.install_dev=/dev/dasda \
coreos.inst.ignition_url=http://<http_server>/worker.ign \
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.
<architecture>.img \
ip=<ip>::<gateway>:<netmask>:<hostname>::none nameserver=<dns> \
rd.znet=qeth,0.0.bdf0,0.0.bdf1,0.0.bdf2,layer2=1,portno=0 \
```

```
rd.dasd=0.0.3490 \
zfcp.allow_lun_scan=0
```

Write all options in the parameter file as a single line and make sure that you have no newline characters.

- For installations on FCP-type disks, complete the following tasks:
 - i. Use **rd.zfcp=<adapter>,<wwpn>,<lun>** to specify the FCP disk where RHCOS is to be installed. For multipathing, repeat this step for each additional path.



NOTE

When you install with multiple paths, you must enable multipathing directly after the installation. Enabling multipathing much later can cause problems for your cluster.

- ii. Set the install device as: **coreos.inst.install_dev=/dev/sda**.



NOTE

If additional LUNs are configured with NPIV, FCP requires **zfcp.allow_lun_scan=0**. If you must enable **zfcp.allow_lun_scan=1** because you use a CSI driver, for example, you must configure your NPIV so that each node cannot access the boot partition of another node.

- iii. You can adjust further parameters if required.



IMPORTANT

Additional postinstallation steps are required to fully enable multipathing. For more information, see "Enabling multipathing with kernel arguments on RHCOS" in *Machine configuration*.

The following is an example parameter file, **additional-worker-fcp.parm** for a worker node with multipathing:

```
cio_ignore=all,!condev rd.neednet=1 \
console=ttySclp0 \
coreos.inst.install_dev=/dev/sda \
coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.
<architecture>.img \
coreos.inst.ignition_url=http://<http_server>/worker.ign \
ip=<ip>::<gateway>:<netmask>:<hostname>::none nameserver=<dns> \
rd.znet=qeth,0.0.bdf0,0.0.bdf1,0.0.bdf2,layer2=1,portno=0 \
zfcp.allow_lun_scan=0 \
rd.zfcp=0.0.1987,0x50050763070bc5e3,0x4008400B00000000 \
rd.zfcp=0.0.19C7,0x50050763070bc5e3,0x4008400B00000000 \
rd.zfcp=0.0.1987,0x50050763071bc5e3,0x4008400B00000000 \
rd.zfcp=0.0.19C7,0x50050763071bc5e3,0x4008400B00000000
```

Write all options in the parameter file as a single line and make sure that you have no newline characters.

7. As an example for FTP, transfer the initramfs, kernel, parameter files, and RHCOS images to the LPAR. For details about how to transfer the files with FTP and boot, see [Booting the installation on IBM Z® to install RHEL in an LPAR](#).
8. Boot the machine.

3.7.2. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

1. Confirm that the cluster recognizes the machines:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
master-0	Ready	master	63m	v1.35.4
master-1	Ready	master	63m	v1.35.4
master-2	Ready	master	64m	v1.35.4

The output lists all of the machines that you created.



NOTE

The preceding output might not include the compute nodes until some CSRs are approved.

2. Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-8b2br	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrap	Pending
csr-8vnps	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstrap	Pending
...			

In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

- If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:



NOTE

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.



NOTE

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrapper** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```



NOTE

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

- Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-bfd72	5m26s	system:node:ip-10-0-50-126.us-east-2.compute.internal	Pending
csr-c57lv	5m26s	system:node:ip-10-0-95-157.us-east-2.compute.internal	Pending
...			

5. If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs oc adm certificate approve
```

6. After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
master-0	Ready	master	73m	v1.35.4
master-1	Ready	master	73m	v1.35.4
master-2	Ready	master	74m	v1.35.4
worker-0	Ready	worker	11m	v1.35.4
worker-1	Ready	worker	11m	v1.35.4

**NOTE**

You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

3.8. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM Z AND IBM LINUXONE WITH RHEL KVM

To create a cluster with multi-architecture compute machines on IBM Z® and IBM® LinuxONE (s390x) with RHEL KVM, you must have an existing single-architecture **x86_64** cluster. You can then add **s390x** compute machines to your OpenShift Container Platform cluster.

Before you can add **s390x** nodes to your cluster, you must upgrade your cluster to one that uses the multi-architecture payload. For more information on migrating to the multi-architecture payload, see "Migrating to a cluster with multi-architecture compute machines".

The following procedures explain how to create a RHCOS compute machine using a RHEL KVM instance. This will allow you to add **s390x** nodes to your cluster and deploy a cluster with multi-architecture compute machines.

To create an IBM Z® or IBM® LinuxONE (**s390x**) cluster with multi-architecture compute machines on **x86_64**, follow the instructions for "Installing a cluster on IBM Z® and IBM® LinuxONE". You can then add **x86_64** compute machines as described in "Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z".



NOTE

Before adding a secondary architecture node to your cluster, install the Multiarch Tuning Operator and then deploy a **ClusterPodPlacementConfig** object. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

3.8.1. Creating RHCOS machines using virt-install

You can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines for your cluster by using **virt-install**.

Prerequisites

- You have at least one LPAR running on RHEL 8.7 or later with KVM, referred to as RHEL KVM host in this procedure.
- The KVM/QEMU hypervisor is installed on the RHEL KVM host.
- You have a domain name server (DNS) that can perform hostname and reverse lookup for the nodes.
- An HTTP or HTTPS server is set up.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URL of this file.
3. You can validate that the Ignition file is available on the URL. The following example gets the Ignition config file for the compute node:

```
$ curl -k http://<HTTP_server>/worker.ign
```

4. Download the RHEL live **kernel**, **initramfs**, and **rootfs** files by running the following commands:

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o
jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.kernel.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o
jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.initramfs.location')
```

```
$ curl -LO $(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o
jsonpath='{.data.stream}' \
| jq -r '.architectures.s390x.artifacts.metal.formats.pxe.rootfs.location')
```

5. Move the downloaded RHEL live **kernel**, **initramfs**, and **rootfs** files to an HTTP or HTTPS server before you launch **virt-install**.
6. Create the new KVM guest nodes using the RHEL **kernel**, **initramfs**, and Ignition files; the new disk image; and adjusted parm line arguments.

```
$ virt-install \
  --connect qemu:///system \
  --name <vm_name> \
  --autostart \
  --os-variant rhel9.4 \
  --cpu host \
  --vcpus <vcpus> \
  --memory <memory_mb> \
  --disk <vm_name>.qcow2,size=<image_size> \
  --network network=<virt_network_parm> \
  --location <media_location>,kernel=<rhcos_kernel>,initrd=<rhcos_initrd> \
  --extra-args "rd.needsnet=1" \
  --extra-args "coreos.inst.install_dev=/dev/vda" \
  --extra-args "coreos.inst.ignition_url=http://<http_server>/worker.ign" \
  --extra-args "coreos.live.rootfs_url=http://<http_server>/rhcos-<version>-live-rootfs.
<architecture>.img" \
  --extra-args "ip=<ip>::<gateway>:<netmask>:<hostname>::none" \
  --extra-args "nameserver=<dns>" \
  --extra-args "console=ttyS0" \
  --noautoconsole \
  --wait
```

where:

os-variant

Specifies the RHEL version for the RHCOS compute machine. **rhel9.4** is the recommended version. To query the supported RHEL version of your operating system, run the following command:

```
$ osinfo-query os -f short-id
```



NOTE

The **os-variant** is case sensitive.

location

Specifies the location of the kernel/initrd on the HTTP or HTTPS server.

coreos.inst.ignition_url

Specifies the location of the **worker.ign** config file. Only HTTP and HTTPS protocols are supported.

coreos.live.rootfs_url

Specifies the location of the **rootfs** artifact for the **kernel** and **initramfs** you are booting. Only HTTP and HTTPS protocols are supported

hostname

Optional parameter. Specifies the fully qualified hostname of the client machine.

**NOTE**

If you are using HAProxy as a load balancer, update your HAProxy rules for **ingress-router-443** and **ingress-router-80** in the **/etc/haproxy/haproxy.cfg** configuration file.

- Continue to create more compute machines for your cluster.

3.8.2. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

- Confirm that the cluster recognizes the machines:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
master-0	Ready	master	63m	v1.35.4
master-1	Ready	master	63m	v1.35.4
master-2	Ready	master	64m	v1.35.4

The output lists all of the machines that you created.

**NOTE**

The preceding output might not include the compute nodes until some CSRs are approved.

- Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

```
NAME      AGE    REQUESTOR                                     CONDITION
csr-8b2br 15m    system:serviceaccount:openshift-machine-config-operator:node-
bootstrapper Pending
csr-8vnps 15m    system:serviceaccount:openshift-machine-config-operator:node-
bootstrapper Pending
...
```

In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

- If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:



NOTE

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.



NOTE

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrapper** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```



NOTE

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

- Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

```
NAME      AGE   REQUESTOR                                CONDITION
csr-bfd72 5m26s system:node:ip-10-0-50-126.us-east-2.compute.internal
Pending
csr-c57lv 5m26s system:node:ip-10-0-95-157.us-east-2.compute.internal
Pending
...
```

- If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs oc adm certificate approve
```

- After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes
```

Example output

```
NAME      STATUS   ROLES    AGE   VERSION
master-0  Ready    master   73m   v1.35.4
master-1  Ready    master   73m   v1.35.4
```


master-2	Ready	master	74m	v1.35.4
worker-0	Ready	worker	11m	v1.35.4
worker-1	Ready	worker	11m	v1.35.4

**NOTE**

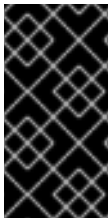
You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

3.8.3. Additional resources

- [Migrating to a cluster with multi-architecture compute machines](#)
- [Installing a cluster on IBM Z® and IBM® LinuxONE](#)
- [Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.9. CREATING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES ON IBM POWER

To create a cluster with multi-architecture compute machines on IBM Power® (**ppc64le**), you must have an existing single-architecture (**x86_64**) cluster. You can then add **ppc64le** compute machines to your OpenShift Container Platform cluster.

**IMPORTANT**

Before you can add **ppc64le** nodes to your cluster, you must upgrade your cluster to one that uses the multi-architecture payload. For more information on migrating to the multi-architecture payload, see "Migrating to a cluster with multi-architecture compute machines".

The following procedures explain how to complete the following tasks:

- Create a RHCOS compute machine by using an ISO image or network PXE booting.
- Add **ppc64le** nodes to your cluster and deploy a cluster with multi-architecture compute machines.

To create an IBM Power® (**ppc64le**) cluster with multi-architecture compute machines on **x86_64**, follow the instructions for "Installing a cluster on IBM Power®". You can then add **x86_64** compute machines as described in "Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z".

**NOTE**

Before adding a secondary architecture node to your cluster, Red Hat recommends that you install the Multiarch Tuning Operator, and deploy a **ClusterPodPlacementConfig** object. For more information, see "Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator".

Additional resources

- [Migrating to a cluster with multi-architecture compute machines](#)
- [Installing a cluster on IBM Power®](#)
- [Creating a cluster with multi-architecture compute machines on bare metal, IBM Power, or IBM Z](#)
- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.9.1. Creating RHCOS machines by using an ISO image

To scale your OpenShift Container Platform cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using an ISO image.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You must have the OpenShift CLI (**oc**) installed.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URLs of these files.
3. You can validate that the ignition files are available on the URLs. The following example gets the Ignition config files for the compute node:

```
$ curl -k http://<HTTP_server>/worker.ign
```

4. You can access the ISO image for booting your new machine by running the following command:

```
RHCOS_VHD_ORIGIN_URL=$(oc -n openshift-machine-config-operator get configmap/coreos-bootimages -o jsonpath='{.data.stream}' | jq -r '.architectures.<architecture>.artifacts.metal.formats.iso.disk.location')
```

5. Use the ISO file to install RHCOS on more compute machines. Use the same method that you used when you created machines before you installed the cluster:
 - Burn the ISO image to a disk and boot it directly.
 - Use ISO redirection with a LOM interface.
6. Boot the RHCOS ISO image without specifying any options, or interrupting the live boot sequence. Wait for the installer to boot into a shell prompt in the RHCOS live environment.

**NOTE**

You can interrupt the RHCOS installation boot process to add kernel arguments. However, for this ISO procedure you must use the **coreos-installer** command as outlined in the following steps, instead of adding kernel arguments.

7. Run the **coreos-installer** command by using **sudo**. The **core** user does not have the root privileges required to perform the installation. Specify the options that meet your installation requirements. At a minimum, you must specify the URL that points to the Ignition config file for the node type, and the device that you are installing to.

```
$ sudo coreos-installer install --ignition-url=http://<HTTP_server>/<node_type>.ign <device>
--ignition-hash=sha512-<digest>
```

where:

<digest>

Specifies the Ignition config file SHA512 digest obtained through an HTTP URL to validate the authenticity of the Ignition config file on the cluster node.

**NOTE**

If you want to provide your Ignition config files through an HTTPS server that uses TLS, you can add the internal certificate authority (CA) to the system trust store before running **coreos-installer**.

The following example initializes a compute node installation to the **/dev/sda** device. The Ignition config file for the compute node is obtained from an HTTP web server with the IP address 192.168.1.2:

```
$ sudo coreos-installer install --ignition-
url=http://192.168.1.2:80/installation_directory/worker.ign /dev/sda --ignition-
hash=sha512-
a5a2d43879223273c9b60af66b44202a1d1248fc01cf156c46d4a79f552b6bad47bc8cc78ddf
0116e80c59d2ea9e32ba53bc807afbca581aa059311def2c3e3b
```

8. Monitor the progress of the RHCOS installation on the console of the machine.

**IMPORTANT**

Ensure that the installation is successful on each node before commencing with the OpenShift Container Platform installation. Observing the installation process can also help to determine the cause of RHCOS installation issues that might arise.

9. Continue to create more compute machines for your cluster.

3.9.2. Creating RHCOS machines by PXE or iPXE booting

To scale your OpenShift Container Platform bare metal cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using PXE or iPXE booting.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You have obtained the URLs of the RHCOS ISO image, compressed metal BIOS, **kernel**, and **initramfs** files that you uploaded to your HTTP server during cluster installation.
- You have access to the PXE booting infrastructure that you used to create the machines for your OpenShift Container Platform cluster during installation. The machines must boot from their local disks after RHCOS is installed on them.
- If you use UEFI, you have access to the **grub.conf** file that you modified during OpenShift Container Platform installation.

Procedure

1. Confirm that your PXE or iPXE installation for the RHCOS images is correct.

- For PXE:

```
DEFAULT pxeboot
TIMEOUT 20
PROMPT 0
LABEL pxeboot
    KERNEL http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture>
    APPEND initrd=http://<HTTP_server>/rhcos-<version>-live-initramfs.
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img
```

where:

KERNEL

Specifies the location of the live **kernel** file that you uploaded to your HTTP server.

APPEND

Specifies the locations of the RHCOS files that you uploaded to your HTTP server:

initrd

Specifies the location of the live **initramfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file. This parameter supports only HTTP and HTTPS.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file. This parameter supports only HTTP and HTTPS.



NOTE

This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **APPEND** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?".

- For iPXE (**x86_64** + **ppc64le**):

```
kernel http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture> initrd=main
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
initrd --name main http://<HTTP_server>/rhcos-<version>-live-initramfs.
<architecture>.img
boot
```

where:

kernel

Specifies the location of the **kernel** file that you uploaded to your HTTP server.

initrd=main

Specifies an argument that is required for booting on UEFI systems.

coreos.live.rootfs_url

Specifies the location of the **rootfs** file that you uploaded to your HTTP server.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file that you uploaded to your HTTP server.

initrd --name main

Specifies the location of the **initramfs** file that you uploaded to your HTTP server.



NOTE

- If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.
- This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **kernel** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?" in the Additional resources section and "Enabling the serial console for PXE and ISO installation" in the "Advanced RHCOS installation configuration" section.

**NOTE**

To network boot the CoreOS **kernel** on **ppc64le** architecture, you need to use a version of iPXE build with the **IMAGE_GZIP** option enabled. For more information, see "IMAGE_GZIP option in iPXE".

- For PXE (with UEFI and GRUB as second stage) on **ppc64le**:

```
menuentry 'Install CoreOS' {
  linux rhcos-<version>-live-kernel-<architecture>
  coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
  <architecture>.img coreos.inst.install_dev=/dev/sda
  coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
  initrd rhcos-<version>-live-initramfs.<architecture>.img
}
```

where:

linux

Specifies the location of the live **kernel** file on your TFTP server.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file.

initrd

Specifies the location of the live **initramfs** file on your TFTP server.

**NOTE**

If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.

2. Use the PXE or iPXE infrastructure to create the required compute machines for your cluster.

3.9.3. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

1. Confirm that the cluster recognizes the machines:

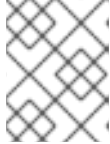
```
$ oc get nodes
```

■

Example output

NAME	STATUS	ROLES	AGE	VERSION
master-0	Ready	master	63m	v1.35.4
master-1	Ready	master	63m	v1.35.4
master-2	Ready	master	64m	v1.35.4

The output lists all of the machines that you created.



NOTE

The preceding output might not include the compute nodes until some CSRs are approved.

2. Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-8b2br	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstraptrapper	Pending
csr-8vnps	15m	system:serviceaccount:openshift-machine-config-operator:node-bootstraptrapper	Pending
...			

In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

3. If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:



NOTE

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.

**NOTE**

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrapper** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{"\n"}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```

**NOTE**

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

- Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

NAME	AGE	REQUESTOR	CONDITION
csr-bfd72	5m26s	system:node:ip-10-0-50-126.us-east-2.compute.internal	Pending
csr-c57lv	5m26s	system:node:ip-10-0-95-157.us-east-2.compute.internal	Pending
...			

- If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```


-

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}
{{end}}{{end}}' | xargs oc adm certificate approve
```

6. After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes -o wide
```

Example output

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
worker-0-ppc64le	Ready	worker	42d	v1.35.4	192.168.200.21	<none>	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.ppc64le	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
worker-1-ppc64le	Ready	worker	42d	v1.35.4	192.168.200.20	<none>	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.ppc64le	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
master-0-x86	Ready	control-plane,master	75d	v1.35.4	10.248.0.38	10.248.0.38	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.x86_64	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
master-1-x86	Ready	control-plane,master	75d	v1.35.4	10.248.0.39	10.248.0.39	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.x86_64	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
master-2-x86	Ready	control-plane,master	75d	v1.35.4	10.248.0.40	10.248.0.40	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.x86_64	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
worker-0-x86	Ready	worker	75d	v1.35.4	10.248.0.43	10.248.0.43	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.x86_64	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9
worker-1-x86	Ready	worker	75d	v1.35.4	10.248.0.44	10.248.0.44	Red Hat Enterprise Linux CoreOS 415.92.202309261919-0 (Plow)	5.14.0-284.34.1.el9_2.x86_64	cri-o://1.35.4-3.rhaos4.15.gitb36169e.el9



NOTE

You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

3.10. MANAGING A CLUSTER WITH MULTI-ARCHITECTURE COMPUTE MACHINES

Managing a cluster that has nodes with multiple architectures requires you to consider node architecture as you monitor the cluster and manage your workloads. This requires you to take additional

considerations into account when you configure cluster resource requirements and behaviors, or schedule workloads in a multi-architecture cluster.

3.10.1. Scheduled workloads on clusters with multi-architecture compute machines

When you deploy workloads on a cluster with compute nodes that use different architectures, you must align pod architecture with the architecture of the underlying node. Your workload might also require additional configuration to particular resources depending on the underlying node architecture.

You can use the Multiarch Tuning Operator to enable architecture-aware scheduling of workloads on clusters with multi-architecture compute machines. The Multiarch Tuning Operator implements additional scheduler predicates in the pod specifications based on the architectures that the pods can support at creation time.

Additional resources

- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.10.1.1. Sample multi-architecture node workload deployments

Scheduling a workload to an appropriate node based on architecture works in the same way as scheduling based on any other node characteristic. Consider the following options when determining how to schedule your workloads.

Using **nodeAffinity** to schedule nodes with specific architectures

You can allow a workload to be scheduled on only a set of nodes with architectures supported by its images. You can set the **spec.affinity.nodeAffinity** field in your pod's template specification.

Example deployment with node affinity set

```
apiVersion: apps/v1
kind: Deployment
metadata: # ...
spec:
  # ...
  template:
    # ...
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/arch
                    operator: In
                    values:
                      - amd64
                      - arm64
            # ...
```

- The **values** parameter specifies the supported architectures. Valid values include **amd64**, **arm64**, or both values.

Tainting each node for a specific architecture

You can taint a node to avoid the node scheduling workloads that are incompatible with its architecture. When your cluster uses a **MachineSet** object, you can add parameters to the **.spec.template.spec.taints** field to avoid workloads being scheduled on nodes with non-supported architectures.

Before you add a taint to a node, you must scale down the **MachineSet** object or remove existing available machines. For more information, see *Modifying a compute machine set*.

Example machine set with taint set

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata: # ...
spec:
  # ...
  template:
    # ...
    spec:
      # ...
      taints:
        - effect: NoSchedule
          key: multiarch.openshift.io/arch
          value: arm64
```

You can also set a taint on a specific node by running the following command:

```
$ oc adm taint nodes <node-name> multiarch.openshift.io/arch=arm64:NoSchedule
```

Creating a default toleration in a namespace

When a node or machine set has a taint, only workloads that tolerate that taint can be scheduled. You can annotate a namespace so all of the workloads get the same default toleration by running the following command:

Example default toleration set on a namespace

```
$ oc annotate namespace my-namespace \
  'scheduler.alpha.kubernetes.io/defaultTolerations'='[{"operator": "Exists", "effect": "NoSchedule",
  "key": "multiarch.openshift.io/arch}]'
```

Tolerating architecture taints in workloads

When a node or machine set has a taint, only workloads that tolerate that taint can be scheduled. You can configure your workload with a **toleration** so that it is scheduled on nodes with specific architecture taints.

Example deployment with toleration set

```
apiVersion: apps/v1
kind: Deployment
metadata: # ...
spec:
  # ...
```

```
template:
  # ...
spec:
  tolerations:
    - key: "multiarch.openshift.io/arch"
      value: "arm64"
      operator: "Equal"
      effect: "NoSchedule"
```

This example deployment can be scheduled on nodes and machine sets that have the **multiarch.openshift.io/arch=arm64** taint specified.

Using node affinity with taints and tolerations

When a scheduler computes the set of nodes to schedule a pod, tolerations can broaden the set while node affinity restricts the set. If you set a taint on nodes that have a specific architecture, you must also add a toleration to workloads that you want to be scheduled there.

Example deployment with node affinity and toleration set

```
apiVersion: apps/v1
kind: Deployment
metadata: # ...
spec:
  # ...
  template:
    # ...
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
              - matchExpressions:
                  - key: kubernetes.io/arch
                    operator: In
                    values:
                      - amd64
                      - arm64
            tolerations:
              - key: "multiarch.openshift.io/arch"
                value: "arm64"
                operator: "Equal"
                effect: "NoSchedule"
```

Additional resources

- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)
- [Controlling pod placement using node taints](#)
- [Controlling pod placement on nodes using node affinity](#)
- [Controlling pod placement using the scheduler](#)

- [Modifying a compute machine set](#)

3.10.2. Enabling 64k pages on the Red Hat Enterprise Linux CoreOS (RHCOS) kernel

You can enable the 64k memory page in the Red Hat Enterprise Linux CoreOS (RHCOS) kernel on the 64-bit ARM compute machines in your cluster. The 64k page size kernel specification can be used for large GPU or high memory workloads.

This configuration is possible by using the Machine Config Operator (MCO), which uses a machine config pool to update the kernel. To enable 64k page sizes on ARM64, create a dedicated **MachineConfigPool** and apply the 64k kernel configuration to it.



IMPORTANT

Using 64k pages is exclusive to 64-bit ARM architecture compute nodes or clusters installed on 64-bit ARM machines. If you configure the 64k pages kernel on a machine config pool using 64-bit x86 machines, the machine config pool and the MCO degrades.

Prerequisites

- You installed the OpenShift CLI (**oc**).
- You created a cluster with compute nodes of different architecture on one of the supported platforms.

Procedure

1. Label the nodes where you want to run the 64k page size kernel:

```
$ oc label node <node_name> <label>
```

Example command

```
$ oc label node worker-arm64-01 node-role.kubernetes.io/worker-64k-pages=
```

2. Create a machine config pool that contains the worker role that uses the ARM64 architecture and the **worker-64k-pages** role:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker-64k-pages
spec:
  machineConfigSelector:
    matchExpressions:
      - key: machineconfiguration.openshift.io/role
        operator: In
        values:
          - worker
          - worker-64k-pages
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/worker-64k-pages: ""
      kubernetes.io/arch: arm64
```

3. Create a machine config on your compute node to enable **64k-pages** with the **64k-pages** parameter.

```
$ oc create -f <filename>.yaml
```

Example MachineConfig

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: "worker-64k-pages"
  name: 99-worker-64kpages
spec:
  kernelType: 64k-pages
```

where:

metadata.labels.machineconfiguration.openshift.io/role

Specifies the value of the **machineconfiguration.openshift.io/role** label in the custom machine config pool. The example MachineConfig uses the **worker-64k-pages** label to enable 64k pages in the **worker-64k-pages** pool.

spec.kernelType

Specifies your desired kernel type. Valid values are **64k-pages** and **default**.



NOTE

The **64k-pages** type is supported on only 64-bit ARM architecture based compute nodes. The **realtime** type is supported on only 64-bit x86 architecture based compute nodes.

Verification

- To view your new **worker-64k-pages** machine config pool, run the following command:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING
DEGRADED	MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT
DEGRADEDMACHINECOUNT	AGE		
master	rendered-master-9d55ac9a91127c36314e1efe7d77fbf8	True	False
False	3 3 3 0 361d		
worker	rendered-worker-e7b61751c4a5b7ff995d64b967c421ff	True	False
False	7 7 7 0 361d		
worker-64k-pages	rendered-worker-64k-pages-e7b61751c4a5b7ff995d64b967c421ff	True	
False	False 2 2 2 0 35m		

3.10.3. Importing manifest lists in image streams on your multi-architecture compute machines

On an OpenShift Container Platform 4.22 cluster with multi-architecture compute machines, the image streams in the cluster do not import manifest lists automatically. You must manually change the default **importMode** option to the **PreserveOriginal** option to import the manifest list.

Prerequisites

- You installed the OpenShift CLI (**oc**).

Procedure

- Enter a command similar to the following example command to patch the **ImageStream** cli-artifacts so that the **cli-artifacts:latest** image stream tag is imported as a manifest list:

```
$ oc patch is/cli-artifacts -n openshift -p '{"spec":{"tags":[{"name":"latest","importPolicy":{"importMode":"PreserveOriginal"}}]}}'
```

Verification

- You can check that the manifest lists imported properly by inspecting the image stream tag. The following command lists the individual architecture manifests for a particular tag.

```
$ oc get istag cli-artifacts:latest -n openshift -oyaml
```

If the **dockerImageManifests** object is present, the manifest list imported successfully.

Example output of the **dockerImageManifests** object

```
dockerImageManifests:
  - architecture: amd64
    digest:
      sha256:16d4c96c52923a9968fbfa69425ec703aff711f1db822e4e9788bf5d2bee5d77
    manifestSize: 1252
    mediaType: application/vnd.docker.distribution.manifest.v2+json
    os: linux
  - architecture: arm64
    digest:
      sha256:6ec8ad0d897bcd727531f7d0b716931728999492709d19d8b09f0d90d57f626
    manifestSize: 1252
    mediaType: application/vnd.docker.distribution.manifest.v2+json
    os: linux
  - architecture: ppc64le
    digest:
      sha256:65949e3a80349cdc42acd8c5b34cde6ebc3241eae8daaeaa458498fedb359a6a
    manifestSize: 1252
    mediaType: application/vnd.docker.distribution.manifest.v2+json
    os: linux
  - architecture: s390x
    digest:
      sha256:75f4fa21224b5d5d511bea8f92dfa8e1c00231e5c81ab95e83c3013d245d1719
```



```
manifestSize: 1252
mediaType: application/vnd.docker.distribution.manifest.v2+json
os: linux
```

3.11. MANAGING WORKLOADS ON MULTI-ARCHITECTURE CLUSTERS BY USING THE MULTIARCH TUNING OPERATOR

The Multiarch Tuning Operator optimizes workload management within multi-architecture clusters and in single-architecture clusters transitioning to multi-architecture environments.

Architecture-aware workload scheduling allows the scheduler to place pods onto nodes that match the architecture of the pod images.

By default, the scheduler does not consider the architecture of a pod's container images when determining the placement of new pods onto nodes.

To enable architecture-aware workload scheduling, you must create the **ClusterPodPlacementConfig** object. When you create the **ClusterPodPlacementConfig** object, the Multiarch Tuning Operator deploys the necessary operands to support architecture-aware workload scheduling. You can also use the **nodeAffinityScoring** plugin in the **ClusterPodPlacementConfig** object to set cluster-wide scores for node architectures. If you enable the **nodeAffinityScoring** plugin, the scheduler first filters nodes with compatible architectures and then places the pod on the node with the highest score.

When a pod is created, the operands perform the following actions:

1. Add the **multiarch.openshift.io/scheduling-gate** scheduling gate that prevents the scheduling of the pod.
2. Compute a scheduling predicate that includes the supported architecture values for the **kubernetes.io/arch** label.
3. Integrate the scheduling predicate as a **nodeAffinity** requirement in the pod specification.
4. Remove the scheduling gate from the pod.

IMPORTANT

Note the following operand behaviors:

- If the **nodeSelector** field is already configured with the **kubernetes.io/arch** label for a workload, the operand does not update the **nodeAffinity** field for that workload.
- If the **nodeSelector** field is not configured with the **kubernetes.io/arch** label for a workload, the operand updates the **nodeAffinity** field for that workload. For the **nodeAffinity** field, the operand updates only the node selector terms that are not configured with the **kubernetes.io/arch** label.
- If the **nodeName** field is already set, the Multiarch Tuning Operator does not process the pod.
- If the pod is owned by a DaemonSet, the operand does not update the **nodeAffinity** field.
- If **nodeSelector** or **nodeAffinity** and **preferredAffinity** fields are set for the **kubernetes.io/arch** label, the operand does not update the **nodeAffinity** field.
- If only the **nodeSelector** or the **nodeAffinity** field is set for the **kubernetes.io/arch** label and the **nodeAffinityScoring** plugin is disabled, the operand does not update the **nodeAffinity** field.
- If the **nodeAffinity.preferredDuringSchedulingIgnoredDuringExecution** field already contains terms that score nodes based on the **kubernetes.io/arch** label, the operand ignores the configuration in the **nodeAffinityScoring** plugin.

3.11.1. Installing the Multiarch Tuning Operator by using the CLI

You can install the Multiarch Tuning Operator by using the OpenShift CLI (**oc**).

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to **oc** as a user with **cluster-admin** privileges.

Procedure

1. Create a new project named **openshift-multiarch-tuning-operator** by running the following command:

```
$ oc create ns openshift-multiarch-tuning-operator
```

2. Create an **OperatorGroup** object:
 - a. Create a YAML file with the configuration for creating an **OperatorGroup** object.

Example YAML configuration for creating an **OperatorGroup** object

```
apiVersion: operators.coreos.com/v1
```

```
kind: OperatorGroup
metadata:
  name: openshift-multiarch-tuning-operator
  namespace: openshift-multiarch-tuning-operator
spec: {}
```

- b. Create the **OperatorGroup** object by running the following command:

```
$ oc create -f <file_name>
```

Replace **<file_name>** with the name of the YAML file that contains the **OperatorGroup** object configuration.

3. Create a **Subscription** object:

- a. Create a YAML file with the configuration for creating a **Subscription** object.

Example YAML configuration for creating a **Subscription** object

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: openshift-multiarch-tuning-operator
  namespace: openshift-multiarch-tuning-operator
spec:
  channel: stable
  name: multiarch-tuning-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  installPlanApproval: Automatic
  startingCSV: multiarch-tuning-operator.<version>
```

- b. Create the **Subscription** object by running the following command:

```
$ oc create -f <file_name>
```

Replace **<file_name>** with the name of the YAML file that contains the **Subscription** object configuration.



NOTE

For more details about configuring the **Subscription** object and **OperatorGroup** object, see "Installing from the software catalog by using the CLI".

Verification

1. To verify that the Multiarch Tuning Operator is installed, run the following command:

```
$ oc get csv -n openshift-multiarch-tuning-operator
```

Example output

NAME	DISPLAY	VERSION	REPLACES
PHASE			
multiarch-tuning-operator.<version>	Multiarch Tuning Operator	<version>	multiarch-tuning-operator.1.0.0
	Succeeded		

The installation is successful if the Operator is in the **Succeeded** phase.

- Optional: To verify that the **OperatorGroup** object is created, run the following command:

```
$ oc get operatorgroup -n openshift-multiarch-tuning-operator
```

Example output

NAME	AGE
openshift-multiarch-tuning-operator-q8zbb	133m

- Optional: To verify that the **Subscription** object is created, run the following command:

```
$ oc get subscription -n openshift-multiarch-tuning-operator
```

Example output

NAME	PACKAGE	SOURCE	CHANNEL
multiarch-tuning-operator	multiarch-tuning-operator	redhat-operators	stable

Additional resources

- [Installing from the software catalog by using the CLI](#)

3.11.2. Installing the Multiarch Tuning Operator by using the web console

You can install the Multiarch Tuning Operator by using the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Navigate to **Ecosystem** → **Software Catalog**.
3. Enter **Multiarch Tuning Operator** in the search field.
4. Click **Multiarch Tuning Operator**.
5. Select the **Multiarch Tuning Operator** version from the **Version** list.
6. Click **Install**.

7. Set the following options on the **Operator Installation** page:
 - a. Set **Update Channel** to **stable**.
 - b. Set **Installation Mode** to **All namespaces on the cluster**.
 - c. Set **Installed Namespace** to **Operator recommended Namespace** or **Select a Namespace**.
 The recommended Operator namespace is **openshift-multiarch-tuning-operator**. If the **openshift-multiarch-tuning-operator** namespace does not exist, the namespace is created during the Operator installation.

 If you select **Select a namespace**, you must select a namespace for the Operator from the **Select Project** list.
 - d. **Update approval** as **Automatic** or **Manual**.
 If you select **Automatic** updates, Operator Lifecycle Manager (OLM) automatically updates the running instance of the Multiarch Tuning Operator without any intervention.

 If you select **Manual** updates, OLM creates an update request. As a cluster administrator, you must manually approve the update request to update the Multiarch Tuning Operator to a newer version.
8. Optional: Select the **Enable Operator recommended cluster monitoring on this Namespace** checkbox.
9. Click **Install**.

Verification

1. Navigate to **Ecosystem → Installed Operators**.
2. Verify that the **Multiarch Tuning Operator** is listed with the **Status** field as **Succeeded** in the **openshift-multiarch-tuning-operator** namespace.

3.11.3. Multiarch Tuning Operator pod labels and architecture support overview

After installing the Multiarch Tuning Operator, you can verify the multi-architecture support for workloads in your cluster. You can identify and manage pods based on their architecture compatibility by using the pod labels.

These labels are automatically set on the newly created pods to provide insights into their architecture support.

The following table describes the labels that the Multiarch Tuning Operator adds when you create a pod:

Table 3.2. Pod labels that the Multiarch Tuning Operator adds when you create a pod

Label	Description
multiarch.openshift.io/multi-arch: ""	The pod supports multiple architectures.
multiarch.openshift.io/single-arch: ""	The pod supports only a single architecture.

Label	Description
multiarch.openshift.io/arm64: ""	The pod supports the arm64 architecture.
multiarch.openshift.io/amd64: ""	The pod supports the amd64 architecture.
multiarch.openshift.io/ppc64le: ""	The pod supports the ppc64le architecture.
multiarch.openshift.io/s390x: ""	The pod supports the s390x architecture.
multiarch.openshift.io/node-affinity: set	The Operator has set the node affinity requirement for the architecture.
multiarch.openshift.io/node-affinity: not-set	The Operator did not set the node affinity requirement. For example, when the pod already has a node affinity for the architecture, the Multiarch Tuning Operator adds this label to the pod.
multiarch.openshift.io/scheduling-gate: gated	The pod is gated.
multiarch.openshift.io/scheduling-gate: removed	The pod gate has been removed.
multiarch.openshift.io/inspection-error: ""	An error has occurred while building the node affinity requirements.
multiarch.openshift.io/preferred-node-affinity: set	The Operator has set the architecture preferences in the pod.
multiarch.openshift.io/preferred-node-affinity: not-set	The Operator did not set the architecture preferences in the pod because the user had already set them in the preferredDuringSchedulingIgnoredDuringExecution node affinity.

3.11.4. The ClusterPodPlacementConfig object

After installing the Multiarch Tuning Operator, you must create a **ClusterPodPlacementConfig** object. The object instructs the Operator to deploy its operand, which enables architecture-aware workload scheduling across your cluster.

The **ClusterPodPlacementConfig** object supports two optional plugins:

- The **node affinity scoring** plugin patches pods to set soft preferences, using weighted affinities, for the architectures specified by the user. Pods are more likely to be scheduled on nodes running architectures with higher weights.
- The **exec format error monitor** plugin detects **ENOEXEC** errors, which occur when a pod attempts to execute a binary incompatible with the architecture of the node. When enabled, this plugin generates events in the affected event stream of the pod. The plugin triggers an

ExecFormatErrorsDetected Prometheus alert if one or more **ENOEXEC** errors are detected within the last six hours. These errors can result from incorrect architecture node selectors, invalid image metadata that affects architecture-aware workload scheduling, an incorrect binary in an image, or an incompatible binary injected at runtime.



NOTE

You can create only one instance of the **ClusterPodPlacementConfig** object.

Example ClusterPodPlacementConfig object configuration

```
apiVersion: multiarch.openshift.io/v1beta1
kind: ClusterPodPlacementConfig
metadata:
  name: cluster
spec:
  logVerbosity: Normal
  namespaceSelector:
    matchExpressions:
      - key: multiarch.openshift.io/exclude-pod-placement
        operator: DoesNotExist
  plugins:
    nodeAffinityScoring:
      enabled: true
    platforms:
      - architecture: amd64
        weight: 100
      - architecture: arm64
        weight: 50
    execFormatErrorMonitor:
      enabled: true
  fallbackArchitecture: amd64
```

where:

metadata.name

Specifies the name of the object. You must set this parameter to **cluster**.

spec.logVerbosity

Optional parameter. Specifies the log verbosity level. You can set the field value to **Normal**, **Debug**, **Trace**, or **TraceAll**. The value is set to **Normal** by default.

spec.namespaceSelector

Optional parameter. You can configure the **namespaceSelector** to select the namespaces in which the Multiarch Tuning Operator's pod placement operand must process the **nodeAffinity** of the pods. All namespaces are considered by default.

spec.plugins.nodeAffinityScoring.enabled

Optional parameter. You can enable the node affinity scoring plugin to set architecture preferences for pod placement. When enabled, the scheduler first filters out nodes that do not meet the pod's requirements. Then, it prioritizes the remaining nodes based on the architecture scores defined in the **nodeAffinityScoring.platforms** field. The default value is false.

spec.plugins.nodeAffinityScoring.platforms

Optional parameter. Defines a list of architectures and their corresponding scores. The scheduler prioritizes nodes for pod placement based on the architecture scores that you set and the scheduling requirements defined in the pod specification.

spec.plugins.nodeAffinityScoring.platforms.architecture

Specifies the architecture for the node affinity scoring plugin. Accepted values are **arm64**, **amd64**, **ppc64le**, or **s390x**.

spec.plugins.nodeAffinityScoring.platforms.weight

Specifies the weight for the architecture you specified in the **spec.plugins.nodeAffinityScoring.platforms.architecture** parameter. The value must be configured in the range of **1** (lowest priority) to **100** (highest priority). The scheduler uses this score to prioritize nodes for pod placement, favoring nodes with architectures that have higher scores.

spec.plugins.execFormatErrorMonitor.enabled

Optional parameter. Set this field to **true** to enable the **execFormatErrorMonitor** plugin. When enabled, the plugin detects **ENOEXEC** errors, caused when a pod executes a binary incompatible with the node's architecture. The plugin generates events in the affected pods, and triggers the **ExecFormatErrorsDetected** Prometheus alert if one or more errors are found in the last six hours.

spec.fallbackArchitecture

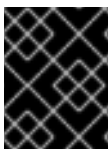
Optional parameter. Specifies an architecture where pods are scheduled if the image inspector cannot determine the architecture of the image. Valid values are **""**, **arm64**, **amd64**, **ppc64le**, or **s390x**. The value is set to **""** by default.

In this example, the **operator** field value is set to **DoesNotExist**. Therefore, if the **key** field value (**multiarch.openshift.io/exclude-pod-placement**) is set as a label in a namespace, the operand does not process the **nodeAffinity** of the pods in that namespace. Instead, the operand processes the **nodeAffinity** of the pods in namespaces that do not contain the label.

If you want the operand to process the **nodeAffinity** of the pods only in specific namespaces, you can configure the **namespaceSelector** as follows:

```
namespaceSelector:
  matchExpressions:
    - key: multiarch.openshift.io/include-pod-placement
      operator: Exists
```

In this example, the **operator** field value is set to **Exists**. Therefore, the operand processes the **nodeAffinity** of the pods only in namespaces that contain the **multiarch.openshift.io/include-pod-placement** label.



IMPORTANT

This Operator excludes pods in namespaces starting with **kube-**. The Operator also excludes pods that are expected to be scheduled on control plane nodes.

3.11.4.1. Creating the ClusterPodPlacementConfig object by using the CLI

To deploy the pod placement operand that enables architecture-aware workload scheduling, you can create the **ClusterPodPlacementConfig** object by using the OpenShift CLI (**oc**).

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to **oc** as a user with **cluster-admin** privileges.
- You have installed the Multiarch Tuning Operator.

Procedure

1. Create a **ClusterPodPlacementConfig** object YAML file:

Example ClusterPodPlacementConfig object configuration

```
apiVersion: multiarch.openshift.io/v1beta1
kind: ClusterPodPlacementConfig
metadata:
  name: cluster
spec:
  logVerbosityLevel: Normal
  namespaceSelector:
    matchExpressions:
      - key: multiarch.openshift.io/exclude-pod-placement
        operator: DoesNotExist
  plugins:
    nodeAffinityScoring:
      enabled: true
    platforms:
      - architecture: amd64
        weight: 100
      - architecture: arm64
        weight: 50
```

2. Create the **ClusterPodPlacementConfig** object by running the following command:

```
$ oc create -f <file_name>
```

Replace **<file_name>** with the name of the **ClusterPodPlacementConfig** object YAML file.

Verification

- To check that the **ClusterPodPlacementConfig** object is created, run the following command:

```
$ oc get clusterpodplacementconfig
```

Example output

```
NAME    AGE
cluster 29s
```

3.11.4.2. Creating the ClusterPodPlacementConfig object by using the web console

To deploy the pod placement operand that enables architecture-aware workload scheduling, you can create the **ClusterPodPlacementConfig** object by using the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.
- You have installed the Multiarch Tuning Operator.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Go to **Ecosystem → Installed Operators**.
3. On the **Installed Operators** page, click **Multiarch Tuning Operator**.
4. Click the **Cluster Pod Placement Config** tab.
5. Select either **Form view** or **YAML view**.
6. Configure the **ClusterPodPlacementConfig** object parameters.
7. Click **Create**.
8. Optional: If you want to edit the **ClusterPodPlacementConfig** object, perform the following actions:
 - a. Click the **Cluster Pod Placement Config** tab.
 - b. Select **Edit ClusterPodPlacementConfig** from the options menu.
 - c. Click **YAML** and edit the **ClusterPodPlacementConfig** object parameters.
 - d. Click **Save**.

Verification

- On the **Cluster Pod Placement Config** page, check that the **ClusterPodPlacementConfig** object is in the **Ready** state.

3.11.5. Creating the namespace-scoped PodPlacementConfig object

After creating the **ClusterPodPlacementConfig** object, you can configure pod placement at the namespace level by creating namespace-scoped **PodPlacementConfig** objects.

PodPlacementConfig objects modify the pod placement controller's behavior at the namespace level, and take precedence over the **ClusterPodPlacementConfig** object.

Prerequisites

- You have created a **ClusterPodPlacementConfig** object.

Procedure

1. Using a text editor, create a YAML file based on the following example and modify the example with your namespace values:

Example **PodPlacementConfig** object configuration

```
apiVersion: multiarch.openshift.io/v1beta1
kind: PodPlacementConfig
metadata:
  name: my-namespace-config
  namespace: my-namespace
spec:
  labelSelector:
    matchExpressions:
      - key: app
        operator: In
        values:
          - my-label-for-apps-performing-better-on-arm64
  priority: 100
  plugins:
    nodeAffinityScoring:
      enabled: true
    platforms:
      - architecture: amd64
        weight: 25
      - architecture: arm64
        weight: 75
```

where:

metadata

Specifies the object name, and namespace name. This parameter is required.

spec.labelSelector

Optionally specifies a label so that the **PodPlacementConfig** only applies to a subset of pods in the namespace. If you do not specify this parameter, the **PodPlacementConfig** applies to all pods in the namespace.

spec.priority

Optionally specifies a priority in case multiple **PodPlacementConfig** objects exist in one namespace. Higher priorities take precedence. Valid values are 0-255, and the default value is 0. If you specify multiple **PodPlacementConfig** objects in one namespace, they must have different priority values.

spec.plugins.nodeAffinityScoring

Specifies architecture preferences for pod placement. The controller prioritizes nodes based on the architecture scores, with higher weights taking precedence. If you enable this plugin by setting **spec.plugins.nodeAffinityScoring.enabled** to **true**, you must specify at least one platform with an architecture and weight value.

spec.plugins.nodeAffinityScoring.platforms[]

Specifies one or more platform configurations, each with a required architecture and weight pair. This parameter is required if **spec.plugins.nodeAffinityScoring** is present. Valid architecture values are **arm64**, **amd64**, **ppc64le**, and **s390x**. Valid weight values are 0-100. Each architecture can only be specified once.

2. Apply the configuration file by running the following command:

```
$ oc create -f <filename>
```

Replace **<filename>** with the name of the **PodPlacementConfig** configuration file.

3.11.6. Deleting the ClusterPodPlacementConfig object by using the CLI

You can create only one instance of the **ClusterPodPlacementConfig** object. If you want to re-create this object, you must first delete the existing instance.

You can delete this object by using the OpenShift CLI (**oc**).

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to **oc** as a user with **cluster-admin** privileges.

Procedure

1. Delete the **ClusterPodPlacementConfig** object by running the following command:

```
$ oc delete clusterpodplacementconfig cluster
```

Verification

- To check that the **ClusterPodPlacementConfig** object is deleted, run the following command:

```
$ oc get clusterpodplacementconfig
```

Example output

```
No resources found
```

3.11.7. Deleting the ClusterPodPlacementConfig object by using the web console

You can create only one instance of the **ClusterPodPlacementConfig** object. If you want to recreate this object, you must first delete the existing instance.

You can delete this object by using the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** privileges.
- You have access to the OpenShift Container Platform web console.
- You have created the **ClusterPodPlacementConfig** object.

Procedure

1. Log in to the OpenShift Container Platform web console.

2. Navigate to **Ecosystem → Installed Operators**.
3. On the **Installed Operators** page, click **Multiarch Tuning Operator**.
4. Click the **Cluster Pod Placement Config** tab.
5. Select **Delete ClusterPodPlacementConfig** from the options menu.
6. Click **Delete**.

Verification

- On the **Cluster Pod Placement Config** page, check that the **ClusterPodPlacementConfig** object has been deleted.

3.11.8. Uninstalling the Multiarch Tuning Operator by using the CLI

You can uninstall the Multiarch Tuning Operator by using the OpenShift CLI (**oc**).

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to **oc** as a user with **cluster-admin** privileges.
- You have deleted the **ClusterPodPlacementConfig** object.



IMPORTANT

You must delete the **ClusterPodPlacementConfig** object before uninstalling the Multiarch Tuning Operator. Uninstalling the Operator without deleting the **ClusterPodPlacementConfig** object leads to unexpected behavior.

Procedure

1. Get the **Subscription** object name for the Multiarch Tuning Operator by running the following command:

```
$ oc get subscription.operators.coreos.com -n <namespace>
```

Replace **<namespace>** with the name of the namespace where you want to uninstall the Multiarch Tuning Operator.

Example output

NAME	PACKAGE	SOURCE	CHANNEL
openshift-multiarch-tuning-operator	multiarch-tuning-operator	redhat-operators	stable

2. Get the **currentCSV** value for the Multiarch Tuning Operator by running the following command:

```
$ oc get subscription.operators.coreos.com <subscription_name> -n <namespace> -o yaml |
grep currentCSV
```

- Replace **<subscription_name>** with the **Subscription** object name. For example, **openshift-multiarch-tuning-operator**.
- Replace **<namespace>** with the name of the namespace where you want to uninstall the Multiarch Tuning Operator.

Example output

```
currentCSV: multiarch-tuning-operator.<version>
```

3. Delete the **Subscription** object by running the following command:

```
$ oc delete subscription.operators.coreos.com <subscription_name> -n <namespace>
```

- Replace **<subscription_name>** with the **Subscription** object name.
- Replace **<namespace>** with the name of the namespace where you want to uninstall the Multiarch Tuning Operator.

Example output

```
subscription.operators.coreos.com "openshift-multiarch-tuning-operator" deleted
```

4. Delete the CSV for the Multiarch Tuning Operator in the target namespace by using the **currentCSV** value by running the following command:

```
$ oc delete clusterserviceversion <currentCSV_value> -n <namespace>
```

- Replace **<currentCSV_value>** with the **currentCSV** value for the Multiarch Tuning Operator. For example: **multiarch-tuning-operator.<version>**.
- Replace **<namespace>** with the name of the namespace where you want to uninstall the Multiarch Tuning Operator.

Example output

```
clusterserviceversion.operators.coreos.com "multiarch-tuning-operator.<version>"
deleted
```

Verification

- To verify that the Multiarch Tuning Operator is uninstalled, run the following command:

```
$ oc get csv -n <namespace>
```

Replace **<namespace>** with the name of the namespace where you have uninstalled the Multiarch Tuning Operator.

Example output

```
No resources found in openshift-multiarch-tuning-operator namespace.
```

3.11.9. Uninstalling the Multiarch Tuning Operator by using the web console

You can uninstall the Multiarch Tuning Operator by using the OpenShift Container Platform web console.

Prerequisites

- You have access to the cluster with **cluster-admin** permissions.
- You have deleted the **ClusterPodPlacementConfig** object.



IMPORTANT

You must delete the **ClusterPodPlacementConfig** object before uninstalling the Multiarch Tuning Operator. Uninstalling the Operator without deleting the **ClusterPodPlacementConfig** object leads to unexpected behavior.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Navigate to **Ecosystem → Software Catalog**.
3. Enter **Multiarch Tuning Operator** in the search field.
4. Click **Multiarch Tuning Operator**.
5. Click the **Details** tab.
6. From the **Actions** menu, select **Uninstall Operator**.
7. When prompted, click **Uninstall**.

Verification

1. Navigate to **Ecosystem → Installed Operators**.
2. On the **Installed Operators** page, verify that the **Multiarch Tuning Operator** is not listed.

3.12. MULTIARCH TUNING OPERATOR RELEASE NOTES

The Multiarch Tuning Operator (MTO) optimizes workload management within multi-architecture clusters and in single-architecture clusters transitioning to multi-architecture environments. Use the release notes to track the development of the Multiarch Tuning Operator.

3.12.1. Additional resources

- [Managing workloads on multi-architecture clusters by using the Multiarch Tuning Operator](#)

3.12.2. Release notes for the Multiarch Tuning Operator 1.3.3

The release notes for the Multiarch Tuning Operator 1.3.3 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 28 July 2026

3.12.2.1. Enhancements

- MTO has been updated to use **go** version 1.26.4.

3.12.3. Release notes for the Multiarch Tuning Operator 1.3.2

The release notes for the Multiarch Tuning Operator 1.3.2 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 13 July 2026

3.12.3.1. Bug fixes

- Previously, when deleting a **ClusterPodPlacementConfig** with the **execFormatErrorMonitor** plugin enabled, the MTO could get stuck in a **Deleting** state because some resources were not deleted. With this update, all relevant resources are removed so that the deletion is successful. ([MULTIARCH-6186](#))
- Previously, a race condition between the **eNoExecEvent** daemon and the handler controller caused the **ENoExecEvent** CR to be rolled back before its status could be set, preventing any exec format errors from being recorded. With this update the conflict so events are captured successfully. ([MULTIARCH-6207](#))
- Previously, deleting a **ClusterPodPlacementConfig** object could tear down the webhook and operand resources while **PodPlacementConfigs** still existed, orphaning them without the resources they depend on. The deletion is now properly gated and blocks if any **PodPlacementConfig** objects remain. ([MULTIARCH-6236](#))

3.12.3.2. Enhancements

- MTO has been updated to use **go** version 1.26.3.
- When the MTO is unable to connect to an image registry, the resulting error message has been improved to better explain the error. ([MULTIARCH-5541](#))
- The MTO API has been updated to use **runtime.NewSchemeBuilder** instead of the deprecated **scheme.Builder**. ([MULTIARCH-6272](#))

3.12.4. Release notes for the Multiarch Tuning Operator 1.3.1

The release notes for the Multiarch Tuning Operator 1.3.1 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 1 July 2026

3.12.4.1. Enhancements

- MTO has been updated to use **go** version 1.25.9.

3.12.5. Release notes for the Multiarch Tuning Operator 1.3.0

The release notes for the Multiarch Tuning Operator 1.3.0 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 6 April 2026

3.12.5.1. New features and enhancements

- With this release, after you create a **ClusterPodPlacementConfig** object, you can create namespace-scoped **PodPlacementConfig** objects for the purposes of configuring pod placement at the namespace level. **PodPlacementConfig** objects modify the behavior of the pod placement controller at the namespace level, and take precedence over the **ClusterPodPlacementConfig** object. For more information, see [Creating the namespace-scoped PodPlacementConfig object](#).
- With this release, you can specify a fallback architecture where pods are scheduled if the image inspector cannot determine the architecture of the image. For more information, see [Creating the ClusterPodPlacementConfig object](#).

3.12.5.2. Bug fixes

- Previously, an error could occur that led to an **ENoExecEvent** custom resource (CR) failing to be deleted. This leftover CR resulted in the failure to uninstall the **execFormatErrorMonitor** plugin. With this update, the **execFormatErrorMonitor** plugin can be uninstalled if there are leftover **ENoExecEvent** CRs. Deleting the **ClusterPodPlacementConfig** object removes all remaining CRs regardless of their state. ([MULTIARCH-5642](#))
- Previously, the Multiarch Tuning Operator (MTO) processed images that contained attestation manifests, leading to the incorrect creation of an "unknown" architecture. Pods could fail to be scheduled when they tried to target the "unknown" architecture. With this update, the MTO does not process attestation manifests, and the "unknown" architecture is not created. ([MULTIARCH-5800](#))

3.12.5.3. Enhancements

- MTO has been updated to use **go** version 1.25.7.

3.12.6. Release notes for the Multiarch Tuning Operator 1.2.2

The release notes for the Multiarch Tuning Operator 1.2.2 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 6 February 2026

3.12.6.1. Enhancements

- With this update, MTO uses the Red Hat Universal Base Image (UBI) 9 minimal image. This change improves compatibility with OpenShift Container Platform ecosystems.
- MTO has been updated to use **go** version 1.25.3, **k8s** version 1.34.1, and Operator SDK v4 version 1.33.

- The **ENoExecEvent.Status.Command** field has been removed from the **ENoExecEvent** custom resource. This field was not in use.

3.12.7. Release notes for the Multiarch Tuning Operator 1.2.1

The release notes for the Multiarch Tuning Operator 1.2.1 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 15 December 2025

3.12.7.1. Bug fixes

- Previously, the Multiarch Tuning Operator image inspector incorrectly processed images whose registry address included a digest, tag, and port number. The port portion of the registry was incorrectly interpreted as an image tag and was trimmed, causing the inspector to construct an invalid image reference. With this update, image references that contain a digest, tag, and registry port are now correctly parsed and handled. ([MULTIARCH-5767](#))

3.12.8. Release notes for the Multiarch Tuning Operator 1.2.0

The release notes for the Multiarch Tuning Operator 1.2.0 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 22 October 2025

3.12.8.1. New features and enhancements

- With this release, you can enable the **exec format error monitor** plugin for the Multiarch Tuning Operator. This plugin detects **ENOEXEC** errors, which occur when a pod attempts to execute a binary incompatible with the node's architecture. You enable this plugin by setting the **plugins.execFormatErrorMonitor.enabled** parameter to **true** in the **ClusterPodPlacementConfig** object. For more information, see [Creating the ClusterPodPlacementConfig object](#).

3.12.8.2. Bug fixes

- Previously, the Multiarch Tuning Operator incorrectly handled the Operator bundle image inspector, restricting the inspector to a single architecture, which could cause OLM to fail when installing Operators. With this update, MTO now sets the bundle image to support all architectures, allowing Operators to be successfully installed on single-architecture clusters when the Multiarch Tuning Operator is deployed. ([MULTIARCH-5546](#))
- Previously, when a cluster global pull secret was changed, stale authentication information could remain in the Multiarch Tuning Operator cache. With this update, the cache is cleared whenever a cluster global pull secret is changed. ([MULTIARCH-5538](#))
- Previously, the Multiarch Tuning Operator failed to process pods if an image reference contained both a tag and a digest. With this update, the image inspector prioritizes the digest if both are present. ([MULTIARCH-5584](#))
- Previously, the Multiarch Tuning Operator did not respect the **.spec.registrySources.containerRuntimeSearchRegistries** field in the

config.openshift.io/Image custom resource when a workload image did not specify a registry URL. With this update, the Operator can now handle this case, allowing workload images without an explicit registry URL to be pulled successfully. ([MULTIARCH-5611](#))

- Previously, if the **ClusterPodPlacementConfig** object was deleted less than 1 second after its creation, some finalizers were not removed in time, causing certain resources to remain. With this update, all finalizers are properly deleted when the **ClusterPodPlacementConfig** object is deleted. ([MULTIARCH-5372](#))

3.12.9. Release notes for the Multiarch Tuning Operator 1.1.1

The release notes for the Multiarch Tuning Operator 1.1.1 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 27 May 2025

3.12.9.1. Bug fixes

- Previously, the pod placement operand did not support authenticating registries using wildcard entries in the hostname of their pull secret. This caused inconsistent behavior with Kubelet when pulling images, because Kubelet supported wildcard entries while the operand required exact hostname matches. As a result, image pulls could fail unexpectedly when registries used wildcard hostnames.
With this release, the pod placement operand supports pull secrets that include wildcard hostnames, ensuring consistent and reliable image authentication and pulling.
- Previously, when image inspection failed after all retries and the **nodeAffinityScoring** plugin was enabled, the pod placement operand applied incorrect **nodeAffinityScoring** labels. With this release, the operand sets **nodeAffinityScoring** labels correctly, even when image inspection fails. The operand now applies these labels independently of the required affinity process to ensure accurate and consistent scheduling.

3.12.10. Release notes for the Multiarch Tuning Operator 1.1.0

The release notes for the Multiarch Tuning Operator 1.1.0 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 18 March 2024

3.12.10.1. New features and enhancements

- The Multiarch Tuning Operator is now supported on managed offerings, including ROSA with Hosted Control Planes (HCP) and other HCP environments.
- With this release, you can configure architecture-aware workload scheduling by using the new **plugins** field in the **ClusterPodPlacementConfig** object. You can use the **plugins.nodeAffinityScoring** field to set architecture preferences for pod placement. If you enable the **nodeAffinityScoring** plugin, the scheduler first filters out nodes that do not meet the pod requirements. The scheduler then prioritizes the remaining nodes based on the architecture scores defined in the **nodeAffinityScoring.platforms** field.

3.12.10.2. Bug fixes

- With this release, the Multiarch Tuning Operator does not update the **nodeAffinity** field for pods that are managed by a daemon set. ([OCPBUGS-45885](#))

3.12.11. Release notes for the Multiarch Tuning Operator 1.0.0

The release notes for the Multiarch Tuning Operator 1.0.0 summarize all new features and enhancements, notable technical changes, major corrections from the previous version, and any known bugs upon general availability.

Issued: 31 October 2024

3.12.11.1. New features and enhancements

- With this release, the Multiarch Tuning Operator supports custom network scenarios and cluster-wide custom registries configurations.
- With this release, you can identify pods based on their architecture compatibility by using the pod labels that the Multiarch Tuning Operator adds to newly created pods.
- With this release, you can monitor the behavior of the Multiarch Tuning Operator by using the metrics and alerts that are registered in the Cluster Monitoring Operator.

CHAPTER 4. POSTINSTALLATION CLUSTER TASKS

After installing OpenShift Container Platform, you can configure, scale, and maintain your cluster to meet operational requirements, including managing nodes and infrastructure workloads, enabling features, applying autoscaling, and maintaining etcd.

After installing OpenShift Container Platform, you can further expand and customize your cluster to your requirements.

4.1. AVAILABLE CLUSTER CUSTOMIZATIONS

You complete most of the cluster configuration and customization after you deploy your OpenShift Container Platform cluster. A number of *configuration resources* are available.



NOTE

If you install your cluster on IBM Z®, not all features and functions are available.

You modify the configuration resources to configure the major features of the cluster, such as the image registry, networking configuration, image build behavior, and the identity provider.

For current documentation of the settings that you control by using these resources, use the **oc explain** command, for example **oc explain builds --api-version=config.openshift.io/v1**

4.1.1. Cluster configuration resources

All cluster configuration resources are globally scoped (not namespaced) and named **cluster**.

Resource name	Description
apiserver.config.openshift.io	Provides API server configuration such as certificates and certificate authorities .
authentication.config.openshift.io	Controls the identity provider and authentication configuration for the cluster.
build.config.openshift.io	Controls default and enforced configuration for all builds on the cluster.
console.config.openshift.io	Configures the behavior of the web console interface, including the logout behavior .
featuregate.config.openshift.io	Enables FeatureGates so that you can use Tech Preview features.
image.config.openshift.io	Configures how specific image registries should be treated (allowed, disallowed, insecure, CA details).

Resource name	Description
ingress.config.openshift.io	Configuration details related to routing such as the default domain for routes.
oauth.config.openshift.io	Configures identity providers and other behavior related to internal OAuth server flows.
project.config.openshift.io	Configures how projects are created including the project template.
proxy.config.openshift.io	Defines proxies to be used by components needing external network access. Note: not all components currently consume this value.
scheduler.config.openshift.io	Configures scheduler behavior such as profiles and default node selectors.

4.1.2. Operator configuration resources

These configuration resources are cluster-scoped instances, named **cluster**, which control the behavior of a specific component as owned by a particular Operator.

Resource name	Description
consoles.operator.openshift.io	Controls console appearance such as branding customizations
config.imageregistry.operator.openshift.io	Configures OpenShift image registry settings such as public routing, log levels, proxy settings, resource constraints, replica counts, and storage type.
config.samples.operator.openshift.io	Configures the Samples Operator to control which example image streams and templates are installed on the cluster.

4.1.3. Additional configuration resources

These configuration resources represent a single instance of a particular component. In some cases, you can request multiple instances by creating multiple instances of the resource. In other cases, the Operator can use only a specific resource instance name in a specific namespace. Reference the component-specific documentation for details on how and when you can create additional resource instances.

Resource name	Instance name	Namespace	Description
alertmanager.monitoring.coreos.com	main	openshift-monitoring	Controls the Alertmanager deployment parameters.
ingresscontroller.operator.openshift.io	default	openshift-ingress-operator	Configures Ingress Operator behavior such as domain, number of replicas, certificates, and controller placement.

4.1.4. Informational Resources

You use these resources to retrieve information about the cluster. Some configurations might require you to edit these resources directly.

Resource name	Instance name	Description
clusterversion.config.openshift.io	version	In OpenShift Container Platform 4.22, you must not customize the ClusterVersion resource for production clusters. Instead, follow the process to update a cluster .
dns.config.openshift.io	cluster	You cannot modify the DNS settings for your cluster. You can check the DNS Operator status .
infrastructure.config.openshift.io	cluster	Configuration details allowing the cluster to interact with its cloud provider.
network.config.openshift.io	cluster	You cannot modify your cluster networking after installation. To customize your network, follow the process to customize networking during installation .

4.2. ADDING WORKER NODES

After you deploy your OpenShift Container Platform cluster, you can add worker nodes to scale cluster resources. There are different ways you can add worker nodes depending on the installation method and the environment of your cluster.

4.2.1. Adding worker nodes to an on-premise cluster

For on-premise clusters, you can add worker nodes by using the OpenShift Container Platform CLI (**oc**) to generate an ISO image, which can then be used to boot one or more nodes in your target cluster. This process can be used regardless of how you installed your cluster.

You can add one or more nodes at a time while customizing each node with more complex configurations, such as static network configuration, or you can specify only the MAC address of each node. Any configurations that are not specified during ISO generation are retrieved from the target cluster and applied to the new nodes.

Preflight validation checks are also performed when booting the ISO image to inform you of failure-causing issues before you attempt to boot each node.

[Adding worker nodes to an on-premise cluster](#)

4.2.2. Adding worker nodes to installer-provisioned infrastructure clusters

For installer-provisioned infrastructure clusters, you can manually or automatically scale the **MachineSet** object to match the number of available bare-metal hosts.

To add a bare-metal host, you must configure all network prerequisites, configure an associated **baremetalhost** object, then provision the worker node to the cluster. You can add a bare-metal host manually or by using the web console.

- [Adding worker nodes using the web console](#)
- [Adding worker nodes using YAML in the web console](#)
- [Manually adding a worker node to an installer-provisioned infrastructure cluster](#)

4.2.3. Adding worker nodes to user-provisioned infrastructure clusters

For user-provisioned infrastructure clusters, you can add worker nodes by using a RHEL or RHCOS ISO image and connecting it to your cluster using cluster Ignition config files. For RHEL worker nodes, the following example uses Ansible playbooks to add worker nodes to the cluster. For RHCOS worker nodes, the following example uses an ISO image and network booting to add worker nodes to the cluster.

- [Adding RHCOS worker nodes to a user-provisioned infrastructure cluster](#)

4.2.4. Adding worker nodes to clusters managed by the Assisted Installer

For clusters managed by the Assisted Installer, you can add worker nodes by using the Red Hat OpenShift Cluster Manager console, the Assisted Installer REST API or you can manually add worker nodes using an ISO image and cluster Ignition config files.

- [Adding worker nodes using the OpenShift Cluster Manager](#)
- [Adding worker nodes using the Assisted Installer REST API](#)
- [Manually adding worker nodes to a SNO cluster](#)

4.2.5. Adding worker nodes to clusters managed by the multicluster engine for Kubernetes

For clusters managed by the multicluster engine for Kubernetes, you can add worker nodes by using the dedicated multicluster engine console.

- [Creating your cluster with the console](#)

4.3. ADJUST WORKER NODES

If you incorrectly sized the worker nodes during deployment, adjust them by creating one or more new compute machine sets, scale them up, then scale the original compute machine set down before removing them.

4.3.1. Understanding the difference between compute machine sets and the machine config pool

Compute machine sets and machine config pools control different aspects of node lifecycle in OpenShift Container Platform. Understanding how each object relates to scaling and upgrades helps you configure nodes correctly.

MachineSet objects describe OpenShift Container Platform nodes with respect to the cloud or machine provider.

The **MachineConfigPool** object allows **MachineConfigController** components to define and provide the status of machines in the context of upgrades.

The **MachineConfigPool** object allows users to configure how upgrades are rolled out to the OpenShift Container Platform nodes in the machine config pool.

The **NodeSelector** object can be replaced with a reference to the **MachineSet** object.

4.3.2. Scaling a compute machine set manually

To add or remove an instance of a machine in a compute machine set, you can manually scale the compute machine set.

This guidance is relevant to fully automated, installer-provisioned infrastructure installations. Customized, user-provisioned infrastructure installations do not have compute machine sets.

Prerequisites

- Install an OpenShift Container Platform cluster and the **oc** command line.
- Log in to **oc** as a user with **cluster-admin** permission.

Procedure

1. View the compute machine sets that are in the cluster by running the following command:

```
$ oc get machinesets.machine.openshift.io -n openshift-machine-api
```

The compute machine sets are listed in the form of **<clusterid>-worker-<aws-region-az>**.

2. View the compute machines that are in the cluster by running the following command:

```
$ oc get machines.machine.openshift.io -n openshift-machine-api
```

3. Set the annotation on the compute machine that you want to delete by running the following command:


```
$ oc annotate machines.machine.openshift.io/<machine_name> -n openshift-machine-api
machine.openshift.io/delete-machine="true"
```

4. Scale the compute machine set by running one of the following commands:

```
$ oc scale --replicas=2 machinesets.machine.openshift.io <machineset> -n openshift-
machine-api
```

Or:

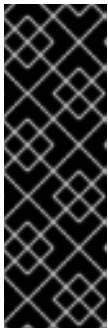
```
$ oc edit machinesets.machine.openshift.io <machineset> -n openshift-machine-api
```

TIP

You can alternatively apply the following YAML to scale the compute machine set:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: <machineset>
  namespace: openshift-machine-api
spec:
  replicas: 2
```

You can scale the compute machine set up or down. It takes several minutes for the new machines to be available.



IMPORTANT

By default, the machine controller tries to drain the node that is backed by the machine until it succeeds. In some situations, such as with a misconfigured pod disruption budget, the drain operation might not be able to succeed. If the drain operation fails, the machine controller cannot proceed removing the machine.

You can skip draining the node by annotating **machine.openshift.io/exclude-node-draining** in a specific machine.

Verification

- Verify the deletion of the intended machine by running the following command:

```
$ oc get machines.machine.openshift.io
```

4.3.3. The compute machine set deletion policy

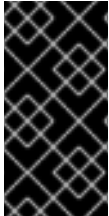
Compute machine sets can be configured to use the **Random**, **Newest**, and **Oldest** deletion options. The default is **Random**, meaning that random machines are chosen and deleted when scaling compute machine sets down.

The deletion policy can be set according to the use case by modifying the particular compute machine set as in the following example:

■

```
spec:
  deletePolicy: <delete_policy>
  replicas: <desired_replica_count>
```

Specific machines can also be prioritized for deletion by adding the annotation **machine.openshift.io/delete-machine=true** to the machine of interest, regardless of the deletion policy.



IMPORTANT

By default, the OpenShift Container Platform router pods are deployed on workers. Because the router is required to access some cluster resources, including the web console, do not scale the worker compute machine set to **0** unless you first relocate the router pods.



NOTE

Custom compute machine sets can be used for use cases requiring that services run on specific nodes and that those services are ignored by the controller when the worker compute machine sets are scaling down. This prevents service disruption.

4.3.4. Creating default cluster-wide node selectors

You can use default cluster-wide node selectors on pods together with labels on nodes to constrain all pods created in a cluster to specific nodes.

With cluster-wide node selectors, when you create a pod in that cluster, OpenShift Container Platform adds the default node selectors to the pod and schedules the pod on nodes with matching labels.

You configure cluster-wide node selectors by editing the Scheduler Operator custom resource (CR). You add labels to a node, a compute machine set, or a machine config. Adding the label to the compute machine set ensures that if the node or machine goes down, new nodes have the label. Labels added to a node or machine config do not persist if the node or machine goes down.



NOTE

You can add additional key/value pairs to a pod. But you cannot add a different value for a default key.

The following procedure adds a default cluster-wide node selector.

Procedure

1. Edit the Scheduler Operator CR to add the default cluster-wide node selectors:

```
$ oc edit scheduler cluster
```

Example Scheduler Operator CR with a node selector

```
apiVersion: config.openshift.io/v1
kind: Scheduler
metadata:
  name: cluster
```

```
...
spec:
  defaultNodeSelector: type=user-node,region=east
  mastersSchedulable: false
```

where:

spec.defaultNodeSelector

Specifies a node selector with the appropriate **<key>:<value>** pairs.

After making this change, wait for the pods in the **openshift-kube-apiserver** project to redeploy. This can take several minutes. The default cluster-wide node selector does not take effect until the pods redeploy.

2. Add labels to a node by using a compute machine set or editing the node directly:

- Use a compute machine set to add labels to nodes managed by the compute machine set when a node is created:
 - a. Run the following command to add labels to a **MachineSet** object:

```
$ oc patch MachineSet <name> --type='json' -
p='[{"op":"add","path":"/spec/template/spec/metadata/labels", "value":{"<key>":"<value>"},"<key>":"<value>"}]' -n openshift-machine-api
```

Add a **<key>/<value>** pair for each label.

For example:

```
$ oc patch MachineSet ci-ln-l8nry52-f76d1-hl7m7-worker-c --type='json' -
p='[{"op":"add","path":"/spec/template/spec/metadata/labels", "value":{"type":"user-node","region":"east"}}]' -n openshift-machine-api
```

TIP

You can alternatively apply the following YAML to add labels to a compute machine set:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: <machineset>
  namespace: openshift-machine-api
spec:
  template:
    spec:
      metadata:
        labels:
          region: "east"
          type: "user-node"
```

- b. Verify that the labels are added to the **MachineSet** object by using the **oc edit** command:
For example:

—

```
$ oc edit MachineSet abc612-msrtw-worker-us-east-1c -n openshift-machine-api
```

Example MachineSet object

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
...
spec:
...
template:
  metadata:
...
  spec:
    metadata:
      labels:
        region: east
        type: user-node
...

```

- c. Redeploy the nodes associated with that compute machine set by scaling down to **0** and scaling up the nodes:

For example:

```
$ oc scale --replicas=0 MachineSet ci-ln-l8nry52-f76d1-hl7m7-worker-c -n openshift-machine-api
```

```
$ oc scale --replicas=1 MachineSet ci-ln-l8nry52-f76d1-hl7m7-worker-c -n openshift-machine-api
```

- d. When the nodes are ready and available, verify that the label is added to the nodes by using the **oc get** command:

```
$ oc get nodes -l <key>=<value>
```

For example:

```
$ oc get nodes -l type=user-node
```

Example output

```
NAME                                STATUS ROLES  AGE  VERSION
ci-ln-l8nry52-f76d1-hl7m7-worker-c-vmqzp Ready  worker  61s  v1.35.4
```

- Add labels directly to a node:
 - a. Edit the **Node** object for the node:

```
$ oc label nodes <name> <key>=<value>
```

For example, to label a node:

```
$ oc label nodes ci-ln-l8nry52-f76d1-hl7m7-worker-b-tgq49 type=user-node
region=east
```

TIP

You can alternatively apply the following YAML to add labels to a node:

```
kind: Node
apiVersion: v1
metadata:
  name: <node_name>
labels:
  type: "user-node"
  region: "east"
```

- b. Verify that the labels are added to the node using the **oc get** command:

```
$ oc get nodes -l <key>=<value>,<key>=<value>
```

For example:

```
$ oc get nodes -l type=user-node,region=east
```

Example output

```
NAME                                STATUS ROLES  AGE  VERSION
ci-ln-l8nry52-f76d1-hl7m7-worker-b-tgq49 Ready  worker  17m  v1.35.4
```

4.4. IMPROVING CLUSTER STABILITY IN HIGH LATENCY ENVIRONMENTS USING WORKER LATENCY PROFILES

If as a cluster administrator, you performed latency tests for platform verification, you might discover the need to adjust the operation of the cluster to ensure stability in cases of high latency.

As a cluster administrator, you need to change only one parameter, recorded in a file, which controls four parameters affecting how supervisory processes read status and interpret the health of the cluster. Changing only the one parameter provides cluster tuning in an easy, supportable manner.

The **Kubelet** process provides the starting point for monitoring cluster health. The **Kubelet** sets status values for all nodes in the OpenShift Container Platform cluster. The Kubernetes Controller Manager (**kube controller**) reads the status values every 10 seconds, by default. If the **kube controller** cannot read a node status value, it loses contact with that node after a configured period. The default behavior is:

1. The node controller on the control plane updates the node health to **Unhealthy** and marks the node **Ready** condition **Unknown**.
2. In response, the scheduler stops scheduling pods to that node.

3. The Node Lifecycle Controller adds a **node.kubernetes.io/unreachable** taint with a **NoExecute** effect to the node and schedules any pods on the node for eviction after five minutes, by default.

This behavior can cause problems if your network is prone to latency issues, especially if you have nodes at the network edge. In some cases, the Kubernetes Controller Manager might not receive an update from a healthy node due to network latency. The **Kubelet** evicts pods from the node even though the node is healthy.

To avoid this problem, you can use *worker latency profiles* to adjust the frequency that the **Kubelet** and the Kubernetes Controller Manager wait for status updates before taking action. These adjustments help to ensure that your cluster runs properly if network latency between the control plane and the worker nodes is not optimal.

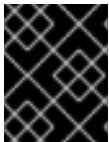
These worker latency profiles contain three sets of parameters that are predefined with carefully tuned values to control the reaction of the cluster to increased latency. There is no need to experimentally find the best values manually.

You can configure worker latency profiles when installing a cluster or at any time you notice increased latency in your cluster network.

4.4.1. Understanding worker latency profiles

Review the following information to learn about worker latency profiles, which allow you to control the reaction of the cluster to latency issues without needing to determine the best values by using manual methods.

Worker latency profiles are four different categories of carefully-tuned parameters. The four parameters which implement these values are **node-status-update-frequency**, **node-monitor-grace-period**, **default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds**.



IMPORTANT

Setting these parameters manually is not supported. Incorrect parameter settings adversely affect cluster stability.

All worker latency profiles configure the following parameters:

node-status-update-frequency

Specifies how often the kubelet posts node status to the API server.

node-monitor-grace-period

Specifies the amount of time in seconds that the Kubernetes Controller Manager waits for an update from a kubelet before marking the node unhealthy and adding the **node.kubernetes.io/not-ready** or **node.kubernetes.io/unreachable** taint to the node.

default-not-ready-toleration-seconds

Specifies the amount of time in seconds after marking a node unhealthy that the Kube API Server Operator waits before evicting pods from that node.

default-unreachable-toleration-seconds

Specifies the amount of time in seconds after marking a node unreachable that the Kube API Server Operator waits before evicting pods from that node.

The following Operators monitor the changes to the worker latency profiles and respond accordingly:

- The Machine Config Operator (MCO) updates the **node-status-update-frequency** parameter on the compute nodes.
- The Kubernetes Controller Manager updates the **node-monitor-grace-period** parameter on the control plane nodes.
- The Kubernetes API Server Operator updates the **default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds** parameters on the control plane nodes.

Although the default configuration works in most cases, OpenShift Container Platform offers two other worker latency profiles for situations where the network is experiencing higher latency than usual. The three worker latency profiles are described in the following sections:

Default worker latency profile

With the **Default** profile, each **Kubelet** updates its status every 10 seconds (**node-status-update-frequency**). The **Kube Controller Manager** checks the statuses of **Kubelet** every 5 seconds. The Kubernetes Controller Manager waits 40 seconds (**node-monitor-grace-period**) for a status update from **Kubelet** before considering the **Kubelet** unhealthy. If no status is made available to the Kubernetes Controller Manager, it then marks the node with the **node.kubernetes.io/not-ready** or **node.kubernetes.io/unreachable** taint and evicts the pods on that node.

If a pod is on a node that has the **NoExecute** taint, the pod runs according to **tolerationSeconds**. If the node has no taint, it will be evicted in 300 seconds (**default-not-ready-toleration-seconds** and **default-unreachable-toleration-seconds** settings of the **Kube API Server**).

Profile	Component	Parameter	Value
Default	kubelet	node-status-update-frequency	10s
	Kubelet Controller Manager	node-monitor-grace-period	40s
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	300s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	300s

Medium worker latency profile

Use the **MediumUpdateAverageReaction** profile if the network latency is slightly higher than usual. The **MediumUpdateAverageReaction** profile reduces the frequency of kubelet updates to 20 seconds and changes the period that the Kubernetes Controller Manager waits for those updates to 2 minutes. The pod eviction period for a pod on that node is reduced to 60 seconds. If the pod has the **tolerationSeconds** parameter, the eviction waits for the period specified by that parameter.

The Kubernetes Controller Manager waits for 2 minutes to consider a node unhealthy. In another minute, the eviction process starts.

Profile	Component	Parameter	Value
MediumUpdateAverageReaction	kubelet	node-status-update-frequency	20s
	Kubelet Controller Manager	node-monitor-grace-period	2m
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	60s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	60s

Low worker latency profile

Use the **LowUpdateSlowReaction** profile if the network latency is extremely high.

The **LowUpdateSlowReaction** profile reduces the frequency of kubelet updates to 1 minute and changes the period that the Kubernetes Controller Manager waits for those updates to 5 minutes. The pod eviction period for a pod on that node is reduced to 60 seconds. If the pod has the **tolerationSeconds** parameter, the eviction waits for the period specified by that parameter.

The Kubernetes Controller Manager waits for 5 minutes to consider a node unhealthy. In another minute, the eviction process starts.

Profile	Component	Parameter	Value
LowUpdateSlowReaction	kubelet	node-status-update-frequency	1m
	Kubelet Controller Manager	node-monitor-grace-period	5m
	Kubernetes API Server Operator	default-not-ready-toleration-seconds	60s
	Kubernetes API Server Operator	default-unreachable-toleration-seconds	60s



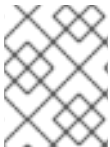
NOTE

The latency profiles do not support custom machine config pools, only the default worker machine config pools.

4.4.2. Using and changing worker latency profiles

You can change a worker latency profile to deal with network latency at any time by editing the **node.config** object. With this configuration, you can ensure that your cluster runs properly if network latency between the control plane and the compute nodes fluctuates.

You must move one worker latency profile at a time. For example, you cannot move directly from the **Default** profile to the **LowUpdateSlowReaction** worker latency profile. You must move from the **Default** worker latency profile to the **MediumUpdateAverageReaction** profile and then to the **LowUpdateSlowReaction** profile. Similarly, when returning to the **Default** profile, you must move from the low profile to the medium profile first, then to **Default**.



NOTE

You can also configure worker latency profiles upon installing an OpenShift Container Platform cluster.

Procedure

1. Move to the medium worker latency profile:
 - a. Edit the **node.config** object:


```
$ oc edit nodes.config/cluster
```
 - b. Add **spec.workerLatencyProfile: MediumUpdateAverageReaction**:

Example node.config object

```
apiVersion: config.openshift.io/v1
kind: Node
metadata:
  annotations:
    include.release.openshift.io/ibm-cloud-managed: "true"
    include.release.openshift.io/self-managed-high-availability: "true"
    include.release.openshift.io/single-node-developer: "true"
    release.openshift.io/create-only: "true"
  creationTimestamp: "2022-07-08T16:02:51Z"
  generation: 1
  name: cluster
  ownerReferences:
    - apiVersion: config.openshift.io/v1
      kind: ClusterVersion
      name: version
      uid: 36282574-bf9f-409e-a6cd-3032939293eb
    resourceVersion: "1865"
    uid: 0c0f7a4c-4307-4187-b591-6155695ac85b
  spec:
    workerLatencyProfile: MediumUpdateAverageReaction
  # ...
```

where:

spec.workerLatencyProfile.MediumUpdateAverageReaction

Specifies that the medium worker latency policy should be used.

Scheduling on each compute node is disabled as the change is being applied.

2. Optional: Move to the low worker latency profile:

- a. Edit the **node.config** object:

```
$ oc edit nodes.config/cluster
```

- b. Change the **spec.workerLatencyProfile** value to **LowUpdateSlowReaction**:

Example node.config object

```
apiVersion: config.openshift.io/v1
kind: Node
metadata:
  annotations:
    include.release.openshift.io/ibm-cloud-managed: "true"
    include.release.openshift.io/self-managed-high-availability: "true"
    include.release.openshift.io/single-node-developer: "true"
    release.openshift.io/create-only: "true"
  creationTimestamp: "2022-07-08T16:02:51Z"
  generation: 1
  name: cluster
  ownerReferences:
    - apiVersion: config.openshift.io/v1
      kind: ClusterVersion
      name: version
      uid: 36282574-bf9f-409e-a6cd-3032939293eb
  resourceVersion: "1865"
  uid: 0c0f7a4c-4307-4187-b591-6155695ac85b
spec:
  workerLatencyProfile: LowUpdateSlowReaction
# ...
```

where:

spec.workerLatencyProfile.LowUpdateSlowReaction

Specifies that the low worker latency policy should be used.

Scheduling on each compute node is disabled as the change is being applied.

Verification

- When all nodes return to the **Ready** condition, you can use the following command to look in the Kubernetes Controller Manager to ensure it was applied:

```
$ oc get KubeControllerManager -o yaml | grep -i workerlatency -A 5 -B 5
```

Example output

```
# ...
- lastTransitionTime: "2022-07-11T19:47:10Z"
```

```

reason: ProfileUpdated
status: "False"
type: WorkerLatencyProfileProgressing
- lastTransitionTime: "2022-07-11T19:47:10Z"
  message: all static pod revision(s) have updated latency profile
reason: ProfileUpdated
status: "True"
type: WorkerLatencyProfileComplete
- lastTransitionTime: "2022-07-11T19:20:11Z"
  reason: AsExpected
status: "False"
type: WorkerLatencyProfileDegraded
- lastTransitionTime: "2022-07-11T19:20:36Z"
  status: "False"
# ...

```

where:

status.message: all static pod revision(s) have updated latency profile

Specifies that the profile is applied and active.

To change the medium profile to default or change the default to medium, edit the **node.config** object and set the **spec.workerLatencyProfile** parameter to the appropriate value.

4.5. MANAGING CONTROL PLANE MACHINES

[Control plane machine sets](#) provide management capabilities for control plane machines that are similar to what compute machine sets provide for compute machines. The availability and initial status of control plane machine sets on your cluster depend on your cloud provider and the version of OpenShift Container Platform that you installed. For more information, see [Getting started with control plane machine sets](#).

4.5.1. Adding a control plane node to your cluster

Add a control plane node to recover from a degraded state, perform deep-level debugging, or ensure stability and security of the control plane in complex bare metal scenarios.

Prerequisites

- You have installed a healthy cluster with at least three control plane nodes.
- You have created a single control plane node that you intend to add to your cluster as a postinstalltion task.

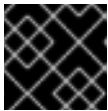
Procedure

1. Retrieve pending Certificate Signing Requests (CSRs) for the new control plane node by entering the following command:

```
$ oc get csr | grep Pending
```

2. Approve all pending CSRs for the control plane node by entering the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}
{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```



IMPORTANT

You must approve the CSRs to complete the installation.

3. Confirm that the control plane node is in the **Ready** status by entering the following command:

```
$ oc get nodes
```



NOTE

On installer-provisioned infrastructure, the etcd Operator relies on the Machine API to manage the control plane and ensure etcd quorum. The Machine API then uses **Machine** CRs to represent and manage the underlying control plane nodes.

4. Create the **BareMetalHost** and **Machine** CRs and link them to the **Node** CR of the control plane node.
 - a. Create the **BareMetalHost** CR with a unique **.metadata.name** value as demonstrated in the following example:

```
apiVersion: metal3.io/v1alpha1
kind: BareMetalHost
metadata:
  name: node-5
  namespace: openshift-machine-api
spec:
  automatedCleaningMode: metadata
  bootMACAddress: 00:00:00:00:00:02
  bootMode: UEFI
  customDeploy:
    method: install_coreos
  externallyProvisioned: true
  online: true
  userData:
    name: master-user-data-managed
    namespace: openshift-machine-api
# ...
```

- b. Apply the **BareMetalHost** CR by entering the following command:

```
$ oc apply -f <filename>
```

where **<filename>** specifies the name of the **BareMetalHost** CR.

- c. Create the **Machine** CR by using the unique **.metadata.name** value as demonstrated in the following example:

```
apiVersion: machine.openshift.io/v1beta1
kind: Machine
```

```

metadata:
  annotations:
    machine.openshift.io/instance-state: externally provisioned
    metal3.io/BareMetalHost: openshift-machine-api/node-5
  finalizers:
  - machine.machine.openshift.io
  labels:
    machine.openshift.io/cluster-api-cluster: <cluster_name>
    machine.openshift.io/cluster-api-machine-role: master
    machine.openshift.io/cluster-api-machine-type: master
  name: node-5
  namespace: openshift-machine-api
spec:
  metadata: {}
  providerSpec:
    value:
      apiVersion: baremetal.cluster.k8s.io/v1alpha1
      customDeploy:
        method: install_coreos
      hostSelector: {}
      image:
        checksum: ""
        url: ""
      kind: BareMetalMachineProviderSpec
      metadata:
        creationTimestamp: null
      userData:
        name: master-user-data-managed
# ...

```

where **<cluster_name>** specifies the name of the specific cluster, for example, **test-day2-1-6qv96**.

- d. Get the cluster name by running the following command:

```
$ oc get infrastructure cluster -o=jsonpath='{.status.infrastructureName}'{"\n"}
```

- e. Apply the **Machine** CR by entering the following command:

```
$ oc apply -f <filename>
```

where **<filename>** specifies the name of the **Machine** CR.

- f. Link **BareMetalHost**, **Machine**, and **Node** objects by running the **link-machine-and-node.sh** script:

- i. Copy the following **link-machine-and-node.sh** script to a local machine:

```

#!/bin/bash

# Credit goes to
# https://bugzilla.redhat.com/show_bug.cgi?id=1801238.
# This script will link Machine object
# and Node object. This is needed
# in order to have IP address of

```

```

# the Node present in the status of the Machine.

set -e

machine="$1"
node="$2"

if [ -z "$machine" ] || [ -z "$node" ]; then
    echo "Usage: $0 MACHINE NODE"
    exit 1
fi

node_name=$(echo "${node}" | cut -f2 -d':')

oc proxy &
proxy_pid=$!
function kill_proxy {
    kill $proxy_pid
}
trap kill_proxy EXIT SIGINT

HOST_PROXY_API_PATH="http://localhost:8001/apis/metal3.io/v1alpha1/namespaces/openshift-machine-api/baremetalhosts"

function print_nics() {
    local ips
    local eob
    declare -a ips

    readarray -t ips <<(echo "${1}" \
        | jq '.[] | select(.type == "InternalIP") | .address' \
        | sed 's"/"/g')

    eob=','
    for (( i=0; i<${#ips[@]}; i++ )); do
        if [ ${i+1} -eq ${#ips[@]} ]; then
            eob=""
        fi
        cat <<- EOF
        {
            "ip": "${ips[i]}",
            "mac": "00:00:00:00:00:00",
            "model": "unknown",
            "speedGbps": 10,
            "vlanId": 0,
            "pxe": true,
            "name": "eth1"
        }${eob}
    EOF
    done
}

function wait_for_json() {
    local name
    local url
    local curl_opts

```

```

local timeout

local start_time
local curr_time
local time_diff

name="$1"
url="$2"
timeout="$3"
shift 3
curl_opts="$@"
echo -n "Waiting for $name to respond"
start_time=$(date +%s)
until curl -g -X GET "$url" "${curl_opts[@]}" 2> /dev/null | jq '.' 2> /dev/null >
/dev/null; do
    echo -n "."
    curr_time=$(date +%s)
    time_diff=$((curr_time - start_time))
    if [[ $time_diff -gt $timeout ]]; then
        printf "\nTimed out waiting for %s" "${name}"
        return 1
    fi
    sleep 5
done
echo " Success!"
return 0
}

wait_for_json oc_proxy "${HOST_PROXY_API_PATH}" 10 -H "Accept:
application/json" -H "Content-Type: application/json"

addresses=$(oc get node -n openshift-machine-api "${node_name}" -o json | jq -c
'.status.addresses')

machine_data=$(oc get machines.machine.openshift.io -n openshift-machine-api -o
json "${machine}")
host=$(echo "$machine_data" | jq
'.metadata.annotations["metal3.io/BareMetalHost"]' | cut -f2 -d/ | sed 's/"//g')

if [ -z "$host" ]; then
    echo "Machine $machine is not linked to a host yet." 1>&2
    exit 1
fi

# The address structure on the host doesn't match the node, so extract
# the values we want into separate variables so we can build the patch
# we need.
hostname=$(echo "${addresses}" | jq '[] | select(. | .type == "Hostname") | .address' |
sed 's/"//g')

set +e
read -r -d " host_patch << EOF
{
    "status": {
        "hardware": {
            "hostname": "${hostname}",
            "nics": [

```

```

$(print_nics "${addresses}")
],
"systemVendor": {
  "manufacturer": "Red Hat",
  "productName": "product name",
  "serialNumber": ""
},
"firmware": {
  "bios": {
    "date": "04/01/2014",
    "vendor": "SeaBIOS",
    "version": "1.11.0-2.el7"
  }
},
"ramMebibytes": 0,
"storage": [],
"cpu": {
  "arch": "x86_64",
  "model": "Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz",
  "clockMegahertz": 2199.998,
  "count": 4,
  "flags": []
}
}
}
}
EOF
set -e

echo "PATCHING HOST"
echo "${host_patch}" | jq .

curl -s \
  -X PATCH \
  "${HOST_PROXY_API_PATH}/${host}/status" \
  -H "Content-type: application/merge-patch+json" \
  -d "${host_patch}"

oc get baremetalhost -n openshift-machine-api -o yaml "${host}"

```

- ii. Make the script executable by entering the following command:

```
$ chmod +x link-machine-and-node.sh
```

- iii. Run the script by entering the following command:

```
$ bash link-machine-and-node.sh node-5 node-5
```



NOTE

The first **node-5** instance represents the machine, and the second instance represents the node.

Verification

1. Confirm members of etcd by executing into one of the pre-existing control plane nodes:
 - a. Open a remote shell session to the control plane node by entering the following command:

```
$ oc rsh -n openshift-etcd etcd-node-0
```

- b. List etcd members:

```
# etcdctl member list -w table
```

2. Check the etcd Operator configuration process until completion by entering the following command. Expected output shows **False** under the **PROGRESSING** column.

```
$ oc get clusteroperator etcd
```

3. Confirm etcd health by running the following commands:

- a. Open a remote shell session to the control plane node:

```
$ oc rsh -n openshift-etcd etcd-node-0
```

- b. Check endpoint health. Expected output shows **is healthy** for the endpoint.

```
# etcdctl endpoint health
```

4. Verify that all nodes are ready by entering the following command. The expected output shows the **Ready** status beside each node entry.

```
$ oc get nodes
```

5. Verify that the cluster Operators are all available by entering the following command. Expected output lists each Operator and shows the available status as **True** beside each listed Operator.

```
$ oc get ClusterOperators
```

6. Verify that the cluster version is correct by entering the following command:

```
$ oc get ClusterVersion
```

Example output

```
NAME      VERSION  AVAILABLE  PROGRESSING  SINCE   STATUS
version   OpenShift Container Platform.5  True      False        5h57m   Cluster version is
OpenShift Container Platform.5
```

4.6. CREATING INFRASTRUCTURE MACHINE SETS FOR PRODUCTION ENVIRONMENTS

You can create a compute machine set to create machines that host only infrastructure components, such as the default router, the integrated container image registry, and components for cluster metrics and monitoring. These infrastructure machines are not counted toward the total number of subscriptions

that are required to run the environment.

For information on infrastructure nodes and which components can run on infrastructure nodes, see [Creating infrastructure machine sets](#).

To create an infrastructure node, you can [use a machine set](#), [assign a label to the nodes](#), or [use a machine config pool](#).

For sample machine sets that you can use with these procedures, see [Creating machine sets for different clouds](#).

Applying a specific node selector to all infrastructure components causes OpenShift Container Platform to [schedule those workloads on nodes with that label](#).

4.6.1. Creating a compute machine set

To dynamically manage machine compute resources, you can create your own compute machine sets in addition to the compute machine sets created by the installation program. Use the OpenShift Container Platform CLI to automate node provisioning.

Prerequisites

- Deploy an OpenShift Container Platform cluster.
- Install the OpenShift CLI (**oc**).
- Log in to **oc** as a user with **cluster-admin** permission.

Procedure

1. Create a new YAML file that contains the compute machine set custom resource (CR) sample and is named **<file_name>.yaml**.
Ensure that you set the **<clusterID>** and **<role>** parameter values.
2. Optional: If you are not sure which value to set for a specific field, you can check an existing compute machine set from your cluster.
 - a. To list the compute machine sets in your cluster, run the following command:

```
$ oc get machinesets -n openshift-machine-api
```

The following is example output:

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
agl030519-vplxk-worker-us-east-1a	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1b	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1c	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1d	0	0			55m
agl030519-vplxk-worker-us-east-1e	0	0			55m
agl030519-vplxk-worker-us-east-1f	0	0			55m

- b. To view values of a specific compute machine set custom resource (CR), run the following command:

```
$ oc get machineset <machineset_name> \
-n openshift-machine-api -o yaml
```

The following is example output:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
    name: <infrastructure_id>-<role>
    namespace: openshift-machine-api
spec:
  replicas: 1
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: <infrastructure_id>
      machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>
  template:
    metadata:
      labels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machine-role: <role>
        machine.openshift.io/cluster-api-machine-type: <role>
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>
    spec:
      providerSpec:
        ...
```

where:

metadata.labels.machine.openshift.io/cluster-api-cluster

Specifies the cluster infrastructure ID.

metadata.labels.name

Specifies a default node label.



NOTE

For clusters that have user-provisioned infrastructure, a compute machine set can only create **worker** and **infra** type machines.

spec.template.metadata.spec.providerSpec

Specifies the values of the compute machine set CR. The values are platform-specific. For more information about **<providerSpec>** parameters in the CR, see the sample compute machine set CR configuration for your provider.

3. Create a **MachineSet** CR by running the following command:

```
$ oc create -f <file_name>.yaml
```

Verification

- View the list of compute machine sets by running the following command:

```
$ oc get machineset -n openshift-machine-api
```

The following is example output:

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
agl030519-vplxk-infra-us-east-1a	1	1	1	1	11m
agl030519-vplxk-worker-us-east-1a	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1b	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1c	1	1	1	1	55m
agl030519-vplxk-worker-us-east-1d	0	0			55m
agl030519-vplxk-worker-us-east-1e	0	0			55m
agl030519-vplxk-worker-us-east-1f	0	0			55m

When the new compute machine set is available, the **DESIRED** and **CURRENT** values match. If the compute machine set is not available, wait a few minutes and run the command again.

4.6.2. Creating an infrastructure node

To reduce subscription costs, you can use labels to configure compute nodes as infrastructure nodes, where you can move infrastructure resources.

After you create the infrastructure nodes, you can move appropriate workloads to those nodes by using taints and tolerations.

You can optionally create a default cluster-wide node selector. The default node selector is applied to pods created in all namespaces and creates an intersection with any existing node selectors on a pod, which additionally constrains the pod's selector.

IMPORTANT

- See "Creating infrastructure machine sets" for installer-provisioned infrastructure environments or for any cluster where the control plane nodes are managed by the Machine API.
- If the default node selector key conflicts with the key of a pod's label, then the default node selector is not applied.
However, do not set a default node selector that might cause a pod to become unschedulable. For example, setting the default node selector to a specific node role, such as **node-role.kubernetes.io/infra=""**, when a pod's label is set to a different node role, such as **node-role.kubernetes.io/master=""**, can cause the pod to become unschedulable. For this reason, use caution when setting the default node selector to specific node roles.

You can alternatively use a project node selector to avoid cluster-wide node selector key conflicts.

Procedure

- Add a label to the compute nodes that you want to act as infrastructure nodes by running the following command:

```
$ oc label node <node-name> node-role.kubernetes.io/infra=""
```

2. Check to see if applicable nodes now have the **infra** role by running the following command:

```
$ oc get nodes
```

3. Optional: Create a default cluster-wide node selector.

- a. Edit the **Scheduler** object by running the following command:

```
$ oc edit scheduler cluster
```

- b. Add the **defaultNodeSelector** field with the appropriate node selector by running the following command:

```
apiVersion: config.openshift.io/v1
kind: Scheduler
metadata:
  name: cluster
spec:
  defaultNodeSelector: node-role.kubernetes.io/infra=""
# ...
```

This example node selector deploys pods on infrastructure nodes by default.

- c. Save the file to apply the changes.

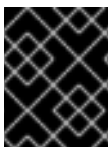
You can now move infrastructure resources to the new infrastructure nodes and remove any workloads that you do not want, or that do not belong, on the new infrastructure node. See the list of workloads supported for use on infrastructure nodes in "OpenShift Container Platform infrastructure components".

Additional resources

- [Project node selectors](#)

4.6.3. Creating a machine config pool for infrastructure machines

You can create a machine configuration pool for infrastructure machines to apply dedicated configuration to infra machines. You might want to apply dedicated configuration to infra machines because they run distinct workloads from other nodes in the cluster.



IMPORTANT

Creating a custom machine configuration pool overrides default worker pool configurations if they refer to the same file or unit.

Procedure

1. Add a label to the node you want to assign as the infra node by running the following command:

```
$ oc label node <node_name> node-role.kubernetes.io/infra=
```

where:

<node_name>

Specifies the name of the node you want to assign as an infra node.

2. Create a YAML file that defines the machine config pool, as in the following example:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: infra
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,infra]}
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/infra: ""
```

- Add the worker role and your custom role in the **spec.machineConfigSelector.matchExpressions[]** field.
- Add the label you added to the node in the **spec.nodeSelector.matchLabels** field.



NOTE

Custom machine config pools inherit machine configs from the worker pool. Custom pools use any machine config targeted for the worker pool, but add the ability to also deploy changes that are targeted at only the custom pool. Because a custom pool inherits resources from the worker pool, any change to the worker pool also affects the custom pool.

3. After you have the YAML file, you can create the machine config pool by specifying the file you created in the following command:

```
$ oc create -f <filename>
```

4. Check the machine configs to ensure that the infrastructure configuration rendered successfully by running the following command:

```
$ oc get machineconfig
```

Example output

NAME	GENERATEDBYCONTROLLER
IGNITIONVERSION CREATED	
00-master	365c1cfd14de5b0e3b85e0fc815b0060f36ab955
3.5.0 31d	
00-worker	365c1cfd14de5b0e3b85e0fc815b0060f36ab955
3.5.0 31d	
# ...	
rendered-infra-4e48906dca84ee702959c71a53ee80e7	
365c1cfd14de5b0e3b85e0fc815b0060f36ab955 3.5.0	23m

You should see a new machine config, with the **rendered-infra-*** prefix.

5. Optional: To deploy changes to a custom pool, create a machine config that uses the custom

pool name as the label, such as **infra**. Note that this is not required and only shown for instructional purposes. In this manner, you can apply any custom configurations specific to only your infra nodes.



NOTE

After you create the new machine config pool, the MCO generates a new rendered config for that pool, and associated nodes of that pool reboot to apply the new configuration.

- a. Create a YAML file that defines the machine config pool, as in the following example:

```
$ cat infra.mc.yaml
```

Example output

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  name: 51-infra
  labels:
    machineconfiguration.openshift.io/role: <role>
spec:
  config:
    ignition:
      version: 3.5.0
    storage:
      files:
        - path: /etc/infratest
          mode: 0644
          contents:
            source: data:,infra
```

where:

role

Specifies the label you added to the node as a **nodeSelector**.

- b. Apply the machine config to the infra-labeled nodes by running the following command:

```
$ oc create -f infra.mc.yaml
```

6. Confirm that your new machine config pool is available by running the following command:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED	MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT	DEGRADEDMACHINECOUNT	AGE
infra	rendered-infra-60e35c2e99f42d976e084fa94da4d0fc	True	False	False	1	1	0	0	4m20s

master	rendered-master-9360fdb895d4c131c7c4bebbae099c90	True	False	False
3	3	3	0	91m
worker	rendered-worker-60e35c2e99f42d976e084fa94da4d0fc	True	False	False
2	2	2	0	91m

In this example, the role of the node was changes from **worker** to **infra**.

Additional resources

- [Node configuration management with machine config pools](#)

4.7. ASSIGNING MACHINE SET RESOURCES TO INFRASTRUCTURE NODES

After creating an infrastructure machine set, the **worker** and **infra** roles are applied to new infra nodes. Nodes with the **infra** role are not counted toward the total number of subscriptions that are required to run the environment, even when the **worker** role is also applied.

However, when an infra node is assigned the worker role, there is a chance that user workloads can get assigned inadvertently to the infra node. To avoid this, you can apply a taint to the infra node and tolerations for the pods that you want to control.

4.7.1. Binding infrastructure node workloads using taints and tolerations

To avoid user workloads being inadvertently assigned to an infra node, you can apply a taint to the infra node and tolerations for the pods you want to control. After creating an infrastructure machine set, the **worker** and **infra** roles are applied to new infra nodes.

Nodes with the **infra** role applied are not counted toward the total number of subscriptions that are required to run the environment, even when the **worker** role is also applied. If you have an infrastructure node that has the **infra** and **worker** roles assigned, you must configure the node so that user workloads are not assigned to it.



IMPORTANT

It is recommended that you preserve the dual **infra,worker** label that is created for infrastructure nodes and use taints and tolerations to manage nodes that user workloads are scheduled on. If you remove the **worker** label from the node, you must create a custom pool to manage it. A node with a label other than **master** or **worker** is not recognized by the MCO without a custom pool. Maintaining the **worker** label allows the node to be managed by the default worker machine config pool, if no custom pools that select the custom label exists. The **infra** label communicates to the cluster that it does not count toward the total number of subscriptions.

Prerequisites

- Configure additional **MachineSet** objects in your OpenShift Container Platform cluster.

Procedure

1. Add a taint to the infrastructure node to prevent scheduling user workloads on it:
 - a. Determine if the node has the taint by running the following command:


```
$ oc describe nodes <node_name>
```

Sample output

```
oc describe node ci-ln-iyhx092-f76d1-nvdfm-worker-b-wln2l
Name:          ci-ln-iyhx092-f76d1-nvdfm-worker-b-wln2l
Roles:         worker
...
Taints:         node-role.kubernetes.io/infra=reserved:NoSchedule
...
```

This example shows that the node has a taint. You can proceed with adding a toleration to your pod in the next step.

- b. If you have not configured a taint to prevent scheduling user workloads on it, configure a taint by running the following command:

```
$ oc adm taint nodes <node_name> <key>=<value>:<effect>
```

For example:

```
$ oc adm taint nodes node1 node-role.kubernetes.io/infra=reserved:NoSchedule
```

TIP

You can alternatively edit the pod specification to add the taint:

```
apiVersion: v1
kind: Node
metadata:
  name: node1
# ...
spec:
  taints:
    - key: node-role.kubernetes.io/infra
      value: reserved
      effect: NoSchedule
# ...
```

These examples place a taint on **node1** that has the **node-role.kubernetes.io/infra** key and the **NoSchedule** taint effect. Nodes with the **NoSchedule** effect schedule only pods that tolerate the taint, but allow existing pods to remain scheduled on the node. If you added a **NoSchedule** taint to the infrastructure node, any pods that are controlled by a daemon set on that node are marked as **misscheduled**. You must either delete the pods or add a toleration to the pods as shown in the Red Hat Knowledgebase solution [add toleration on misscheduled DNS pods](#). Note that you cannot add a toleration to a daemon set object that is managed by an operator.



NOTE

If a descheduler is used, pods violating node taints could be evicted from the cluster.

2. Add tolerations to the pods that you want to schedule on the infrastructure node, such as the router, registry, and monitoring workloads. Referencing the previous examples, add the following tolerations to the **Pod** object specification:

```

apiVersion: v1
kind: Pod
metadata:
  annotations:

# ...
spec:
# ...
  tolerations:
    - key: <node_taint_key>
      value: <node_taint_value>
      effect: <taint_effect>
      operator: Equal

```

where:

<node_taint_key>

Specifies the key that you added to the taint on the node.

<node_taint_value>

Specifies the value of the key-value pair taint that you added to the node.

<taint_effect>

Specifies the effect that you added to the node.

Equal

Specifies that a taint with a key matching **<node_taint_key>** is required to be present on the node.

This toleration matches the taint created by the **oc adm taint** command. A pod with this toleration can be scheduled onto the infrastructure node.



NOTE

Moving pods for an Operator installed via OLM to an infrastructure node is not always possible. The capability to move Operator pods depends on the configuration of each Operator.

3. Schedule the pod to the infrastructure node by using a scheduler. See the documentation for "Controlling pod placement using the scheduler" for details.
4. Remove any workloads that you do not want, or that do not belong, on the new infrastructure node. See the list of workloads supported for use on infrastructure nodes in "OpenShift Container Platform infrastructure components".

Additional resources

- [Controlling pod placement using the scheduler](#)

4.8. MOVING RESOURCES TO INFRASTRUCTURE MACHINE SETS

Some of the infrastructure resources are deployed in your cluster by default. You can move them to the infrastructure machine sets that you created.

4.8.1. Moving the router

Deploying the router pod on an infrastructure node can reduce your OpenShift Container Platform subscription size. Move the router pod by editing the **IngressController** object in the **openshift-ingress-operator** namespace. By default, the pod is deployed to a worker node.

Prerequisites

- Configure additional compute machine sets in your OpenShift Container Platform cluster.

Procedure

1. View the **IngressController** custom resource for the router Operator by running the following command:

```
$ oc get ingresscontroller default -n openshift-ingress-operator -o yaml
```

Example output

```
apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  creationTimestamp: 2019-04-18T12:35:39Z
  finalizers:
  - ingresscontroller.operator.openshift.io/finalizer-ingresscontroller
  generation: 1
  name: default
  namespace: openshift-ingress-operator
  resourceVersion: "11341"
  selfLink: /apis/operator.openshift.io/v1/namespaces/openshift-ingress-operator/ingresscontrollers/default
  uid: 79509e05-61d6-11e9-bc55-02ce4781844a
spec: {}
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: 2019-04-18T12:36:15Z
    status: "True"
    type: Available
  domain: apps.<cluster>.example.com
  endpointPublishingStrategy:
    type: LoadBalancerService
  selector: ingresscontroller.operator.openshift.io/deployment-ingresscontroller=default
```

2. Edit the **ingresscontroller** resource and change the **nodeSelector** to use the **infra** label by running the following command:

```
$ oc edit ingresscontroller default -n openshift-ingress-operator
```

Example output

```
-
```

```

apiVersion: operator.openshift.io/v1
kind: IngressController
metadata:
  creationTimestamp: "2025-03-26T21:15:43Z"
  finalizers:
  - ingresscontroller.operator.openshift.io/finalizer-ingresscontroller
  generation: 1
  name: default
# ...
spec:
  nodePlacement:
    nodeSelector:
      matchLabels:
        node-role.kubernetes.io/infra: ""
  tolerations:
  - effect: NoSchedule
    key: node-role.kubernetes.io/infra
    value: reserved
# ...

```

Add a **nodeSelector** parameter with the appropriate value to the component you want to move. You can use a **nodeSelector** parameter in the format shown or use **<key>: <value>** pairs, based on the value specified for the node. If you added a taint to the infrastructure node, also add a matching toleration.

Verification

- Confirm that the router pod is running on the **infra** node.
 - a. View the list of router pods and note the node name of the running pod by running the following command:

```
$ oc get pod -n openshift-ingress -o wide
```

Example output

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
NOMINATED NODE READINESS GATES						
router-default-86798b4b5d-bdlvd	1/1	Running	0	28s	10.130.2.4	ip-10-0-217-226.ec2.internal
router-default-955d875f4-255g8	0/1	Terminating	0	19h	10.129.2.4	ip-10-0-148-172.ec2.internal

In this example, the running pod is on the **ip-10-0-217-226.ec2.internal** node.

- b. View the node status of the running pod by running the following command:

```
$ oc get node <node_name>
```

Specify the **<node_name>** that you obtained from the pod list.

Example output

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-217-226.ec2.internal	Ready	infra,worker	17h	v1.35.4

Because the role list includes **infra**, the pod is running on the correct node.

4.8.2. Moving the default registry

Deploying the registry pod on an infrastructure node can reduce your OpenShift Container Platform subscription size. Move the registry pod by editing the **configs.imageregistry.operator.openshift.io/cluster** config object.

Prerequisites

- Configure additional compute machine sets in your OpenShift Container Platform cluster.

Procedure

1. Edit the **configs.imageregistry.operator.openshift.io/cluster** object by running the following command:

```
$ oc edit configs.imageregistry.operator.openshift.io/cluster
```

2. Add a **nodeSelector** parameter with the appropriate value to the component you want to move, as shown in the following example.

```
apiVersion: imageregistry.operator.openshift.io/v1
kind: Config
metadata:
  name: cluster
# ...
spec:
  logLevel: Normal
  managementState: Managed
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
    - effect: NoSchedule
      key: node-role.kubernetes.io/infra
      value: reserved
```

You can use a **nodeSelector** parameter in the format shown or use **<key>: <value>** pairs, based on the value specified for the node. If you added a taint to the infrastructure node, also add a matching toleration.

Verification

- Verify the registry pod has been moved to the infrastructure node.
 - a. Identify the node where the registry pod is located by running the following command:

```
$ oc get pods -o wide -n openshift-image-registry
```

- b. Confirm the node has the label you specified:

```
$ oc describe node <node_name>
```

where:

<node_name>

Specifies the name of the node that you modified. Review the command output and confirm that **node-role.kubernetes.io/infra** is in the **LABELS** list.

4.8.3. Moving the monitoring solution

Redeploy the monitoring stack to infrastructure nodes to reduce your subscription requirements. Create and apply a custom config map to move the monitoring stack to infrastructure nodes. The monitoring stack includes Prometheus, Thanos Querier, and Alertmanager, and is managed by the Cluster Monitoring Operator (CMO).

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** cluster role.
- You have created the **cluster-monitoring-config ConfigMap** object.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Edit the **cluster-monitoring-config** config map and change the **nodeSelector** to use the **infra** label by running the following command:

```
$ oc edit configmap cluster-monitoring-config -n openshift-monitoring
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |+
    alertmanagerMain:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
      tolerations:
        - key: node-role.kubernetes.io/infra
          value: reserved
          effect: NoSchedule
    prometheusK8s:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
      tolerations:
        - key: node-role.kubernetes.io/infra
          value: reserved
          effect: NoSchedule
    prometheusOperator:
      nodeSelector:
        node-role.kubernetes.io/infra: ""
```

```

tolerations:
- key: node-role.kubernetes.io/infra
  value: reserved
  effect: NoSchedule
metricsServer:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule
kubeStateMetrics:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule
telemetryClient:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule
openshiftStateMetrics:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule
thanosQuerier:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule
monitoringPlugin:
  nodeSelector:
    node-role.kubernetes.io/infra: ""
  tolerations:
  - key: node-role.kubernetes.io/infra
    value: reserved
    effect: NoSchedule

```

Add a **nodeSelector** parameter with the appropriate value to the component you want to move. You can use a **nodeSelector** parameter in the format shown or use **<key>: <value>** pairs, based on the value specified for the node. If you added a taint to the infrastructure node, also add a matching toleration.

2. Watch the monitoring pods move to the new machines by running the following command:

```
$ watch 'oc get pod -n openshift-monitoring -o wide'
```

3. If a component has not moved to the **infra** node, delete the pod with this component by running the following command:

```
$ oc delete pod -n openshift-monitoring <pod>
```

The component from the deleted pod is re-created on the **infra** node.

4.9. THE CLUSTER AUTOSCALER

The cluster autoscaler adjusts the size of an OpenShift Container Platform cluster to meet its current deployment needs. It uses declarative, Kubernetes-style arguments to provide infrastructure management that does not rely on objects of a specific cloud provider.

The cluster autoscaler increases the size of the cluster when there are pods that fail to schedule on any of the current worker nodes due to insufficient resources or when another node is necessary to meet deployment needs. The cluster autoscaler does not increase the cluster resources beyond the limits that you specify.

The cluster autoscaler computes the total memory, CPU, and GPU on all nodes the cluster, even though it does not manage the control plane nodes. These values are not single-machine oriented. They are an aggregation of all the resources in the entire cluster. For example, if you set the maximum memory resource limit, the cluster autoscaler includes all the nodes in the cluster when calculating the current memory usage. That calculation is then used to determine if the cluster autoscaler has the capacity to add more worker resources.



IMPORTANT

Ensure that the **maxNodesTotal** value in the **ClusterAutoscaler** custom resource (CR) that you create is large enough to account for the total possible number of machines in your cluster. This value must encompass the number of control plane machines and the possible number of compute machines that you might scale to.

4.9.1. Automatic node removal

Every 10 seconds, the cluster autoscaler checks which nodes are unnecessary in the cluster and removes them. The cluster autoscaler considers a node for removal if the following conditions apply:

- The node utilization is less than the *node utilization level* threshold for the cluster. The node utilization level is the sum of the requested resources divided by the allocated resources for the node. If you do not specify a value in the **ClusterAutoscaler** custom resource, the cluster autoscaler uses a default value of **0.5**, which corresponds to 50% utilization.
- The cluster autoscaler can move all pods running on the node to the other nodes. The Kubernetes scheduler is responsible for scheduling pods on the nodes.
- The cluster autoscaler does not have scale down disabled annotation.

If the following types of pods are present on a node, the cluster autoscaler will not remove the node:

- Pods with restrictive pod disruption budgets (PDBs).
- Kube-system pods that do not run on the node by default.
- Kube-system pods that do not have a PDB or have a PDB that is too restrictive.

- Pods that are not backed by a controller object such as a deployment, replica set, or stateful set.
- Pods with local storage.
- Pods that cannot be moved elsewhere because of a lack of resources, incompatible node selectors or affinity, matching anti-affinity, and so on.
- Unless they also have a **"cluster-autoscaler.kubernetes.io/safe-to-evict": "true"** annotation, pods that have a **"cluster-autoscaler.kubernetes.io/safe-to-evict": "false"** annotation.

For example, you set the maximum CPU limit to 64 cores and configure the cluster autoscaler to only create machines that have 8 cores each. If your cluster starts with 30 cores, the cluster autoscaler can add up to 4 more nodes with 32 cores, for a total of 62.



NOTE

By default, when the cluster autoscaler removes a node, it does not cordon the node when draining the pods from the node. You can configure the cluster autoscaler to cordon the node before draining and moving the pods by setting the **spec.scaleDown.cordonNodeBeforeTerminating** parameter to **enabled** in the **ClusterAutoscaler** CR. This parameter is disabled by default. It is recommended to enable this parameter in production clusters because of the risk of data loss, application errors, pods getting stuck in the terminating state, or other issues if the cluster autoscaler removes a node when the parameter is disabled. Leaving this parameter disabled, which can result in faster node removal, might be appropriate in clusters that run only stateless workloads.

4.9.2. Limitations

If you configure the cluster autoscaler, additional usage restrictions apply:

- Do not modify the nodes that are in autoscaled node groups directly. All nodes within the same node group have the same capacity and labels and run the same system pods.
- Specify requests for your pods.
- If you have to prevent pods from being deleted too quickly, configure appropriate PDBs.
- Confirm that your cloud provider quota is large enough to support the maximum node pools that you configure.
- Do not run additional node group autoscalers, especially the ones offered by your cloud provider.



NOTE

The cluster autoscaler only adds nodes in autoscaled node groups if doing so would result in a schedulable pod. If the available node types cannot meet the requirements for a pod request, or if the node groups that could meet these requirements are at their maximum size, the cluster autoscaler cannot scale up.

4.9.3. Interaction with other scheduling features

The horizontal pod autoscaler (HPA) and the cluster autoscaler modify cluster resources in different ways. The HPA changes the deployment's or replica set's number of replicas based on the current CPU

load. If the load increases, the HPA creates new replicas, regardless of the amount of resources available to the cluster. If there are not enough resources, the cluster autoscaler adds resources so that the HPA-created pods can run. If the load decreases, the HPA stops some replicas. If this action causes some nodes to be underutilized or completely empty, the cluster autoscaler deletes the unnecessary nodes.

The cluster autoscaler takes pod priorities into account. The Pod Priority and Preemption feature enables scheduling pods based on priorities if the cluster does not have enough resources, but the cluster autoscaler ensures that the cluster has resources to run all pods. To honor the intention of both features, the cluster autoscaler includes a priority cutoff function. You can use this cutoff to schedule "best-effort" pods, which do not cause the cluster autoscaler to increase resources but instead run only when spare resources are available.

Pods with priority lower than the cutoff value do not cause the cluster to scale up or prevent the cluster from scaling down. No new nodes are added to run the pods, and nodes running these pods might be deleted to free resources.

4.9.4. Cluster autoscaler resource definition

This **ClusterAutoscaler** resource definition shows the parameters and sample values for the cluster autoscaler.



NOTE

When you change the configuration of an existing cluster autoscaler, it restarts.

```
apiVersion: "autoscaling.openshift.io/v1"
kind: "ClusterAutoscaler"
metadata:
  name: "default"
spec:
  podPriorityThreshold: -10
  resourceLimits:
    maxNodesTotal: 24
    cores:
      min: 8
      max: 128
    memory:
      min: 4
      max: 256
    gpus:
      - type: <gpu_type>
        min: 0
        max: 16
  logVerbosity: 4
  scaleDown:
    cordonNodeBeforeTerminating: Enabled
    enabled: true
    delayAfterAdd: 10m
    delayAfterDelete: 5m
    delayAfterFailure: 30s
    unneededTime: 5m
    utilizationThreshold: "0.4"
  scaleUp:
    newPodScaleUpDelay: "10s"
  expanders: ["Random"]
```

Table 4.1. Cluster autoscaler parameters

Parameter	Description
podPriorityThreshold	Specify the priority that a pod must exceed to cause the cluster autoscaler to deploy additional nodes. Enter a 32-bit integer value. The podPriorityThreshold value is compared to the value of the PriorityClass that you assign to each pod.
maxNodesTotal	Specify the maximum number of nodes to deploy. This value is the total number of machines that are deployed in your cluster, not just the ones that the autoscaler controls. Ensure that this value is large enough to account for all of your control plane and compute machines and the total number of replicas that you specify in your MachineAutoscaler resources.
cores.min	Specify the minimum number of cores to deploy in the cluster.
cores.max	Specify the maximum number of cores to deploy in the cluster.
memory.min	Specify the minimum amount of memory, in GiB, in the cluster.
memory.max	Specify the maximum amount of memory, in GiB, in the cluster.
gpus.type	Optional: To configure the cluster autoscaler to deploy GPU-enabled nodes, specify a type value. This value must match the value of the spec.template.spec.metadata.labels[cluster-api/accelerator] label in the machine set that manages the GPU-enabled nodes of that type. For example, this value might be nvidia-t4 to represent Nvidia T4 GPUs, or nvidia-a10g for A10G GPUs. For more information, see "Labeling GPU machine sets for the cluster autoscaler".
gpus.min	Specify the minimum number of GPUs of the specified type to deploy in the cluster.
gpus.max	Specify the maximum number of GPUs of the specified type to deploy in the cluster.
logVerbosity	Specify the logging verbosity level between 0 and 10. The following log level thresholds are provided for guidance: ● 1: (Default) Basic information about changes. ● 4: Debug-level verbosity for troubleshooting typical issues. ● 9: Extensive, protocol-level debugging information. If you do not specify a value, the default value of 1 is used.
scaleDown	In this section, you can specify the period to wait for each action by using any valid ParseDuration interval, including ns , us , ms , s , m , and h .

Parameter	Description
scaleDown.cordonNodeBeforeTerminating	Optional: Specify whether the cluster autoscaler should cordon a node before removing that node by using one of the following values: ● Enabled: The cluster autoscaler cordons the node before draining any pods and removing that node. ● Disabled: The cluster autoscaler does not cordon the node before draining any pods and removing that node. This is the default.
scaleDown.enabled	Specify whether the cluster autoscaler can remove unnecessary nodes.
scaleDown.delayAfterAdd	Optional: Specify the period to wait before deleting a node after a node has recently been <i>added</i> . If you do not specify a value, the default value of 10m is used.
scaleDown.delayAfterDelete	Optional: Specify the period to wait before deleting a node after a node has recently been <i>deleted</i> . If you do not specify a value, the default value of 0s is used.
scaleDown.delayAfterFailure	Optional: Specify the period to wait before deleting a node after a scale down failure occurred. If you do not specify a value, the default value of 3m is used.
scaleDown.unneededTime	Optional: Specify a period of time before an unnecessary node is eligible for deletion. If you do not specify a value, the default value of 10m is used.
scaleDown.utilizationThreshold	Optional: Specify the <i>node utilization level</i> . Nodes below this utilization level are eligible for deletion. The node utilization level is the sum of the requested resources divided by the allocated resources for the node, and must be a value greater than "0" but less than "1" . If you do not specify a value, the cluster autoscaler uses a default value of "0.5" , which corresponds to 50% utilization. You must express this value as a string.
scaleUp	In this section, you can specify the period to wait before recognizing newly pending pods by using any valid ParseDuration interval, including ns , us , ms , s , m , and h .
scaleUp.newPodScaleUpDelay	Optional: Specify the period to ignore a new unschedulable pod before adding a new node. If you do not specify a value, the default value of 0s is used.

Parameter	Description
expanders	Optional: Specify any expanders that you want the cluster autoscaler to use. The following values are valid: ● LeastWaste: Selects the machine set that minimizes the idle CPU after scaling. If multiple machine sets would yield the same amount of idle CPU, the selection minimizes unused memory. ● Priority: Selects the machine set with the highest user-assigned priority. To use this expander, you must create a config map that defines the priority of your machine sets. For more information, see "Configuring a priority expander for the cluster autoscaler." ● Random: (Default) Selects the machine set randomly. If you do not specify a value, the default value of Random is used. You can specify multiple expanders by using the [LeastWaste, Priority] format. The cluster autoscaler applies each expander according to the specified order. In the [LeastWaste, Priority] example, the cluster autoscaler first evaluates according to the LeastWaste criteria. If more than one machine set satisfies the LeastWaste criteria equally well, the cluster autoscaler then evaluates according to the Priority criteria. If more than one machine set satisfies all of the specified expanders equally well, the cluster autoscaler selects one to use at random.



NOTE

When performing a scaling operation, the cluster autoscaler remains within the ranges set in the **ClusterAutoscaler** resource definition, such as the minimum and maximum number of cores to deploy or the amount of memory in the cluster. However, the cluster autoscaler does not correct the current values in your cluster to be within those ranges.

The minimum and maximum CPUs, memory, and GPU values are determined by calculating those resources on all nodes in the cluster, even if the cluster autoscaler does not manage the nodes. For example, the control plane nodes are considered in the total memory in the cluster, even though the cluster autoscaler does not manage the control plane nodes.

4.9.5. Deploying a cluster autoscaler

To deploy a cluster autoscaler, you create an instance of the **ClusterAutoscaler** resource.

Procedure

1. Create a YAML file for a **ClusterAutoscaler** resource that contains the custom resource definition.
2. Create the custom resource in the cluster by running the following command:

```
$ oc create -f <filename>.yaml
```

where:

<filename>

Specifies the name of the YAML file you created.

4.10. APPLYING AUTOSCALING TO YOUR CLUSTER

Applying autoscaling to an OpenShift Container Platform cluster involves deploying a cluster autoscaler and then deploying machine autoscalers for each machine type in your cluster.

For more information, see [Applying autoscaling to an OpenShift Container Platform cluster](#) .

4.11. ENABLING TECHNOLOGY PREVIEW FEATURES USING FEATUREGATES

You can turn on a subset of the current Technology Preview features on for all nodes in the cluster by editing the **FeatureGate** custom resource (CR).

4.11.1. Understanding feature gates

You can use the **FeatureGate** custom resource (CR) to enable specific feature sets so that you can use specific non-default features in your cluster.

A feature set is a collection of OpenShift Container Platform features that are not enabled by default.

You can activate the following feature set by using the **FeatureGate** CR:

- **TechPreviewNoUpgrade**. This feature set is a subset of the current Technology Preview features. This feature set allows you to enable these Technology Preview features on test clusters, where you can fully test them, while leaving the features disabled on production clusters.



WARNING

Enabling the **TechPreviewNoUpgrade** feature set on your cluster cannot be undone and prevents minor version updates. You should not enable this feature set on production clusters.

The following Technology Preview features are enabled by this feature set:

- **AdditionalStorageConfig**
- **AutomatedEtcdBackup**
- **AWSClusterHostedDNS**
- **AWSClusterHostedDNSInstall**
- **AWSDedicatedHosts**

- **AWSDualStackInstall**
- **AWSEuropeanSovereignCloudInstall**
- **AWSServiceLBNetworkSecurityGroup**
- **AzureClusterHostedDNSInstall**
- **AzureDedicatedHosts**
- **AzureDualStackInstall**
- **AzureMultiDisk**
- **AzureWorkloadIdentity**
- **BootcNodeManagement**
- **BootImageSkewEnforcement**
- **BuildCSIVolumes**
- **CBORServingAndStorage**
- **ClientsPreferCBOR**
- **ClusterAPIInstallIBMCloud**
- **ClusterAPIMachineManagement**
- **ClusterAPIMachineManagementAWS**
- **ClusterAPIMachineManagementAzure**
- **ClusterAPIMachineManagementBareMetal**
- **ClusterAPIMachineManagementGCP**
- **ClusterAPIMachineManagementOpenStack**
- **ClusterAPIMachineManagementPowerVS**
- **ClusterAPIMachineManagementVSphere**
- **ClusterMonitoringConfig**
- **ClusterUpdateAcceptRisks**
- **ClusterVersionOperatorConfiguration**
- **ConfigurablePKI**
- **ConsolePluginContentSecurityPolicy**
- **CRDCompatibilityRequirementOperator**

- **CRIOCredentialProviderConfig**
- **DNSNameResolver**
- **DRAPartitionableDevices**
- **DualReplica**
- **DynamicServiceEndpointIBMCloud**
- **EtcBackendQuota**
- **EventTTL**
- **Example**
- **ExternalOIDC**
- **ExternalOIDCWithUIDAndExtraClaimMappings**
- **ExternalOIDCWithUpstreamParity**
- **GatewayAPIWithoutOLM**
- **GCPCustomAPIEndpoints**
- **GCPCustomAPIEndpointsInstall**
- **GCPDualStackInstall**
- **HyperShiftOnlyDynamicResourceAllocation**
- **ImageModeStatusReporting**
- **ImageStreamImportMode**
- **IngressControllerDynamicConfigurationManager**
- **InsightsConfig**
- **InsightsOnDemandDataGather**
- **IrreconcilableMachineConfig**
- **KMSEncryption**
- **KMSv1**
- **MachineAPIMigration**
- **MachineAPIMigrationAWS**
- **MachineAPIMigrationOpenStack**
- **ManagedBootImagesCPMS**

- **MaxUnavailableStatefulSet**
- **MetricsCollectionProfiles**
- **MinimumKubeletVersion**
- **MixedCPUsAllocation**
- **MultiDiskSetup**
- **MutableCSINodeAllocatableCount**
- **MutatingAdmissionPolicy**
- **NewOLM**
- **NewOLMBoxCutterRuntime**
- **NewOLMCatalogdAPIV1Metas**
- **NewOLMConfigAPI**
- **NewOLMOwnSingleNamespace**
- **NewOLMPreflightPermissionChecks**
- **NewOLMWebhookProviderOpenshiftServiceCA**
- **NoOverlayMode**
- **NoRegistryClusterInstall**
- **NutanixMultiSubnets**
- **OnPremDNSRecords**
- **OpenShiftPodSecurityAdmission**
- **OSStreams**
- **OVNObservability**
- **RouteExternalCertificate**
- **SELinuxMount**
- **ServiceAccountTokenNodeBinding**
- **SignatureStores**
- **SigstoreImageVerification**
- **SigstoreImageVerificationPKI**
- **StoragePerformantSecurityPolicy**

- **TLSSAdherence**
- **UpgradeStatus**
- **UserNamespacesPodSecurityStandards**
- **UserNamespacesSupport**
- **VolumeGroupSnapshot**
- **VSphereConfigurableMaxAllowedBlockVolumesPerNode**
- **VSphereHostVMGroupZonal**
- **VSphereMixedNodeEnv**
- **VSphereMultiDisk**
- **VSphereMultiNetworks**

See the *Additional resources* sections for information on some of these features.

4.11.2. Enabling feature sets using the web console

You can use the OpenShift Container Platform web console to enable feature sets for all of the nodes in a cluster by editing the **FeatureGate** custom resource (CR). Completing this task enables non-default features in your cluster.

Procedure

1. In the OpenShift Container Platform web console, switch to the **Administration → Custom Resource Definitions** page.
2. On the **Custom Resource Definitions** page, click **FeatureGate**.
3. On the **Custom Resource Definition Details** page, click the **Instances** tab.
4. Click the **cluster** feature gate, then click the **YAML** tab.
5. Edit the **cluster** instance to add specific feature sets:



WARNING

Enabling the **TechPreviewNoUpgrade** feature set on your cluster cannot be undone and prevents minor version updates. You should not enable this feature set on production clusters.

Sample Feature Gate custom resource

```
apiVersion: config.openshift.io/v1
```

```
kind: FeatureGate
metadata:
  name: cluster 1
# ...
spec:
  featureSet: TechPreviewNoUpgrade 2
```

where:

metadata.name

Specifies the name of the **FeatureGate** CR. You must specify **cluster** for the name.

spec.featureSet

Specifies the feature set that you want to enable:

- **TechPreviewNoUpgrade** enables specific Technology Preview features.

After you save the changes, new machine configs are created, the machine config pools are updated, and scheduling on each node is disabled while the change is being applied.

Verification

You can verify that the feature gates are enabled by looking at the **kubelet.conf** file on a node after the nodes return to the ready state.

1. From the **Administrator** perspective in the web console, navigate to **Compute → Nodes**.
2. Select a node.
3. In the **Node details** page, click **Terminal**.
4. In the terminal window, change your root directory to **/host**:

```
sh-4.2# chroot /host
```

5. View the **kubelet.conf** file:

```
sh-4.2# cat /etc/kubernetes/kubelet.conf
```

Sample output

```
# ...
featureGates:
  InsightsOperatorPullingSCA: true,
  LegacyNodeRoleBehavior: false
# ...
```

The features that are listed as **true** are enabled on your cluster.



NOTE

The features listed vary depending upon the OpenShift Container Platform version.

4.11.3. Enabling feature sets using the CLI

You can use the OpenShift CLI (**oc**) to enable feature sets for all of the nodes in a cluster by editing the **FeatureGate** custom resource (CR). Completing this task enables non-default features in your cluster.

Prerequisites

- You have installed the OpenShift CLI (**oc**).

Procedure

- Edit the **FeatureGate** CR named **cluster**:

```
$ oc edit featuregate cluster
```



WARNING

Enabling the **TechPreviewNoUpgrade** feature set on your cluster cannot be undone and prevents minor version updates. You should not enable this feature set on production clusters.

Sample FeatureGate custom resource

```
apiVersion: config.openshift.io/v1
kind: FeatureGate
metadata:
  name: cluster
# ...
spec:
  featureSet: TechPreviewNoUpgrade
```

where:

metadata.name

Specifies the name of the **FeatureGate** CR. This must be **cluster**.

spec.featureSet

Specifies the feature set that you want to enable:

- **TechPreviewNoUpgrade** enables specific Technology Preview features.

After you save the changes, new machine configs are created, the machine config pools are updated, and scheduling on each node is disabled while the change is being applied.

Verification

You can verify that the feature gates are enabled by looking at the **kubelet.conf** file on a node after the nodes return to the ready state.

1. From the **Administrator** perspective in the web console, navigate to **Compute → Nodes**.
2. Select a node.
3. In the **Node details** page, click **Terminal**.
4. In the terminal window, change your root directory to **/host**:

```
sh-4.2# chroot /host
```

5. View the **kubelet.conf** file:

```
sh-4.2# cat /etc/kubernetes/kubelet.conf
```

Sample output

```
# ...
featureGates:
  InsightsOperatorPullingSCA: true,
  LegacyNodeRoleBehavior: false
# ...
```

The features that are listed as **true** are enabled on your cluster.



NOTE

The features listed vary depending upon the OpenShift Container Platform version.

4.12. ETCD TASKS

Back up etcd, enable or disable etcd encryption, or defragment etcd data.



NOTE

If you deployed a bare-metal cluster, you can scale the cluster up to 5 nodes as part of your post-installation tasks. For more information, see [Node scaling for etcd](#).

4.12.1. etcd encryption

You can encrypt sensitive resource data in etcd to provide an additional layer of protection if an etcd backup or storage data is exposed.

By default, etcd data is not encrypted in OpenShift Container Platform. You can enable etcd encryption for your cluster to provide an additional layer of data security. For example, it can help protect the loss of sensitive data if an etcd backup is exposed to the incorrect parties.

When you enable etcd encryption, the following OpenShift API server and Kubernetes API server resources are encrypted:

- Secrets
- Config maps

- Routes
- OAuth access tokens
- OAuth authorize tokens

When you enable etcd encryption, encryption keys are created. You must have these keys to restore from an etcd backup.



NOTE

etcd encryption only encrypts values, not keys. Resource types, namespaces, and object names are unencrypted.

If etcd encryption is enabled during a backup, the **`static_kuberesources_<datetimestamp>.tar.gz`** file contains the encryption keys for the etcd snapshot. For security reasons, store this file separately from the etcd snapshot. However, this file is required to restore a previous state of etcd from the respective etcd snapshot.

4.12.2. Supported encryption types

OpenShift Container Platform supports AES-CBC and AES-GCM encryption types to protect etcd data at rest.

The following encryption types are supported for encrypting etcd data in OpenShift Container Platform:

AES-CBC

Uses AES-CBC with PKCS#7 padding and a 32-byte key to perform the encryption.

AES-GCM

Uses AES-GCM with a random nonce and a 32-byte key to perform the encryption.

The etcd encryption keys are rotated every 7 days. Up to 10 historical encryption keys are preserved after rotation to help decrypt older backups and provide an extra layer of data recovery safety.

4.12.3. Enabling etcd encryption

Enable etcd encryption to protect sensitive cluster resources such as secrets, config maps, routes, and OAuth tokens at rest.



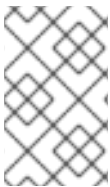
WARNING

Do not back up etcd resources until the initial encryption process is completed. If the encryption process is not completed, the backup might be only partially encrypted.

After you enable etcd encryption, several changes can occur:

- The etcd encryption might affect the memory consumption of a few resources.
- You might notice a transient effect on backup performance because the leader must serve the backup.
- A disk I/O can affect the node that receives the backup state.

You can encrypt the etcd database in either AES-GCM or AES-CBC encryption.



NOTE

To migrate your etcd database from one encryption type to the other, you can modify the API server's **spec.encryption.type** field. Migration of the etcd data to the new encryption type occurs automatically.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Modify the **APIServer** object:

```
$ oc edit apiserver
```

2. Set the **spec.encryption.type** field to **aesgcm** or **aescbc**:

```
spec:
  encryption:
    type: aesgcm
```

- The **aesgcm** value specifies AES-GCM encryption. Alternatively, set the **type** field to **aescbc** for AES-CBC encryption.
3. Save the file to apply the changes.
The encryption process starts. It can take 20 minutes or longer for this process to complete, depending on the size of the etcd database.

Verification

- Review the **Encrypted** status condition for the OpenShift API server to verify that its resources were successfully encrypted:

```
$ oc get openshiftapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}{"\n"}{.message}{"\n"}'
```

The output shows **EncryptionCompleted** upon successful encryption:

```
EncryptionCompleted
All resources encrypted: routes.route.openshift.io
```

If the output shows **EncryptionInProgress**, encryption is still in progress. Wait a few minutes and try again.

- Review the **Encrypted** status condition for the Kubernetes API server to verify that its resources were successfully encrypted:

```
$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}{"\n"}{.message}{"\n"}'
```

The output shows **EncryptionCompleted** upon successful encryption:

```
EncryptionCompleted
All resources encrypted: secrets, configmaps
```

If the output shows **EncryptionInProgress**, encryption is still in progress. Wait a few minutes and try again.

- Review the **Encrypted** status condition for the OpenShift OAuth API server to verify that its resources were successfully encrypted:

```
$ oc get authentication.operator.openshift.io -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}{"\n"}{.message}{"\n"}'
```

The output shows **EncryptionCompleted** upon successful encryption:

```
EncryptionCompleted
All resources encrypted: oauthtokens.oauth.openshift.io,
oauthauthorizetokens.oauth.openshift.io
```

If the output shows **EncryptionInProgress**, encryption is still in progress. Wait a few minutes and try again.

4.12.4. Disabling etcd encryption

Disable etcd encryption when you no longer need to encrypt sensitive cluster resources at rest.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Modify the **APIServer** object:

```
$ oc edit apiserver
```

2. Set the **encryption** field type to **identity**:

```
spec:
  encryption:
    type: identity
```

The **identity** value specifies that no encryption is performed. This is the default value.

3. Save the file to apply the changes.

The decryption process starts. It can take 20 minutes or longer for this process to complete, depending on the size of your cluster.

Verification

- Review the **Encrypted** status condition for the OpenShift API server to verify that its resources were successfully decrypted:

```
$ oc get openshiftapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}\n'{.message}\n'
```

The output shows **DecryptionCompleted** upon successful decryption:

```
DecryptionCompleted
Encryption mode set to identity and everything is decrypted
```

If the output shows **DecryptionInProgress**, decryption is still in progress. Wait a few minutes and try again.

- Review the **Encrypted** status condition for the Kubernetes API server to verify that its resources were successfully decrypted:

```
$ oc get kubeapiserver -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}\n'{.message}\n'
```

The output shows **DecryptionCompleted** upon successful decryption:

```
DecryptionCompleted
Encryption mode set to identity and everything is decrypted
```

If the output shows **DecryptionInProgress**, decryption is still in progress. Wait a few minutes and try again.

- Review the **Encrypted** status condition for the OpenShift OAuth API server to verify that its resources were successfully decrypted:

```
$ oc get authentication.operator.openshift.io -o=jsonpath='{range .items[0].status.conditions[?(@.type=="Encrypted")]}{.reason}\n'{.message}\n'
```

The output shows **DecryptionCompleted** upon successful decryption:

```
DecryptionCompleted
Encryption mode set to identity and everything is decrypted
```

If the output shows **DecryptionInProgress**, decryption is still in progress. Wait a few minutes and try again.

4.12.5. Backing up etcd data

Follow these steps to back up etcd data by creating an etcd snapshot and backing up the resources for the static pods. This backup can be saved and used at a later time if you need to restore etcd.



IMPORTANT

Only save a backup from a single control plane host. Do not take a backup from each control plane host in the cluster.

For a Two-Node with Fencing (TNF) setup, follow the steps to back up etcd data on only one node in the cluster. The cluster restore process is driven by data from a single node, so you can perform the etcd backup steps on only one node.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have checked whether the cluster-wide proxy is enabled.

TIP

You can check whether the proxy is enabled by reviewing the output of **oc get proxy cluster -o yaml**. The proxy is enabled if the **httpProxy**, **httpsProxy**, and **noProxy** fields have values set.

Procedure

1. Start a debug session as root for a control plane node:

```
$ oc debug --as-root node/<node_name>
```

2. Change your root directory to **/host** in the debug shell:

```
sh-4.4# chroot /host
```

3. If the cluster-wide proxy is enabled, export the **NO_PROXY**, **HTTP_PROXY**, and **HTTPS_PROXY** environment variables by running the following commands:

```
$ export HTTP_PROXY=http://<your_proxy.example.com>:8080
```

```
$ export HTTPS_PROXY=https://<your_proxy.example.com>:8080
```

```
$ export NO_PROXY=<example.com>
```

4. Run the **cluster-backup.sh** script in the debug shell and pass in the location to save the backup to.

TIP

The **cluster-backup.sh** script is maintained as a component of the etcd Cluster Operator and is a wrapper around the **etcdctl snapshot save** command.

```
sh-4.4# /usr/local/bin/cluster-backup.sh /home/core/assets/backup
```

Example script output

```
found latest kube-apiserver: /etc/kubernetes/static-pod-resources/kube-apiserver-pod-6
found latest kube-controller-manager: /etc/kubernetes/static-pod-resources/kube-controller-
manager-pod-7
found latest kube-scheduler: /etc/kubernetes/static-pod-resources/kube-scheduler-pod-6
found latest etcd: /etc/kubernetes/static-pod-resources/etcd-pod-3
ede95fe6b88b87ba86a03c15e669fb4aa5bf0991c180d3c6895ce72eaade54a1
etcdctl version: 3.4.14
API version: 3.4
{"level":"info","ts":1624647639.0188997,"caller":"snapshot/v3_snapshot.go:119","msg":"created
temporary db file","path":"/home/core/assets/backup/snapshot_2021-06-25_190035.db.part"}
{"level":"info","ts":"2021-06-
25T19:00:39.030Z","caller":"clientv3/maintenance.go:200","msg":"opened snapshot stream;
downloading"}
{"level":"info","ts":1624647639.0301006,"caller":"snapshot/v3_snapshot.go:127","msg":"fetching
snapshot","endpoint":"https://10.0.0.5:2379"}
{"level":"info","ts":"2021-06-
25T19:00:40.215Z","caller":"clientv3/maintenance.go:208","msg":"completed snapshot read;
closing"}
{"level":"info","ts":1624647640.6032252,"caller":"snapshot/v3_snapshot.go:142","msg":"fetched
snapshot","endpoint":"https://10.0.0.5:2379","size":"114 MB","took":1.584090459}
{"level":"info","ts":1624647640.6047094,"caller":"snapshot/v3_snapshot.go:152","msg":"saved",
"path":"/home/core/assets/backup/snapshot_2021-06-25_190035.db"}
Snapshot saved at /home/core/assets/backup/snapshot_2021-06-25_190035.db
{"hash":3866667823,"revision":31407,"totalKey":12828,"totalSize":114446336}
snapshot db and kube resources are successfully saved to /home/core/assets/backup
```

In this example, two files are created in the **/home/core/assets/backup/** directory on the control plane host:

- **snapshot_<datetimestamp>.db**: This file is the etcd snapshot. The **cluster-backup.sh** script confirms its validity.
- **static_kuberresources_<datetimestamp>.tar.gz**: This file contains the resources for the static pods. If etcd encryption is enabled, it also contains the encryption keys for the etcd snapshot.

**NOTE**

If etcd encryption is enabled, store this second file separately from the etcd snapshot for security reasons. However, this file is required to restore from the etcd snapshot.

The etcd encryption only encrypts values, not keys. This means that resource types, namespaces, and object names are not encrypted.

4.12.6. Data defragmentation for etcd

To prevent etcd performance degradation and cluster-wide maintenance alarms on large clusters, monitor etcd database metrics and defragment the data store when the key space grows too large.

For large and dense clusters, etcd can suffer from poor performance if the key space grows too large and exceeds the space quota. Periodically maintain and defragment etcd to free up space in the data store. Monitor Prometheus for etcd metrics and defragment it when required. Otherwise, etcd can raise a cluster-wide alarm that puts the cluster into a maintenance mode, which accepts only key reads and deletes.

Monitor these key metrics:

- **etcd_server_quota_backend_bytes**, which is the current quota limit
- **etcd_mvcc_db_total_size_in_use_in_bytes**, which indicates the actual database usage after a history compaction
- **etcd_mvcc_db_total_size_in_bytes**, which shows the database size, including free space waiting for defragmentation

Defragment etcd data to reclaim disk space after events that cause disk fragmentation, such as etcd history compaction.

History compaction is performed automatically every five minutes and leaves gaps in the back-end database. This fragmented space is available for use by etcd, but is not available to the host file system. You must defragment etcd to make this space available to the host file system.

Defragmentation occurs automatically, but you can also trigger it manually.

4.12.7. Automatic defragmentation

When etcd database growth affects performance, the etcd Operator can automatically defragment member disks based on cluster metrics.

**NOTE**

Automatic defragmentation works well in most cases because the etcd Operator uses cluster metrics to choose the most efficient defragmentation approach.

The etcd Operator automatically defragments disks. No manual intervention is needed.

Verify that defragmentation succeeded by checking one of these logs:

- etcd logs

- cluster-etcd-operator pod
- operator status error log



WARNING

Automatic defragmentation can cause leader election failure in various OpenShift Container Platform core components, such as the Kubernetes controller manager, which triggers a restart of the failing component. The restart is harmless and either triggers failover to the next running instance or the component resumes work again after the restart.

The following is example log output for successful defragmentation:

```
etcd member has been defragmented: <member_name>, memberID: <member_id>
```

The following is example log output for unsuccessful defragmentation:

```
failed defrag on member: <member_name>, memberID: <member_id>: <error_message>
```

4.12.8. Manually defragmenting etcd data

When automatic etcd defragmentation cannot reclaim enough space, manually defragment etcd on each member to restore disk availability and normal cluster operation.

A Prometheus alert indicates when you need to use manual defragmentation. The alert is displayed in two cases:

- When etcd uses more than 50% of its available space for more than 10 minutes
- When etcd is actively using less than 50% of its total database size for more than 10 minutes

You can also determine whether defragmentation is needed by checking the etcd database size in MB that will be freed by defragmentation with the PromQL expression:

`(etcd_mvcc_db_total_size_in_bytes - etcd_mvcc_db_total_size_in_use_in_bytes)/1024/1024`



WARNING

Defragmenting etcd is a blocking action. The etcd member does not respond until defragmentation is complete. For this reason, wait at least one minute between defragmentation actions on each of the pods to allow the cluster to recover.

Follow this procedure to defragment etcd data on each etcd member.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

- Determine which etcd member is the leader, because the leader should be defragmented last.
 - Get the list of etcd pods:

```
$ oc -n openshift-etcd get pods -l k8s-app=etcd -o wide
```

The following is example output:

```
etcd-ip-10-0-159-225.example.redhat.com      3/3   Running   0       175m
10.0.159.225 ip-10-0-159-225.example.redhat.com <none>   <none>
etcd-ip-10-0-191-37.example.redhat.com      3/3   Running   0       173m
10.0.191.37 ip-10-0-191-37.example.redhat.com <none>   <none>
etcd-ip-10-0-199-170.example.redhat.com     3/3   Running   0       176m
10.0.199.170 ip-10-0-199-170.example.redhat.com <none>   <none>
```

- Choose a pod and run the following command to determine which etcd member is the leader:

```
$ oc rsh -n openshift-etcd etcd-ip-10-0-159-225.example.redhat.com etcdctl endpoint
status --cluster -w table
```

The following is example output:

```
Defaulting container name to etcdctl.
Use 'oc describe pod/etcd-ip-10-0-159-225.example.redhat.com -n openshift-etcd' to see
all of the containers in this pod.
+-----+-----+-----+-----+-----+-----+
+-----+-----+
|   ENDPOINT   |   ID   | VERSION | DB SIZE | IS LEADER | IS LEARNER |
| RAFT TERM | RAFT INDEX | RAFT APPLIED INDEX | ERRORS |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| https://10.0.191.37:2379 | 251cd44483d811c3 | 3.5.9 | 104 MB | false | false |
7 | 91624 | 91624 |
| https://10.0.159.225:2379 | 264c7c58ecbdabee | 3.5.9 | 104 MB | false | false |
7 | 91624 | 91624 |
| https://10.0.199.170:2379 | 9ac311f93915cc79 | 3.5.9 | 104 MB | true | false |
7 | 91624 | 91624 |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
```

Based on the **IS LEADER** column of this output, the **https://10.0.199.170:2379** endpoint is the leader. Matching this endpoint with the output of the previous step, the pod name of the leader is **etcd-ip-10-0-199-170.example.redhat.com**.

- Defragment an etcd member.
 - Connect to the running etcd container, passing in the name of a pod that is *not* the leader:

```
$ oc rsh -n openshift-etcd etcd-ip-10-0-159-225.example.redhat.com
```

- b. Unset the **ETCDCTL_ENDPOINTS** environment variable:

```
sh-4.4# unset ETCDCTL_ENDPOINTS
```

- c. Defragment the etcd member:

```
sh-4.4# etcdctl --command-timeout=30s --endpoints=https://localhost:2379 defrag
```

The following is example output:

```
Finished defragmenting etcd member[https://localhost:2379]
```

If a timeout error occurs, increase the value for **--command-timeout** until the command succeeds.

- d. Verify that the database size was reduced:

```
sh-4.4# etcdctl endpoint status -w table --cluster
```

The following is example output:

```
+-----+-----+-----+-----+-----+-----+
+-----+-----+
| ENDPOINT | ID | VERSION | DB SIZE | IS LEADER | IS LEARNER |
| RAFT TERM | RAFT INDEX | RAFT APPLIED INDEX | ERRORS |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
| https://10.0.191.37:2379 | 251cd44483d811c3 | 3.5.9 | 104 MB | false | false |
7 | 91624 | 91624 | |
| https://10.0.159.225:2379 | 264c7c58ecbdabee | 3.5.9 | 41 MB | false | false |
7 | 91624 | 91624 | 1
| https://10.0.199.170:2379 | 9ac311f93915cc79 | 3.5.9 | 104 MB | true | false |
7 | 91624 | 91624 | |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+
```

This example shows that the database size for this etcd member is now 41 MB as opposed to the starting size of 104 MB.

- e. Repeat these steps to connect to each of the other etcd members and defragment them. Always defragment the leader last.
Wait at least one minute between defragmentation actions to allow the etcd pod to recover. Until the etcd pod recovers, the etcd member does not respond.

3. If any **NOSPACE** alarms were triggered due to the space quota being exceeded, clear them.

- a. Check if there are any **NOSPACE** alarms:

```
sh-4.4# etcdctl alarm list
```

The following is example output:

```
memberID:12345678912345678912 alarm:NOSPACE
```

- b. Clear the alarms:

```
sh-4.4# etcdctl alarm disarm
```

4.12.9. Restoring to an earlier cluster state for more than one node

You can use a saved etcd backup to restore an earlier cluster state or restore a cluster that has lost the majority of control plane hosts.

For a Two-Node with Fencing (TNF) setup, a single surviving node can continue to operate in degraded mode. Use a saved etcd backup to restore an earlier cluster state if only one node is operational, or when both nodes have failed and you need to restart the cluster from a known safe state. In both cases, perform the restore procedure on a single node. The peer node automatically synchronizes its data with the restored node when it rejoins the cluster.

For a 3-node HA cluster, shut down etcd on the following number of hosts:

- For 3-node clusters: Shut down etcd on 2 hosts.
- For 4-node and 5-node clusters: Shut down etcd on 3 hosts.

Quorum requires a simple majority of nodes. The minimum number of nodes required for a quorum is as follows:

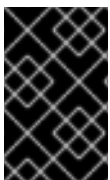
- For 3-node HA cluster: 2.
- For 4-node and 5-node HA clusters: 3.

If you start a new cluster from backup on your recovery host, the other etcd members might still form a quorum and continue service.



NOTE

If your cluster uses a control plane machine set, see "Recovering a degraded etcd Operator" in "Troubleshooting the control plane machine set" for an etcd recovery procedure. For OpenShift Container Platform on a single node, see "Restoring to an earlier cluster state for a single node".



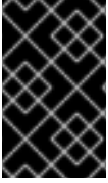
IMPORTANT

When you restore your cluster, you must use an etcd backup that was taken from the same z-stream release. For example, an OpenShift Container Platform 4.22.2 cluster must use an etcd backup that was taken from 4.22.2.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role through a certificate-based **kubeconfig** file, like the one that was used during installation.
- A healthy control plane host to use as the recovery host.
- You have SSH access to control plane hosts.

- A backup directory containing both the **etcd** snapshot and the resources for the static pods, which were from the same backup. The file names in the directory must be in the following formats: **snapshot_<timestamp>.db** and **static_kubernetes_<timestamp>.tar.gz**.
- Nodes must be accessible or bootable.

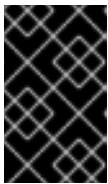


IMPORTANT

For non-recovery control plane nodes, it is not required to establish SSH connectivity or to stop the static pods. You can delete and re-create other non-recovery, control plane machines, one by one.

Procedure

1. Select a control plane host to use as the recovery host. This is the host that you run the restore operation on.
2. Establish SSH connectivity to each of the control plane nodes, including the recovery host. **kube-apiserver** becomes inaccessible after the restore process starts, so you cannot access the control plane nodes. For this reason, it is recommended to establish SSH connectivity to each control plane host in a separate terminal.



IMPORTANT

If you do not complete this step, you cannot access the control plane hosts to complete the restore procedure, and you cannot recover your cluster from this state.

3. Using SSH, connect to each control plane node and run the following command to disable etcd:

```
$ sudo -E /usr/local/bin/disable-etcd.sh
```

4. Copy the etcd backup directory to the recovery control plane host.
This procedure assumes that you copied the **backup** directory containing the etcd snapshot and the resources for the static pods to the **/home/core/** directory of your recovery control plane host.
5. Use SSH to connect to the recovery host. To restore the cluster from an earlier backup, run the following command:

```
$ sudo -E /usr/local/bin/cluster-restore.sh /home/core/<etcd-backup-directory>
```

6. Exit the SSH session.
7. When the API responds, to turn off the etcd Operator quorum guard, run the following command:



IMPORTANT

For a TNF setup, do not:

- Change the etcd Operator quorum setting.
- Turn the etcd Operator quorum off.
- Turn the etcd Operator quorum on back.

```
$ oc patch etcd/cluster --type=merge -p '{"spec": {"unsupportedConfigOverrides": {"useUnsupportedUnsafeNonHANonProductionUnstableEtcd": true}}}'
```

8. Monitor the recovery progress of the control plane by running the following command:

```
$ oc adm wait-for-stable-cluster
```



NOTE

It can take up to 15 minutes for the control plane to recover. Wait for the control plane to recover before using the next step.

9. Enable the quorum guard by running the following command:

```
$ oc patch etcd/cluster --type=merge -p '{"spec": {"unsupportedConfigOverrides": null}}'
```

Troubleshooting

If the etcd static pods do not roll out, you can manually force an etcd redeployment from the **cluster-etcd-operator** by running the following command:

```
$ oc patch etcd cluster -p='{"spec": {"forceRedeploymentReason": "recovery-"$(date --rfc-3339=ns)""}}' --type=merge
```

Additional resources

- [Recommended etcd practices](#)
- [Installing a user-provisioned cluster on bare metal](#)
- [Replacing a bare-metal control plane node](#)

4.12.10. Issues and workarounds for restoring a persistent storage state

After restoring a cluster from an etcd snapshot, persistent storage resources might no longer match the current storage provider state. Identify and resolve outdated volume, credential, attachment, or device references to restore workloads safely.

If your OpenShift Container Platform cluster uses persistent storage of any form, a state of the cluster is typically stored outside etcd. When you restore from an etcd backup, the status of the workloads in OpenShift Container Platform is also restored. However, if the etcd snapshot is old, the status might be invalid or outdated.



IMPORTANT

The contents of persistent volumes (PVs) are never part of the etcd snapshot. When you restore an OpenShift Container Platform cluster from an etcd snapshot, non-critical workloads might gain access to critical data, or vice-versa.

The following are some example scenarios that produce an out-of-date status:

- MySQL database is running in a pod backed up by a PV object. Restoring OpenShift Container Platform from an etcd snapshot does not bring back the volume on the storage provider, and does not produce a running MySQL pod, despite the pod repeatedly attempting to start. You must manually restore this pod by restoring the volume on the storage provider, and then editing the PV to point to the new volume.
- Pod P1 is using volume A, which is attached to node X. If the etcd snapshot is taken while another pod uses the same volume on node Y, then when the etcd restore is performed, pod P1 might not be able to start correctly due to the volume still being attached to node Y. OpenShift Container Platform is not aware of the attachment, and does not automatically detach it. When this occurs, the volume must be manually detached from node Y so that the volume can attach on node X, and then pod P1 can start.
- Cloud provider or storage provider credentials were updated after the etcd snapshot was taken. This causes any CSI drivers or Operators that depend on the those credentials to not work. You might have to manually update the credentials required by those drivers or Operators.
- A device is removed or renamed from OpenShift Container Platform nodes after the etcd snapshot is taken. The Local Storage Operator creates symlinks for each PV that it manages from `/dev/disk/by-id` or `/dev` directories. This situation might cause the local PVs to refer to devices that no longer exist.

To fix this problem, an administrator must:

1. Manually remove the PVs with invalid devices.
2. Remove symlinks from respective nodes.
3. Delete **LocalVolume** or **LocalVolumeSet** objects (see *Storage → Configuring persistent storage → Persistent storage using local volumes → Deleting the Local Storage Operator Resources*).

4.13. POD DISRUPTION BUDGETS

Understand and configure pod disruption budgets.

4.13.1. Understanding how to use pod disruption budgets to specify the number of pods that must be up

To ensure pod availability during voluntary disruptions such as node maintenance or cluster updates, you can use pod disruption budgets to define safety constraints for your applications.

A **PodDisruptionBudget** is an API object that specifies the minimum number or percentage of replicas that must be up at a time. Setting these in projects can be helpful during node maintenance (such as scaling a cluster down or a cluster upgrade) and is only honored on voluntary evictions (not on node failures).

A **PodDisruptionBudget** object's configuration consists of the following key parts:

- A label selector, which is a label query over a set of pods.
- An availability level, which specifies the minimum number of pods that must be available simultaneously, either:
 - **minAvailable** is the number of pods must always be available, even during a disruption.
 - **maxUnavailable** is the number of pods can be unavailable during a disruption.



NOTE

Available refers to the number of pods that has condition **Ready=True**. **Ready=True** refers to the pod that is able to serve requests and should be added to the load balancing pools of all matching services.

A **maxUnavailable** of **0%** or **0** or a **minAvailable** of **100%** or equal to the number of replicas is permitted but can block nodes from being drained.



WARNING

The default setting for **maxUnavailable** is **1** for all the machine config pools in OpenShift Container Platform. It is recommended to not change this value and update one control plane node at a time. Do not change this value to **3** for the control plane pool.

You can check for pod disruption budgets across all projects with the following:

```
$ oc get poddisruptionbudget --all-namespaces
```



NOTE

The following example contains some values that are specific to OpenShift Container Platform on AWS.

Example output

NAMESPACE	NAME	MIN AVAILABLE	MAX UNAVAILABLE
ALLOWED DISRUPTIONS	AGE		
openshift-apiserver	openshift-apiserver-pdb	N/A	1
121m			1
openshift-cloud-controller-manager	aws-cloud-controller-manager	1	N/A
125m			1
openshift-cloud-credential-operator	pod-identity-webhook	1	N/A
117m			1
openshift-cluster-csi-drivers	aws-ebs-csi-driver-controller-pdb	N/A	1
121m			1
openshift-cluster-storage-operator	csi-snapshot-controller-pdb	N/A	1
122m			1
openshift-cluster-storage-operator	csi-snapshot-webhook-pdb	N/A	1

122m				
openshift-console	console	N/A	1	1
116m				
#...				

The **PodDisruptionBudget** is considered healthy when there are at least **minAvailable** pods running in the system. Every pod above that limit can be evicted.



NOTE

Depending on your pod priority and preemption settings, lower-priority pods might be removed despite their pod disruption budget requirements.

4.13.2. Specifying the number of pods that must be up with pod disruption budgets

You can use a **PodDisruptionBudget** object to specify the minimum number or percentage of replicas that must be up at a time. This ensures pod availability during voluntary disruptions such as node maintenance or cluster updates.

The following procedure shows how to configure a pod disruption budget.

Procedure

1. Create a YAML file with the an object definition similar to the following:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      name: my-pod
```

where:

apiVersion

Specifies the **policy/v1** API group.

spec.minAvailable

Specifies the minimum number of pods that must be available simultaneously. This can be either an integer or a string specifying a percentage, for example, **20%**.

spec.selector

Specifies a label query over a set of resources. The result of **matchLabels** and **matchExpressions** are logically conjoined. Leave this parameter blank, for example **selector {}**, to select all pods in the project.

Or:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
```

```

name: my-pdb
spec:
  maxUnavailable: 25%
  selector:
    matchLabels:
      name: my-pod

```

where:

apiVersion

Specifies the **policy/v1** API group.

spec.maxUnavailable

Specifies the maximum number of pods that can be unavailable simultaneously. This can be either an integer or a string specifying a percentage, for example, **20%**.

spec.selector

Specifies a label query over a set of resources. The result of **matchLabels** and **matchExpressions** are logically conjoined. Leave this parameter blank, for example **selector {}**, to select all pods in the project.

2. Run the following command to add the object to project:

```
$ oc create -f </path/to/file> -n <project_name>
```

4.13.3. Specifying the eviction policy for unhealthy pods

When you use pod disruption budgets (PDBs) to specify how many pods must be available simultaneously, you can also define the criteria for how unhealthy pods are considered for eviction. The eviction policy determines which pods the cluster can evict.

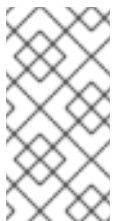
You can choose one of the following policies:

IfHealthyBudget

Running pods that are not yet healthy can be evicted only if the guarded application is not disrupted.

AlwaysAllow

Running pods that are not yet healthy can be evicted regardless of whether the criteria in the pod disruption budget is met. This policy can help evict malfunctioning applications, such as ones with pods stuck in the **CrashLoopBackOff** state or failing to report the **Ready** status.



NOTE

It is recommended to set the **unhealthyPodEvictionPolicy** field to **AlwaysAllow** in the **PodDisruptionBudget** object to support the eviction of misbehaving applications during a node drain. The default behavior is to wait for the application pods to become healthy before the drain can proceed.

Procedure

1. Create a YAML file that defines a **PodDisruptionBudget** object and specify the unhealthy pod eviction policy:

Example pod-disruption-budget.yaml file

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      name: my-pod
  unhealthyPodEvictionPolicy: AlwaysAllow
```

where:

spec.unhealthyPodEvictionPolicy

Specifies the unhealthy pod eviction policy, either **IfHealthyBudget** or **AlwaysAllow**. The default is **IfHealthyBudget** when the **unhealthyPodEvictionPolicy** field is empty.

2. Create the **PodDisruptionBudget** object by running the following command:

```
$ oc create -f pod-disruption-budget.yaml
```

With a PDB that has the **AlwaysAllow** unhealthy pod eviction policy set, you can now drain nodes and evict the pods for a malfunctioning application guarded by this PDB.

Additional resources

- [Enabling features using feature gates](#)
- [Unhealthy Pod Eviction Policy in the Kubernetes documentation](#)

CHAPTER 5. POSTINSTALLATION NODE TASKS

You can perform postinstallation node tasks to add and manage compute machines, configure node resources and hardware, improve availability, and control workload scheduling.

After installing OpenShift Container Platform, you can further expand and customize your cluster to your requirements through certain node tasks.

5.1. ADDING RHCOS COMPUTE MACHINES TO AN OPENSIFT CONTAINER PLATFORM CLUSTER

You can add more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines to your OpenShift Container Platform cluster on bare metal.

Before you add more compute machines to a cluster that you installed on bare metal infrastructure, you must create RHCOS machines for it to use. You can either use an ISO image or network PXE booting to create the machines.

5.1.1. Prerequisites

- You installed a cluster on bare metal.
- You have installation media and Red Hat Enterprise Linux CoreOS (RHCOS) images that you used to create your cluster. If you do not have these files, you must obtain them by following the instructions in the installation procedure.

5.1.2. Creating RHCOS machines by using an ISO image

To scale your OpenShift Container Platform bare metal cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using an ISO image.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You must have the OpenShift CLI (**oc**) installed.

Procedure

1. Extract the Ignition config file from the cluster by running the following command:

```
$ oc extract -n openshift-machine-api secret/worker-user-data-managed --keys=userData --to=- > worker.ign
```

2. Upload the **worker.ign** Ignition config file you exported from your cluster to your HTTP server. Note the URLs of these files.
3. You can validate that the ignition files are available on the URLs. The following example gets the Ignition config files for the compute node:

```
$ curl -k http://<HTTP_server>/worker.ign
```


- You can access the ISO image for booting your new machine by running the following command:

```
RHCOS_VHD_ORIGIN_URL=$(oc -n openshift-machine-config-operator get
configmap/coreos-bootimages -o jsonpath='{.data.stream}' | jq -r '.architectures.
<architecture>.artifacts.metal.formats.iso.disk.location')
```

- Use the ISO file to install RHCOS on more compute machines. Use the same method that you used when you created machines before you installed the cluster:
 - Burn the ISO image to a disk and boot it directly.
 - Use ISO redirection with a LOM interface.
- Boot the RHCOS ISO image without specifying any options, or interrupting the live boot sequence. Wait for the installer to boot into a shell prompt in the RHCOS live environment.



NOTE

You can interrupt the RHCOS installation boot process to add kernel arguments. However, for this ISO procedure you must use the **coreos-installer** command as outlined in the following steps, instead of adding kernel arguments.

- Run the **coreos-installer** command by using **sudo**. The **core** user does not have the root privileges required to perform the installation. Specify the options that meet your installation requirements. At a minimum, you must specify the URL that points to the Ignition config file for the node type, and the device that you are installing to.

```
$ sudo coreos-installer install --ignition-url=http://<HTTP_server>/<node_type>.ign <device>
--ignition-hash=sha512-<digest>
```

where:

<digest>

Specifies the Ignition config file SHA512 digest obtained through an HTTP URL to validate the authenticity of the Ignition config file on the cluster node.



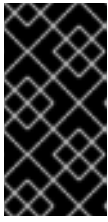
NOTE

If you want to provide your Ignition config files through an HTTPS server that uses TLS, you can add the internal certificate authority (CA) to the system trust store before running **coreos-installer**.

The following example initializes a compute node installation to the **/dev/sda** device. The Ignition config file for the compute node is obtained from an HTTP web server with the IP address 192.168.1.2:

```
$ sudo coreos-installer install --ignition-
url=http://192.168.1.2:80/installation_directory/worker.ign /dev/sda --ignition-
hash=sha512-
a5a2d43879223273c9b60af66b44202a1d1248fc01cf156c46d4a79f552b6bad47bc8cc78ddf
0116e80c59d2ea9e32ba53bc807afbca581aa059311def2c3e3b
```

8. Monitor the progress of the RHCOS installation on the console of the machine.



IMPORTANT

Ensure that the installation is successful on each node before commencing with the OpenShift Container Platform installation. Observing the installation process can also help to determine the cause of RHCOS installation issues that might arise.

9. Continue to create more compute machines for your cluster.

Additional resources

- [Installing a cluster on bare metal](#)

5.1.3. Creating RHCOS machines by PXE or iPXE booting

To scale your OpenShift Container Platform bare metal cluster, you can create more Red Hat Enterprise Linux CoreOS (RHCOS) compute machines by using PXE or iPXE booting.

Prerequisites

- You have obtained the URL of the Ignition config file for the compute machines for your cluster. You uploaded this file to your HTTP server during installation.
- You have obtained the URLs of the RHCOS ISO image, compressed metal BIOS, **kernel**, and **initramfs** files that you uploaded to your HTTP server during cluster installation.
- You have access to the PXE booting infrastructure that you used to create the machines for your OpenShift Container Platform cluster during installation. The machines must boot from their local disks after RHCOS is installed on them.
- If you use UEFI, you have access to the **grub.conf** file that you modified during OpenShift Container Platform installation.

Procedure

1. Confirm that your PXE or iPXE installation for the RHCOS images is correct.

- For PXE:

```

DEFAULT pxeboot
TIMEOUT 20
PROMPT 0
LABEL pxeboot
  KERNEL http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture>
  APPEND initrd=http://<HTTP_server>/rhcos-<version>-live-initramfs.
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img
  
```

where:

KERNEL

Specifies the location of the live **kernel** file that you uploaded to your HTTP server.

APPEND

Specifies the locations of the RHCOS files that you uploaded to your HTTP server:

initrd

Specifies the location of the live **initramfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file. This parameter supports only HTTP and HTTPS.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file. This parameter supports only HTTP and HTTPS.

**NOTE**

This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **APPEND** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?".

- For iPXE (**x86_64** + **aarch64**):

```
kernel http://<HTTP_server>/rhcos-<version>-live-kernel-<architecture> initrd=main
coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
<architecture>.img coreos.inst.install_dev=/dev/sda
coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
initrd --name main http://<HTTP_server>/rhcos-<version>-live-initramfs.
<architecture>.img
boot
```

where:

kernel

Specifies the location of the **kernel** file that you uploaded to your HTTP server.

initrd=main

Specifies an argument that is required for booting on UEFI systems.

coreos.live.rootfs_url

Specifies the location of the **rootfs** file that you uploaded to your HTTP server.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file that you uploaded to your HTTP server.

initrd --name main

Specifies the location of the **initramfs** file that you uploaded to your HTTP server.



NOTE

- If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.
- This configuration does not enable serial console access on machines with a graphical console. To configure a different console, add one or more **console=** arguments to the **kernel** line. For example, add **console=tty0 console=ttyS0** to set the first PC serial port as the primary console and the graphical console as a secondary console. For more information on setting up a serial terminal and/or console in RHCOS, see "How does one set up a serial terminal and/or console in Red Hat Enterprise Linux?" in the Additional resources section and "Enabling the serial console for PXE and ISO installation" in the "Advanced RHCOS installation configuration" section.



NOTE

To network boot the CoreOS **kernel** on **aarch64** architecture, you need to use a version of iPXE build with the **IMAGE_GZIP** option enabled. For more information, see "IMAGE_GZIP option in iPXE".

- For PXE (with UEFI and GRUB as second stage) on **aarch64**:

```
menuentry 'Install CoreOS' {
    linux rhcos-<version>-live-kernel-<architecture>
    coreos.live.rootfs_url=http://<HTTP_server>/rhcos-<version>-live-rootfs.
    <architecture>.img coreos.inst.install_dev=/dev/sda
    coreos.inst.ignition_url=http://<HTTP_server>/worker.ign
    initrd rhcos-<version>-live-initramfs.<architecture>.img
}
```

where:

linux

Specifies the location of the live **kernel** file on your TFTP server.

coreos.live.rootfs_url

Specifies the location of the live **rootfs** file.

coreos.inst.ignition_url

Specifies the location of the worker Ignition config file.

initrd

Specifies the location of the live **initramfs** file on your TFTP server.



NOTE

If you use multiple NICs, specify a single interface in the **ip** option. For example, to use DHCP on a NIC named **eno1**, set **ip=eno1:dhcp**.

2. Use the PXE or iPXE infrastructure to create the required compute machines for your cluster.

Additional resources

- [How does one set up a serial terminal and/or console in Red Hat Enterprise Linux? \(Red Hat Knowledgebase article\)](#)
- [IMAGE_GZIP option in iPXE \(iPXE documentation\)](#)

5.1.4. Approving the certificate signing requests for your machines

To allow newly added machines to join your OpenShift Container Platform cluster, you can confirm that pending certificate signing requests (CSRs) are approved or approve them yourself. Approve client requests first, then server requests.

Prerequisites

- You added machines to your cluster.

Procedure

1. Confirm that the cluster recognizes the machines:

```
$ oc get nodes
```

Example output

```
NAME      STATUS    ROLES    AGE   VERSION
master-0  Ready    master   63m   v1.35.4
master-1  Ready    master   63m   v1.35.4
master-2  Ready    master   64m   v1.35.4
```

The output lists all of the machines that you created.



NOTE

The preceding output might not include the compute nodes until some CSRs are approved.

2. Review the pending CSRs and ensure that you see the client requests with the **Pending** or **Approved** status for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

```
NAME      AGE   REQUESTOR                                CONDITION
csr-8b2br  15m   system:serviceaccount:openshift-machine-config-operator:node-
bootstrapper  Pending
csr-8vnps  15m   system:serviceaccount:openshift-machine-config-operator:node-
bootstrapper  Pending
...
```

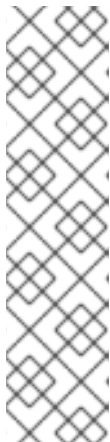
In this example, two machines are joining the cluster. You might see more approved CSRs in the list.

3. If the CSRs were not approved, after all of the pending CSRs for the machines you added are in **Pending** status, approve the CSRs for your cluster machines:



NOTE

You must approve your CSRs within an hour of adding the machines to the cluster. If you do not approve them within an hour, the certificates rotate, and more than two certificates are present for each node. You must approve all of these certificates. After the client CSR is approved, the kubelet creates a secondary CSR for the serving certificate, which requires manual approval. The subsequent serving certificate renewal requests are then automatically approved by the **machine-approver** if the Kubelet requests a new certificate with identical parameters.



NOTE

For clusters running on platforms that are not machine API enabled, such as bare metal and other user-provisioned infrastructure, you must implement a method of automatically approving the kubelet serving certificate requests (CSRs). If a request is not approved, then the **oc exec**, **oc rsh**, and **oc logs** commands cannot succeed, because a serving certificate is required when the API server connects to the kubelet. Any operation that contacts the Kubelet endpoint requires this certificate approval to be in place. The method must watch for new CSRs, confirm that the CSR was submitted by the **node-bootstrapper** service account in the **system:node** or **system:admin** groups, and confirm the identity of the node.

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}{{end}}{{end}}' | xargs --no-run-if-empty oc adm certificate approve
```



NOTE

Some Operators might not become available until some CSRs are approved. Each node submits two CSRs, so you might need to run the command to approve CSRs multiple times.

4. Now that your client requests are approved, you must review the server requests for each machine that you added to the cluster:

```
$ oc get csr
```

Example output

```
NAME      AGE   REQUESTOR                                CONDITION
csr-bfd72  5m26s system:node:ip-10-0-50-126.us-east-2.compute.internal
Pending
csr-c57lv  5m26s system:node:ip-10-0-95-157.us-east-2.compute.internal
Pending
...
```

- If the remaining CSRs are not approved, and are in the **Pending** status, approve the CSRs for your cluster machines:

- To approve them individually, run the following command for each valid CSR:

```
$ oc adm certificate approve <csr_name>
```

where:

<csr_name>

Specifies the name of a CSR from the list of current CSRs.

- To approve all pending CSRs, run the following command:

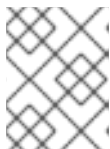
```
$ oc get csr -o go-template='{{range .items}}{{if not .status}}{{.metadata.name}}{{"\n"}}
{{end}}{{end}}' | xargs oc adm certificate approve
```

- After all client and server CSRs have been approved, the machines have the **Ready** status. Verify this by running the following command:

```
$ oc get nodes
```

Example output

```
NAME      STATUS  ROLES  AGE  VERSION
master-0  Ready   master  73m  v1.35.4
master-1  Ready   master  73m  v1.35.4
master-2  Ready   master  74m  v1.35.4
worker-0  Ready   worker  11m  v1.35.4
worker-1  Ready   worker  11m  v1.35.4
```



NOTE

You might need to wait a few minutes after approval of the server CSRs for the machines to transition to the **Ready** status.

5.1.5. Adding a new RHCOS worker node with a custom /var partition in AWS

You can add an RHCOS worker node with a custom **/var** partition in AWS by creating a user data secret and a compute machine set that use the required device naming schema.

OpenShift Container Platform supports partitioning devices during installation by using machine configs that are processed during the bootstrap. However, if you use **/var** partitioning, the device name must be determined at installation and cannot be changed. You cannot add different instance types as nodes if they have a different device naming schema. For example, if you configured the **/var** partition with the default AWS device name for **m4.large** instances, **dev/xvdb**, you cannot directly add an AWS **m5.large** instance, as **m5.large** instances use a **/dev/nvme1n1** device by default. The device might fail to partition due to the different naming schema.

The procedure in this section shows how to add a new Red Hat Enterprise Linux CoreOS (RHCOS) compute node with an instance that uses a different device name from what was configured at installation. You create a custom user data secret and configure a new compute machine set. These steps are specific to an AWS cluster. The principles apply to other cloud deployments also. However, the device naming schema is different for other deployments and should be determined on a per-case basis.

Procedure

1. On a command line, change to the **openshift-machine-api** namespace:

```
$ oc project openshift-machine-api
```

2. Create a new secret from the **worker-user-data** secret:

- a. Export the **userData** section of the secret to a text file:

```
$ oc get secret worker-user-data --template='{{index .data.userData | base64decode}}' |
jq > userData.txt
```

- b. Edit the text file to add the **storage**, **filesystems**, and **systemd** stanzas for the partitions you want to use for the new node. You can specify any [Ignition configuration parameters](#) as needed.



NOTE

Do not change the values in the **ignition** stanza.

```
{
  "ignition": {
    "config": {
      "merge": [
        {
          "source": "https:...."
        }
      ]
    },
    "security": {
      "tls": {
        "certificateAuthorities": [
          {
            "source": "data:text/plain;charset=utf-8;base64,.....=="
          }
        ]
      }
    },
    "version": "3.2.0"
  }
}
```



```

    },
    "storage": {
      "disks": [
        {
          "device": "/dev/nvme1n1", ❶
          "partitions": [
            {
              "label": "var",
              "sizeMiB": 50000, ❷
              "startMiB": 0 ❸
            }
          ]
        }
      ],
      "filesystems": [
        {
          "device": "/dev/disk/by-partlabel/var", ❹
          "format": "xfs", ❺
          "path": "/var" ❻
        }
      ]
    },
    "systemd": {
      "units": [ ❼
        {
          "contents": "[Unit]\nBefore=local-
fs.target\n[Mount]\nWhere=/var\nWhat=/dev/disk/by-
partlabel/var\nOptions=defaults,pquota\n[Install]\nWantedBy=local-fs.target\n",
          "enabled": true,
          "name": "var.mount"
        }
      ]
    }
  }
}

```

❶ ❶ ❶ Specifies an absolute path to the AWS block device.

❷ ❷ Specifies the size of the data partition in Mebibytes.

❸ Specifies the start of the partition in Mebibytes. When adding a data partition to the boot disk, a minimum value of 25000 MB (Mebibytes) is recommended. The root file system is automatically resized to fill all available space up to the specified offset. If no value is specified, or if the specified value is smaller than the recommended minimum, the resulting root file system will be too small, and future reinstalls of RHCOS might overwrite the beginning of the data partition.

❹ Specifies an absolute path to the **/var** partition.

❺ Specifies the filesystem format.

❻ Specifies the mount-point of the filesystem while Ignition is running relative to where the root filesystem will be mounted. This is not necessarily the same as where it should be mounted in the real root, but it is encouraged to make it the same.

❼

Defines a systemd mount unit that mounts the **/dev/disk/by-partlabel/var** device to the **/var** partition.

- c. Extract the **disableTemplating** section from the **work-user-data** secret to a text file:

```
$ oc get secret worker-user-data --template='{{index .data.disableTemplating |
base64decode}}' | jq > disableTemplating.txt
```

- d. Create the new user data secret file from the two text files. This user data secret passes the additional node partition information in the **userData.txt** file to the newly created node.

```
$ oc create secret generic worker-user-data-x5 --from-file=userData=userData.txt --
from-file=disableTemplating=disableTemplating.txt
```

3. Create a new compute machine set for the new node:

- a. Create a new compute machine set YAML file, similar to the following, which is configured for AWS. Add the required partitions and the newly-created user data secret:

TIP

Use an existing compute machine set as a template and change the parameters as needed for the new node.

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  labels:
    machine.openshift.io/cluster-api-cluster: auto-52-92tf4
  name: worker-us-east-2-nvme1n1 1
  namespace: openshift-machine-api
spec:
  replicas: 1
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-cluster: auto-52-92tf4
      machine.openshift.io/cluster-api-machineset: auto-52-92tf4-worker-us-east-2b
  template:
    metadata:
      labels:
        machine.openshift.io/cluster-api-cluster: auto-52-92tf4
        machine.openshift.io/cluster-api-machine-role: worker
        machine.openshift.io/cluster-api-machine-type: worker
        machine.openshift.io/cluster-api-machineset: auto-52-92tf4-worker-us-east-2b
    spec:
      metadata: {}
      providerSpec:
        value:
          ami:
            id: ami-0c2dbd95931a
            apiVersion: awsproviderconfig.openshift.io/v1beta1
            blockDevices:
              - DeviceName: /dev/nvme1n1 2
```

```

ebs:
  encrypted: true
  iops: 0
  volumeSize: 120
  volumeType: gp2
- DeviceName: /dev/nvme1n2 3
ebs:
  encrypted: true
  iops: 0
  volumeSize: 50
  volumeType: gp2
credentialsSecret:
  name: aws-cloud-credentials
deviceIndex: 0
iamInstanceProfile:
  id: auto-52-92tf4-worker-profile
instanceType: m6i.large
kind: AWSMachineProviderConfig
metadata:
  creationTimestamp: null
placement:
  availabilityZone: us-east-2b
  region: us-east-2
securityGroups:
- filters:
  - name: tag:Name
    values:
    - auto-52-92tf4-worker-sg
subnet:
  id: subnet-07a90e5db1
tags:
- name: kubernetes.io/cluster/auto-52-92tf4
  value: owned
userDataSecret:
  name: worker-user-data-x5 4

```

- 1** Specifies a name for the new node.
- 2** Specifies an absolute path to the AWS block device, here an encrypted EBS volume.
- 3** Optional. Specifies an additional EBS volume.
- 4** Specifies the user data secret file.

b. Create the compute machine set:

```
$ oc create -f <file-name>.yaml
```

The machines might take a few moments to become available.

4. Verify that the new partition and nodes are created:

a. Verify that the compute machine set is created:

```
$ oc get machineset
```

Example output

NAME	DESIRED	CURRENT	READY	AVAILABLE	AGE
ci-ln-2675bt2-76ef8-bdgsc-worker-us-east-1a	1	1	1	1	124m
ci-ln-2675bt2-76ef8-bdgsc-worker-us-east-1b	2	2	2	2	124m
worker-us-east-2-nvme1n1	1	1	1	1	2m35s 1

- 1** This is the new compute machine set.

- b. Verify that the new node is created:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-128-78.ec2.internal	Ready	worker	117m	v1.35.4
ip-10-0-146-113.ec2.internal	Ready	master	127m	v1.35.4
ip-10-0-153-35.ec2.internal	Ready	worker	118m	v1.35.4
ip-10-0-176-58.ec2.internal	Ready	master	126m	v1.35.4
ip-10-0-217-135.ec2.internal	Ready	worker	2m57s	v1.35.4 1
ip-10-0-225-248.ec2.internal	Ready	master	127m	v1.35.4
ip-10-0-245-59.ec2.internal	Ready	worker	116m	v1.35.4

- 1** This is new new node.

- c. Verify that the custom **/var** partition is created on the new node:

```
$ oc debug node/<node-name> -- chroot /host lsblk
```

For example:

```
$ oc debug node/ip-10-0-217-135.ec2.internal -- chroot /host lsblk
```

Example output

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINT
nvme0n1	202:0	0	120G	0	disk	
nvme0n1p1	202:1	0	1M	0	part	
nvme0n1p2	202:2	0	127M	0	part	
nvme0n1p3	202:3	0	384M	0	part	/boot
`nvme0n1p4	202:4	0	119.5G	0	part	/sysroot
nvme1n1	202:16	0	50G	0	disk	
`nvme1n1p1	202:17	0	48.8G	0	part	/var 1

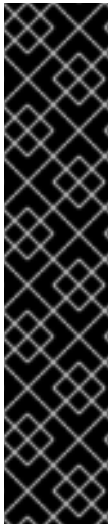
- 1** The **nvme1n1** device is mounted to the **/var** partition.

Additional resources

- [Disk partitioning for OpenShift Container Platform](#)

5.2. DEPLOYING MACHINE HEALTH CHECKS

Understand and deploy machine health checks.



IMPORTANT

You can use the advanced machine management and scaling capabilities only in clusters where the Machine API is operational. Clusters with user-provisioned infrastructure require additional validation and configuration to use the Machine API.

Clusters with the infrastructure platform type **none** cannot use the Machine API. This limitation applies even if the compute machines that are attached to the cluster are installed on a platform that supports the feature. This parameter cannot be changed after installation.

To view the platform type for your cluster, run the following command:

```
$ oc get infrastructure cluster -o jsonpath='{.status.platform}'
```

5.2.1. About machine health checks

You can use machine health checks to detect and remediate unhealthy machines automatically while limiting disruption to the targeted machine pool.



NOTE

You can only apply a machine health check to machines that are managed by compute machine sets or control plane machine sets.

To monitor machine health, create a resource to define the configuration for a controller. Set a condition to check, such as staying in the **NotReady** status for five minutes or displaying a permanent condition in the node-problem-detector, and a label for the set of machines to monitor.

The controller that observes a **MachineHealthCheck** resource checks for the defined condition. If a machine fails the health check, the machine is automatically deleted and one is created to take its place. When a machine is deleted, you see a **machine deleted** event.

To limit disruptive impact of the machine deletion, the controller drains and deletes only one node at a time. If there are more unhealthy machines than the **maxUnhealthy** threshold allows for in the targeted pool of machines, remediation stops and therefore enables manual intervention.



NOTE

Consider the timeouts carefully, accounting for workloads and requirements.

- Long timeouts can result in long periods of downtime for the workload on the unhealthy machine.
- Too short timeouts can result in a remediation loop. For example, the timeout for checking the **NotReady** status must be long enough to allow the machine to complete the startup process.

To stop the check, remove the resource.

5.2.1.1. Limitations when deploying machine health checks

There are limitations to consider before deploying a machine health check:

- Only machines owned by a machine set are remediated by a machine health check.
- If the node for a machine is removed from the cluster, a machine health check considers the machine to be unhealthy and remediates it immediately.
- If the corresponding node for a machine does not join the cluster after the **nodeStartupTimeout**, the machine is remediated.
- A machine is remediated immediately if the **Machine** resource phase is **Failed**.

Additional resources

- [About control plane machine sets](#)

5.2.2. Sample MachineHealthCheck resource

You can use the sample **MachineHealthCheck** resource to configure health criteria, remediation limits, and startup timeouts for machines in a targeted pool.

The **MachineHealthCheck** resource for all cloud-based installation types, and other than bare metal, resembles the following YAML file:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineHealthCheck
metadata:
  name: example 1
  namespace: openshift-machine-api
spec:
  selector:
    matchLabels:
      machine.openshift.io/cluster-api-machine-role: <role> 2
      machine.openshift.io/cluster-api-machine-type: <role> 3
      machine.openshift.io/cluster-api-machineset: <cluster_name>-<label>-<zone> 4
  unhealthyConditions:
  - type: "Ready"
    timeout: "300s" 5
    status: "False"
  - type: "Ready"
    timeout: "300s" 6
    status: "Unknown"
  maxUnhealthy: "40%" 7
  nodeStartupTimeout: "10m" 8
```

- 1 Specify the name of the machine health check to deploy.
- 2 3 Specify a label for the machine pool that you want to check.
- 4

Specify the machine set to track in **<cluster_name>-<label>-<zone>** format. For example, **prod-node-us-east-1a**.

- 5** **6** Specify the timeout duration for a node condition. If a condition is met for the duration of the timeout, the machine will be remediated. Long timeouts can result in long periods of downtime for a workload on an unhealthy machine.
- 7** Specify the amount of machines allowed to be concurrently remediated in the targeted pool. This can be set as a percentage or an integer. If the number of unhealthy machines exceeds the limit set by **maxUnhealthy**, remediation is not performed.
- 8** Specify the timeout duration that a machine health check must wait for a node to join the cluster before a machine is determined to be unhealthy.



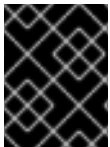
NOTE

The **matchLabels** are examples only; you must map your machine groups based on your specific needs.

5.2.2.1. Short-circuiting machine health check remediation

Short-circuiting ensures that machine health checks remediate machines only when the cluster is healthy. Short-circuiting is configured through the **maxUnhealthy** field in the **MachineHealthCheck** resource.

If the user defines a value for the **maxUnhealthy** field, before remediating any machines, the **MachineHealthCheck** compares the value of **maxUnhealthy** with the number of machines within its target pool that it has determined to be unhealthy. Remediation is not performed if the number of unhealthy machines exceeds the **maxUnhealthy** limit.



IMPORTANT

If **maxUnhealthy** is not set, the value defaults to **100%** and the machines are remediated regardless of the state of the cluster.

The appropriate **maxUnhealthy** value depends on the scale of the cluster you deploy and how many machines the **MachineHealthCheck** covers. For example, you can use the **maxUnhealthy** value to cover multiple compute machine sets across multiple availability zones so that if you lose an entire zone, your **maxUnhealthy** setting prevents further remediation within the cluster. In global Azure regions that do not have multiple availability zones, you can use availability sets to ensure high availability.



IMPORTANT

If you configure a **MachineHealthCheck** resource for the control plane, set the value of **maxUnhealthy** to **1**.

This configuration ensures that the machine health check takes no action when multiple control plane machines appear to be unhealthy. Multiple unhealthy control plane machines can indicate that the etcd cluster is degraded or that a scaling operation to replace a failed machine is in progress.

If the etcd cluster is degraded, manual intervention might be required. If a scaling operation is in progress, the machine health check should allow it to finish.

The **maxUnhealthy** field can be set as either an integer or percentage. There are different remediation implementations depending on the **maxUnhealthy** value.

5.2.2.1.1. Setting maxUnhealthy by using an absolute value

If **maxUnhealthy** is set to **2**:

- Remediation will be performed if 2 or fewer nodes are unhealthy
- Remediation will not be performed if 3 or more nodes are unhealthy

These values are independent of how many machines are being checked by the machine health check.

5.2.2.1.2. Setting maxUnhealthy by using percentages

If **maxUnhealthy** is set to **40%** and there are 25 machines being checked:

- Remediation will be performed if 10 or fewer nodes are unhealthy
- Remediation will not be performed if 11 or more nodes are unhealthy

If **maxUnhealthy** is set to **40%** and there are 6 machines being checked:

- Remediation will be performed if 2 or fewer nodes are unhealthy
- Remediation will not be performed if 3 or more nodes are unhealthy



NOTE

The allowed number of machines is rounded down when the percentage of **maxUnhealthy** machines that are checked is not a whole number.

5.2.3. Creating a machine health check resource

You can create a **MachineHealthCheck** resource to monitor and automatically remediate unhealthy machines in a machine set.

You can create a **MachineHealthCheck** resource for machine sets in your cluster.



NOTE

You can only apply a machine health check to machines that are managed by compute machine sets or control plane machine sets.

Prerequisites

- Install the **oc** command-line interface.

Procedure

1. Create a **healthcheck.yml** file that contains the definition of your machine health check.
2. Apply the **healthcheck.yml** file to your cluster:


```
$ oc apply -f healthcheck.yml
```

5.2.4. Scaling a compute machine set manually

To add or remove an instance of a machine in a compute machine set, you can manually scale the compute machine set.

This guidance is relevant to fully automated, installer-provisioned infrastructure installations. Customized, user-provisioned infrastructure installations do not have compute machine sets.

Prerequisites

- Install an OpenShift Container Platform cluster and the **oc** command line.
- Log in to **oc** as a user with **cluster-admin** permission.

Procedure

1. View the compute machine sets that are in the cluster by running the following command:

```
$ oc get machinesets.machine.openshift.io -n openshift-machine-api
```

The compute machine sets are listed in the form of **<clusterid>-worker-<aws-region-az>**.

2. View the compute machines that are in the cluster by running the following command:

```
$ oc get machines.machine.openshift.io -n openshift-machine-api
```

3. Set the annotation on the compute machine that you want to delete by running the following command:

```
$ oc annotate machines.machine.openshift.io/<machine_name> -n openshift-machine-api  
machine.openshift.io/delete-machine="true"
```

4. Scale the compute machine set by running one of the following commands:

```
$ oc scale --replicas=2 machinesets.machine.openshift.io <machineset> -n openshift-  
machine-api
```

Or:

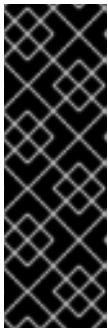
```
$ oc edit machinesets.machine.openshift.io <machineset> -n openshift-machine-api
```

TIP

You can alternatively apply the following YAML to scale the compute machine set:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: <machineset>
  namespace: openshift-machine-api
spec:
  replicas: 2
```

You can scale the compute machine set up or down. It takes several minutes for the new machines to be available.

**IMPORTANT**

By default, the machine controller tries to drain the node that is backed by the machine until it succeeds. In some situations, such as with a misconfigured pod disruption budget, the drain operation might not be able to succeed. If the drain operation fails, the machine controller cannot proceed removing the machine.

You can skip draining the node by annotating **machine.openshift.io/exclude-node-draining** in a specific machine.

Verification

- Verify the deletion of the intended machine by running the following command:

```
$ oc get machines.machine.openshift.io
```

5.2.5. Understanding the difference between compute machine sets and the machine config pool

Compute machine sets and machine config pools control different aspects of node lifecycle in OpenShift Container Platform. Understanding how each object relates to scaling and upgrades helps you configure nodes correctly.

MachineSet objects describe OpenShift Container Platform nodes with respect to the cloud or machine provider.

The **MachineConfigPool** object allows **MachineConfigController** components to define and provide the status of machines in the context of upgrades.

The **MachineConfigPool** object allows users to configure how upgrades are rolled out to the OpenShift Container Platform nodes in the machine config pool.

The **NodeSelector** object can be replaced with a reference to the **MachineSet** object.

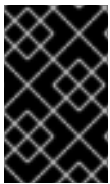
5.3. RECOMMENDED NODE HOST PRACTICES

You can configure the **podsPerCore** and **maxPods** parameters to control the maximum number of pods that can be scheduled on a node.

The OpenShift Container Platform node configuration file contains important options. For example, two parameters control the maximum number of pods that can be scheduled to a node: **podsPerCore** and **maxPods**.

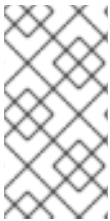
When both options are in use, the lower of the two values limits the number of pods on a node. Exceeding these values can result in the following conditions:

- Increased CPU utilization.
- Slow pod scheduling.
- Potential out-of-memory scenarios, depending on the amount of memory in the node.
- Exhausting the pool of IP addresses.
- Resource overcommitting, leading to poor user application performance.



IMPORTANT

In Kubernetes, a pod that is holding a single container actually uses two containers. The second container is used to set up networking prior to the actual container starting. Therefore, a system running 10 pods will actually have 20 containers running.



NOTE

Disk IOPS throttling from the cloud provider might have an impact on CRI-O and kubelet. They might get overloaded when there are large number of I/O intensive pods running on the nodes. It is recommended that you monitor the disk I/O on the nodes and use volumes with sufficient throughput for the workload.

The **podsPerCore** parameter sets the number of pods that the node can run based on the number of processor cores on the node. For example, if **podsPerCore** is set to **10** on a node with 4 processor cores, the maximum number of pods allowed on the node is **40**.

```
kubeletConfig:
  podsPerCore: 10
```

Setting **podsPerCore** to **0** disables this limit. The default is **0**. The value of the **podsPerCore** parameter cannot exceed the value of the **maxPods** parameter.

The **maxPods** parameter sets the number of pods that the node can run to a fixed value, regardless of the properties of the node.

```
kubeletConfig:
  maxPods: 250
```

5.3.1. Creating a KubeletConfig CR to edit kubelet parameters

You can use a **KubeletConfig** custom resource (CR) to edit a kubelet parameters without modifying the kubelet configuration directly.



NOTE

As the fields in the **kubeletConfig** object are passed directly to the kubelet from upstream Kubernetes, the kubelet validates those values directly. Invalid values in the **kubeletConfig** object might cause cluster nodes to become unavailable. For valid values, see the [Kubernetes documentation](#).

Consider the following guidance:

- Edit an existing **KubeletConfig** CR to modify existing settings or add new settings, instead of creating a CR for each change. It is recommended that you create a CR only to modify a different machine config pool, or for changes that are intended to be temporary, so that you can revert the changes.
- Create one **KubeletConfig** CR for each machine config pool with all the config changes you want for that pool.
- As needed, create multiple **KubeletConfig** CRs with a limit of 10 per cluster. For the first **KubeletConfig** CR, the Machine Config Operator (MCO) creates a machine config appended with **kubelet**. With each subsequent CR, the controller creates another **kubelet** machine config with a numeric suffix. For example, if you have a **kubelet** machine config with a **-2** suffix, the next **kubelet** machine config is appended with **-3**.



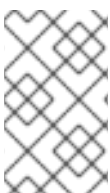
NOTE

If you are applying a kubelet or container runtime config to a custom machine config pool, the custom role in the **machineConfigSelector** must match the name of the custom machine config pool.

For example, because the following custom machine config pool is named **infra**, the custom role must also be **infra**:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: infra
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,infra]}
  # ...
```

If you want to delete the machine configs, delete them in reverse order to avoid exceeding the limit. For example, you delete the **kubelet-3** machine config before deleting the **kubelet-2** machine config.



NOTE

If you have a machine config with a **kubelet-9** suffix, and you create another **KubeletConfig** CR, a new machine config is not created, even if there are fewer than 10 **kubelet** machine configs.

The following example command and output show a **KubeletConfig** CR:

■

```
$ oc get kubeletconfig
```

```
NAME          AGE
set-kubelet-config 15m
```

The following example command and output show a **KubeletConfig** machine config:

```
$ oc get mc | grep kubelet
```

```
...
99-worker-generated-kubelet-1      b5c5119de007945b6fe6fb215db3b8e2ceb12511  3.5.0
26m
...
```

The following procedure is an example to show how to configure the maximum number of pods per node, the maximum PIDs per node, and the maximum container log size size on the worker nodes.

Prerequisites

1. Obtain the label associated with the static **MachineConfigPool** CR for the type of node you want to configure. Perform one of the following steps:
 - a. View the machine config pool:

```
$ oc describe machineconfigpool <name>
```

For example:

```
$ oc describe machineconfigpool worker
```

Example output

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  creationTimestamp: 2019-02-08T14:52:39Z
  generation: 1
  labels:
    custom-kubelet: set-kubelet-config
```

The **metadata.labels** parameter specifies labels you can use with a **KubeletConfig** CR.

- b. If the label is not present, add a key/value pair:

```
$ oc label machineconfigpool worker custom-kubelet=set-kubelet-config
```

Procedure

1. View the available machine configuration objects that you can select:

```
$ oc get machineconfig
```

By default, the two kubelet-related configs are **01-master-kubelet** and **01-worker-kubelet**.

2. Check the current value for the maximum pods per node:

```
$ oc describe node <node_name>
```

For example:

```
$ oc describe node ci-ln-5grqprb-f76d1-ncnqq-worker-a-mdv94
```

Look for **value: pods: <value>** in the **Allocatable** stanza:

Example output

```
Allocatable:
attachable-volumes-aws-ebs: 25
cpu:                        3500m
hugepages-1Gi:             0
hugepages-2Mi:             0
memory:                    15341844Ki
pods:                      250
```

3. Configure the worker nodes as needed:
 - a. Create a YAML file similar to the following that contains the kubelet configuration:



IMPORTANT

Kubelet configurations that target a specific machine config pool also affect any dependent pools. For example, creating a kubelet configuration for the pool containing worker nodes will also apply to any subset pools, including the pool containing infrastructure nodes. To avoid this, you must create a new machine config pool with a selection expression that only includes worker nodes, and have your kubelet configuration target this new pool.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: set-kubelet-config
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: set-kubelet-config
  kubeletConfig:
    podPidsLimit: 8192
    containerLogMaxSize: 50Mi
    maxPods: 500
```

where:

spec.machineConfigPoolSelector.matchLabels

Specifies the label from the machine config pool.

spec.kubeletConfig

Specifies the kubelet configuration. For example:

- Use **podPidsLimit** to set the maximum number of PIDs in any pod.
- Use **containerLogMaxSize** to set the maximum size of the container log file before it is rotated.
- Use **maxPods** to set the maximum pods per node.

NOTE

The rate at which the kubelet talks to the API server depends on queries per second (QPS) and burst values. The default values, **50** for **kubeAPIQPS** and **100** for **kubeAPIBurst**, are sufficient if there are limited pods running on each node. It is recommended to update the kubelet QPS and burst rates if there are enough CPU and memory resources on the node.

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: set-kubelet-config
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: set-kubelet-config
  kubeletConfig:
    maxPods: <pod_count>
    kubeAPIBurst: <burst_rate>
    kubeAPIQPS: <QPS>
```

- b. Update the machine config pool for workers with the label:

```
$ oc label machineconfigpool worker custom-kubelet=set-kubelet-config
```

- c. Create the **KubeletConfig** object:

```
$ oc create -f change-maxPods-cr.yaml
```

Verification

1. Verify that the **KubeletConfig** object is created:

```
$ oc get kubeletconfig
```

Example output

```
NAME          AGE
set-kubelet-config  15m
```

Depending on the number of worker nodes in the cluster, wait for the worker nodes to be rebooted one by one. For a cluster with 3 worker nodes, this could take about 10 to 15 minutes.

2. Verify that the changes are applied to the node:
 - a. Check on a worker node that the **maxPods** value changed:

```
$ oc describe node <node_name>
```

- b. Locate the **Allocatable** stanza:

```
...
Allocatable:
  attachable-volumes-gce-pd: 127
  cpu: 3500m
  ephemeral-storage: 123201474766
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  memory: 14225400Ki
  pods: 500
...
```

In this example, the **pods** parameter should report the value you set in the **KubeletConfig** object.

3. Verify the change in the **KubeletConfig** object:

```
$ oc get kubeletconfigs set-kubelet-config -o yaml
```

This should show a status of **True** and **type:Success**, as shown in the following example:

```
spec:
  kubeletConfig:
    containerLogMaxSize: 50Mi
    maxPods: 500
    podPidsLimit: 8192
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: set-kubelet-config
status:
  conditions:
  - lastTransitionTime: "2021-06-30T17:04:07Z"
    message: Success
    status: "True"
    type: Success
```

5.3.2. Modifying the number of unavailable worker nodes

You can speed up kubelet configuration rollouts on large clusters by increasing the number of worker nodes that can be unavailable during machine config pool updates.

By default, only one machine is allowed to be unavailable when applying the kubelet-related configuration to the available worker nodes. For a large cluster, it can take a long time for the configuration change to be reflected. At any time, you can adjust the number of machines that are

updating to speed up the process.

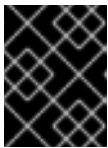
Procedure

1. Edit the **worker** machine config pool:

```
$ oc edit machineconfigpool worker
```

2. Add the **maxUnavailable** field and set the value:

```
spec:
  maxUnavailable: <node_count>
```



IMPORTANT

When setting the value, consider the number of worker nodes that can be unavailable without affecting the applications running on the cluster.

5.3.3. Control plane node sizing

The control plane node resource requirements depend on the number and type of nodes and objects in the cluster. Reference the control plane node size recommendations to better understand your sizing needs.

The following control plane node size recommendations are based on the results of a control plane density focused testing, or *Cluster-density*. This test creates the following objects across a given number of namespaces:

- 1 image stream
- 1 build
- 5 deployments, with 2 pod replicas in a **sleep** state, mounting 4 secrets, 4 config maps, and 1 downward API volume each
- 5 services, each one pointing to the TCP/8080 and TCP/8443 ports of one of the previous deployments
- 1 route pointing to the first of the previous services
- 10 secrets containing 2048 random string characters
- 10 config maps containing 2048 random string characters

Number of compute nodes	Cluster-density (namespaces)	CPU cores	Memory (GB)
24	500	4	16
120	1000	8	32

Number of compute nodes	Cluster-density (namespaces)	CPU cores	Memory (GB)
252	4000	16, but 24 if using the OVN-Kubernetes network plug-in	64, but 128 if using the OVN-Kubernetes network plug-in
501, but untested with the OVN-Kubernetes network plug-in	4000	16	96

The data from the table above is based on an OpenShift Container Platform running on top of AWS, using r5.4xlarge instances as control-plane nodes and m5.2xlarge instances as compute nodes.

On a large and dense cluster with three control plane nodes, the CPU and memory usage will spike up when one of the nodes is stopped, rebooted, or fails. The failures can be due to unexpected issues with power, network, underlying infrastructure, or intentional cases where the cluster is restarted after shutting it down to save costs. The remaining two control plane nodes must handle the load in order to be highly available, which leads to increase in the resource usage. This is also expected during upgrades because the control plane nodes are cordoned, drained, and rebooted serially to apply the operating system updates, as well as the control plane Operators update. To avoid cascading failures, keep the overall CPU and memory resource usage on the control plane nodes to at most 60% of all available capacity to handle the resource usage spikes. Increase the CPU and memory on the control plane nodes accordingly to avoid potential downtime due to lack of resources.



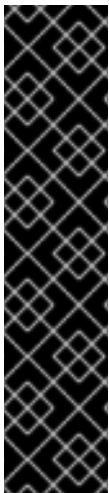
IMPORTANT

The node sizing varies depending on the number of nodes and object counts in the cluster. It also depends on whether the objects are actively being created on the cluster. During object creation, the control plane is more active in terms of resource usage compared to when the objects are in the **Running** phase.

Operator Lifecycle Manager (OLM) runs on the control plane nodes and its memory footprint depends on the number of namespaces and user installed operators that OLM needs to manage on the cluster. Control plane nodes need to be sized accordingly to avoid OOM kills. Following data points are based on the results from cluster maximums testing.

Number of namespaces	OLM memory at idle state (GB)	OLM memory with 5 user operators installed (GB)
500	0.823	1.7
1000	1.2	2.5
1500	1.7	3.2
2000	2	4.4
3000	2.7	5.6

Number of namespaces	OLM memory at idle state (GB)	OLM memory with 5 user operators installed (GB)
4000	3.8	7.6
5000	4.2	9.02
6000	5.8	11.3
7000	6.6	12.9
8000	6.9	14.8
9000	8	17.7
10,000	9.9	21.6



IMPORTANT

You can modify the control plane node size in a running OpenShift Container Platform 4.22 cluster for the following configurations only:

- Clusters installed with a user-provisioned installation method.
- AWS clusters installed with an installer-provisioned infrastructure installation method.
- Clusters that use a control plane machine set to manage control plane machines.

For all other configurations, you must estimate your total node count and use the suggested control plane node size during installation.



NOTE

In OpenShift Container Platform 4.22, half of a CPU core (500 millicore) is now reserved by the system by default compared to OpenShift Container Platform 3.11 and previous versions. The sizes are determined taking that into consideration.

5.3.4. Setting up CPU Manager

To configure CPU manager, create a **KubeletConfig** custom resource (CR) and apply it to the required set of nodes.

Procedure

1. Label a node by running the following command:

```
# oc label node perf-node.example.com cpumanager=true
```

2. To enable CPU Manager for all compute nodes, edit the CR by running the following command:

```
# oc edit machineconfigpool worker
```

3. Add the **custom-kubelet: cpumanager-enabled** label to **metadata.labels** section.

```
metadata:
  creationTimestamp: 2020-xx-xxx
  generation: 3
  labels:
    custom-kubelet: cpumanager-enabled
```

4. Create a **KubeletConfig**, **cpumanager-kubeletconfig.yaml**, custom resource (CR). Refer to the label created in the previous step to have the correct nodes updated with the new kubelet config. See the **machineConfigPoolSelector** section:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: cpumanager-enabled
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: cpumanager-enabled
  kubeletConfig:
    cpuManagerPolicy: static
    cpuManagerReconcilePeriod: 5s
```

- **cpuManagerPolicy** specifies a policy:
 - **none**. This policy explicitly enables the existing default CPU affinity scheme, providing no affinity beyond what the scheduler does automatically. This is the default policy.
 - **static**. This policy allows containers in guaranteed pods with integer CPU requests. It also limits access to exclusive CPUs on the node. If **static**, you must use a lowercase **s**.
- **cpuManagerReconcilePeriod** is optional. Specify the CPU Manager reconcile frequency. The default is **5s**.

5. Create the dynamic kubelet config by running the following command:

```
# oc create -f cpumanager-kubeletconfig.yaml
```

This adds the CPU Manager feature to the kubelet config and, if needed, the Machine Config Operator (MCO) reboots the node. To enable CPU Manager, a reboot is not needed.

6. Check for the merged kubelet config by running the following command:

```
# oc get machineconfig 99-worker-XXXXXX-XXXXX-XXXX-XXXXX-kubelet -o json | grep ownerReference -A7
```

Example output

```
"ownerReferences": [
  {
    "apiVersion": "machineconfiguration.openshift.io/v1",
```

```

        "kind": "KubeletConfig",
        "name": "cpumanager-enabled",
        "uid": "7ed5616d-6b72-11e9-aae1-021e1ce18878"
      }
    ]

```

7. Check the compute node for the updated **kubelet.conf** file by running the following command:

```

# oc debug node/perf-node.example.com
sh-4.2# cat /host/etc/kubernetes/kubelet.conf | grep cpuManager

```

The following is example output:

```

cpuManagerPolicy: static
cpuManagerReconcilePeriod: 5s

```

- **cpuManagerPolicy** is defined when you create the **KubeletConfig** CR.
- **cpuManagerReconcilePeriod** is defined when you create the **KubeletConfig** CR.

8. Create a project by running the following command:

```

$ oc new-project <project_name>

```

9. Create a pod that requests a core or multiple cores. Both limits and requests must have their CPU value set to a whole integer. That is the number of cores that will be dedicated to this pod:

```

# cat cpumanager-pod.yaml

```

Example output

```

apiVersion: v1
kind: Pod
metadata:
  generateName: cpumanager-
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault
  containers:
  - name: cpumanager
    image: gcr.io/google_containers/pause:3.2
    resources:
      requests:
        cpu: 1
        memory: "1G"
      limits:
        cpu: 1
        memory: "1G"
    securityContext:
      allowPrivilegeEscalation: false
    capabilities:

```

```
drop: [ALL]
nodeSelector:
  cpumanager: "true"
```

10. Create the pod:

```
# oc create -f cpumanager-pod.yaml
```

Verification

1. Verify that the pod is scheduled to the node that you labeled by running the following command:

```
# oc describe pod cpumanager
```

Example output

```
Name:          cpumanager-6cqz7
Namespace:     default
Priority:       0
PriorityClassName: <none>
Node: perf-node.example.com/xxx.xx.xx.xxx
...
Limits:
  cpu: 1
  memory: 1G
Requests:
  cpu: 1
  memory: 1G
...
QoS Class:     Guaranteed
Node-Selectors: cpumanager=true
```

2. Verify that a CPU has been exclusively assigned to the pod by running the following command:

```
# oc describe node --selector='cpumanager=true' | grep -i cpumanager- -B2
```

Example output

NAMESPACE	NAME	CPU Requests	CPU Limits	Memory Requests	Memory Limits	Age
cpuman	cpumanager-mlrrz	1 (28%)	1 (28%)	1G (13%)	1G (13%)	27m

3. Verify that the **cgroups** are set up correctly. Get the process ID (PID) of the **pause** process by running the following commands:

```
# oc debug node/perf-node.example.com
```

```
sh-4.2# systemctl status | grep -B5 pause
```

**NOTE**

If the output returns multiple pause process entries, you must identify the correct pause process.

Example output

```
# |—init.scope
|  |—1 /usr/lib/systemd/systemd --switched-root --system --deserialize 17
|  |—kubepods.slice
|     |—kubepods-pod69c01f8e_6b74_11e9_ac0f_0a2b62178a22.slice
|         |—crio-b5437308f1a574c542bdf08563b865c0345c8f8c0b0a655612c.scope
|             |—32706 /pause
```

4. Verify that pods of quality of service (QoS) tier **Guaranteed** are placed within the **kubepods.slice** subdirectory by running the following commands:

```
# cd /sys/fs/cgroup/kubepods.slice/kubepods-
pod69c01f8e_6b74_11e9_ac0f_0a2b62178a22.slice/crio-
b5437308f1ad1a7db0574c542bdf08563b865c0345c86e9585f8c0b0a655612c.scope
```

```
# for i in `ls cpuset.cpus cgroup.procs` ; do echo -n "$i "; cat $i ; done
```

**NOTE**

Pods of other QoS tiers end up in child **cgroups** of the parent **kubepods**.

Example output

```
cpuset.cpus 1
tasks 32706
```

5. Check the allowed CPU list for the task by running the following command:

```
# grep ^Cpus_allowed_list /proc/32706/status
```

Example output

```
Cpus_allowed_list: 1
```

6. Verify that another pod on the system cannot run on the core allocated for the **Guaranteed** pod. For example, to verify the pod in the **besteffort** QoS tier, run the following commands:

```
# cat /sys/fs/cgroup/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-
podc494a073_6b77_11e9_98c0_06bba5c387ea.slice/crio-
c56982f57b75a2420947f0afc6cafe7534c5734efc34157525fa9abbf99e3849.scope/cpuset.cpus
```

```
# oc describe node perf-node.example.com
```

Example output

```

...
Capacity:
  attachable-volumes-aws-ebs: 39
  cpu: 2
  ephemeral-storage: 124768236Ki
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  memory: 8162900Ki
  pods: 250
Allocatable:
  attachable-volumes-aws-ebs: 39
  cpu: 1500m
  ephemeral-storage: 124768236Ki
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  memory: 7548500Ki
  pods: 250
-----
-
  default          cpumanager-6cqz7      1 (66%)    1 (66%)    1G (12%)
1G (12%)    29m
Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)
Resource           Requests          Limits
-----
cpu                 1440m (96%)      1 (66%)

```

This VM has two CPU cores. The **system-reserved** setting reserves 500 millicores, meaning that half of one core is subtracted from the total capacity of the node to arrive at the **Node Allocatable** amount. You can see that **Allocatable CPU** is 1500 millicores. This means you can run one of the CPU Manager pods since each will take one whole core. A whole core is equivalent to 1000 millicores. If you try to schedule a second pod, the system will accept the pod, but it will never be scheduled:

NAME	READY	STATUS	RESTARTS	AGE
cpumanager-6cqz7	1/1	Running	0	33m
cpumanager-7qc2t	0/1	Pending	0	11s

5.4. HUGE PAGES

Understand and configure huge pages.

5.4.1. What huge pages do

To optimize memory mapping efficiency, understand the function of huge pages. Unlike standard 4Ki blocks, huge pages are larger memory segments that reduce the tracking load on the translation lookaside buffer (TLB) hardware cache.

Memory is managed in blocks known as pages. On most systems, a page is 4Ki; 1Mi of memory is equal to 256 pages; 1Gi of memory is 256,000 pages, and so on. CPUs have a built-in memory management unit that manages a list of these pages in hardware. The translation lookaside buffer (TLB) is a small hardware cache of virtual-to-physical page mappings. If the virtual address passed in a hardware instruction can be found in the TLB, the mapping can be determined quickly. If not, a TLB miss occurs,

and the system falls back to slower, software-based address translation, resulting in performance issues. Since the size of the TLB is fixed, the only way to reduce the chance of a TLB miss is to increase the page size.

A huge page is a memory page that is larger than 4Ki. On x86_64 architectures, there are two common huge page sizes: 2Mi and 1Gi. Sizes vary on other architectures. To use huge pages, code must be written so that applications are aware of them. Transparent huge pages (THP) attempt to automate the management of huge pages without application knowledge, but they have limitations. In particular, they are limited to 2Mi page sizes. THP can lead to performance degradation on nodes with high memory utilization or fragmentation because of defragmenting efforts of THP, which can lock memory pages. For this reason, some applications might be designed to or recommend usage of pre-allocated huge pages instead of THP.

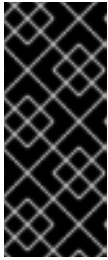
5.4.2. How huge pages are consumed by apps

You must ensure that nodes pre-allocate huge pages in order for the node to report its huge page capacity. A node can only pre-allocate huge pages for a single size.

Huge pages can be consumed through container-level resource requirements by using the resource name **hugepages-<size>**, where size is the most compact binary notation by using integer values supported on a particular node. For example, if a node supports 2048 KiB page sizes, the node exposes a schedulable resource **hugepages-2Mi**. Unlike CPU or memory, huge pages do not support over-commitment.

```
apiVersion: v1
kind: Pod
metadata:
  generateName: hugepages-volume-
spec:
  containers:
  - securityContext:
      privileged: true
    image: rhel7:latest
    command:
    - sleep
    - inf
    name: example
    volumeMounts:
    - mountPath: /dev/hugepages
      name: hugepage
    resources:
      limits:
        hugepages-2Mi: 100Mi
        memory: "1Gi"
        cpu: "1"
    volumes:
    - name: hugepage
      emptyDir:
        medium: HugePages
```

- **spec.containers.resources.limits.hugepages-2Mi**: Specifies the amount of memory for **hugepages** as the exact amount to be allocated.



IMPORTANT

Do not specify this value as the amount of memory for **hugepages** multiplied by the size of the page. For example, given a huge page size of 2 MB, if you want to use 100 MB of huge-page-backed RAM for your application, then you would allocate 50 huge pages. OpenShift Container Platform handles the math for you. As in the above example, you can specify **100MB** directly.

5.4.2.1. Allocating huge pages of a specific size

Some platforms support multiple huge page sizes. To allocate huge pages of a specific size, precede the huge pages boot command parameters with a huge page size selection parameter **hugepagesz=<size>**. The **<size>** value must be specified in bytes with an optional scale suffix [**kKmMgG**]. The default huge page size can be defined with the **default_hugepagesz=<size>** boot parameter.

5.4.2.2. Huge page requirements

- Huge page requests must equal the limits. This is the default if limits are specified, but requests are not.
- Huge pages are isolated at a pod scope. Container isolation is planned in a future iteration.
- **EmptyDir** volumes backed by huge pages must not consume more huge page memory than the pod request.
- Applications that consume huge pages via **shmget()** with **SHM_HUGETLB** must run with a supplemental group that matches *proc/sys/vm/hugetlb_shm_group*.

5.4.3. Configuring huge pages at boot time

To ensure nodes in your OpenShift Container Platform cluster pre-allocate memory for specific workloads, reserve huge pages at boot time.

There are two ways of reserving huge pages: at boot time and at run time. Reserving at boot time increases the possibility of success because the memory has not yet been significantly fragmented. The Node Tuning Operator currently supports boot-time allocation of huge pages on specific nodes.



NOTE

The Tuned boot-loader plugin only supports Red Hat Enterprise Linux CoreOS (RHCOS) compute nodes.

Procedure

1. Label all nodes that need the same huge pages setting by a label by entering the following command:

```
$ oc label node <node_using_hugepages> node-role.kubernetes.io/worker-hp=
```

2. Create a file with the following content and name it **hugepages-tuned-boottime.yaml**:

```
apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
```

```

name: hugepages
namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Boot time configuration for hugepages
        include=openshift-node
        [bootloader]
        cmdline_openshift_node_hugepages=hugepagesz=2M hugepages=50
      name: openshift-node-hugepages

  recommend:
    - machineConfigLabels:
        machineconfiguration.openshift.io/role: "worker-hp"
      priority: 30
      profile: openshift-node-hugepages
# ...

```

where:

metadata.name

Specifies the **name** of the Tuned resource to **hugepages**.

spec.profile

Specifies the **profile** section to allocate huge pages.

spec.profile.data

Specifies the order of parameters. The order is important as some platforms support huge pages of various sizes.

spec.recommend.machineConfigLabels

Specifies the enablement of a machine config pool based matching.

3. Create the Tuned **hugepages** object by entering the following command:

```
$ oc create -f hugepages-tuned-boottime.yaml
```

4. Create a file with the following content and name it **hugepages-mcp.yaml**:

```

apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: worker-hp
  labels:
    worker-hp: ""
spec:
  machineConfigSelector:
    matchExpressions:
      - {key: machineconfiguration.openshift.io/role, operator: In, values: [worker,worker-hp]}
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/worker-hp: ""

```

5. Create the machine config pool by entering the following command:

—

```
$ oc create -f hugepages-mcp.yaml
```

Verification

- To check that enough non-fragmented memory exists and that all the nodes in the **worker-hp** machine config pool now have 50 2Mi huge pages allocated, enter the following command:

```
$ oc get node <node_using_hugepages> -o jsonpath="{.status.allocatable.hugepages-2Mi}"
100Mi
```

5.5. UNDERSTANDING DEVICE PLUGINS

A device plugin is a gRPC service running on nodes that manages specific hardware resources through an extension mechanism, enabling containers to consume these devices.

The device plugin provides a consistent and portable solution to consume hardware devices across clusters. The device plugin provides support for these devices through an extension mechanism, which makes these devices available to Containers, provides health checks of these devices, and securely shares them.



IMPORTANT

OpenShift Container Platform supports the device plugin API, but the device plugin Containers are supported by individual vendors.

A device plugin is a gRPC service running on the nodes (external to the **kubelet**) that is responsible for managing specific hardware resources. Any device plugin must support following remote procedure calls (RPCs):

```
service DevicePlugin {
    // GetDevicePluginOptions returns options to be communicated with Device
    // Manager
    rpc GetDevicePluginOptions(Empty) returns (DevicePluginOptions) {}

    // ListAndWatch returns a stream of List of Devices
    // Whenever a Device state change or a Device disappears, ListAndWatch
    // returns the new list
    rpc ListAndWatch(Empty) returns (stream ListAndWatchResponse) {}

    // Allocate is called during container creation so that the Device
    // Plug-in can run device specific operations and instruct Kubelet
    // of the steps to make the Device available in the container
    rpc Allocate(AllocateRequest) returns (AllocateResponse) {}

    // PreStartcontainer is called, if indicated by Device Plug-in during
    // registration phase, before each container start. Device plug-in
    // can run device specific operations such as resetting the device
    // before making devices available to the container
    rpc PreStartcontainer(PreStartcontainerRequest) returns (PreStartcontainerResponse) {}
}
```

5.5.1. Example device plugins

- Nvidia GPU device plugin for COS-based operating system
- Nvidia official GPU device plugin
- Solarflare device plugin
- KubeVirt device plugins: vfio and kvm
- Kubernetes device plugin for IBM® Crypto Express (CEX) cards



NOTE

For easy device plugin reference implementation, there is a stub device plugin in the Device Manager code:

vendor/k8s.io/kubernetes/pkg/kubelet/cm/deviceplugin/device_plugin_stub.go.

5.5.2. Methods for deploying a device plugin

- Daemon sets are the recommended approach for device plugin deployments.
- Upon start, the device plugin will try to create a UNIX domain socket at */var/lib/kubelet/device-plugin/* on the node to serve RPCs from Device Manager.
- Since device plugins must manage hardware resources, access to the host file system, as well as socket creation, they must be run in a privileged security context.
- More specific details regarding deployment steps can be found with each device plugin implementation.

Additional resources

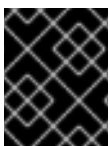
- [Nvidia GPU device plugin for COS-based operating system](#)
- [Nvidia official GPU device plugin](#)
- [Solarflare device plugin](#)
- [KubeVirt device plugins: vfio and kvm](#)
- [Kubernetes device plugin for IBM® Crypto Express \(CEX\) cards](#)

5.5.3. Understanding the Device Manager

Device Manager advertises specialized node hardware resources through device plugins, enabling pods to consume hardware devices without requiring upstream code changes.

Device Manager provides a mechanism for advertising specialized node hardware resources with the help of plugins known as device plugins.

You can advertise specialized hardware without requiring any upstream code changes.



IMPORTANT

OpenShift Container Platform supports the device plugin API, but the device plugin Containers are supported by individual vendors.

Device Manager advertises devices as **Extended Resources**. User pods can consume devices, advertised by Device Manager, using the same **Limit/Request** mechanism, which is used for requesting any other **Extended Resource**.

Upon start, the device plugin registers itself with Device Manager invoking **Register** on the `/var/lib/kubelet/device-plugins/kubelet.sock` and starts a gRPC service at `/var/lib/kubelet/device-plugins/<plugin>.sock` for serving Device Manager requests.

Device Manager, while processing a new registration request, invokes **ListAndWatch** remote procedure call (RPC) at the device plugin service. In response, Device Manager gets a list of **Device** objects from the plugin over a gRPC stream. Device Manager will keep watching on the stream for new updates from the plugin. On the plugin side, the plugin will also keep the stream open and whenever there is a change in the state of any of the devices, a new device list is sent to the Device Manager over the same streaming connection.

While handling a new pod admission request, Kubelet passes requested **Extended Resources** to the Device Manager for device allocation. Device Manager checks in its database to verify if a corresponding plugin exists or not. If the plugin exists and there are free allocatable devices as well as per local cache, **Allocate** RPC is invoked at that particular device plugin.

Additionally, device plugins can also perform several other device-specific operations, such as driver installation, device initialization, and device resets. These functionalities vary from implementation to implementation.

5.5.4. Enabling Device Manager

Enable Device Manager to allow device plugins to advertise specialized node hardware resources and make them available to pods without requiring code changes.

Enable Device Manager to implement a device plugin to advertise specialized hardware without any upstream code changes.

Device Manager provides a mechanism for advertising specialized node hardware resources with the help of plugins known as device plugins.

Procedure

1. Obtain the label associated with the static **MachineConfigPool** CRD for the type of node you want to configure by entering the following command. Perform one of the following steps:
 - a. View the machine config:

```
# oc describe machineconfig <name>
```

For example:

```
# oc describe machineconfig 00-worker
```

Example output

```
Name:      00-worker
Namespace:
Labels:    machineconfiguration.openshift.io/role=worker
```

machineconfiguration.openshift.io is the label required for the Device Manager.

2. Create a custom resource (CR) for your configuration change.

Sample configuration for a Device Manager CR

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: devicemgr
spec:
  machineConfigPoolSelector:
    matchLabels:
      machineconfiguration.openshift.io: devicemgr
  kubeletConfig:
    feature-gates:
      - DevicePlugins=true
```

where:

metadata.name

Specifies a name to assign to the CR.

spec.machineConfigPoolSelector.matchLabels

Specifies the label from the Machine Config Pool.

spec.kubeletConfig.feature-gates

Specifies the **DevicePlugins** feature gate. Set to **true**.

3. Create the Device Manager:

```
$ oc create -f devicemgr.yaml
```

Example output

```
kubeletconfig.machineconfiguration.openshift.io/devicemgr created
```

4. Ensure that Device Manager was actually enabled by confirming that ***/var/lib/kubelet/device-plugins/kubelet.sock*** is created on the node. This is the UNIX domain socket on which the Device Manager gRPC server listens for new plugin registrations. This sock file is created when the Kubelet is started only if Device Manager is enabled.

5.6. TAINTS AND TOLERATIONS

Understand and work with taints and tolerations.

5.6.1. Understanding taints and tolerations

You can use a *taint* on a node to allow that node to refuse a pod to be scheduled unless that pod has a matching *toleration*.

You apply taints to a node through the **Node** specification (**NodeSpec**) and apply tolerations to a pod through the **Pod** specification (**PodSpec**). When you apply a taint to a node, the scheduler cannot place a pod on that node unless the pod can tolerate the taint.

Example taint in a node specification

```
apiVersion: v1
kind: Node
metadata:
  name: my-node
#...
spec:
  taints:
  - effect: NoExecute
    key: key1
    value: value1
#...
```

Example toleration in a Pod spec

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
  - key: "key1"
    operator: "Equal"
    value: "value1"
    effect: "NoExecute"
    tolerationSeconds: 3600
#...
```

Taints and tolerations consist of a key, value, and effect.

Table 5.1. Taint and toleration components

Parameter	Description
key	The key is any string, up to 253 characters. The key must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.
value	The value is any string, up to 63 characters. The value must begin with a letter or number, and may contain letters, numbers, hyphens, dots, and underscores.

Parameter	Description						
effect	The effect is one of the following: <table> <tr> <td>NoSchedule ^[1]</td><td> - New pods that do not match the taint are not scheduled onto that node. - Existing pods on the node remain. </td></tr> <tr> <td>PreferNoSchedule</td><td> - New pods that do not match the taint might be scheduled onto that node, but the scheduler tries not to. - Existing pods on the node remain. </td></tr> <tr> <td>NoExecute</td><td> - New pods that do not match the taint cannot be scheduled onto that node. - Existing pods on the node that do not have a matching toleration are removed. </td></tr> </table>	NoSchedule ^[1]	- New pods that do not match the taint are not scheduled onto that node. - Existing pods on the node remain.	PreferNoSchedule	- New pods that do not match the taint might be scheduled onto that node, but the scheduler tries not to. - Existing pods on the node remain.	NoExecute	- New pods that do not match the taint cannot be scheduled onto that node. - Existing pods on the node that do not have a matching toleration are removed.
NoSchedule ^[1]	- New pods that do not match the taint are not scheduled onto that node. - Existing pods on the node remain.						
PreferNoSchedule	- New pods that do not match the taint might be scheduled onto that node, but the scheduler tries not to. - Existing pods on the node remain.						
NoExecute	- New pods that do not match the taint cannot be scheduled onto that node. - Existing pods on the node that do not have a matching toleration are removed.						
operator	<table> <tr> <td>Equal</td><td>The key/value/effect parameters must match. This is the default.</td></tr> <tr> <td>Exists</td><td>The key/effect parameters must match. You must leave a blank value parameter, which matches any.</td></tr> </table>	Equal	The key/value/effect parameters must match. This is the default.	Exists	The key/effect parameters must match. You must leave a blank value parameter, which matches any.		
Equal	The key/value/effect parameters must match. This is the default.						
Exists	The key/effect parameters must match. You must leave a blank value parameter, which matches any.						

1. If you add a **NoSchedule** taint to a control plane node, the node must have the **node-role.kubernetes.io/master=:NoSchedule** taint, which is added by default.

For example:

```

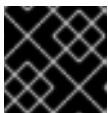
apiVersion: v1
kind: Node
metadata:
  annotations:
    machine.openshift.io/machine: openshift-machine-api/ci-ln-62s7gtb-f76d1-v8jxv-master-0
    machineconfiguration.openshift.io/currentConfig: rendered-master-
cdc1ab7da414629332cc4c3926e6e59c
    name: my-node
  #...
spec:
  taints:
    - effect: NoSchedule
      key: node-role.kubernetes.io/master
  #...
```

A toleration matches a taint:

- If the **operator** parameter is set to **Equal**:
 - the **key** parameters are the same;
 - the **value** parameters are the same;
 - the **effect** parameters are the same.
- If the **operator** parameter is set to **Exists**:
 - the **key** parameters are the same;
 - the **effect** parameters are the same.

The following taints are built into OpenShift Container Platform:

- **node.kubernetes.io/not-ready**: The node is not ready. This corresponds to the node condition **Ready=False**.
- **node.kubernetes.io/unreachable**: The node is unreachable from the node controller. This corresponds to the node condition **Ready=Unknown**.
- **node.kubernetes.io/memory-pressure**: The node has memory pressure issues. This corresponds to the node condition **MemoryPressure=True**.
- **node.kubernetes.io/disk-pressure**: The node has disk pressure issues. This corresponds to the node condition **DiskPressure=True**.
- **node.kubernetes.io/network-unavailable**: The node network is unavailable.
- **node.kubernetes.io/unschedulable**: The node is unschedulable.
- **node.cloudprovider.kubernetes.io/uninitialized**: When the node controller is started with an external cloud provider, this taint is set on a node to mark it as unusable. After a controller from the cloud-controller-manager initializes this node, the kubelet removes this taint.
- **node.kubernetes.io/pid-pressure**: The node has pid pressure. This corresponds to the node condition **PIDPressure=True**.



IMPORTANT

OpenShift Container Platform does not set a default pid.available **evictionHard**.

5.6.2. Adding taints and tolerations

You can add tolerations to pods and taints to nodes to allow the node to control which pods should or should not be scheduled on that node.

For existing pods and nodes, you should add the toleration to the pod first, then add the taint to the node to avoid pods being removed from the node before you can add the toleration.

Procedure

1. Add a toleration to a pod by editing the **Pod** spec to include a **tolerations** stanza:

Sample pod configuration file with an Equal operator

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
  - key: "key1"
    value: "value1"
    operator: "Equal"
    effect: "NoExecute"
    tolerationSeconds: 3600
#...
```

where:

spec.tolerations

Specifies the toleration parameters, as described in the **Taint and toleration components** table.

spec.tolerations.tolerationSeconds`

Specifies how long a pod can remain bound to a node before being evicted.

For example:

Sample pod configuration file with an Exists operator

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
  - key: "key1"
    operator: "Exists"
    effect: "NoExecute"
    tolerationSeconds: 3600
#...
```

The **Exists** operator does not take a **value**.

This example places a taint on **node1** that has key **key1**, value **value1**, and taint effect **NoExecute**.

2. Add a taint to a node by using the following command with the parameters described in the **Taint and toleration components** table:

```
$ oc adm taint nodes <node_name> <key>=<value>:<effect>
```

For example:

```
$ oc adm taint nodes node1 key1=value1:NoExecute
```

This command places a taint on **node1** that has key **key1**, value **value1**, and effect **NoExecute**.

NOTE

If you add a **NoSchedule** taint to a control plane node, the node must have the **node-role.kubernetes.io/master=:NoSchedule** taint, which is added by default.

For example:

```
apiVersion: v1
kind: Node
metadata:
  annotations:
    machine.openshift.io/machine: openshift-machine-api/ci-ln-62s7gtb-f76d1-
v8jxv-master-0
    machineconfiguration.openshift.io/currentConfig: rendered-master-
cdc1ab7da414629332cc4c3926e6e59c
  name: my-node
#...
spec:
  taints:
    - effect: NoSchedule
      key: node-role.kubernetes.io/master
#...
```

The tolerations on the pod match the taint on the node. A pod with either toleration can be scheduled onto **node1**.

5.6.3. Adding taints and tolerations using a compute machine set

You can add taints to groups of nodes by using a compute machine set. All nodes associated with the **MachineSet** object are updated with the taint.

Tolerations respond to taints added by a compute machine set in the same manner as taints added directly to the nodes.

Procedure

1. Add a toleration to a pod by editing the **Pod** spec to include a **tolerations** stanza:

Sample pod configuration file with **Equal** operator

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
```

```
- key: "key1"
  value: "value1"
  operator: "Equal"
  effect: "NoExecute"
  tolerationSeconds: 3600
#...
```

where:

spec.tolerations

Specifies the toleration parameters, as described in the **Taint and toleration components** table.

spec.tolerations.tolerationSeconds

Specifies how long a pod can remain bound to a node before being evicted.

For example:

Sample pod configuration file with Exists operator

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
    - key: "key1"
      operator: "Exists"
      effect: "NoExecute"
      tolerationSeconds: 3600
#...
```

2. Add the taint to the **MachineSet** object:

- a. Edit the **MachineSet** YAML for the nodes you want to taint or you can create a new **MachineSet** object:

```
$ oc edit machineset <machineset>
```

- b. Add the taint to the **spec.template.spec** section:

Example taint in a compute machine set specification

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: my-machineset
#...
spec:
  #...
  template:
    #...
    spec:
      taints:
```

```
- effect: NoExecute
  key: key1
  value: value1
#...
```

This example places a taint that has the key **key1**, value **value1**, and taint effect **NoExecute** on the nodes.

- c. Scale down the compute machine set to 0:

```
$ oc scale --replicas=0 machineset <machineset> -n openshift-machine-api
```

TIP

You can alternatively apply the following YAML to scale the compute machine set:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  name: <machineset>
  namespace: openshift-machine-api
spec:
  replicas: 0
```

Wait for the machines to be removed.

- d. Scale up the compute machine set as needed:

```
$ oc scale --replicas=2 machineset <machineset> -n openshift-machine-api
```

Or:

```
$ oc edit machineset <machineset> -n openshift-machine-api
```

Wait for the machines to start. The taint is added to the nodes associated with the **MachineSet** object.

5.6.4. Binding a user to a node using taints and tolerations

You can use taints and tolerations to dedicate a set of nodes for exclusive use by a particular set of users.

First, add a toleration to their pods. Then, add a corresponding taint to those nodes. The pods with the tolerations are allowed to use the tainted nodes or any other nodes in the cluster.

If you want ensure the pods are scheduled to only those tainted nodes, also add a label to the same set of nodes and add a node affinity to the pods so that the pods can only be scheduled onto nodes with that label.

Use the following procedure to configure a node so that users can use only that node.

Procedure

1. Add a corresponding taint to those nodes:

For example:

```
$ oc adm taint nodes node1 dedicated=groupName:NoSchedule
```

TIP

You can alternatively apply the following YAML to add the taint:

```
kind: Node
apiVersion: v1
metadata:
  name: my-node
#...
spec:
  taints:
    - key: dedicated
      value: groupName
      effect: NoSchedule
#...
```

2. Add a toleration to the pods by writing a custom admission controller.

5.6.5. Controlling nodes with special hardware using taints and tolerations

In a cluster that has specialized hardware, you can use taints and tolerations to either keep pods that do not need the specialized hardware off of those nodes or require pods that need specialized hardware to use specific nodes.

You can achieve this by adding a toleration to pods that need the special hardware and tainting the nodes that have the specialized hardware.

Use the following procedure to ensure nodes with specialized hardware are reserved for specific pods.

Procedure

1. Add a toleration to pods that need the special hardware.

For example:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
#...
spec:
  tolerations:
    - key: "disktype"
      value: "ssd"
      operator: "Equal"
      effect: "NoSchedule"
      tolerationSeconds: 3600
#...
```

2. Taint the nodes that have the specialized hardware using one of the following commands:

```
$ oc adm taint nodes <node-name> disktype=ssd:NoSchedule
```

Or:

```
$ oc adm taint nodes <node-name> disktype=ssd:PreferNoSchedule
```

TIP

You can alternatively apply the following YAML to add the taint:

```
kind: Node
apiVersion: v1
metadata:
  name: my_node
#...
spec:
  taints:
    - key: disktype
      value: ssd
      effect: PreferNoSchedule
#...
```

5.6.6. Removing taints and tolerations

You can remove taints from nodes and tolerations from pods as needed if you no longer want the scheduling behavior.

You should add the toleration to the pod first, then add the taint to the node to avoid pods being removed from the node before you can add the toleration.

Use the following procedure to remove taints and tolerations.

Procedure

1. To remove a taint from a node:

```
$ oc adm taint nodes <node-name> <key>-
```

For example:

```
$ oc adm taint nodes ip-10-0-132-248.ec2.internal key1-
```

Example output

```
node/ip-10-0-132-248.ec2.internal untainted
```

2. To remove a toleration from a pod, edit the **Pod** spec to remove the toleration:

```
apiVersion: v1
kind: Pod
```



```

metadata:
  name: my-pod
#...
spec:
  tolerations:
    - key: "key2"
      operator: "Exists"
      effect: "NoExecute"
      tolerationSeconds: 3600
#...

```

5.7. TOPOLOGY MANAGER

Understand and work with Topology Manager.

5.7.1. Topology Manager policies

Topology Manager aligns **Pod** resources of all Quality of Service (QoS) classes by collecting topology hints from Hint Providers, such as CPU Manager and Device Manager, and using the collected hints to align the **Pod** resources.

Topology Manager supports four allocation policies, which you assign in the **KubeletConfig** custom resource (CR) named **cpumanager-enabled**:

none policy

This is the default policy and does not perform any topology alignment.

best-effort policy

For each container in a pod with the **best-effort** topology management policy, kubelet tries to align all the required resources on a NUMA node according to the preferred NUMA node affinity for that container. Even if the allocation is not possible due to insufficient resources, the Topology Manager still admits the pod but the allocation is shared with other NUMA nodes.

restricted policy

For each container in a pod with the **restricted** topology management policy, kubelet determines the theoretical minimum number of NUMA nodes that can fulfill the request. If the actual allocation requires more than the that number of NUMA nodes, the Topology Manager rejects the admission, placing the pod in a **Terminated** state. If the number of NUMA nodes can fulfill the request, the Topology Manager admits the pod and the pod starts running.

single-numa-node policy

For each container in a pod with the **single-numa-node** topology management policy, kubelet admits the pod if all the resources required by the pod can be allocated on the same NUMA node. If a single NUMA node affinity is not possible, the Topology Manager rejects the pod from the node. This results in a pod in a **Terminated** state with a pod admission failure.

5.7.2. Setting up Topology Manager

To use Topology Manager, you must configure an allocation policy in the **KubeletConfig** custom resource (CR) named **cpumanager-enabled**. This file might exist if you have set up CPU Manager. If the file does not exist, you can create the file.

Prerequisites

- Configure the CPU Manager policy to be **static**.

Procedure

1. To activate Topology Manager, configure the Topology Manager allocation policy in the custom resource.

```
$ oc edit KubeletConfig cpumanager-enabled
```

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: cpumanager-enabled
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet: cpumanager-enabled
  kubeletConfig:
    cpuManagerPolicy: static
    cpuManagerReconcilePeriod: 5s
    topologyManagerPolicy: single-numa-node
```

- **cpuManagerPolicy** must be **static** with a lowercase **s**.
- **topologyManagerPolicy** specifies your selected Topology Manager allocation policy. In this example, the policy is **single-numa-node**. Acceptable values are: **default**, **best-effort**, **restricted**, **single-numa-node**.

5.7.3. Pod interactions with Topology Manager policies

The example **Pod** specs illustrate pod interactions with Topology Manager.

The following pod runs in the **BestEffort** QoS class because no resource requests or limits are specified.

```
spec:
  containers:
  - name: nginx
    image: nginx
```

The next pod runs in the **Burstable** QoS class because requests are less than limits.

```
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
      requests:
        memory: "100Mi"
```

If the selected policy is anything other than **none**, Topology Manager would process all the pods and it enforces resource alignment only for the **Guaranteed** QoS **Pod** specification. When the Topology

Manager policy is set to **none**, the relevant containers are pinned to any available CPU without considering NUMA affinity. This is the default behavior and it does not optimize for performance-sensitive workloads. Other values enable the use of topology awareness information from device plugins core resources, such as CPU and memory. The Topology Manager attempts to align the CPU, memory, and device allocations according to the topology of the node when the policy is set to other values than **none**. For more information about the available values, see *Topology Manager policies*.

The following example pod runs in the **Guaranteed** QoS class because requests are equal to limits.

```
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      limits:
        memory: "200Mi"
        cpu: "2"
        example.com/device: "1"
      requests:
        memory: "200Mi"
        cpu: "2"
        example.com/device: "1"
```

Topology Manager would consider this pod. The Topology Manager would consult the Hint Providers, which are the CPU Manager, the Device Manager, and the Memory Manager, to get topology hints for the pod.

Topology Manager will use this information to store the best topology for this container. In the case of this pod, CPU Manager and Device Manager will use this stored information at the resource allocation stage.

5.8. RESOURCE REQUESTS AND OVERCOMMITMENT

You can use resource requests in an overcommitted environment help you ensure that your cluster is properly configured.

For each compute resource, a container can specify a resource request and limit. Scheduling decisions are made based on the request to ensure that a node has enough capacity available to meet the requested value. If a container specifies limits, but omits requests, the requests are defaulted to the limits. A container is not able to exceed the specified limit on the node.

The enforcement of limits is dependent upon the compute resource type. If a container makes no request or limit, the container is scheduled to a node with no resource guarantees. In practice, the container is able to consume as much of the specified resource as is available with the lowest local priority. In low resource situations, containers that specify no resource requests are given the lowest quality of service.

Scheduling is based on resources requested, where quota and hard limits refer to resource limits, which can be set higher than requested resources. The difference between the request and the limit determines the level of overcommit. For example, if a container is given a memory request of 1Gi and a memory limit of 2Gi, the container is scheduled based on the 1Gi request being available on the node, but could use up to 2Gi; so it is 100% overcommitted.

5.9. CLUSTER-LEVEL OVERCOMMIT USING THE CLUSTER RESOURCE OVERRIDE OPERATOR

You can use the Cluster Resource Override Operator to control the level of overcommit and manage container density across all the nodes in your cluster. The Operator, which is an admission webhook, controls how nodes in specific projects can exceed defined memory and CPU limits.

The Operator modifies the ratio between the requests and limits that are set on developer containers. In conjunction with a per-project limit range that specifies limits and defaults, you can achieve the desired level of overcommit.

You must install the Cluster Resource Override Operator by using the OpenShift Container Platform console or CLI as shown in the following sections. After you deploy the Cluster Resource Override Operator, the Operator modifies all new pods in specific namespaces. The Operator does not edit pods that existed before you deployed the Operator.

During the installation, you create a **ClusterResourceOverride** custom resource (CR), where you set the level of overcommit, as shown in the following example:

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  name: cluster
spec:
  podResourceOverride:
    spec:
      memoryRequestToLimitPercent: 50
      cpuRequestToLimitPercent: 25
      limitCPUToMemoryPercent: 200
# ...
```

where:

metadata.name

Specifies a name for the object. The name must be **cluster**.

spec.podResourceOverride.spec.memoryRequestToLimitPercent

If a container memory limit has been specified or defaulted, the memory request is overridden to this percentage of the limit, between 1-100. The default is 50.

spec.podResourceOverride.spec.cpuRequestToLimitPercent

If a container CPU limit has been specified or defaulted, the CPU request is overridden to this percentage of the limit, between 1-100. The default is 25.

spec.podResourceOverride.spec.limitCPUToMemoryPercent

If a container memory limit has been specified or defaulted, the CPU limit is overridden to a percentage of the memory limit, if specified. Scaling 1Gi of RAM at 100 percent is equal to 1 CPU core. This is processed before overriding the CPU request (if configured). The default is 200.



NOTE

The Cluster Resource Override Operator overrides have no effect if limits have not been set on containers. Create a **LimitRange** object with default limits per individual project or configure limits in **Pod** specs for the overrides to apply.

When configured, you can enable overrides on a per-project basis by applying the following label to the **Namespace** object for each project where you want the overrides to apply. For example, you can configure override so that infrastructure components are not subject to the overrides.

```
apiVersion: v1
kind: Namespace
metadata:

# ...

labels:
  clusterresourceoverrides.admission.autoscaling.openshift.io/enabled: "true"

# ...
```

The Operator watches for the **ClusterResourceOverride** CR and ensures that the **ClusterResourceOverride** admission webhook is installed into the same namespace as the operator.

For example, a pod has the following resources limits:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
# ...
spec:
  containers:
    - name: hello-openshift
      image: openshift/hello-openshift
      resources:
        limits:
          memory: "512Mi"
          cpu: "2000m"
# ...
```

The Cluster Resource Override Operator intercepts the original pod request, then overrides the resources according to the configuration set in the **ClusterResourceOverride** object.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: my-namespace
# ...
spec:
  containers:
    - image: openshift/hello-openshift
      name: hello-openshift
      resources:
        limits:
          cpu: "1"
          memory: 512Mi
        requests:
```

```
cpu: 250m
memory: 256Mi
# ...
```

where:

spec.containers.resources.limits.cpu

Specifies that the CPU limit has been overridden to **1** because the **limitCPUToMemoryPercent** parameter is set to **200** in the **ClusterResourceOverride** object. As such, 200% of the memory limit, 512Mi in CPU terms, is 1 CPU core.

spec.containers.resources.memory.cpu

Specifies that the CPU request is now **250m** because the **cpuRequestToLimit** is set to **25** in the **ClusterResourceOverride** object. As such, 25% of the 1 CPU core is 250m.

5.9.1. Installing the Cluster Resource Override Operator using the web console

You can use the OpenShift Container Platform web console to install the Cluster Resource Override Operator to help you control overcommit in your cluster.

By default, the installation process creates a Cluster Resource Override Operator pod on a worker node in the **clusterresourceoverride-operator** namespace. You can move this pod to another node, such as an infrastructure node, as needed. Infrastructure nodes are not counted toward the total number of subscriptions that are required to run the environment. For more information, see "Moving the Cluster Resource Override Operator pods".

Prerequisites

- The Cluster Resource Override Operator has no effect if limits have not been set on containers. You must specify default limits for a project using a **LimitRange** object or configure limits in **Pod** specs for the overrides to apply.

Procedure

1. In the OpenShift Container Platform web console, navigate to **Home → Projects**
 - a. Click **Create Project**.
 - b. Specify **clusterresourceoverride-operator** as the name of the project.
 - c. Click **Create**.
2. Navigate to **Ecosystem → Software Catalog**.
 - a. Choose **ClusterResourceOverride Operator** from the list of available Operators and click **Install**.
 - b. On the **Install Operator** page, make sure **A specific Namespace on the cluster** is selected for **Installation Mode**.
 - c. Make sure **clusterresourceoverride-operator** is selected for **Installed Namespace**.
 - d. Select an **Update Channel** and **Approval Strategy**.
 - e. Click **Install**.

3. On the **Installed Operators** page, click **ClusterResourceOverride**.
 - a. On the **ClusterResourceOverride Operator** details page, click **Create ClusterResourceOverride**.
 - b. On the **Create ClusterResourceOverride** page, click **YAML view** and edit the YAML template to set the overcommit values as needed:

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  name: cluster
spec:
  podResourceOverride:
    spec:
      memoryRequestToLimitPercent: 50
      cpuRequestToLimitPercent: 25
      limitCPUToMemoryPercent: 200
```

where:

metadata.name

Specifies a name for the CR. The name must be **cluster**.

spec.podResourceOverride.spec.memoryRequestToLimitPercent

Specifies the percentage to override the container memory limit, if used, between 1-100. The default is **50**. This parameter is optional.

spec.podResourceOverride.spec.cpuRequestToLimitPercent

Specifies the percentage to override the container CPU limit, if used, between 1-100. The default is **25**. This parameter is optional.

spec.podResourceOverride.spec.limitCPUToMemoryPercent

Specifies the percentage to override the container memory limit, if used. Scaling 1 Gi of RAM at 100 percent is equal to 1 CPU core. This is processed before overriding the CPU request, if configured. The default is **200**. This parameter is optional.

- c. Click **Create**.
4. Check the current state of the admission webhook by checking the status of the cluster custom resource:
 - a. On the **ClusterResourceOverride Operator** page, click **cluster**.
 - b. On the **ClusterResourceOverride Details** page, click **YAML**. The **mutatingWebhookConfigurationRef** section displays when the webhook is called.

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  annotations:
    kubectrl.kubernetes.io/last-applied-configuration: |

{"apiVersion":"operator.autoscaling.openshift.io/v1","kind":"ClusterResourceOverride","met
adata":{"annotations":{},"name":"cluster"},"spec":{"podResourceOverride":{"spec":
{"cpuRequestToLimitPercent":25,"limitCPUToMemoryPercent":200,"memoryRequestToLi
```

```

    mitPercent":50}}}}
    creationTimestamp: "2019-12-18T22:35:02Z"
    generation: 1
    name: cluster
    resourceVersion: "127622"
    selfLink: /apis/operator.autoscaling.openshift.io/v1/clusterresourceoverrides/cluster
    uid: 978fc959-1717-4bd1-97d0-ae00ee111e8d
  spec:
    podResourceOverride:
      spec:
        cpuRequestToLimitPercent: 25
        limitCPUToMemoryPercent: 200
        memoryRequestToLimitPercent: 50
  status:

# ...

  mutatingWebhookConfigurationRef:
    apiVersion: admissionregistration.k8s.io/v1
    kind: MutatingWebhookConfiguration
    name: clusterresourceoverrides.admission.autoscaling.openshift.io
    resourceVersion: "127621"
    uid: 98b3b8ae-d5ce-462b-8ab5-a729ea8f38f3

# ...

```

where:

status.mutatingWebhookConfigurationRef

Specifies the **ClusterResourceOverride** admission webhook.

5.9.2. Installing the Cluster Resource Override Operator using the CLI

You can use the OpenShift CLI to install the Cluster Resource Override Operator to help you control overcommit in your cluster.

By default, the installation process creates a Cluster Resource Override Operator pod on a worker node in the **clusterresourceoverride-operator** namespace. You can move this pod to another node, such as an infrastructure node, as needed. Infrastructure nodes are not counted toward the total number of subscriptions that are required to run the environment. For more information, see "Moving the Cluster Resource Override Operator pods".

Prerequisites

- The Cluster Resource Override Operator has no effect if limits have not been set on containers. You must specify default limits for a project using a **LimitRange** object or configure limits in **Pod** specs for the overrides to apply.

Procedure

1. Create a namespace for the Cluster Resource Override Operator:
 - a. Create a **Namespace** object YAML file (for example, **cro-namespace.yaml**) for the Cluster Resource Override Operator:


```
apiVersion: v1
kind: Namespace
metadata:
  name: clusterresourceoverride-operator
```

- b. Create the namespace:

```
$ oc create -f <file-name>.yaml
```

For example:

```
$ oc create -f cro-namespace.yaml
```

2. Create an Operator group:

- a. Create an **OperatorGroup** object YAML file (for example, cro-og.yaml) for the Cluster Resource Override Operator:

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: clusterresourceoverride-operator
  namespace: clusterresourceoverride-operator
spec:
  targetNamespaces:
    - clusterresourceoverride-operator
```

- b. Create the Operator Group:

```
$ oc create -f <file-name>.yaml
```

For example:

```
$ oc create -f cro-og.yaml
```

3. Create a subscription:

- a. Create a **Subscription** object YAML file (for example, cro-sub.yaml) for the Cluster Resource Override Operator:

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: clusterresourceoverride
  namespace: clusterresourceoverride-operator
spec:
  channel: "stable"
  name: clusterresourceoverride
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

- b. Create the subscription:

```
$ oc create -f <file-name>.yaml
```

For example:

```
$ oc create -f cro-sub.yaml
```

4. Create a **ClusterResourceOverride** custom resource (CR) object in the **clusterresourceoverride-operator** namespace:

- a. Change to the **clusterresourceoverride-operator** namespace.

```
$ oc project clusterresourceoverride-operator
```

- b. Create a **ClusterResourceOverride** object YAML file (for example, cro-cr.yaml) for the Cluster Resource Override Operator:

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  name: cluster
spec:
  podResourceOverride:
    spec:
      memoryRequestToLimitPercent: 50
      cpuRequestToLimitPercent: 25
      limitCPUToMemoryPercent: 200
```

where

metadata.name

Specifies a name for the CR. The name must be **cluster**.

spec.podResourceOverride.spec.memoryRequestToLimitPercent

Specifies the percentage to override the container memory limit, if used, between 1-100. The default is **50**. This parameter is optional.

spec.podResourceOverride.spec.cpuRequestToLimitPercent

Specifies the percentage to override the container CPU limit, if used, between 1-100. The default is **25**. This parameter is optional.

spec.podResourceOverride.spec.limitCPUToMemoryPercent

Specifies the percentage to override the container memory limit, if used. Scaling 1 Gi of RAM at 100 percent is equal to 1 CPU core. This is processed before overriding the CPU request, if configured. The default is **200**. This parameter is optional.

- c. Create the **ClusterResourceOverride** object:

```
$ oc create -f <file-name>.yaml
```

For example:

```
$ oc create -f cro-cr.yaml
```

5. Verify the current state of the admission webhook by checking the status of the cluster custom resource.

```
$ oc get clusterresourceoverride cluster -n clusterresourceoverride-operator -o yaml
```

The **mutatingWebhookConfigurationRef** section displays when the webhook is called.

Example output

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |

{"apiVersion":"operator.autoscaling.openshift.io/v1","kind":"ClusterResourceOverride","metadat
a":{"annotations":{},"name":"cluster"},"spec":{"podResourceOverride":{"spec":
{"cpuRequestToLimitPercent":25,"limitCPUToMemoryPercent":200,"memoryRequestToLimitPe
rcent":50}}}}
creationTimestamp: "2019-12-18T22:35:02Z"
generation: 1
name: cluster
resourceVersion: "127622"
selfLink: /apis/operator.autoscaling.openshift.io/v1/clusterresourceoverrides/cluster
uid: 978fc959-1717-4bd1-97d0-ae00ee111e8d
spec:
  podResourceOverride:
    spec:
      cpuRequestToLimitPercent: 25
      limitCPUToMemoryPercent: 200
      memoryRequestToLimitPercent: 50
status:

# ...

mutatingWebhookConfigurationRef:
  apiVersion: admissionregistration.k8s.io/v1
  kind: MutatingWebhookConfiguration
  name: clusterresourceoverrides.admission.autoscaling.openshift.io
  resourceVersion: "127621"
  uid: 98b3b8ae-d5ce-462b-8ab5-a729ea8f38f3

# ...
```

where:

status.mutatingWebhookConfigurationRef

Specifies the **ClusterResourceOverride** admission webhook.

5.9.3. Configuring cluster-level overcommit

You can use the OpenShift CLI to configure the Cluster Resource Override Operator to help control overcommit in your cluster.

The Cluster Resource Override Operator requires a **ClusterResourceOverride** custom resource (CR) and a label for each project where you want the Operator to control overcommit.

By default, the installation process creates two Cluster Resource Override pods on the control plane nodes in the **clusterresourceoverride-operator** namespace. You can move these pods to other nodes, such as infrastructure nodes, as needed. Infrastructure nodes are not counted toward the total number of subscriptions that are required to run the environment. For more information, see "Moving the Cluster Resource Override Operator pods".

Prerequisites

- The Cluster Resource Override Operator has no effect if limits have not been set on containers. You must specify default limits for a project using a **LimitRange** object or configure limits in **Pod** specs for the overrides to apply.

Procedure

1. Edit the **ClusterResourceOverride** CR:

```
apiVersion: operator.autoscaling.openshift.io/v1
kind: ClusterResourceOverride
metadata:
  name: cluster
spec:
  podResourceOverride:
    spec:
      memoryRequestToLimitPercent: 50
      cpuRequestToLimitPercent: 25
      limitCPUToMemoryPercent: 200
# ...
```

where:

spec.podResourceOverride.spec.memoryRequestToLimitPercent

Specifies the percentage to override the container memory limit, if used, between 1-100. The default is **50**. This parameter is optional.

spec.podResourceOverride.spec.cpuRequestToLimitPercent

Specifies the percentage to override the container CPU limit, if used, between 1-100. The default is **25**. This parameter is optional.

spec.podResourceOverride.spec.limitCPUToMemoryPercent

Specifies the percentage to override the container memory limit, if used. Scaling 1Gi of RAM at 100 percent is equal to 1 CPU core. This is processed before overriding the CPU request, if configured. The default is **200**. This parameter is optional.

2. Ensure the following label has been added to the Namespace object for each project where you want the Cluster Resource Override Operator to control overcommit:

```
apiVersion: v1
kind: Namespace
metadata:
# ...
```

```
labels:
  clusterresourceoverrides.admission.autoscaling.openshift.io/enabled: "true"

# ...
```

where:

metadata.labels.clusterresourceoverrides.admission.autoscaling.openshift.io/enabled: "true"

Specifies that you want to use the Cluster Resource Override Operator with this project.

5.10. NODE-LEVEL OVERCOMMIT

You can use various ways to control overcommit on specific nodes, such as quality of service (QoS) guarantees, CPU limits, or reserve resources. You can also disable overcommit for specific nodes and specific projects.

5.10.1. Understanding container CPU and memory requests

Review the following information to learn about container CPU and memory requests to help you ensure that your cluster is properly configured.

A container is guaranteed the amount of CPU it requests and is additionally able to consume excess CPU available on the node, up to any limit specified by the container. If multiple containers are attempting to use excess CPU, CPU time is distributed based on the amount of CPU requested by each container.

For example, if one container requested 500m of CPU time and another container requested 250m of CPU time, any extra CPU time available on the node is distributed among the containers in a 2:1 ratio. If a container specified a limit, it will be throttled not to use more CPU than the specified limit. CPU requests are enforced using the CFS shares support in the Linux kernel. By default, CPU limits are enforced using the CFS quota support in the Linux kernel over a 100ms measuring interval, though this can be disabled.

A container is guaranteed the amount of memory it requests. A container can use more memory than requested, but once it exceeds its requested amount, it could be terminated in a low memory situation on the node. If a container uses less memory than requested, it will not be terminated unless system tasks or daemons need more memory than was accounted for in the node's resource reservation. If a container specifies a limit on memory, it is immediately terminated if it exceeds the limit amount.

5.10.2. Understanding overcommitment and quality of service classes

You can use Quality of Service (QoS) classes in an overcommitted environment to help you ensure that your cluster is properly configured.

A node is *overcommitted* when it has a pod scheduled that makes no request, or when the sum of limits across all pods on that node exceeds available machine capacity.

In an overcommitted environment, the pods on the node might attempt to use more compute resource than is available at any given point in time. When this occurs, the node must give priority to one pod over another. The facility used to make this decision is referred to as a Quality of Service (QoS) class.

A pod is designated as one of three QoS classes with decreasing order of priority:

Table 5.2. Quality of Service classes

Priority	Class Name	Description
1 (highest)	Guaranteed	If limits and optionally requests are set (not equal to 0) for all resources and they are equal, then the pod is classified as Guaranteed .
2	Burstable	If requests and optionally limits are set (not equal to 0) for all resources, and they are not equal, then the pod is classified as Burstable .
3 (lowest)	BestEffort	If requests and limits are not set for any of the resources, then the pod is classified as BestEffort .

Memory is an incompressible resource, so in low memory situations, containers that have the lowest priority are terminated first:

- **Guaranteed** containers are considered top priority, and are guaranteed to only be terminated if they exceed their limits, or if the system is under memory pressure and there are no lower priority containers that can be evicted.
- **Burstable** containers under system memory pressure are more likely to be terminated when they exceed their requests and no other **BestEffort** containers exist.
- **BestEffort** containers are treated with the lowest priority. Processes in these containers are first to be terminated if the system runs out of memory.

5.10.2.1. Understanding how to reserve memory across quality of service tiers

You can use the **qos-reserved** parameter to specify a percentage of memory to be reserved by a pod in a particular QoS level. This feature attempts to reserve requested resources to exclude pods from lower QoS classes from using resources requested by pods in higher QoS classes.

OpenShift Container Platform uses the **qos-reserved** parameter as follows:

- A value of **qos-reserved=memory=100%** prevents the **Burstable** and **BestEffort** QoS classes from consuming memory that was requested by a higher QoS class. This increases the risk of inducing OOM on **BestEffort** and **Burstable** workloads in favor of increasing memory resource guarantees for **Guaranteed** and **Burstable** workloads.
- A value of **qos-reserved=memory=50%** allows the **Burstable** and **BestEffort** QoS classes to consume half of the memory requested by a higher QoS class.
- A value of **qos-reserved=memory=0%** allows a **Burstable** and **BestEffort** QoS classes to consume up to the full node allocatable amount if available, but increases the risk that a **Guaranteed** workload does not have access to requested memory. This condition effectively disables this feature.

5.10.3. Understanding swap memory and QoS

Review the following information to learn how swap memory and QoS interact in an overcommitted environment to help you ensure that your cluster is properly configured.

You can disable swap by default on your nodes to preserve quality of service (QoS) guarantees. Otherwise, physical resources on a node can oversubscribe, affecting the resource guarantees the Kubernetes scheduler makes during pod placement.

For example, if two guaranteed pods have reached their memory limit, each container could start using swap memory. Eventually, if there is not enough swap space, processes in the pods can be terminated due to the system being oversubscribed.

Failing to disable swap causes nodes to not recognize that they are experiencing **MemoryPressure**, resulting in pods not receiving the memory they made in their scheduling request. As a result, additional pods are placed on the node to further increase memory pressure, ultimately increasing your risk of experiencing a system out of memory (OOM) event.



IMPORTANT

If swap is enabled, any out-of-resource handling eviction thresholds for available memory will not work as expected. Out-of-resource handling allows pods to be evicted from a node when it is under memory pressure, and rescheduled on an alternative node that has no such pressure.

5.10.4. Understanding nodes overcommitment

To maintain optimal system performance and stability in an overcommitted environment in OpenShift Container Platform, configure your nodes to manage resource contention effectively.

When the node starts, it ensures that the kernel tunable flags for memory management are set properly. The kernel should never fail memory allocations unless it runs out of physical memory.

To ensure this behavior, OpenShift Container Platform configures the kernel to always overcommit memory by setting the **vm.overcommit_memory** parameter to **1**, overriding the default operating system setting.

OpenShift Container Platform also configures the kernel to not panic when it runs out of memory by setting the **vm.panic_on_oom** parameter to **0**. A setting of 0 instructs the kernel to call the OOM killer in an Out of Memory (OOM) condition, which kills processes based on priority.

You can view the current setting by running the following commands on your nodes:

```
$ sysctl -a |grep commit
```

Example output

```
#...
vm.overcommit_memory = 0
#...
```

```
$ sysctl -a |grep panic
```

Example output

```
#...
vm.panic_on_oom = 0
#...
```



NOTE

The previous commands should already be set on nodes, so no further action is required.

You can also perform the following configurations for each node:

- Disable or enforce CPU limits using CPU CFS quotas
- Reserve resources for system processes
- Reserve memory across quality of service tiers

Additional resources

- [Disabling or enforcing CPU limits using CPU CFS quotas](#)
- [Reserving resources for system processes](#)
- [Understanding how to reserve memory across quality of service tiers](#)

5.10.5. Disabling or enforcing CPU limits using CPU CFS quotas

You can disable the default enforcement of CPU limits for nodes in a machine config pool.

By default, nodes enforce specified CPU limits using the Completely Fair Scheduler (CFS) quota support in the Linux kernel.

If you disable CPU limit enforcement, it is important to understand the impact on your node:

- If a container has a CPU request, the request continues to be enforced by CFS shares in the Linux kernel.
- If a container does not have a CPU request, but does have a CPU limit, the CPU request defaults to the specified CPU limit, and is enforced by CFS shares in the Linux kernel.
- If a container has both a CPU request and limit, the CPU request is enforced by CFS shares in the Linux kernel, and the CPU limit has no impact on the node.

Prerequisites

- You have the label associated with the static **MachineConfigPool** CRD for the type of node you want to configure.

Procedure

1. Create a custom resource (CR) for your configuration change.

Sample configuration for a disabling CPU limits

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: disable-cpu-units
spec:
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
  kubeletConfig:
    cpuCfsQuota: false
```


where:

metadata.name

Specifies a name for the CR.

spec.machineConfigPoolSelector.matchLabels

Specifies the label from the machine config pool.

spec.kubeletConfig.cpuCfsQuota

Specifies the **cpuCfsQuota** parameter to **false**.

2. Run the following command to create the CR:

```
$ oc create -f <file_name>.yaml
```

5.10.6. Reserving resources for system processes

You can explicitly reserve resources for non-pod processes by allocating node resources through specifying resources available for scheduling.

To provide more reliable scheduling and minimize node resource overcommitment, each node can reserve a portion of its resources for use by the system daemons that are required to run on your node for your cluster to function.



NOTE

It is recommended that you reserve resources for incompressible resources such as memory.

For more details, see "Allocating Resources for Nodes".

Additional resources

- [Allocating resources for nodes](#)

5.10.7. Disabling overcommitment for a node

When overcommitment is enabled on a node, you can disable overcommitment on that node. Disabling overcommit can help ensure predictability, stability, and high performance in your cluster.

Procedure

- Run the following command on a node to disable overcommitment on that node:

```
$ sysctl -w vm.overcommit_memory=0
```

5.11. PROJECT-LEVEL LIMITS

To help control overcommit, you can set per-project resource limit ranges, specifying memory and CPU limits and defaults for a project that overcommit cannot exceed.

For information on project-level resource limits, see the *Additional resources* section.

Alternatively, you can disable overcommitment for specific projects.

5.11.1. Disabling overcommitment for a project

If overcommitment is enabled on a project, you can disable overcommitment for that projects. This allows infrastructure components to be configured independently of overcommitment.

Procedure

1. Create or edit the namespace object file.
2. Add the following annotation:

```
apiVersion: v1
kind: Namespace
metadata:
  annotations:
    quota.openshift.io/cluster-resource-override-enabled: "false"
# ...
```

where:

metadata.annotations.quota.openshift.io/cluster-resource-override-enabled.false

Specifies that overcommit is disabled for this namespace.

5.12. FREEING NODE RESOURCES USING GARBAGE COLLECTION

Understand and use garbage collection.

5.12.1. Understanding how terminated containers are removed through garbage collection

You can help ensure that your nodes are running efficiently by using container garbage collection to remove terminated containers.

When eviction thresholds are set for garbage collection, the node tries to keep any container for any pod accessible from the API. If the pod has been deleted, the containers will be as well. Containers are preserved as long the pod is not deleted and the eviction threshold is not reached. If the node is under disk pressure, it will remove containers and their logs will no longer be accessible using **oc logs**.

- **eviction-soft** – A soft eviction threshold pairs an eviction threshold with a required administrator-specified grace period.
- **eviction-hard** – A hard eviction threshold has no grace period, and if observed, OpenShift Container Platform takes immediate action.

The following table lists the eviction thresholds:

Table 5.3. Variables for configuring container garbage collection

Node condition	Eviction signal	Description
----------------	-----------------	-------------

Node condition	Eviction signal	Description
MemoryPressure	memory.available	The available memory on the node.
DiskPressure	• nodefs.available • nodefs.inodesFree • imagefs.available • imagefs.inodesFree	The available disk space or inodes on the node root file system, nodefs , or image file system, imagefs .

**NOTE**

For **evictionHard** you must specify all of these parameters. If you do not specify all parameters, only the specified parameters are applied and the garbage collection will not function properly.

If a node is oscillating above and below a soft eviction threshold, but not exceeding its associated grace period, the corresponding node would constantly oscillate between **true** and **false**. As a consequence, the scheduler could make poor scheduling decisions.

To protect against this oscillation, use the **evictionpressure-transition-period** flag to control how long OpenShift Container Platform must wait before transitioning out of a pressure condition. OpenShift Container Platform will not set an eviction threshold as being met for the specified pressure condition for the period specified before toggling the condition back to false.

**NOTE**

Setting the **evictionPressureTransitionPeriod** parameter to **0** configures the default value of 5 minutes. You cannot set an eviction pressure transition period to zero seconds.

5.12.2. Understanding how images are removed through garbage collection

You can help ensure that your nodes are running efficiently by using image garbage collection to removes images that are not referenced by any running pods.

OpenShift Container Platform determines which images to remove from a node based on the disk usage that is reported by **cAdvisor**.

The policy for image garbage collection is based on two conditions:

- The percent of disk usage (expressed as an integer) which triggers image garbage collection. The default is **85**.
- The percent of disk usage (expressed as an integer) to which image garbage collection attempts to free. Default is **80**.

For image garbage collection, you can modify any of the following variables using a custom resource.

Table 5.4. Variables for configuring image garbage collection

Setting	Description
imageMinimumGCAge	The minimum age for an unused image before the image is removed by garbage collection. The default is 2m .
imageGCHighThresholdPercent	The percent of disk usage, expressed as an integer, which triggers image garbage collection. The default is 85 . This value must be greater than the imageGCLowThresholdPercent value.
imageGCLowThresholdPercent	The percent of disk usage, expressed as an integer, to which image garbage collection attempts to free. The default is 80 . This value must be less than the imageGCHighThresholdPercent value.

Two lists of images are retrieved in each garbage collector run:

1. A list of images currently running in at least one pod.
2. A list of images available on a host.

As new containers are run, new images appear. All images are marked with a time stamp. If the image is running (the first list above) or is newly detected (the second list above), it is marked with the current time. The remaining images are already marked from the previous spins. All images are then sorted by the time stamp.

Once the collection starts, the oldest images get deleted first until the stopping criterion is met.

5.12.3. Configuring garbage collection for containers and images

As an administrator, you can configure how OpenShift Container Platform performs garbage collection by creating a **kubeletConfig** object for each machine config pool. Performing garbage collection helps ensure that your nodes are running efficiently.



NOTE

OpenShift Container Platform supports only one **kubeletConfig** object for each machine config pool.

You can configure any combination of the following:

- Soft eviction for containers
- Hard eviction for containers
- Eviction for images

Container garbage collection removes terminated containers. Image garbage collection removes images that are not referenced by any running pods.

Prerequisites

1. Obtain the label associated with the static **MachineConfigPool** CRD for the type of node you want to configure by entering the following command:

```
$ oc edit machineconfigpool <name>
```

For example:

```
$ oc edit machineconfigpool worker
```

Example output

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  creationTimestamp: "2022-11-16T15:34:25Z"
  generation: 4
  labels:
    pools.operator.machineconfiguration.openshift.io/worker: ""
  name: worker
#...
```

where:

metadata.labels

Specifies a label to use with the kubelet configuration.

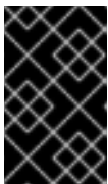
TIP

If the label is not present, add a key/value pair such as:

```
$ oc label machineconfigpool worker custom-kubelet=small-pods
```

Procedure

1. Create a custom resource (CR) for your configuration change.



IMPORTANT

If there is one file system, or if `/var/lib/kubelet` and `/var/lib/containers/` are in the same file system, the settings with the highest values trigger evictions, as those are met first. The file system triggers the eviction.

Sample configuration for a container garbage collection CR

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: worker-kubeconfig
spec:
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
  kubeletConfig:
```

```

evictionSoft:
  memory.available: "500Mi"
  nodefs.available: "10%"
  nodefs.inodesFree: "5%"
  imagefs.available: "15%"
  imagefs.inodesFree: "10%"
evictionSoftGracePeriod:
  memory.available: "1m30s"
  nodefs.available: "1m30s"
  nodefs.inodesFree: "1m30s"
  imagefs.available: "1m30s"
  imagefs.inodesFree: "1m30s"
evictionHard:
  memory.available: "200Mi"
  nodefs.available: "5%"
  nodefs.inodesFree: "4%"
  imagefs.available: "10%"
  imagefs.inodesFree: "5%"
evictionPressureTransitionPeriod: 3m
imageMinimumGCAge: 5m
imageGCHighThresholdPercent: 80
imageGCLowThresholdPercent: 75
#...

```

where:

metadata.name

Specifies a name for the object.

spec.machineConfigPoolSelector.matchLabels

Specifies the label from the machine config pool.

spec.kubeletConfig.evictionSoft

Specifies a soft eviction and eviction thresholds for container garbage collection.

spec.kubeletConfig.evictionSoftGracePeriod

Specifies a grace period for the soft eviction of containers. This parameter does not apply to **eviction-hard**.

spec.kubeletConfig.evictionHard

Specifies a soft eviction and eviction thresholds for container garbage collection. For **evictionHard** you must specify all of these parameters. If you do not specify all parameters, only the specified parameters are applied and the garbage collection will not function properly.

spec.kubeletConfig.evictionPressureTransitionPeriod

Specifies the duration to wait before transitioning out of an eviction pressure condition for container garbage collection. Setting the **evictionPressureTransitionPeriod** parameter to **0** configures the default value of 5 minutes.

spec.kubeletConfig.imageMinimumGCAge

Specifies the minimum age for an unused image before the image is removed by garbage collection.

spec.kubeletConfig.imageGCHighThresholdPercent

Specifies the percent of disk usage, expressed as an integer, that triggers image garbage collection. This value must be greater than the **imageGCLowThresholdPercent** value.

spec.kubeletConfig.imageGCHighThresholdPercent

Specifies the percent of disk usage, expressed as an integer, to which image garbage collection attempts to free. This value must be less than the **imageGCHighThresholdPercent** value.

2. Run the following command to create the CR:

```
$ oc create -f <file_name>.yaml
```

For example:

```
$ oc create -f gc-container.yaml
```

Example output

```
kubeletconfig.machineconfiguration.openshift.io/gc-container created
```

Verification

- Verify that garbage collection is active by entering the following command. The Machine Config Pool you specified in the custom resource appears with **UPDATING** as 'true' until the change is fully implemented:

```
$ oc get machineconfigpool
```

Example output

NAME	CONFIG	UPDATED	UPDATING
master	rendered-master-546383f80705bd5aeaba93	True	False
worker	rendered-worker-b4c51bb33ccaae6fc4a6a5	False	True

5.13. USING THE NODE TUNING OPERATOR

Understand and use the Node Tuning Operator.

The Node Tuning Operator helps you manage node-level tuning by orchestrating the TuneD daemon and achieves low latency performance by using the Performance Profile controller. The majority of high-performance applications require some level of kernel tuning. The Node Tuning Operator provides a unified management interface to users of node-level sysctls and more flexibility to add custom tuning specified by user needs.

The Operator manages the containerized TuneD daemon for OpenShift Container Platform as a Kubernetes daemon set. It ensures the custom tuning specification is passed to all containerized TuneD daemons running in the cluster in the format that the daemons understand. The daemons run on all nodes in the cluster, one per node.

Node-level settings applied by the containerized TuneD daemon are rolled back on an event that triggers a profile change or when the containerized TuneD daemon is terminated gracefully by receiving and handling a termination signal.

The Node Tuning Operator uses the Performance Profile controller to implement automatic tuning to achieve low latency performance for OpenShift Container Platform applications.

The cluster administrator configures a performance profile to define node-level settings such as the following:

- Updating the kernel to kernel-rt.
- Choosing CPUs for housekeeping.
- Choosing CPUs for running workloads.

The Node Tuning Operator is part of a standard OpenShift Container Platform installation in version 4.1 and later.



NOTE

In earlier versions of OpenShift Container Platform, the Performance Addon Operator was used to implement automatic tuning to achieve low latency performance for OpenShift applications. In OpenShift Container Platform 4.11 and later, this functionality is part of the Node Tuning Operator.

5.13.1. Accessing an example Node Tuning Operator specification

Use this process to access an example Node Tuning Operator specification.

Procedure

- Run the following command to access an example Node Tuning Operator specification:

```
oc get tuned.tuned.openshift.io/default -o yaml -n openshift-cluster-node-tuning-operator
```

The default CR is meant for delivering standard node-level tuning for the OpenShift Container Platform platform and it can only be modified to set the Operator Management state. Any other custom changes to the default CR will be overwritten by the Operator. For custom tuning, create your own Tuned CRs. Newly created CRs will be combined with the default CR and custom tuning applied to OpenShift Container Platform nodes based on node or pod labels and profile priorities.



WARNING

While in certain situations the support for pod labels can be a convenient way of automatically delivering required tuning, this practice is discouraged and strongly advised against, especially in large-scale clusters. The default Tuned CR ships without pod label matching. If a custom profile is created with pod label matching, then the functionality will be enabled at that time. The pod label functionality will be deprecated in future versions of the Node Tuning Operator.

5.13.2. Custom tuning specification

The custom resource (CR) for the Operator has two major sections. The first section, **profile:**, is a list of Tuned profiles and their names. The second, **recommend:**, defines the profile selection logic.

Multiple custom tuning specifications can co-exist as multiple CRs in the Operator's namespace. The existence of new CRs or the deletion of old CRs is detected by the Operator. All existing custom tuning specifications are merged and appropriate objects for the containerized TuneD daemons are updated.

Management state

The Operator Management state is set by adjusting the default Tuned CR. By default, the Operator is in the Managed state and the **spec.managementState** field is not present in the default Tuned CR. Valid values for the Operator Management state are as follows:

- Managed: the Operator will update its operands as configuration resources are updated
- Unmanaged: the Operator will ignore changes to the configuration resources
- Removed: the Operator will remove its operands and resources the Operator provisioned

Profile data

The **profile:** section lists TuneD profiles and their names.

```
profile:
- name: tuned_profile_1
  data: |
    # TuneD profile specification
    [main]
    summary=Description of tuned_profile_1 profile

    [sysctl]
    net.ipv4.ip_forward=1
    # ... other sysctl's or other TuneD daemon plugins supported by the containerized TuneD

# ...

- name: tuned_profile_n
  data: |
    # TuneD profile specification
    [main]
    summary=Description of tuned_profile_n profile

    # tuned_profile_n profile settings
```

Recommended profiles

The **profile:** selection logic is defined by the **recommend:** section of the CR. The **recommend:** section is a list of items to recommend the profiles based on a selection criteria.

```
recommend:
<recommend-item-1>
# ...
<recommend-item-n>
```

The individual items of the list:

```
- machineConfigLabels:
  <mcLabels>
```

```
match:
  <match>
priority: <priority>
profile: <tuned_profile_name>
operand:
  debug: <bool>
  tunedConfig:
    reapply_sysctl: <bool>
```

where:

machineConfigLabels

Optional.

<mcLabels>

A dictionary of key/value **MachineConfig** labels. The keys must be unique.

match

If omitted, profile match is assumed unless a profile with a higher priority matches first or **machineConfigLabels** is set.

<match>

An optional list.

<priority>

Profile ordering priority. Lower numbers mean higher priority (**0** is the highest priority).

<tuned_profile_name>

A TuneD profile to apply on a match. For example **tuned_profile_1**.

operand

Optional operand configuration.

debug

Turn debugging on or off for the TuneD daemon. Options are **true** for on or **false** for off. The default is **false**.

reapply_sysctl

Turn **reapply_sysctl** functionality on or off for the TuneD daemon. Options are **true** for on and **false** for off.

The **<match>** parameter is an optional list recursively defined as follows:

```
- label: <label_name>
  value: <label_value>
  type: <label_type>
  <match>
```

where:

<label_name>

Node or pod label name.

<label_value>

Optional node or pod label value. If omitted, the presence of **<label_name>** is enough to match.

<label_type>

Optional object type (**node** or **pod**). If omitted, **node** is assumed.

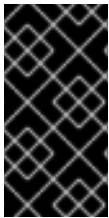
<match>

An optional **<match>** list.

If **<match>** is not omitted, all nested **<match>** sections must also evaluate to **true**. Otherwise, **false** is assumed and the profile with the respective **<match>** section will not be applied or recommended. Therefore, the nesting (child **<match>** sections) works as logical AND operator. Conversely, if any item of the **<match>** list matches, the entire **<match>** list evaluates to **true**. Therefore, the list acts as logical OR operator.

If **machineConfigLabels** is defined, machine config pool based matching is turned on for the given **recommend:** list item. **<mcLabels>** specifies the labels for a machine config. The machine config is created automatically to apply host settings, such as kernel boot parameters, for the profile **<tuned_profile_name>**. This involves finding all machine config pools with machine config selector matching **<mcLabels>** and setting the profile **<tuned_profile_name>** on all nodes that are assigned the found machine config pools. To target nodes that have both master and worker roles, you must use the master role.

The list items **match** and **machineConfigLabels** are connected by the logical OR operator. The **match** item is evaluated first in a short-circuit manner. Therefore, if it evaluates to **true**, the **machineConfigLabels** item is not considered.



IMPORTANT

When using machine config pool based matching, it is advised to group nodes with the same hardware configuration into the same machine config pool. Not following this practice might result in TuneD operands calculating conflicting kernel parameters for two or more nodes sharing the same machine config pool.

Example: Node or pod label based matching

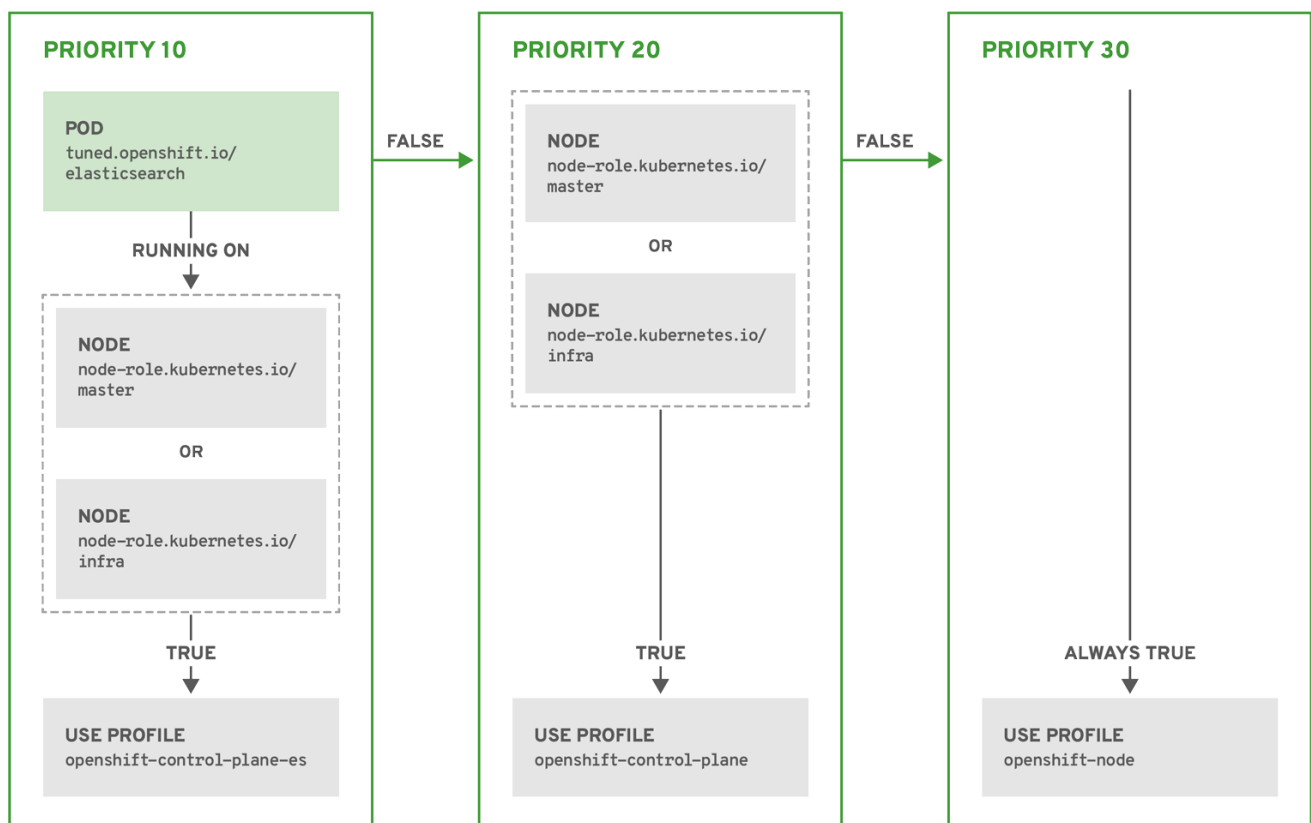
```
- match:
  - label: tuned.openshift.io/elasticsearch
    match:
      - label: node-role.kubernetes.io/master
      - label: node-role.kubernetes.io/infra
    type: pod
  priority: 10
  profile: openshift-control-plane-es
- match:
  - label: node-role.kubernetes.io/master
  - label: node-role.kubernetes.io/infra
  priority: 20
  profile: openshift-control-plane
- priority: 30
  profile: openshift-node
```

The CR above is translated for the containerized TuneD daemon into its **recommend.conf** file based on the profile priorities. The profile with the highest priority (**10**) is **openshift-control-plane-es** and, therefore, it is considered first. The containerized TuneD daemon running on a given node looks to see if there is a pod running on the same node with the **tuned.openshift.io/elasticsearch** label set. If not, the

entire **<match>** section evaluates as **false**. If there is such a pod with the label, in order for the **<match>** section to evaluate to **true**, the node label also needs to be **node-role.kubernetes.io/master** or **node-role.kubernetes.io/infra**.

If the labels for the profile with priority **10** matched, **openshift-control-plane-es** profile is applied and no other profile is considered. If the node/pod label combination did not match, the second highest priority profile (**openshift-control-plane**) is considered. This profile is applied if the containerized Tuned pod runs on a node with labels **node-role.kubernetes.io/master** or **node-role.kubernetes.io/infra**.

Finally, the profile **openshift-node** has the lowest priority of **30**. It lacks the **<match>** section and, therefore, will always match. It acts as a profile catch-all to set **openshift-node** profile, if no other profile with higher priority matches on a given node.



OPENSIFT_10_0319

Example: Machine config pool based matching

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: openshift-node-custom
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Custom OpenShift node profile with an additional kernel parameter
        include=openshift-node
        [bootloader]
        cmdline_openshift_node_custom=+skew_tick=1
  
```

```

name: openshift-node-custom

recommend:
- machineConfigLabels:
    machineconfiguration.openshift.io/role: "worker-custom"
priority: 20
profile: openshift-node-custom

```

To minimize node reboots, label the target nodes with a label the machine config pool's node selector will match, then create the Tuned CR above and finally create the custom machine config pool itself.

Cloud provider-specific Tuned profiles

With this functionality, all Cloud provider-specific nodes can conveniently be assigned a Tuned profile specifically tailored to a given Cloud provider on a OpenShift Container Platform cluster. This can be accomplished without adding additional node labels or grouping nodes into machine config pools.

This functionality takes advantage of **spec.providerID** node object values in the form of **<cloud-provider>://<cloud-provider-specific-id>** and writes the file **/var/lib/ocp-tuned/provider** with the value **<cloud-provider>** in NTO operand containers. The content of this file is then used by Tuned to load **provider-<cloud-provider>** profile if such profile exists.

The **openshift** profile that both **openshift-control-plane** and **openshift-node** profiles inherit settings from is now updated to use this functionality through the use of conditional profile loading. Neither NTO nor Tuned currently include any Cloud provider-specific profiles. However, it is possible to create a custom profile **provider-<cloud-provider>** that will be applied to all Cloud provider-specific cluster nodes.

Example GCE Cloud provider profile

```

apiVersion: tuned.openshift.io/v1
kind: Tuned
metadata:
  name: provider-gce
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=GCE Cloud provider-specific profile
        # Your tuning for GCE Cloud provider goes here.
    name: provider-gce

```



NOTE

Due to profile inheritance, any setting specified in the **provider-<cloud-provider>** profile will be overwritten by the **openshift** profile and its child profiles.

5.13.3. Default profiles set on a cluster

The following are the default profiles set on a cluster.

```

apiVersion: tuned.openshift.io/v1
kind: Tuned

```

```
metadata:
  name: default
  namespace: openshift-cluster-node-tuning-operator
spec:
  profile:
    - data: |
        [main]
        summary=Optimize systems running OpenShift (provider specific parent profile)
        include=-provider-$(f:exec:cat:/var/lib/ocp-tuned/provider),openshift
        name: openshift
  recommend:
    - profile: openshift-control-plane
      priority: 30
      match:
        - label: node-role.kubernetes.io/master
        - label: node-role.kubernetes.io/infra
    - profile: openshift-node
      priority: 40
```

Starting with OpenShift Container Platform 4.9, all OpenShift Tuned profiles are shipped with the Tuned package. You can use the **oc exec** command to view the contents of these profiles:

```
$ oc exec $tuned_pod -n openshift-cluster-node-tuning-operator -- find /usr/lib/tuned/openshift{-control-plane,-node} -name tuned.conf -exec grep -H ^ {} \;
```

5.13.4. Supported Tuned daemon plugins

Excluding the **[main]** section, the following Tuned plugins are supported when using custom profiles defined in the **profile:** section of the Tuned CR:

- audio
- cpu
- disk
- eeepc_she
- modules
- mounts
- net
- scheduler
- scsi_host
- selinux
- sysctl
- sysfs
- usb

- video
- vm
- bootloader

There is some dynamic tuning functionality provided by some of these plugins that is not supported. The following TuneD plugins are currently not supported:

- script
- systemd



NOTE

The TuneD bootloader plugin only supports Red Hat Enterprise Linux CoreOS (RHCOS) worker nodes.

Additional resources

- [Available TuneD Plugins](#)
- [Getting Started with TuneD](#)

5.14. CONFIGURING THE MAXIMUM NUMBER OF PODS PER NODE

You can use the **podsPerCore** and **maxPods** parameters in a kubelet configuration to control the maximum number of pods that can be scheduled to a node. If you use both options, the lower of the two limits the number of pods on a node. Setting an appropriate maximum can help ensure your nodes run efficiently.

For example, if **podsPerCore** is set to **10** on a node with 4 processor cores, the maximum number of pods allowed on the node will be 40.

Prerequisites

- You have the label associated with the static **MachineConfigPool** CRD for the type of node you want to configure.

Procedure

1. Create a custom resource (CR) for your configuration change.

Sample configuration for a **max-pods** CR

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: set-max-pods
spec:
  machineConfigPoolSelector:
    matchLabels:
      pools.operator.machineconfiguration.openshift.io/worker: ""
  kubeletConfig:
    podsPerCore: 10
```

```
maxPods: 250
#...
```

where:

metadata.name

Specifies a name for the CR.

spec.machineConfigPoolSelector.matchLabels

Specifies the label from the machine config pool.

spec.kubeletConfig.podsPerCore

Specifies the number of pods the node can run based on the number of processor cores on the node.

spec.kubeletConfig.maxPods

Specifies the number of pods the node can run to a fixed value, regardless of the properties of the node.



NOTE

Setting **podsPerCore** to **0** disables this limit.

In the above example, the default value for **podsPerCore** is **10** and the default value for **maxPods** is **250**. This means that unless the node has 25 cores or more, by default, **podsPerCore** will be the limiting factor.

2. Run the following command to create the CR:

```
$ oc create -f <file_name>.yaml
```

Verification

- List the **MachineConfigPool** CRDs to check if the change is applied. The **UPDATING** column reports **True** if the change is picked up by the Machine Config Controller:

```
$ oc get machineconfigpools
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
master	master-9cc2c72f205e103bb534	False	False	False
worker	worker-8cecd1236b33ee3f8a5e	False	True	False

After the change is complete, the **UPDATED** column reports **True**.

```
$ oc get machineconfigpools
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
master	master-9cc2c72f205e103bb534	False	True	False
worker	worker-8cecd1236b33ee3f8a5e	True	False	False

5.15. MACHINE SCALING WITH STATIC IP ADDRESSES

After you deployed your cluster to run nodes with static IP addresses, you can scale an instance of a machine or a machine set to use one of these static IP addresses.

Additional resources

- [Static IP addresses for vSphere nodes](#)

5.15.1. Scaling machines to use static IP addresses

You can add machines with pre-defined static IP addresses by creating machine resources that specify static network configuration in the machine YAML file.

You can scale additional machine sets to use pre-defined static IP addresses on your cluster. For this configuration, you need to create a machine resource YAML file and then define static IP addresses in this file.

Prerequisites

- You deployed a cluster that runs at least one node with a configured static IP address.

Procedure

1. Create a machine resource YAML file and define static IP address network information in the **network** parameter.

Example of a machine resource YAML file with static IP address information defined in the network parameter.

```
apiVersion: machine.openshift.io/v1beta1
kind: Machine
metadata:
  creationTimestamp: null
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
    machine.openshift.io/cluster-api-machine-role: <role>
    machine.openshift.io/cluster-api-machine-type: <role>
    machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>
  name: <infrastructure_id>-<role>
  namespace: openshift-machine-api
spec:
  lifecycleHooks: {}
  metadata: {}
  providerSpec:
    value:
      apiVersion: machine.openshift.io/v1beta1
      credentialsSecret:
        name: vsphere-cloud-credentials
      diskGiB: 120
```

```

kind: VSphereMachineProviderSpec
memoryMiB: 8192
metadata:
  creationTimestamp: null
network:
  devices:
    - gateway: 192.168.204.1 ❶
      ipAdrs:
        - 192.168.204.8/24 ❷
      nameservers: ❸
        - 192.168.204.1
      networkName: qe-segment-204
  numCPUs: 4
  numCoresPerSocket: 2
  snapshot: ""
  template: <vm_template_name>
  userDataSecret:
    name: worker-user-data
  workspace:
    datacenter: <vcenter_data_center_name>
    datastore: <vcenter_datastore_name>
    folder: <vcenter_vm_folder_path>
    resourcepool: <vsphere_resource_pool>
    server: <vcenter_server_ip>
status: {}

```

- ❶ The IP address for the default gateway for the network interface.
 - ❷ Lists IPv4, IPv6, or both IP addresses that installation program passes to the network interface. Both IP families must use the same network interface for the default network.
 - ❸ Lists a DNS nameserver. You can define up to 3 DNS nameservers. Consider defining more than one DNS nameserver to take advantage of DNS resolution if that one DNS nameserver becomes unreachable.
- Create a **machine** custom resource (CR) by entering the following command in your terminal:

```
$ oc create -f <file_name>.yaml
```

5.15.2. Machine set scaling of machines with configured static IP addresses

You can scale machines with static IP addresses by configuring machine sets that request addresses through **IPAddressClaim** resources managed by your IP address management (IPAM) service.

You can use a machine set to scale machines with configured static IP addresses.

After you configure a machine set to request a static IP address for a machine, the machine controller creates an **IPAddressClaim** resource in the **openshift-machine-api** namespace. The external controller then creates an **IPAddress** resource and binds any static IP addresses to the **IPAddressClaim** resource.



IMPORTANT

Your organization might use numerous types of IP address management (IPAM) services. If you want to enable a particular IPAM service on OpenShift Container Platform, you might need to manually create the **IPAddressClaim** resource in a YAML definition and then bind a static IP address to this resource by entering the following command in your **oc** CLI:

```
$ oc create -f <ipaddressclaim_filename>
```

The following demonstrates an example of an **IPAddressClaim** resource:

```
kind: IPAddressClaim
metadata:
  finalizers:
  - machine.openshift.io/ip-claim-protection
  name: cluster-dev-9n5wg-worker-0-m7529-claim-0-0
  namespace: openshift-machine-api
spec:
  poolRef:
    apiGroup: ipamcontroller.example.io
    kind: IPPool
    name: static-ci-pool
status: {}
```

The machine controller updates the machine with a status of **IPAddressClaimed** to indicate that a static IP address has successfully bound to the **IPAddressClaim** resource. The machine controller applies the same status to a machine with multiple **IPAddressClaim** resources that each contain a bound static IP address. The machine controller then creates a virtual machine and applies static IP addresses to any nodes listed in the **providerSpec** of a machine's configuration.

5.15.3. Using a machine set to scale machines with configured static IP addresses

You can scale machines that use static IP addresses by configuring a machine set to request addresses from an IP address pool.

You can use a machine set to scale machines with configured static IP addresses.

The example in the procedure demonstrates the use of controllers for scaling machines in a machine set.

Prerequisites

- You deployed a cluster that runs at least one node with a configured static IP address.

Procedure

1. Configure a machine set by specifying IP pool information in the **network.devices.addressesFromPools** schema of the machine set's YAML file:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
metadata:
  annotations:
    machine.openshift.io/memoryMb: "8192"
```

```

    machine.openshift.io/vCPU: "4"
  labels:
    machine.openshift.io/cluster-api-cluster: <infrastructure_id>
  name: <infrastructure_id>-<role>
  namespace: openshift-machine-api
  spec:
    replicas: 0
    selector:
      matchLabels:
        machine.openshift.io/cluster-api-cluster: <infrastructure_id>
        machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>
    template:
      metadata:
        labels:
          ipam: "true"
          machine.openshift.io/cluster-api-cluster: <infrastructure_id>
          machine.openshift.io/cluster-api-machine-role: worker
          machine.openshift.io/cluster-api-machine-type: worker
          machine.openshift.io/cluster-api-machineset: <infrastructure_id>-<role>
      spec:
        lifecycleHooks: {}
        metadata: {}
        providerSpec:
          value:
            apiVersion: machine.openshift.io/v1beta1
            credentialsSecret:
              name: vsphere-cloud-credentials
            diskGiB: 120
            kind: VSphereMachineProviderSpec
            memoryMiB: 8192
            metadata: {}
            network:
              devices:
                - addressesFromPools: 1
                - group: ipamcontroller.example.io
                  name: static-ci-pool
                  resource: IPPool
                  nameservers:
                    - "192.168.204.1" 2
                  networkName: qe-segment-204
            numCPUs: 4
            numCoresPerSocket: 2
            snapshot: ""
            template: rvanderp4-dev-9n5wg-rhcos-generated-region-generated-zone
            userDataSecret:
              name: worker-user-data
            workspace:
              datacenter: IBMCDatacenter
              datastore: /IBMCDatacenter/datastore/vsanDatastore
              folder: /IBMCDatacenter/vm/rvanderp4-dev-9n5wg
              resourcePool: /IBMCDatacenter/host/IBMCCluster/Resources
              server: vcenter.ibmcc.devcluster.openshift.com

```

1

Specifies an IP pool, which lists a static IP address or a range of static IP addresses. The IP Pool can either be a reference to a custom resource definition (CRD) or a resource supported by the **IPAddressClaims** resource handler. The machine controller accesses

static IP addresses listed in the machine set's configuration and then allocates each address to each machine.

- 2 Lists a nameserver. You must specify a nameserver for nodes that receive static IP address, because the Dynamic Host Configuration Protocol (DHCP) network configuration does not support static IP addresses.

2. Scale the machine set by entering the following commands in your **oc** CLI:

```
$ oc scale --replicas=2 machineset <machineset> -n openshift-machine-api
```

Or:

```
$ oc edit machineset <machineset> -n openshift-machine-api
```

After each machine is scaled up, the machine controller creates an **IPAddressClaim** resource.

3. Optional: Check that the **IPAddressClaim** resource exists in the **openshift-machine-api** namespace by entering the following command:

```
$ oc get ipaddressclaims.ipam.cluster.x-k8s.io -n openshift-machine-api
```

Example oc CLI output that lists two IP pools listed in the **openshift-machine-api namespace**

NAME	POOL NAME	POOL KIND
cluster-dev-9n5wg-worker-0-m7529-claim-0-0	static-ci-pool	IPPool
cluster-dev-9n5wg-worker-0-wdqkt-claim-0-0	static-ci-pool	IPPool

4. Create an **IPAddress** resource by entering the following command:

```
$ oc create -f ipaddress.yaml
```

The following example shows an **IPAddress** resource with defined network configuration information and one defined static IP address:

```
apiVersion: ipam.cluster.x-k8s.io/v1alpha1
kind: IPAddress
metadata:
  name: cluster-dev-9n5wg-worker-0-m7529-ipaddress-0-0
  namespace: openshift-machine-api
spec:
  address: 192.168.204.129
  claimRef: 1
    name: cluster-dev-9n5wg-worker-0-m7529-claim-0-0
  gateway: 192.168.204.1
  poolRef: 2
    apiGroup: ipamcontroller.example.io
    kind: IPPool
    name: static-ci-pool
  prefix: 23
```

- 1 The name of the target **IPAddressClaim** resource.
- 2 Details information about the static IP address or addresses from your nodes.

**NOTE**

By default, the external controller automatically scans any resources in the machine set for recognizable address pool types. When the external controller finds **kind: IPPool** defined in the **IPAddress** resource, the controller binds any static IP addresses to the **IPAddressClaim** resource.

5. Update the **IPAddressClaim** status with a reference to the **IPAddress** resource:

```
$ oc --type=merge patch IPAddressClaim cluster-dev-9n5wg-worker-0-m7529-claim-0-0 -
p='{"status":{"addressRef": {"name": "cluster-dev-9n5wg-worker-0-m7529-ipaddress-0-0"}}}' -
n openshift-machine-api --subresource=status
```

CHAPTER 6. POSTINSTALLATION NETWORK CONFIGURATION

You can configure networking after installation to manage cluster traffic, security, connectivity, and default network policies for new projects.

After installing OpenShift Container Platform, you can further expand and customize your network to your requirements.

6.1. USING THE CLUSTER NETWORK OPERATOR

You can use the Cluster Network Operator (CNO) to deploy and manage cluster network components on an OpenShift Container Platform cluster, including the Container Network Interface (CNI) network plugin selected for the cluster during installation.

For more information, see "Cluster Network Operator in OpenShift Container Platform".

6.2. NETWORK CONFIGURATION TASKS

- Configuring the cluster-wide proxy
- Configuring ingress cluster traffic overview
- Configuring the node port service range
- Configuring IPsec encryption
- Create a network policy or configure multitenant isolation with network policies
- Optimizing routing
- Understanding multiple networks

6.2.1. Creating default network policies for a new project

As a cluster administrator, you can modify the new project template to automatically include **NetworkPolicy** objects when you create a new project.

6.2.1.1. Modifying the template for new projects

To modify the default project template to customize the resources and settings applied when users create new projects, you can create a custom project template.

As a cluster administrator, you can modify the default project template so that new projects are created using your custom requirements.

To create your own custom project template:

Prerequisites

- You have access to an OpenShift Container Platform cluster using an account with **cluster-admin** permissions.

Procedure

1. Log in as a user with **cluster-admin** privileges.

2. Generate the default project template:

```
$ oc adm create-bootstrap-project-template -o yaml > template.yaml
```

3. Use a text editor to modify the generated **template.yaml** file by adding objects or modifying existing objects.
4. The project template must be created in the **openshift-config** namespace. Load your modified template:

```
$ oc create -f template.yaml -n openshift-config
```

5. Edit the project configuration resource using the web console or CLI.

- Using the web console, complete the following tasks:
 - i. Navigate to the **Administration → Cluster Settings** page.
 - ii. Click **Configuration** to view all configuration resources.
 - iii. Find the entry for **Project** and click **Edit YAML**.
- Using the CLI, complete the following tasks:
 - i. Edit the **project.config.openshift.io/cluster** resource:

```
$ oc edit project.config.openshift.io/cluster
```

6. Update the **spec** section to include the **projectRequestTemplate** and **name** parameters. Ensure you set the name of your uploaded project template. The default name is **project-request**.

Project configuration resource with custom project template

```
apiVersion: config.openshift.io/v1
kind: Project
metadata:
  # ...
spec:
  projectRequestTemplate:
    name: <template_name>
  # ...
```

7. After you save your changes, create a new project to verify that your changes were successfully applied.

6.2.1.2. Adding network policies to the new project template

You can add **NetworkPolicy** objects to the default project template so that new projects automatically include predefined network isolation rules. Applying network policies through templates helps enforce consistent network security controls across projects.

Prerequisites

- Your cluster uses a default container network interface (CNI) network plugin that supports **NetworkPolicy** objects, such as the OVN-Kubernetes.
- You installed the OpenShift CLI (**oc**).
- You must log in to the cluster with a user with **cluster-admin** privileges.
- You must have created a custom default project template for new projects.

Procedure

1. Edit the default template for a new project by running the following command:

```
$ oc edit template <project_template> -n openshift-config
```

Replace **<project_template>** with the name of the default template that you configured for your cluster. The default template name is **project-request**.

2. In the template, add each **NetworkPolicy** object as an element to the **objects** parameter. The **objects** parameter accepts a collection of one or more objects.

In the following example, the **objects** parameter collection includes several **NetworkPolicy** objects.

```
objects:
- apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: allow-from-same-namespace
  spec:
    podSelector: {}
    ingress:
    - from:
      - podSelector: {}
- apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: allow-from-openshift-ingress
  spec:
    ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            policy-group.network.openshift.io/ingress:
    podSelector: {}
    policyTypes:
    - Ingress
- apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: allow-from-kube-apiserver-operator
  spec:
    ingress:
    - from:
```

```
- namespaceSelector:
  matchLabels:
    kubernetes.io/metadata.name: openshift-kube-apiserver-operator
podSelector:
  matchLabels:
    app: kube-apiserver-operator
policyTypes:
- Ingress
...
```

3. Optional: Create a new project and confirm the successful creation of your network policy objects.

- a. Create a new project:

```
$ oc new-project <project>
```

Replace **<project>** with the name of the project you want to create.

- b. Confirm that the network policy objects in the new project template exist in the new project:

```
$ oc get networkpolicy
```

Expected output:

```
NAME                                POD-SELECTOR  AGE
allow-from-openshift-ingress       <none>        7s
allow-from-same-namespace          <none>        7s
```

6.3. ADDITIONAL RESOURCES

- [Cluster Network Operator in OpenShift Container Platform](#)
- [Configuring the cluster-wide proxy](#)
- [Configuring ingress cluster traffic overview](#)
- [Configuring the node port service range](#)
- [Configuring IPsec encryption](#)
- [Create a network policy](#)
- [Configure multitenant isolation with network policies](#)
- [Optimizing routing](#)
- [Understanding multiple networks](#)

CHAPTER 7. CONFIGURING IMAGE STREAMS AND IMAGE REGISTRIES

You can configure image streams, image registries, and pull secrets after installation to control how your cluster accesses, imports, and stores container images.

You can update the global pull secret for your cluster by either replacing the current pull secret or appending a new pull secret. The procedure is required when users use a separate registry to store images than the registry used during installation. For more information, see "Using image pull secrets".

For information about images and configuring image streams or image registries, see the following documentation:

- Overview of images
- Image Registry Operator in OpenShift Container Platform
- Configuring image registry settings

7.1. CONFIGURING IMAGE STREAMS FOR A DISCONNECTED CLUSTER

After installing OpenShift Container Platform in a disconnected environment, configure the image streams for the Cluster Samples Operator and the **must-gather** image stream.

7.1.1. Cluster Samples Operator assistance for mirroring

During installation, OpenShift Container Platform creates a config map named **imagestreamtag-to-image** in the **openshift-cluster-samples-operator** namespace.

The **imagestreamtag-to-image** config map contains an entry, the populating image, for each image stream tag.

The format of the key for each entry in the data field in the config map is **<image_stream_name>_<image_stream_tag_name>**.

During a disconnected installation of OpenShift Container Platform, the status of the Cluster Samples Operator is set to **Removed**. If you choose to change it to **Managed**, it installs samples.



NOTE

The use of samples in a network-restricted or discontinued environment might require access to services external to your network. Some example services include: Github, Maven Central, npm, RubyGems, PyPi and others. There might be additional steps to take that allow the Cluster Samples Operators objects to reach the services they require.

Use the following principles to determine which images you need to mirror for your image streams to import:

- While the Cluster Samples Operator is set to **Removed**, you can create your mirrored registry, or determine which existing mirrored registry you want to use.
- Mirror the samples you want to the mirrored registry using the new config map as your guide.

- Add any of the image streams you did not mirror to the **skippedImagestreams** list of the Cluster Samples Operator configuration object.
- Set **samplesRegistry** of the Cluster Samples Operator configuration object to the mirrored registry.
- Then set the Cluster Samples Operator to **Managed** to install the image streams you have mirrored.

7.1.2. Using Cluster Samples Operator image streams with alternate or mirrored registries

You can use an alternate or mirror registry to host your images streams instead of using the Red Hat registry.

Most image streams in the **openshift** namespace managed by the Cluster Samples Operator point to images located in the Red Hat registry at registry.redhat.io.



NOTE

The **cli**, **installer**, **must-gather**, and **tests** image streams, while part of the install payload, are not managed by the Cluster Samples Operator. These are not addressed in this procedure.



IMPORTANT

The Cluster Samples Operator must be set to **Managed** in a disconnected environment. To install the image streams, you must have a mirrored registry.

Prerequisites

- Access to the cluster as a user with the **cluster-admin** role.
- Create a pull secret for your mirror registry.

Procedure

1. Access the images of a specific image stream to mirror, for example:

```
$ oc get is <imagestream> -n openshift -o json | jq .spec.tags[].from.name | grep registry.redhat.io
```

2. Mirror images from registry.redhat.io associated with any image streams you need

```
$ oc image mirror registry.redhat.io/rhsc/ruby-25-rhel7:latest ${MIRROR_ADDR}/rhsc/ruby-25-rhel7:latest
```

3. Create the image configuration object for the cluster by running the following command:

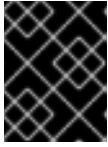
```
$ oc create configmap registry-config --from-file=${MIRROR_ADDR_HOSTNAME}..5000=$path/ca.crt -n openshift-config
```

4. Add the required trusted CAs for the mirror in the image configuration object:

```
$ oc patch image.config.openshift.io/cluster --patch '{"spec":{"additionalTrustedCA":
{"name":"registry-config"}}}' --type=merge
```

5. Update the **samplesRegistry** field in the Cluster Samples Operator configuration object to contain the **hostname** portion of the mirror location defined in the mirror configuration:

```
$ oc edit configs.samples.operator.openshift.io -n openshift-cluster-samples-operator
```



IMPORTANT

This step is required because the image stream import process does not use the mirror or search mechanism at this time.

6. Add any image streams that are not mirrored into the **skippedImagestreams** field of the Cluster Samples Operator configuration object. Or if you do not want to support any of the sample image streams, set the Cluster Samples Operator to **Removed** in the Cluster Samples Operator configuration object.



NOTE

The Cluster Samples Operator issues alerts if image stream imports are failing but the Cluster Samples Operator is either periodically retrying or does not appear to be retrying them.

Many of the templates in the **openshift** namespace reference the image streams. You can use **Removed** to purge both the image streams and templates. This eliminates the possibility of attempts to use the templates if they are not functional because of any missing image streams.

7.1.3. Preparing your cluster to gather support data

You can prepare a cluster on a restricted network to gather support data by importing the default must-gather image and configuring access to the mirror registry.

Clusters using a restricted network must import the default must-gather image to gather debugging data for Red Hat support. The must-gather image is not imported by default, and clusters on a restricted network do not have access to the internet to pull the latest image from a remote repository.

Procedure

1. If you have not added your mirror registry's trusted CA to your cluster's image configuration object as part of the Cluster Samples Operator configuration, perform the following steps:
 - a. Create the cluster's image configuration object:

```
$ oc create configmap registry-config --from-
file=${MIRROR_ADDR_HOSTNAME}..5000=$path/ca.crt -n openshift-config
```

- b. Add the required trusted CAs for the mirror in the cluster's image configuration object:

```
$ oc patch image.config.openshift.io/cluster --patch '{"spec":{"additionalTrustedCA":
{"name":"registry-config"}}}' --type=merge
```

2. Import the default must-gather image from your installation payload:

```
$ oc import-image is/must-gather -n openshift
```

When running the **oc adm must-gather** command, use the **--image** flag and point to the payload image, as in the following example:

```
$ oc adm must-gather --image=$(oc adm release info --image-for must-gather)
```

7.2. CONFIGURING PERIODIC IMPORTING OF CLUSTER SAMPLE OPERATOR IMAGE STREAM TAGS

You can configure scheduled imports for Cluster Sample Operator image stream tags to periodically retrieve updated images and retry imports after temporary failures.

You can ensure that you always have access to the latest versions of the Cluster Sample Operator images by periodically importing the image stream tags when new versions become available.

Procedure

1. Fetch all the imagestreams in the **openshift** namespace by running the following command:

```
oc get imagestreams -n openshift
```

2. Fetch the tags for every imagestream in the **openshift** namespace by running the following command:

```
$ oc get is <image-stream-name> -o jsonpath="{range .spec.tags[*]}.{name}{'\t'}{.from.name}{'\n'}{end}" -n openshift
```

For example:

```
$ oc get is ubi8-openjdk-17 -o jsonpath="{range .spec.tags[*]}.{name}{'\t'}{.from.name}{'\n'}{end}" -n openshift
```

Example output

```
1.11 registry.access.redhat.com/ubi8/openjdk-17:1.11
1.12 registry.access.redhat.com/ubi8/openjdk-17:1.12
```

3. Schedule periodic importing of images for each tag present in the image stream by running the following command:

```
$ oc tag <repository/image> <image-stream-name:tag> --scheduled -n openshift
```

For example:

```
$ oc tag registry.access.redhat.com/ubi8/openjdk-17:1.11 ubi8-openjdk-17:1.11 --scheduled -n openshift
```

```
$ oc tag registry.access.redhat.com/ubi8/openjdk-17:1.12 ubi8-openjdk-17:1.12 --scheduled -n openshift
```



NOTE

Using the **--scheduled** flag is recommended to periodically re-import an image when working with an external container image registry. The **--scheduled** flag helps to ensure that you receive the latest versions and security updates. Additionally, this setting allows the import process to automatically retry if a temporary error initially prevents the image from being imported.

By default, scheduled image imports occur every 15 minutes cluster-wide.

4. Verify the scheduling status of the periodic import by running the following command:

```
oc get imagestream <image-stream-name> -o jsonpath="{range .spec.tags[*]}Tag: {.name}\n\t}Scheduled: {.importPolicy.scheduled}\n\t}{end}" -n openshift
```

For example:

```
oc get imagestream ubi8-openjdk-17 -o jsonpath="{range .spec.tags[*]}Tag: {.name}\n\t}Scheduled: {.importPolicy.scheduled}\n\t}{end}" -n openshift
```

Example output

```
Tag: 1.11 Scheduled: true
Tag: 1.12 Scheduled: true
```

7.3. ADDITIONAL RESOURCES

- [Using image pull secrets](#)
- [Overview of images](#)
- [Image Registry Operator in OpenShift Container Platform](#)
- [Configuring image registry settings](#)

CHAPTER 8. POSTINSTALLATION STORAGE CONFIGURATION

You can configure persistent storage after installation by using dynamic or static provisioning to retain application data beyond the lifetime of individual containers.

After installing OpenShift Container Platform, you can further expand and customize your cluster to your requirements, including storage configuration.

By default, containers operate by using the ephemeral storage or transient local storage. The ephemeral storage has a lifetime limitation. To store the data for a long time, you must configure persistent storage. You can configure storage by using one of the following methods:

Dynamic provisioning

You can dynamically provision storage on-demand by defining and creating storage classes that control different levels of storage, including storage access.

Static provisioning

You can use Kubernetes persistent volumes to make existing storage available to a cluster. Static provisioning can support various device configurations and mount options.

8.1. DYNAMIC PROVISIONING

Dynamic Provisioning allows you to create storage volumes on-demand, eliminating the need for cluster administrators to pre-provision storage. See [Dynamic provisioning](#).

8.2. RECOMMENDED CONFIGURABLE STORAGE TECHNOLOGY

Review the recommended and configurable storage technologies for the given OpenShift Container Platform cluster application.

Table 8.1. Recommended and configurable storage technology

Storage type	Block	File	Object
ROX	Yes	Yes	Yes
RWX	No	Yes	Yes
Registry	Configurable	Configurable	Recommended
Scaled registry	Not configurable	Configurable	Recommended
Metrics	Recommended	Configurable	Not configurable
Elasticsearch Logging	Recommended	Configurable	Not supported
Loki Logging	Not configurable	Not configurable	Recommended
Apps	Recommended	Recommended	Not configurable

where:

ROX

Specifies **ReadOnlyMany** access mode.

ROX.Yes

Specifies that this access mode

RWX

Specifies **ReadWriteMany** access mode.

Metrics

Specifies Prometheus as the underlying technology used for metrics.

Metrics.Configurable

For metrics, using file storage with the **ReadWriteMany** (RWX) access mode is unreliable. If you use file storage, do not configure the RWX access mode on any persistent volume claims (PVCs) that are configured for use with metrics.

Elasticsearch Logging.Configurable

For logging, review the recommended storage solution in Configuring persistent storage for the log store section. Using NFS storage as a persistent volume or through NAS, such as Gluster, can corrupt the data. Therefore, NFS is not supported for Elasticsearch storage and LokiStack log store in OpenShift Container Platform Logging. You must use one persistent volume type per log store.

Apps.Not configurable

Specifies that object storage is not consumed through PVs or PVCs of OpenShift Container Platform. Apps must integrate with the object storage REST API.



NOTE

A scaled registry is an OpenShift image registry where two or more pod replicas are running.

8.3. DEPLOY RED HAT OPENSIFT DATA FOUNDATION

Red Hat OpenShift Data Foundation is a provider of agnostic persistent storage for OpenShift Container Platform supporting file, block, and object storage, either in-house or in hybrid clouds. As a Red Hat storage solution, Red Hat OpenShift Data Foundation is completely integrated with OpenShift Container Platform for deployment, management, and monitoring. For more information, see the [Red Hat OpenShift Data Foundation documentation](#).



IMPORTANT

OpenShift Data Foundation on top of Red Hat Hyperconverged Infrastructure (RHII) for Virtualization, which uses hyperconverged nodes that host virtual machines installed with OpenShift Container Platform, is not a supported configuration. For more information about supported platforms, see the [Red Hat OpenShift Data Foundation Supportability and Interoperability Guide](#).

If you are looking for Red Hat OpenShift Data Foundation information about...	See the following Red Hat OpenShift Data Foundation documentation:
What's new, known issues, notable bug fixes, and Technology Previews	OpenShift Data Foundation 4.12 Release Notes
Supported workloads, layouts, hardware and software requirements, sizing and scaling recommendations	Planning your OpenShift Data Foundation 4.12 deployment
Instructions on deploying OpenShift Data Foundation to use an external Red Hat Ceph Storage cluster	Deploying OpenShift Data Foundation 4.12 in external mode
Instructions on deploying OpenShift Data Foundation to local storage on bare metal infrastructure	Deploying OpenShift Data Foundation 4.12 using bare metal infrastructure
Instructions on deploying OpenShift Data Foundation on Red Hat OpenShift Container Platform VMware vSphere clusters	Deploying OpenShift Data Foundation 4.12 on VMware vSphere
Instructions on deploying OpenShift Data Foundation using Amazon Web Services for local or cloud storage	Deploying OpenShift Data Foundation 4.12 using Amazon Web Services
Instructions on deploying and managing OpenShift Data Foundation on existing Red Hat OpenShift Container Platform Google Cloud clusters	Deploying and managing OpenShift Data Foundation 4.12 using Google Cloud
Instructions on deploying and managing OpenShift Data Foundation on existing Red Hat OpenShift Container Platform Azure clusters	Deploying and managing OpenShift Data Foundation 4.12 using Microsoft Azure
Instructions on deploying OpenShift Data Foundation to use local storage on IBM Power® infrastructure	Deploying OpenShift Data Foundation on IBM Power®
Instructions on deploying OpenShift Data Foundation to use local storage on IBM Z® infrastructure	Deploying OpenShift Data Foundation on IBM Z® infrastructure
Allocating storage to core services and hosted applications in Red Hat OpenShift Data Foundation, including snapshot and clone	Managing and allocating resources
Managing storage resources across a hybrid cloud or multicloud environment using the Multicloud Object Gateway (NooBaa)	Managing hybrid and multicloud resources

If you are looking for Red Hat OpenShift Data Foundation information about...	See the following Red Hat OpenShift Data Foundation documentation:
Safely replacing storage devices for Red Hat OpenShift Data Foundation	Replacing devices
Safely replacing a node in a Red Hat OpenShift Data Foundation cluster	Replacing nodes
Scaling operations in Red Hat OpenShift Data Foundation	Scaling storage
Monitoring a Red Hat OpenShift Data Foundation 4.12 cluster	Monitoring Red Hat OpenShift Data Foundation 4.12
Resolve issues encountered during operations	Troubleshooting OpenShift Data Foundation 4.12
Migrating your OpenShift Container Platform cluster from version 3 to version 4	Migration

CHAPTER 9. PREPARING FOR USERS

You can prepare your cluster for users by configuring authentication and permissions, managing initial administrative access, and making Operators available through the software catalog.

After installing OpenShift Container Platform, you can further expand and customize your cluster to your requirements, including taking steps to prepare for users.

9.1. UNDERSTANDING IDENTITY PROVIDER CONFIGURATION

The OpenShift Container Platform control plane includes a built-in OAuth server. Developers and administrators obtain OAuth access tokens to authenticate themselves to the API.

As an administrator, you can configure OAuth to specify an identity provider after you install your cluster.

9.1.1. Identity providers in OpenShift Container Platform

You can configure identity providers by creating a custom resource (CR) that describes the provider and adding it to the cluster. Identity providers enable user authentication in OpenShift Container Platform beyond the default **kubeadmin** user.



NOTE

OpenShift Container Platform usernames containing `/`, `:`, and `%` are not supported.

9.1.2. Supported identity providers

You can configure the following types of identity providers:

Identity provider	Description
htpasswd	Configure the htpasswd identity provider to validate user names and passwords against a flat file generated using htpasswd .
Keystone	Configure the keystone identity provider to integrate your OpenShift Container Platform cluster with Keystone to enable shared authentication with an OpenStack Keystone v3 server configured to store users in an internal database.
LDAP	Configure the ldap identity provider to validate user names and passwords against an LDAPv3 server, using simple bind authentication.
Basic authentication	Configure a basic-authentication identity provider for users to log in to OpenShift Container Platform with credentials validated against a remote identity provider. Basic authentication is a generic backend integration mechanism.
Request header	Configure a request-header identity provider to identify users from request header values, such as X-Remote-User . It is typically used in combination with an authenticating proxy, which sets the request header value.

Identity provider	Description
GitHub or GitHub Enterprise	Configure a github identity provider to validate user names and passwords against GitHub or GitHub Enterprise's OAuth authentication server.
GitLab	Configure a gitlab identity provider to use GitLab.com or any other GitLab instance as an identity provider.
Google	Configure a google identity provider using Google's OpenID Connect integration .
OpenID Connect	Configure an oidc identity provider to integrate with an OpenID Connect identity provider using an Authorization Code Flow .

After you define an identity provider, you can [use RBAC to define and apply permissions](#) .

9.1.3. Identity provider parameters

The following parameters are common to all identity providers:

Parameter	Description
name	The provider name is prefixed to provider user names to form an identity name.
mappingMethod	Defines how new identities are mapped to users when they log in. Enter one of the following values: claim The default value. Provisions a user with the identity's preferred user name. Fails if a user with that user name is already mapped to another identity. lookup Looks up an existing identity, user identity mapping, and user, but does not automatically provision users or identities. This allows cluster administrators to set up identities and users manually, or using an external process. Using this method requires you to manually provision users. add Provisions a user with the identity's preferred user name. If a user with that user name already exists, the identity is mapped to the existing user, adding to any existing identity mappings for the user. Required when multiple identity providers are configured that identify the same set of users and map to the same user names.



NOTE

When adding or changing identity providers, you can map identities from the new provider to existing users by setting the **mappingMethod** parameter to **add**.

9.1.4. Sample identity provider CR

You can use a custom resource (CR) to see the parameters and default values that you use to configure an identity provider.

The following example uses the `htpasswd` identity provider.

Sample identity provider CR

```
apiVersion: config.openshift.io/v1
kind: OAuth
metadata:
  name: cluster
spec:
  identityProviders:
  - name: my_identity_provider
    mappingMethod: claim
    type: HTPasswd
    htpasswd:
      fileData:
        name: htpass-secret
```

where:

`spec.identityProviders.name`

Specifies the provider name, which is prefixed to provider user names to form an identity name.

`spec.identityProviders.mappingMethod`

Specifies how mappings are established between this provider's identities and **User** objects.

`spec.identityProviders.htpasswd.fileData.name`

Specifies an existing secret containing a file generated using [htpasswd](#).

9.2. USING RBAC TO DEFINE AND APPLY PERMISSIONS

Understand and apply role-based access control.

9.2.1. RBAC overview

You can use role-based access control to configure whether users and groups can perform specific actions on cluster or project resources by evaluating roles, rules, and bindings.

Role-based access control (RBAC) objects determine whether a user is allowed to perform a given action within a project.

Cluster administrators can use the cluster roles and bindings to control who has various access levels to the OpenShift Container Platform platform itself and all projects.

Developers can use local roles and bindings to control who has access to their projects. Note that authorization is a separate step from authentication, which is more about determining the identity of who is taking the action.

Authorization is managed using:

Authorization object	Description
Rules	Sets of permitted verbs on a set of objects. For example, whether a user or service account can create pods.
Roles	Collections of rules. You can associate, or bind, users and groups to multiple roles.
Bindings	Associations between users and/or groups with a role.

There are two levels of RBAC roles and bindings that control authorization:

RBAC level	Description
Cluster RBAC	Roles and bindings that are applicable across all projects. <i>Cluster roles</i> exist cluster-wide, and <i>cluster role bindings</i> can reference only cluster roles.
Local RBAC	Roles and bindings that are scoped to a given project. While <i>local roles</i> exist only in a single project, local role bindings can reference <i>both</i> cluster and local roles.

A cluster role binding is a binding that exists at the cluster level. A role binding exists at the project level. The cluster role *view* must be bound to a user using a local role binding for that user to view the project. Create local roles only if a cluster role does not provide the set of permissions needed for a particular situation.

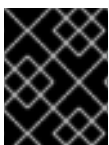
This two-level hierarchy allows reuse across multiple projects through the cluster roles while allowing customization inside of individual projects through local roles.

During evaluation, both the cluster role bindings and the local role bindings are used. For example:

1. Cluster-wide "allow" rules are checked.
2. Locally-bound "allow" rules are checked.
3. Deny by default.

9.2.1.1. Default cluster roles

OpenShift Container Platform includes a set of default cluster roles that you can bind to users and groups cluster-wide or locally.



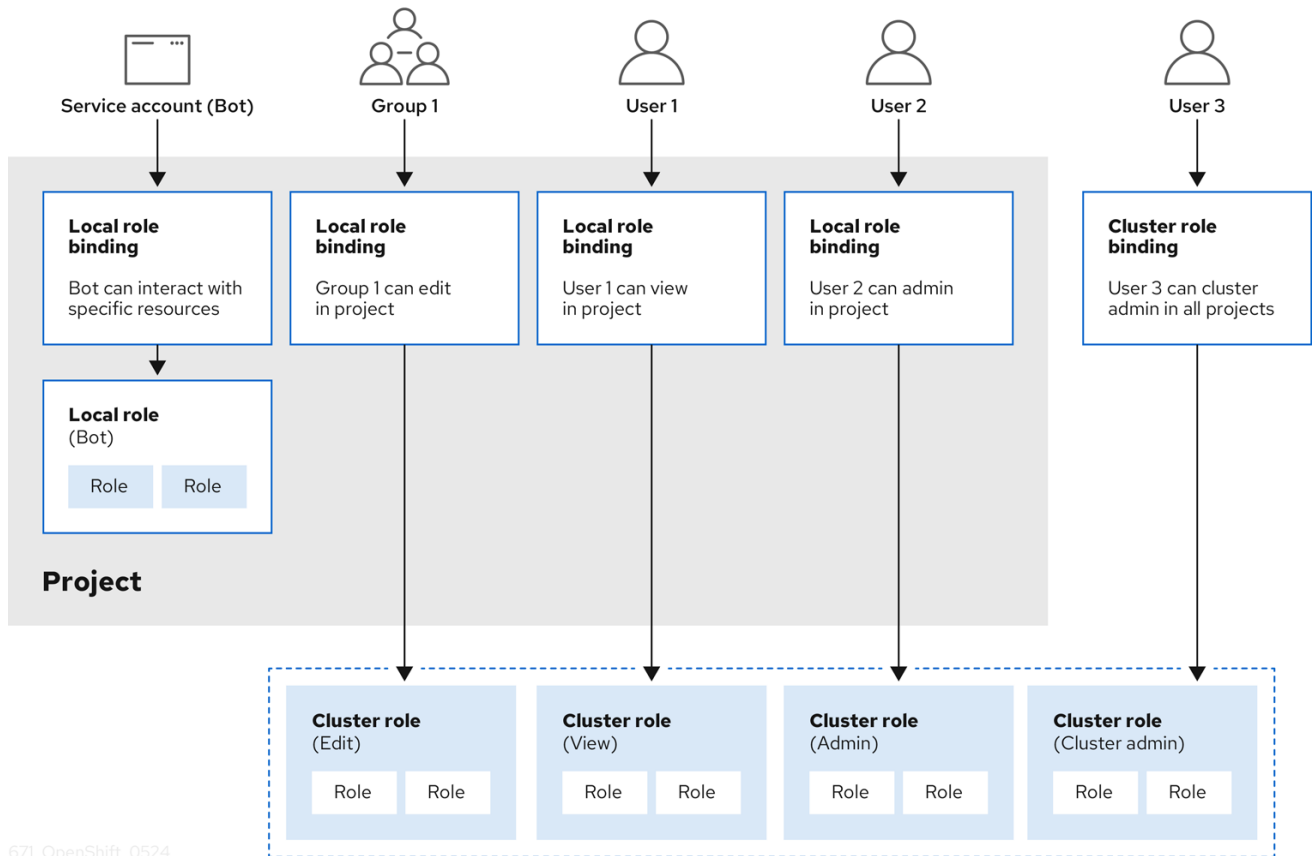
IMPORTANT

It is not recommended to manually modify the default cluster roles. Modifications to these system roles can prevent a cluster from functioning properly.

Default cluster role	Description
admin	A project manager. If used in a local binding, an admin has rights to view any resource in the project and modify any resource in the project except for quota.
basic-user	A user that can get basic information about projects and users.
cluster-admin	A super-user that can perform any action in any project. When bound to a user with a local binding, they have full control over quota and every action on every resource in the project.
cluster-status	A user that can get basic cluster status information.
cluster-reader	A user that can get or view most of the objects but cannot modify them.
edit	A user that can modify most objects in a project but does not have the power to view or modify roles or bindings.
self-provisioner	A user that can create their own projects.
view	A user who cannot make any modifications, but can see most objects in a project. They cannot view or modify roles or bindings.

Be mindful of the difference between local and cluster bindings. For example, if you bind the **cluster-admin** role to a user by using a local role binding, it might appear that this user has the privileges of a cluster administrator. This is not the case. Binding the **cluster-admin** to a user in a project grants super administrator privileges for only that project to the user. That user has the permissions of the cluster role **admin**, plus a few additional permissions like the ability to edit rate limits, for that project. This binding can be confusing via the web console UI, which does not list cluster role bindings that are bound to true cluster administrators. However, it does list local role bindings that you can use to locally bind **cluster-admin**.

The relationships between cluster roles, local roles, cluster role bindings, local role bindings, users, groups and service accounts are illustrated below.



WARNING

The **get pods/exec**, **get pods/***, and **get *** rules grant execution privileges when they are applied to a role. Apply the principle of least privilege and assign only the minimal RBAC rights required for users and agents. For more information, see "RBAC rules allow execution privileges".

9.2.1.2. Evaluating authorization

OpenShift Container Platform evaluates authorization by using:

Identity

The user name and list of groups that the user belongs to.

Action

The action you perform. In most cases, this consists of:

- **Project:** The project you access. A project is a Kubernetes namespace with additional annotations that allows a community of users to organize and manage their content in isolation from other communities.
- **Verb :** The action itself: **get**, **list**, **create**, **update**, **delete**, **deletecollection**, or **watch**.
- **Resource name:** The API endpoint that you access.

Bindings

The full list of bindings, the associations between users or groups with a role.

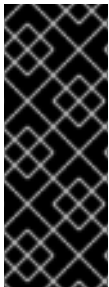
OpenShift Container Platform evaluates authorization by using the following steps:

1. The identity and the project-scoped action is used to find all bindings that apply to the user or their groups.
2. Bindings are used to locate all the roles that apply.
3. Roles are used to find all the rules that apply.
4. The action is checked against each rule to find a match.
5. If no matching rule is found, the action is then denied by default.

TIP

Remember that users and groups can be associated with, or bound to, multiple roles at the same time.

Project administrators can use the CLI to view local roles and bindings, including a matrix of the verbs and resources each are associated with.



IMPORTANT

The cluster role bound to the project administrator is limited in a project through a local binding. It is not bound cluster-wide like the cluster roles granted to the **cluster-admin** or **system:admin**.

Cluster roles are roles defined at the cluster level but can be bound either at the cluster level or at the project level.

9.2.1.3. Cluster role aggregation

The default admin, edit, view, and cluster-reader cluster roles support cluster role aggregation, where the cluster rules for each role are dynamically updated as new rules are created. This feature is relevant only if you extend the Kubernetes API by creating custom resources.

Additional resources

- [RBAC rules allow execution privileges](#)
- [Aggregated ClusterRoles \(Kubernetes documentation\)](#)

9.2.2. Projects and namespaces

You can use projects and namespaces to organize and isolate cluster resources. These resources provide boundaries for access control, policies, quotas, and service accounts.

A Kubernetes *namespace* provides a mechanism to scope resources in a cluster. The Kubernetes documentation has more information on namespaces.

Namespaces provide a unique scope for:

- Named resources to avoid basic naming collisions.

- Delegated management authority to trusted users.
- The ability to limit community resource consumption.

Most objects in the system are scoped by namespace, but some are excepted and have no namespace, including nodes and users.

A *project* is a Kubernetes namespace with additional annotations and is the central vehicle by which access to resources for regular users is managed. A project allows a community of users to organize and manage their content in isolation from other communities. Users must be given access to projects by administrators, or if allowed to create projects, automatically have access to their own projects.

Projects can have a separate **name**, **displayName**, and **description**.

- The mandatory **name** is a unique identifier for the project and is most visible when using the CLI tools or API. The maximum name length is 63 characters.
- The optional **displayName** is how the project is displayed in the web console (defaults to **name**).
- The optional **description** can be a more detailed description of the project and is also visible in the web console.

Each project scopes its own set of:

Object	Description
Objects	Pods, services, replication controllers, etc.
Policies	Rules for which users can or cannot perform actions on objects.
Constraints	Quotas for each kind of object that can be limited.
Service accounts	Service accounts act automatically with designated access to objects in the project.

Cluster administrators can create projects and delegate administrative rights for the project to any member of the user community. Cluster administrators can also allow developers to create their own projects.

Developers and administrators can interact with projects by using the CLI or the web console.

Additional resources

- [Kubernetes documentation on namespaces](#)

9.2.3. Default projects

Default projects host critical cluster and infrastructure components. By understanding their purpose, you can avoid making changes that could disrupt essential cluster services.

OpenShift Container Platform includes several default projects, and projects starting with **openshift-**

are the most essential to users. These projects host master components that run as pods and other infrastructure components. The pods created in these namespaces that have a critical pod annotation are considered critical, and they have guaranteed admission by kubelet. Pods created for master components in these namespaces are already marked as critical.



IMPORTANT

Do not run workloads in or share access to default projects. Default projects are reserved for running core cluster components.

The following default projects are considered highly privileged: **default**, **kube-public**, **kube-system**, **openshift**, **openshift-infra**, **openshift-node**, and other system-created projects that have the **openshift.io/run-level** label set to **0** or **1**. Functionality that relies on admission plugins, such as pod security admission, security context constraints, cluster resource quotas, and image reference resolution, does not work in highly privileged projects.

Additional resources

- [Guaranteed Scheduling For Critical Add-On Pods \(Kubernetes documentation\)](#)

9.2.4. Viewing cluster roles and bindings

You can view cluster roles and bindings by using the **oc** CLI to determine the permissions associated with roles and identify the users, groups, and service accounts assigned to them.

You can use the **oc** CLI to view cluster roles and bindings by using the **oc describe** command.

Prerequisites

- Install the **oc** CLI.
- Obtain permission to view the cluster roles and bindings.

Users with the **cluster-admin** default cluster role bound cluster-wide can perform any action on any resource, including viewing cluster roles and bindings.

Procedure

1. To view the cluster roles and their associated rule sets:

```
$ oc describe clusterrole.rbac
```

Example output

```
Name:      admin
Labels:    kubernetes.io/bootstrapping=rbac-defaults
Annotations: rbac.authorization.kubernetes.io/autoupdate: true
PolicyRule:
  Resources                                Non-Resource URLs  Resource Names  Verbs
-----
.packages.apps.redhat.com                  []                 []              [* create update
patch delete get list watch]
imagestreams                              []                 []              [create delete
```

```

deletecollection get list patch update watch create get list watch]
  imagestreams.image.openshift.io [] [] [create delete
deletecollection get list patch update watch create get list watch]
  secrets [] [] [create delete deletecollection
get list patch update watch get list watch create delete deletecollection patch update]
  buildconfigs/webhooks [] [] [create delete
deletecollection get list patch update watch get list watch]
  buildconfigs [] [] [create delete
deletecollection get list patch update watch get list watch]
  buildlogs [] [] [create delete deletecollection
get list patch update watch get list watch]
  deploymentconfigs/scale [] [] [create delete
deletecollection get list patch update watch get list watch]
  deploymentconfigs [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreamimages [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreammappings [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreamtags [] [] [create delete
deletecollection get list patch update watch get list watch]
  processedtemplates [] [] [create delete
deletecollection get list patch update watch get list watch]
  routes [] [] [create delete deletecollection
get list patch update watch get list watch]
  templateconfigs [] [] [create delete
deletecollection get list patch update watch get list watch]
  templateinstances [] [] [create delete
deletecollection get list patch update watch get list watch]
  templates [] [] [create delete
deletecollection get list patch update watch get list watch]
  deploymentconfigs.apps.openshift.io/scale [] [] [create delete
deletecollection get list patch update watch get list watch]
  deploymentconfigs.apps.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  buildconfigs.build.openshift.io/webhooks [] [] [create delete
deletecollection get list patch update watch get list watch]
  buildconfigs.build.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  buildlogs.build.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreamimages.image.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreammappings.image.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  imagestreamtags.image.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  routes.route.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  processedtemplates.template.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  templateconfigs.template.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  templateinstances.template.openshift.io [] [] [create delete
deletecollection get list patch update watch get list watch]
  templates.template.openshift.io [] [] [create delete

```

```

deletecollection get list patch update watch get list watch]
  serviceaccounts [] [] [create delete
deletecollection get list patch update watch impersonate create delete deletecollection patch
update get list watch]
  imagestreams/secrets [] [] [create delete
deletecollection get list patch update watch]
  rolebindings [] [] [create delete
deletecollection get list patch update watch]
  roles [] [] [create delete deletecollection
get list patch update watch]
  rolebindings.authorization.openshift.io [] [] [create delete
deletecollection get list patch update watch]
  roles.authorization.openshift.io [] [] [create delete
deletecollection get list patch update watch]
  imagestreams.image.openshift.io/secrets [] [] [create delete
deletecollection get list patch update watch]
  rolebindings.rbac.authorization.k8s.io [] [] [create delete
deletecollection get list patch update watch]
  roles.rbac.authorization.k8s.io [] [] [create delete
deletecollection get list patch update watch]
  networkpolicies.extensions [] [] [create delete
deletecollection patch update create delete deletecollection get list patch update watch get
list watch]
  networkpolicies.networking.k8s.io [] [] [create delete
deletecollection patch update create delete deletecollection get list patch update watch get
list watch]
  configmaps [] [] [create delete
deletecollection patch update get list watch]
  endpoints [] [] [create delete
deletecollection patch update get list watch]
  persistentvolumeclaims [] [] [create delete
deletecollection patch update get list watch]
  pods [] [] [create delete deletecollection
patch update get list watch]
  replicationcontrollers/scale [] [] [create delete
deletecollection patch update get list watch]
  replicationcontrollers [] [] [create delete
deletecollection patch update get list watch]
  services [] [] [create delete deletecollection
patch update get list watch]
  daemonsets.apps [] [] [create delete
deletecollection patch update get list watch]
  deployments.apps/scale [] [] [create delete
deletecollection patch update get list watch]
  deployments.apps [] [] [create delete
deletecollection patch update get list watch]
  replicaset.apps/scale [] [] [create delete
deletecollection patch update get list watch]
  replicaset.apps [] [] [create delete
deletecollection patch update get list watch]
  statefulsets.apps/scale [] [] [create delete
deletecollection patch update get list watch]
  statefulsets.apps [] [] [create delete
deletecollection patch update get list watch]
  horizontalpodautoscalers.autoscaling [] [] [create delete
deletecollection patch update get list watch]

```

cronjobs.batch	[]	[]	[create delete]
deletecollection patch update get list watch]			
jobs.batch	[]	[]	[create delete]
deletecollection patch update get list watch]			
daemonsets.extensions	[]	[]	[create delete]
deletecollection patch update get list watch]			
deployments.extensions/scale	[]	[]	[create delete]
deletecollection patch update get list watch]			
deployments.extensions	[]	[]	[create delete]
deletecollection patch update get list watch]			
ingresses.extensions	[]	[]	[create delete]
deletecollection patch update get list watch]			
replicasets.extensions/scale	[]	[]	[create delete]
deletecollection patch update get list watch]			
replicasets.extensions	[]	[]	[create delete]
deletecollection patch update get list watch]			
replicationcontrollers.extensions/scale	[]	[]	[create delete]
deletecollection patch update get list watch]			
poddisruptionbudgets.policy	[]	[]	[create delete]
deletecollection patch update get list watch]			
deployments.apps/rollback	[]	[]	[create delete]
deletecollection patch update]			
deployments.extensions/rollback	[]	[]	[create delete]
deletecollection patch update]			
catalogsources.operators.coreos.com	[]	[]	[create update]
patch delete get list watch]			
clusterserviceversions.operators.coreos.com	[]	[]	[create update]
patch delete get list watch]			
installplans.operators.coreos.com	[]	[]	[create update]
patch delete get list watch]			
packagemanifests.operators.coreos.com	[]	[]	[create update]
patch delete get list watch]			
subscriptions.operators.coreos.com	[]	[]	[create update]
patch delete get list watch]			
buildconfigs/instantiate	[]	[]	[create]
buildconfigs/instantiatebinary	[]	[]	[create]
builds/clone	[]	[]	[create]
deploymentconfigrollbacks	[]	[]	[create]
deploymentconfigs/instantiate	[]	[]	[create]
deploymentconfigs/rollback	[]	[]	[create]
imagestreamimports	[]	[]	[create]
localresourceaccessreviews	[]	[]	[create]
localsubjectaccessreviews	[]	[]	[create]
podsecuritypolicyreviews	[]	[]	[create]
podsecuritypolicyselfsubjectreviews	[]	[]	[create]
podsecuritypolicysubjectreviews	[]	[]	[create]
resourceaccessreviews	[]	[]	[create]
routes/custom-host	[]	[]	[create]
subjectaccessreviews	[]	[]	[create]
subjectrulesreviews	[]	[]	[create]
deploymentconfigrollbacks.apps.openshift.io	[]	[]	[create]
deploymentconfigs.apps.openshift.io/instantiate	[]	[]	[create]
deploymentconfigs.apps.openshift.io/rollback	[]	[]	[create]
localsubjectaccessreviews.authorization.k8s.io	[]	[]	[create]
localresourceaccessreviews.authorization.openshift.io	[]	[]	[create]
localsubjectaccessreviews.authorization.openshift.io	[]	[]	[create]

resourceaccessreviews.authorization.openshift.io			[create]
subjectaccessreviews.authorization.openshift.io			[create]
subjectrulesreviews.authorization.openshift.io			[create]
buildconfigs.build.openshift.io/instantiate			[create]
buildconfigs.build.openshift.io/instantiatebinary			[create]
builds.build.openshift.io/clone			[create]
imagestreamimports.image.openshift.io			[create]
routes.route.openshift.io/custom-host			[create]
podsecuritypolicyreviews.security.openshift.io			[create]
podsecuritypolicyselfsubjectreviews.security.openshift.io			[create]
podsecuritypolicysubjectreviews.security.openshift.io			[create]
jenkins.build.openshift.io			[edit view view admin]
edit view]			
builds			[get create delete]
deletecollection get list patch update watch get list watch]			
builds.build.openshift.io			[get create delete]
deletecollection get list patch update watch get list watch]			
projects			[get delete get delete get patch update]
projects.project.openshift.io			[get delete get delete]
get patch update]			
namespaces			[get get list watch]
pods/attach			[get list watch create delete]
deletecollection patch update]			
pods/exec			[get list watch create delete]
deletecollection patch update]			
pods/portforward			[get list watch create]
delete deletecollection patch update]			
pods/proxy			[get list watch create delete]
deletecollection patch update]			
services/proxy			[get list watch create delete]
deletecollection patch update]			
routes/status			[get list watch update]
routes.route.openshift.io/status			[get list watch update]
appliedclusterresourcequotas			[get list watch]
bindings			[get list watch]
builds/log			[get list watch]
deploymentconfigs/log			[get list watch]
deploymentconfigs/status			[get list watch]
events			[get list watch]
imagestreams/status			[get list watch]
limitranges			[get list watch]
namespaces/status			[get list watch]
pods/log			[get list watch]
pods/status			[get list watch]
replicationcontrollers/status			[get list watch]
resourcequotas/status			[get list watch]
resourcequotas			[get list watch]
resourcequotausages			[get list watch]
rolebindingrestrictions			[get list watch]
deploymentconfigs.apps.openshift.io/log			[get list watch]
deploymentconfigs.apps.openshift.io/status			[get list watch]
controllerrevisions.apps			[get list watch]
rolebindingrestrictions.authorization.openshift.io			[get list watch]
builds.build.openshift.io/log			[get list watch]
imagestreams.image.openshift.io/status			[get list watch]


```

appliedclusterresourcequotas.quota.openshift.io      []      []      [get list watch]
imagestreams/layers                                  []      []      [get update get]
imagestreams.image.openshift.io/layers                []      []      [get update get]
builds/details                                       []      []      [update]
builds.build.openshift.io/details                    []      []      [update]

```

Name: basic-user

Labels: <none>

Annotations: openshift.io/description: A user that can get basic information about projects.
rbac.authorization.kubernetes.io/autoupdate: true

PolicyRule:

Resources	Non-Resource URLs	Resource Names	Verbs
selfsubjectrulesreviews			[create]
selfsubjectaccessreviews.authorization.k8s.io			[create]
selfsubjectrulesreviews.authorization.openshift.io			[create]
clusterroles.rbac.authorization.k8s.io			[get list watch]
clusterroles			[get list]
clusterroles.authorization.openshift.io			[get list]
storageclasses.storage.k8s.io			[get list]
users	[~]		[get]
users.user.openshift.io	[~]		[get]
projects			[list watch]
projects.project.openshift.io			[list watch]
projectrequests			[list]
projectrequests.project.openshift.io			[list]

Name: cluster-admin

Labels: kubernetes.io/bootstrapping=rbac-defaults

Annotations: rbac.authorization.kubernetes.io/autoupdate: true

PolicyRule:

Resources	Non-Resource URLs	Resource Names	Verbs
.			[*]
	[*]		[*]

...

- To view the current set of cluster role bindings, which shows the users and groups that are bound to various roles:

```
$ oc describe clusterrolebinding.rbac
```

Example output

Name: alertmanager-main

Labels: <none>

Annotations: <none>

Role:

Kind: ClusterRole

Name: alertmanager-main

Subjects:

Kind	Name	Namespace
----	----	-----

ServiceAccount alertmanager-main openshift-monitoring

Name: basic-users
 Labels: <none>
 Annotations: rbac.authorization.kubernetes.io/autoupdate: true
 Role:
 Kind: ClusterRole
 Name: basic-user
 Subjects:

Kind	Name	Namespace
Group	system:authenticated	

Name: cloud-credential-operator-rolebinding
 Labels: <none>
 Annotations: <none>
 Role:
 Kind: ClusterRole
 Name: cloud-credential-operator-role
 Subjects:

Kind	Name	Namespace
ServiceAccount	default	openshift-cloud-credential-operator

Name: cluster-admin
 Labels: kubernetes.io/bootstrapping=rbac-defaults
 Annotations: rbac.authorization.kubernetes.io/autoupdate: true
 Role:
 Kind: ClusterRole
 Name: cluster-admin
 Subjects:

Kind	Name	Namespace
Group	system:masters	

Name: cluster-admins
 Labels: <none>
 Annotations: rbac.authorization.kubernetes.io/autoupdate: true
 Role:
 Kind: ClusterRole
 Name: cluster-admin
 Subjects:

Kind	Name	Namespace
Group	system:cluster-admins	
User	system:admin	

Name: cluster-api-manager-rolebinding
 Labels: <none>
 Annotations: <none>
 Role:

```

Kind: ClusterRole
Name: cluster-api-manager-role
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount default openshift-machine-api
...

```

9.2.5. Viewing local roles and bindings

You can view local role bindings by using the **oc** CLI to identify the users, groups, and service accounts that have roles within the current project or another project.

You can use the **oc** CLI to view local roles and bindings by using the **oc describe** command.

Prerequisites

- Install the **oc** CLI.
- Obtain permission to view the local roles and bindings:
 - Users with the **cluster-admin** default cluster role bound cluster-wide can perform any action on any resource, including viewing local roles and bindings.
 - Users with the **admin** default cluster role bound locally can view and manage roles and bindings in that project.

Procedure

1. To view the current set of local role bindings, which show the users and groups that are bound to various roles for the current project:

```
$ oc describe rolebinding.rbac
```

2. To view the local role bindings for a different project, add the **-n** flag to the command:

```
$ oc describe rolebinding.rbac -n joe-project
```

Example output

```

Name:      admin
Labels:    <none>
Annotations: <none>
Role:
  Kind: ClusterRole
  Name: admin
Subjects:
  Kind Name      Namespace
  ---- -
  User kube:admin

Name:      system:deployers

```

```

Labels:      <none>
Annotations: openshift.io/description:
              Allows deploymentconfigs in this namespace to rollout pods in
              this namespace. It is auto-managed by a controller; remove
              subjects to disa...

Role:
  Kind: ClusterRole
  Name: system:deployer
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount  deployer  joe-project

Name:      system:image-builders
Labels:    <none>
Annotations: openshift.io/description:
              Allows builds in this namespace to push images to this
              namespace. It is auto-managed by a controller; remove subjects
              to disable.

Role:
  Kind: ClusterRole
  Name: system:image-builder
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount  builder  joe-project

Name:      system:image-pullers
Labels:    <none>
Annotations: openshift.io/description:
              Allows all pods in this namespace to pull images from this
              namespace. It is auto-managed by a controller; remove subjects
              to disable.

Role:
  Kind: ClusterRole
  Name: system:image-puller
Subjects:
  Kind  Name      Namespace
  ----  -
  Group  system:serviceaccounts:joe-project

```

9.2.6. Adding roles to users

To grant a user access within a project, you can bind an appropriate role to the user and verify the resulting role binding.

You can use the **oc adm** administrator CLI to manage the roles and bindings.

Binding, or adding, a role to users or groups gives the user or group the access that is granted by the role. You can add and remove roles to and from users and groups using **oc adm policy** commands.

You can bind any of the default cluster roles to local users or groups in your project.

Procedure

1. Add a role to a user in a specific project:

```
$ oc adm policy add-role-to-user <role> <user> -n <project>
```

For example, you can add the **admin** role to the **alice** user in **joe** project by running:

```
$ oc adm policy add-role-to-user admin alice -n joe
```

TIP

You can alternatively apply the following YAML to add the role to the user:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: admin-0
  namespace: joe
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: admin
subjects:
- apiGroup: rbac.authorization.k8s.io
  kind: User
  name: alice
```

2. View the local role bindings and verify the addition in the output:

```
$ oc describe rolebinding.rbac -n <project>
```

For example, to view the local role bindings for the **joe** project:

```
$ oc describe rolebinding.rbac -n joe
```

Example output

```
Name:      admin
Labels:    <none>
Annotations: <none>
Role:
  Kind: ClusterRole
  Name: admin
Subjects:
  Kind Name      Namespace
  ----
  User kube:admin

Name:      admin-0
Labels:    <none>
Annotations: <none>
```

```

Role:
  Kind: ClusterRole
  Name: admin
Subjects:
  Kind Name Namespace
  ----
  User alice

Name:      system:deployers
Labels:    <none>
Annotations: openshift.io/description:
            Allows deploymentconfigs in this namespace to rollout pods in
            this namespace. It is auto-managed by a controller; remove
            subjects to disa...

Role:
  Kind: ClusterRole
  Name: system:deployer
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount deployer joe

Name:      system:image-builders
Labels:    <none>
Annotations: openshift.io/description:
            Allows builds in this namespace to push images to this
            namespace. It is auto-managed by a controller; remove subjects
            to disable.

Role:
  Kind: ClusterRole
  Name: system:image-builder
Subjects:
  Kind      Name      Namespace
  ----      -
  ServiceAccount builder joe

Name:      system:image-pullers
Labels:    <none>
Annotations: openshift.io/description:
            Allows all pods in this namespace to pull images from this
            namespace. It is auto-managed by a controller; remove subjects
            to disable.

Role:
  Kind: ClusterRole
  Name: system:image-puller
Subjects:
  Kind Name      Namespace
  ----
  Group system:serviceaccounts:joe

```

The **alice** user has been added to the **admins RoleBinding**.

9.2.7. Creating a local role

You can create a local role and bind it to a user to define custom permissions within a project.

Procedure

1. To create a local role for a project, run the following command:

```
$ oc create role <name> --verb=<verb> --resource=<resource> -n <project>
```

In this command, specify:

- **<name>**, the local role's name
- **<verb>**, a comma-separated list of the verbs to apply to the role
- **<resource>**, the resources that the role applies to
- **<project>**, the project name

For example, to create a local role that allows a user to view pods in the **blue** project, run the following command:

```
$ oc create role podview --verb=get --resource=pod -n blue
```

2. To bind the new role to a user, run the following command:

```
$ oc adm policy add-role-to-user podview user2 --role-namespace=blue -n blue
```

9.2.8. Creating a cluster role

To define custom cluster-wide permissions, you can create a cluster role that specifies the verbs and resources users can access.

Procedure

- To create a cluster role, run the following command:

```
$ oc create clusterrole <name> --verb=<verb> --resource=<resource>
```

In this command, specify:

- **<name>**, the local role's name
- **<verb>**, a comma-separated list of the verbs to apply to the role
- **<resource>**, the resources that the role applies to

For example, to create a cluster role that allows a user to view pods, run the following command:

```
$ oc create clusterrole podviewonly --verb=get --resource=pod
```

9.2.9. Local role binding commands

You can use local role binding commands to review, grant, or remove user and group permissions within the current or a specified project.

When you manage a user or group's associated roles for local role bindings using the following operations, a project may be specified with the **-n** flag. If it is not specified, then the current project is used.

You can use the following commands for local RBAC management.

Table 9.1. Local role binding operations

Command	Description
\$ oc adm policy who-can <verb> <resource>	Indicates which users can perform an action on a resource.
\$ oc adm policy add-role-to-user <role> <username>	Binds a specified role to specified users in the current project.
\$ oc adm policy remove-role-from-user <role> <username>	Removes a given role from specified users in the current project.
\$ oc adm policy remove-user <username>	Removes specified users and all of their roles in the current project.
\$ oc adm policy add-role-to-group <role> <groupname>	Binds a given role to specified groups in the current project.
\$ oc adm policy remove-role-from-group <role> <groupname>	Removes a given role from specified groups in the current project.
\$ oc adm policy remove-group <groupname>	Removes specified groups and all of their roles in the current project.

9.2.10. Cluster role binding commands

You can use cluster role binding commands to grant or remove roles for users and groups across all projects in the cluster.

You can also manage cluster role bindings using the following operations. The **-n** flag is not used for these operations because cluster role bindings use non-namespaced resources.

Table 9.2. Cluster role binding operations

Command	Description
\$ oc adm policy add-cluster-role-to-user <role> <username>	Binds a given role to specified users for all projects in the cluster.

Command	Description
\$ oc adm policy remove-cluster-role-from-user <role> <username>	Removes a given role from specified users for all projects in the cluster.
\$ oc adm policy add-cluster-role-to-group <role> <groupname>	Binds a given role to specified groups for all projects in the cluster.
\$ oc adm policy remove-cluster-role-from-group <role> <groupname>	Removes a given role from specified groups for all projects in the cluster.

9.2.11. Creating a cluster admin

To grant a user full administrative access to the cluster, you can bind the **cluster-admin** cluster role to that user.

The **cluster-admin** role is required to perform administrator level tasks on the OpenShift Container Platform cluster, such as modifying cluster resources.

Prerequisites

- You must have created a user to define as the cluster admin.

Procedure

- Define the user as a cluster admin:

```
$ oc adm policy add-cluster-role-to-user cluster-admin <user>
```

9.2.12. Cluster role bindings for unauthenticated groups

Unauthenticated groups do not have default access to cluster roles. As a cluster administrator, you can grant limited unauthenticated access when required, while ensuring that the change complies with organizational security standards.



NOTE

Before OpenShift Container Platform 4.17, unauthenticated groups were allowed access to some cluster roles. Clusters updated from versions before OpenShift Container Platform 4.17 retain this access for unauthenticated groups.

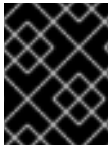
For security reasons OpenShift Container Platform 4.22 does not allow unauthenticated groups to have default access to cluster roles.

There are use cases where it might be necessary to add **system:unauthenticated** to a cluster role.

Cluster administrators can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**

- **system:oauth-token-deleter**
- **self-access-reviewer**



IMPORTANT

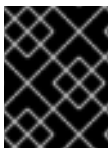
Always verify compliance with your organization's security standards when modifying unauthenticated access.

9.2.13. Adding unauthenticated groups to cluster roles

Grant unauthenticated users access to specific cluster roles to enable features that require cluster access without authentication, such as external webhooks or automated token management.

You can add unauthenticated users to the following cluster roles:

- **system:scope-impersonation**
- **system:webhook**
- **system:oauth-token-deleter**
- **self-access-reviewer**



IMPORTANT

Always verify compliance with your organization's security standards when modifying unauthenticated access.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Create a YAML file named **add-<cluster_role>-unauth.yaml** and add the following content:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    rbac.authorization.kubernetes.io/autoupdate: "true"
  name: <cluster_role>access-unauthenticated
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: <cluster_role>
subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: Group
    name: system:unauthenticated
```

2. Apply the configuration by running the following command:

```
$ oc apply -f add-<cluster_role>.yaml
```

9.3. THE KUBEADMIN USER

OpenShift Container Platform creates a cluster administrator, **kubeadmin**, after the installation process completes. This user has the **cluster-admin** role automatically applied and is treated as the root user for the cluster.

The password is dynamically generated and unique to your OpenShift Container Platform environment. After the installation completes, the password is provided in the installation program's output. For example:

```
INFO Install complete!
INFO Run 'export KUBECONFIG=<your working directory>/auth/kubeconfig' to manage the cluster
with 'oc', the OpenShift CLI.
INFO The cluster is ready when 'oc login -u kubeadmin -p <provided>' succeeds (wait a few minutes).
INFO Access the OpenShift web-console here: https://console-openshift-console.apps.demo1.openshift4-beta-abc.com
INFO Login to the console with user: kubeadmin, password: <provided>
```

9.3.1. Removing the kubeadmin user

After you define an identity provider and create a new **cluster-admin** user, you can remove the **kubeadmin** to improve cluster security.



WARNING

If you follow this procedure before another user is a **cluster-admin**, then OpenShift Container Platform must be reinstalled. It is not possible to undo this command.

Prerequisites

- You must have configured at least one identity provider.
- You must have added the **cluster-admin** role to a user.
- You must be logged in as an administrator.

Procedure

- Remove the **kubeadmin** secrets:

```
$ oc delete secrets kubeadmin -n kube-system
```

9.4. POPULATING THE SOFTWARE CATALOG FROM MIRRORED OPERATOR CATALOGS

If you mirrored Operator catalogs for use with disconnected clusters, you can populate the software catalog with the Operators from your mirrored catalogs. You can use the generated manifests from the mirroring process to create the required **ImageContentSourcePolicy** and **CatalogSource** objects.

9.4.1. Prerequisites

- [Mirroring Operator catalogs for use with disconnected clusters](#)

9.4.1.1. Creating the ImageContentSourcePolicy object

To make mirrored Operator images available to a disconnected cluster, you can create an **ImageContentSourcePolicy** object that redirects image references to your mirror registry.

After mirroring Operator catalog content to your mirror registry, create the required **ImageContentSourcePolicy** (ICSP) object. The ICSP object configures nodes to translate between the image references stored in Operator manifests and the mirrored registry.

Procedure

- On a host with access to the disconnected cluster, create the ICSP by running the following command to specify the **imageContentSourcePolicy.yaml** file in your manifests directory:

```
$ oc create -f <path/to/manifests/dir>/imageContentSourcePolicy.yaml
```

where **<path/to/manifests/dir>** is the path to the manifests directory for your mirrored content.

You can now create a **CatalogSource** object to reference your mirrored index image and Operator content.

9.4.1.2. Adding a catalog source to a cluster

To make Operators from a custom index image available for installation, create a catalog source that adds the catalog content to your cluster.

Cluster administrators can create a **CatalogSource** object that references an index image. The software catalog uses catalog sources to populate the user interface.

TIP

Alternatively, you can use the web console to manage catalog sources. From the **Administration → Cluster Settings → Configuration → OperatorHub** page, click the **Sources** tab, where you can create, update, delete, disable, and enable individual sources.

Prerequisites

- You built and pushed an index image to a registry.
- You have access to the cluster as a user with the **cluster-admin** role.

Procedure

1. Create a **CatalogSource** object that references your index image. If you used the **oc adm catalog mirror** command to mirror your catalog to a target registry, you can use the generated **catalogSource.yaml** file in your manifests directory as a starting point.
 - a. Modify the following to your specifications and save it as a **catalogSource.yaml** file:

```
apiVersion: operators.coreos.com/v1alpha1
kind: CatalogSource
metadata:
  name: my-operator-catalog
  namespace: openshift-marketplace
spec:
  sourceType: grpc
  grpcPodConfig:
    securityContextConfig: <security_mode>
  image: <registry>/<namespace>/redhat-operator-index:v4.22
  displayName: My Operator Catalog
  publisher: <publisher_name>
  updateStrategy:
    registryPoll:
      interval: 30m
```

where:

metadata.name

Specifies the value for the **metadata.name** parameter. If you mirrored content to local files before uploading to a registry, remove any backslash (/) characters from the **metadata.name** field to avoid an "invalid resource name" error when you create the object.

metadata.namespace

Specifies the value for the **metadata.namespace** parameter. If you want the catalog source to be available globally to users in all namespaces, specify the **openshift-marketplace** namespace. Otherwise, you can specify a different namespace for the catalog to be scoped and available only for that namespace.

spec.grpcPodConfig.securityContextConfig

Specifies the value of **legacy** or **restricted**. If the field is not set, the default value is **legacy**. In a future OpenShift Container Platform release, it is planned that the default value will be **restricted**.



NOTE

If your catalog cannot run with **restricted** permissions, it is recommended that you manually set this field to **legacy**.

spec.image

Specifies your index image. If you specify a tag after the image name, for example **:v4.22**, the catalog source pod uses an image pull policy of **Always**, meaning the pod always pulls the image before starting the container. If you specify a digest, for example **@sha256:<id>**, the image pull policy is **IfNotPresent**, meaning the pod pulls the image only if it does not already exist on the node.

spec.publisher

Specifies your name or an organization name publishing the catalog.

spec.updateStrategy.registryPoll

Specifies the value for the **spec.updateStrategy.registryPoll** parameter. The catalog sources can automatically check for new versions to keep up to date.

- b. Use the file to create the **CatalogSource** object:

```
$ oc apply -f catalogSource.yaml
```

2. Verify the following resources are created successfully.

- a. Check the pods:

```
$ oc get pods -n openshift-marketplace
```

The following is example output:

NAME	READY	STATUS	RESTARTS	AGE
my-operator-catalog-6njx6	1/1	Running	0	28s
marketplace-operator-d9f549946-96sgr	1/1	Running	0	26h

- b. Check the catalog source:

```
$ oc get catalogsource -n openshift-marketplace
```

The following is example output:

NAME	DISPLAY	TYPE	PUBLISHER	AGE
my-operator-catalog	My Operator Catalog	grpc		5s

- c. Check the package manifest:

```
$ oc get packagemanifest -n openshift-marketplace
```

The following is example output:

NAME	CATALOG	AGE
jaeger-product	My Operator Catalog	93s

You can now install the Operators from the **Software Catalog** page on your OpenShift Container Platform web console.

Additional resources

- [Accessing images for Operators from private registries](#)
- [Image template for custom catalog sources](#)
- [Image pull policy](#)

9.5. ABOUT OPERATOR INSTALLATION FROM THE SOFTWARE CATALOG

When installing an Operator from the software catalog, you can choose where it is available, which update channel to follow, and whether OLM applies updates automatically or requires manual approval.

The software catalog is a user interface for discovering Operators; it works in conjunction with Operator Lifecycle Manager (OLM), which installs and manages Operators on a cluster.

As a cluster administrator, you can install an Operator from the software catalog by using the OpenShift Container Platform web console or CLI. Subscribing an Operator to one or more namespaces makes the Operator available to developers on your cluster.

During installation, you must determine the following initial settings for the Operator:

Installation Mode

Choose **All namespaces on the cluster (default)** to have the Operator installed on all namespaces or choose individual namespaces, if available, to only install the Operator on selected namespaces. This example chooses **All namespaces...** to make the Operator available to all users and projects.

Update Channel

If an Operator is available through multiple channels, you can choose which channel you want to subscribe to. For example, to deploy from the **stable** channel, if available, select it from the list.

Approval Strategy

You can choose automatic or manual updates.

If you choose automatic updates for an installed Operator, when a new version of that Operator is available in the selected channel, Operator Lifecycle Manager (OLM) automatically upgrades the running instance of your Operator without human intervention.

If you select manual updates, when a newer version of an Operator is available, OLM creates an update request. As a cluster administrator, you must then manually approve that update request to have the Operator updated to the new version.

9.5.1. Installing from the software catalog by using the web console

To make an Operator available in selected namespaces, you can use the web console to choose its version, installation mode, and update approval strategy, and then install it from the software catalog.

You can install and subscribe to an Operator from software catalog by using the OpenShift Container Platform web console. .Prerequisites

- Access to an OpenShift Container Platform cluster using an account with **cluster-admin** permissions.

Procedure

1. Navigate in the web console to the **Ecosystem → Software Catalog** page.
2. Scroll or type a keyword into the **Filter by keyword** box to find the Operator you want. For example, type **jaeger** to find the Jaeger Operator.
You can also filter options by **Infrastructure Features**. For example, select **Disconnected** if you want to see Operators that work in disconnected environments, also known as restricted network environments.

3. Select the Operator to display additional information.



NOTE

Choosing a Community Operator warns that Red Hat does not certify Community Operators; you must acknowledge the warning before continuing.

4. Read the information about the Operator and click **Install**.
5. On the **Install Operator** page, configure your Operator installation:
 - a. If you want to install a specific version of an Operator, select an **Update channel** and **Version** from the lists. You can browse the various versions of an Operator across any channels it might have, view the metadata for that channel and version, and select the exact version you want to install.



NOTE

The version selection defaults to the latest version for the channel selected. If the latest version for the channel is selected, the **Automatic** approval strategy is enabled by default. Otherwise, **Manual** approval is required when not installing the latest version for the selected channel.

Installing an Operator with **Manual** approval causes all Operators installed within the namespace to function with the **Manual** approval strategy and all Operators are updated together. If you want to update Operators independently, install Operators into separate namespaces.

- b. Confirm the installation mode for the Operator:
 - **All namespaces on the cluster (default)** installs the Operator in the default **openshift-operators** namespace to watch and be made available to all namespaces in the cluster. This option is not always available.
 - **A specific namespace on the cluster** allows you to choose a specific, single namespace in which to install the Operator. The Operator will only watch and be made available for use in this single namespace.
 - c. For clusters on cloud providers with token authentication enabled:
 - If the cluster uses AWS Security Token Service (**STS Mode** in the web console), enter the Amazon Resource Name (ARN) of the AWS IAM role of your service account in the **role ARN** field. To create the role's ARN, follow the procedure described in [Preparing AWS account](#).
 - If the cluster uses Microsoft Entra Workload ID (**Workload Identity / Federated Identity Mode** in the web console), add the client ID, tenant ID, and subscription ID in the appropriate fields.
 - If the cluster uses Google Cloud Platform Workload Identity (**GCP Workload Identity / Federated Identity Mode** in the web console), add the project number, pool ID, provider ID, and service account email in the appropriate fields.
 - d. For **Update approval**, select either the **Automatic** or **Manual** approval strategy.



IMPORTANT

If the web console shows that the cluster uses AWS STS, Microsoft Entra Workload ID, or GCP Workload Identity, you must set **Update approval** to **Manual**.

Subscriptions with automatic approvals for updates are not recommended because there might be permission changes to make before updating. Subscriptions with manual approvals for updates ensure that administrators have the opportunity to verify the permissions of the later version, take any necessary steps, and then update.

6. Click **Install** to make the Operator available to the selected namespaces on this OpenShift Container Platform cluster:
 - a. If you selected a **Manual** approval strategy, the upgrade status of the subscription remains **Upgrading** until you review and approve the install plan. After approving on the **Install Plan** page, the subscription upgrade status moves to **Up to date**.
 - b. If you selected an **Automatic** approval strategy, the upgrade status should resolve to **Up to date** without intervention.

Verification

- After the upgrade status of the subscription is **Up to date**, select **Ecosystem → Installed Operators** to verify that the cluster service version (CSV) of the installed Operator eventually shows up. The **Status** should eventually resolve to **Succeeded** in the relevant namespace.



NOTE

For the **All namespaces...** installation mode, the status resolves to **Succeeded** in the **openshift-operators** namespace, but the status is **Copied** if you check in other namespaces.

If it does not:

- Check the logs in any pods in the **openshift-operators** project (or other relevant namespace if **A specific namespace...** installation mode was selected) on the **Workloads → Pods** page that are reporting issues to troubleshoot further.
- When the Operator is installed, the metadata indicates which channel and version are installed.



NOTE

The **Channel** and **Version** dropdown menus are still available for viewing other version metadata in this catalog context.

9.5.2. Installing from the software catalog by using the CLI

To install an Operator from the software catalog by using the CLI, you can identify the appropriate catalog, channel, and install mode, and then create the required **OperatorGroup** and **Subscription** objects.

Instead of using the OpenShift Container Platform web console, you can install an Operator from the software catalog by using the CLI. Use the **oc** command to create or update a **Subscription** object.

For **SingleNamespace** install mode, you must also ensure an appropriate Operator group exists in the related namespace. An Operator group, defined by an **OperatorGroup** object, selects target namespaces in which to generate required RBAC access for all Operators in the same namespace as the Operator group.

TIP

In most cases, the web console method of this procedure is preferred because it automates tasks in the background, such as handling the creation of **OperatorGroup** and **Subscription** objects automatically when choosing **SingleNamespace** mode.

Prerequisites

- Access to your OpenShift Container Platform cluster using an account with **cluster-admin** permissions.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. View the list of Operators available to the cluster from the software catalog:

```
$ oc get packagemanifests -n openshift-marketplace
```

Example 9.1. Example output

NAME	CATALOG	AGE
3scale-operator	Red Hat Operators	91m
advanced-cluster-management	Red Hat Operators	91m
amq7-cert-manager	Red Hat Operators	91m
# ...		
couchbase-enterprise-certified	Certified Operators	91m
crunchy-postgres-operator	Certified Operators	91m
mongodb-enterprise	Certified Operators	91m
# ...		
etcd	Community Operators	91m
jaeger	Community Operators	91m
kubefed	Community Operators	91m
# ...		

Note the catalog for your desired Operator.

2. Inspect your desired Operator to verify its supported install modes and available channels:

```
$ oc describe packagemanifests <operator_name> -n openshift-marketplace
```

Example 9.2. Example output

```
# ...
```

```

Kind:      PackageManifest
# ...
  Install Modes: ❶
    Supported: true
    Type:      OwnNamespace
    Supported: true
    Type:      SingleNamespace
    Supported: false
    Type:      MultiNamespace
    Supported: true
    Type:      AllNamespaces
# ...
  Entries:
    Name:      example-operator.v3.7.11
    Version:   3.7.11
    Name:      example-operator.v3.7.10
    Version:   3.7.10
    Name:      stable-3.7 ❷
# ...
  Entries:
    Name:      example-operator.v3.8.5
    Version:   3.8.5
    Name:      example-operator.v3.8.4
    Version:   3.8.4
    Name:      stable-3.8 ❸
Default Channel: stable-3.8 ❹

```

- ❶ Indicates which install modes are supported.
- ❷❸ Example channel names.
- ❹ The channel selected by default if one is not specified.

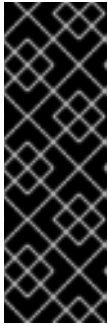
TIP

You can print an Operator's version and channel information in YAML format by running the following command:

```
$ oc get packagemanifests <operator_name> -n <catalog_namespace> -o yaml
```

3. If more than one catalog is installed in a namespace, run the following command to look up the available versions and channels of an Operator from a specific catalog:

```
$ oc get packagemanifest \
  --selector=catalog=<catalogsource_name> \
  --field-selector metadata.name=<operator_name> \
  -n <catalog_namespace> -o yaml
```



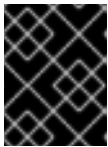
IMPORTANT

If you do not specify the Operator's catalog, running the **oc get packagemanifest** and **oc describe packagemanifest** commands might return a package from an unexpected catalog if the following conditions are met:

- Multiple catalogs are installed in the same namespace.
- The catalogs contain the same Operators or Operators with the same name.

4. If the Operator you intend to install supports the **AllNamespaces** install mode, and you choose to use this mode, skip this step, because the **openshift-operators** namespace already has an appropriate Operator group in place by default, called **global-operators**.

If the Operator you intend to install supports the **SingleNamespace** install mode, and you choose to use this mode, you must ensure an appropriate Operator group exists in the related namespace. If one does not exist, you can create one by following these steps:



IMPORTANT

You can only have one Operator group per namespace. For more information, see "Operator groups".

- a. Create an **OperatorGroup** object YAML file, for example **operatorgroup.yaml**, for **SingleNamespace** install mode:

Example OperatorGroup object for SingleNamespace install mode

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: <operatorgroup_name>
  namespace: <namespace> 1
spec:
  targetNamespaces:
    - <namespace> 2
```

- 1 2 For **SingleNamespace** install mode, use the same **<namespace>** value for both the **metadata.namespace** and **spec.targetNamespaces** fields.

- b. Create the **OperatorGroup** object:

```
$ oc apply -f operatorgroup.yaml
```

5. Create a **Subscription** object to subscribe a namespace to an Operator:

- a. Create a YAML file for the **Subscription** object, for example **subscription.yaml**:



NOTE

If you want to subscribe to a specific version of an Operator, set the **startingCSV** field to the desired version and set the **installPlanApproval** field to **Manual** to prevent the Operator from automatically upgrading if a later version exists in the catalog. For details, see the following "Example **Subscription** object with a specific starting Operator version".

Example 9.3. Example **Subscription** object

```

apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: <subscription_name>
  namespace: <namespace_per_install_mode> ❶
spec:
  channel: <channel_name> ❷
  name: <operator_name> ❸
  source: <catalog_name> ❹
  sourceNamespace: <catalog_source_namespace> ❺
  config:
    env: ❻
    - name: ARGS
      value: "-v=10"
    envFrom: ❼
    - secretRef:
        name: license-secret
  volumes: ❽
  - name: <volume_name>
    configMap:
      name: <configmap_name>
  volumeMounts: ❾
  - mountPath: <directory_name>
    name: <volume_name>
  tolerations: ❿
  - operator: "Exists"
  resources: ⓫
  requests:
    memory: "64Mi"
    cpu: "250m"
  limits:
    memory: "128Mi"
    cpu: "500m"
  nodeSelector: ⓬
    foo: bar

```

- ❶ For default **AllNamespaces** install mode usage, specify the **openshift-operators** namespace. Alternatively, you can specify a custom global namespace, if you have created one. For **SingleNamespace** install mode usage, specify the relevant single namespace.
- ❷ Name of the channel to subscribe to.

- 3 Name of the Operator to subscribe to.
- 4 Name of the catalog source that provides the Operator.
- 5 Namespace of the catalog source. Use **openshift-marketplace** for the default software catalog sources.
- 6 The **env** parameter defines a list of environment variables that must exist in all containers in the pod created by OLM.
- 7 The **envFrom** parameter defines a list of sources to populate environment variables in the container.
- 8 The **volumes** parameter defines a list of volumes that must exist on the pod created by OLM.
- 9 The **volumeMounts** parameter defines a list of volume mounts that must exist in all containers in the pod created by OLM. If a **volumeMount** references a **volume** that does not exist, OLM fails to deploy the Operator.
- 10 The **tolerations** parameter defines a list of tolerations for the pod created by OLM.
- 11 The **resources** parameter defines resource constraints for all the containers in the pod created by OLM.
- 12 The **nodeSelector** parameter defines a **NodeSelector** for the pod created by OLM.

Example 9.4. Example **Subscription** object with a specific starting Operator version

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: example-operator
  namespace: example-operator
spec:
  channel: stable-3.7
  installPlanApproval: Manual 1
  name: example-operator
  source: custom-operators
  sourceNamespace: openshift-marketplace
  startingCSV: example-operator.v3.7.10 2
```

- 1 Set the approval strategy to **Manual** in case your specified version is superseded by a later version in the catalog. This plan prevents an automatic upgrade to a later version and requires manual approval before the starting CSV can complete the installation.
- 2 Set a specific version of an Operator CSV.

b. For clusters on cloud providers with token authentication enabled, such as Amazon Web

Services (AWS) Security Token Service (STS), Microsoft Entra Workload ID, or Google Cloud Platform Workload Identity, configure your **Subscription** object by following these steps:

- i. Ensure the **Subscription** object is set to manual update approvals:

Example 9.5. Example **Subscription** object with manual update approvals

```
kind: Subscription
# ...
spec:
  installPlanApproval: Manual 1
```

- 1 Subscriptions with automatic approvals for updates are not recommended because there might be permission changes to make before updating. Subscriptions with manual approvals for updates ensure that administrators have the opportunity to verify the permissions of the later version, take any necessary steps, and then update.

- ii. Include the relevant cloud provider-specific fields in the **Subscription** object's **config** section:

If the cluster is in AWS STS mode, include the following fields:

Example 9.6. Example **Subscription** object with AWS STS variables

```
kind: Subscription
# ...
spec:
  config:
    env:
      - name: ROLEARN
        value: "<role_arn>" 1
```

- 1 Include the role ARN details.

If the cluster is in Workload ID mode, include the following fields:

Example 9.7. Example **Subscription** object with Workload ID variables

```
kind: Subscription
# ...
spec:
  config:
    env:
      - name: CLIENTID
        value: "<client_id>" 1
      - name: TENANTID
        value: "<tenant_id>" 2
      - name: SUBSCRIPTIONID
        value: "<subscription_id>" 3
```

- 1 Include the client ID.
- 2 Include the tenant ID.
- 3 Include the subscription ID.

If the cluster is in GCP Workload Identity mode, include the following fields:

Example 9.8. Example **Subscription** object with GCP Workload Identity variables

```
kind: Subscription
# ...
spec:
  config:
    env:
      - name: AUDIENCE
        value: "<audience_url>" 1
      - name: SERVICE_ACCOUNT_EMAIL
        value: "<service_account_email>" 2
```

where:

<audience>

Created in Google Cloud by the administrator when they set up GCP Workload Identity, the **AUDIENCE** value must be a preformatted URL in the following format:

```
//iam.googleapis.com/projects/<project_number>/locations/global/workloadIdentityPools/<pool_id>/providers/<provider_id>
```

<service_account_email>

The **SERVICE_ACCOUNT_EMAIL** value is a Google Cloud service account email that is impersonated during Operator operation, for example:

```
<service_account_name>@<project_id>.iam.gserviceaccount.com
```

- c. Create the **Subscription** object by running the following command:

```
$ oc apply -f subscription.yaml
```

6. If you set the **installPlanApproval** field to **Manual**, manually approve the pending install plan to complete the Operator installation. For more information, see "Manually approving a pending Operator update".
- At this point, OLM is now aware of the selected Operator. A cluster service version (CSV) for the Operator should appear in the target namespace, and APIs provided by the Operator should be available for creation.

Verification

1. Check the status of the **Subscription** object for your installed Operator by running the following command:

```
$ oc describe subscription <subscription_name> -n <namespace>
```

2. If you created an Operator group for **SingleNamespace** install mode, check the status of the **OperatorGroup** object by running the following command:

```
$ oc describe operatorgroup <operatorgroup_name> -n <namespace>
```

Additional resources

- [About OperatorGroups](#)

CHAPTER 10. CHANGING THE CLOUD PROVIDER CREDENTIALS CONFIGURATION

You can change your cluster's cloud provider credentials configuration to meet security and authentication requirements. You can rotate or remove credentials, or enable supported short-term credential methods.

For supported configurations, you can change how OpenShift Container Platform authenticates with your cloud provider.

To determine which cloud credentials strategy your cluster uses, see [Determining the Cloud Credential Operator mode](#).

10.1. ROTATING CLOUD PROVIDER SERVICE KEYS WITH THE CLOUD CREDENTIAL OPERATOR UTILITY

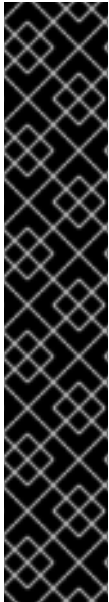
Some organizations require the rotation of the service keys that authenticate the cluster. You can use the Cloud Credential Operator (CCO) utility (**ccctl**) to update keys for clusters installed on the following cloud providers:

- [Amazon Web Services \(AWS\) with Security Token Service \(STS\)](#)
- [Google Cloud with GCP Workload Identity](#)
- [Microsoft Azure with Workload ID](#)
- [IBM Cloud](#)

10.1.1. Rotating AWS OIDC bound service account signer keys

You can rotate the bound service account signer key for an OpenShift Container Platform cluster on Amazon Web Services (AWS) that uses the Cloud Credential Operator (CCO) in manual mode with STS.

To rotate the key, you delete the existing key on your cluster, which causes the Kubernetes API server to create a new key. To reduce authentication failures during this process, you must immediately add the new public key to the existing issuer file. After the cluster is using the new key for authentication, you can remove any remaining keys.



IMPORTANT

The process to rotate OIDC bound service account signer keys is disruptive and takes a significant amount of time. Some steps are time-sensitive. Before proceeding, observe the following considerations:

- Read the following steps and ensure that you understand and accept the time requirement. The exact time requirement varies depending on the individual cluster, but it is likely to require at least one hour.
- To reduce the risk of authentication failures, ensure that you understand and prepare for the time-sensitive steps.
- During this process, you must refresh all service accounts and restart all pods on the cluster. These actions are disruptive to workloads. To mitigate this impact, you can temporarily halt these services and then redeploy them when the cluster is ready.

Prerequisites

- You have access to the OpenShift CLI (**oc**) as a user with the **cluster-admin** role.
- You have created an AWS account for the **ccocctl** utility to use with the following permissions:
 - **s3:GetObject**
 - **s3:PutObject**
 - **s3:PutObjectTagging**
 - For clusters that store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, the AWS account that runs the **ccocctl** utility requires the **cloudfront:ListDistributions** permission.
- You have configured the **ccocctl** utility.
- Your cluster is in a stable state. You can confirm that the cluster is stable by running the following command:

```
$ oc adm wait-for-stable-cluster --minimum-stable-period=5s
```

Procedure

1. Configure the following environment variables:

```
INFRA_ID=$(oc get infrastructures cluster -o jsonpath='{.status.infrastructureName}')
CLUSTER_NAME=${INFRA_ID%-*} 1
```

- 1** **1** This value should match the name of the cluster that was specified in the **metadata.name** field of the **install-config.yaml** file during installation.

**NOTE**

Your cluster might differ from this example, and the resource names might not be derived identically from the cluster name. Ensure that you specify the correct corresponding resource names for your cluster.

- For AWS clusters that store the OIDC configuration in a public S3 bucket, configure the following environment variable:

```
AWS_BUCKET=$(oc get authentication cluster -o jsonpath=
{'spec.serviceAccountIssuer'} | awk -F'://' '{print$2}' |awk -F'.' '{print$1}')
```

- For AWS clusters that store the OIDC configuration in a private S3 bucket that is accessed by the IAM identity provider through a public CloudFront distribution URL, complete the following steps:
 - i. Extract the public CloudFront distribution URL by running the following command:

```
$ basename $(oc get authentication cluster -o jsonpath=
{'spec.serviceAccountIssuer'} )
```

Example output

```
<subdomain>.cloudfront.net
```

where **<subdomain>** is an alphanumeric string.

- ii. Determine the private S3 bucket name by running the following command:

```
$ aws cloudfront list-distributions --query "DistributionList.Items[.DomainName:
DomainName, OriginDomainName: Origins.Items[0].DomainName][?
contains(DomainName, '<subdomain>.cloudfront.net')]"
```

Example output

```
[
  {
    "DomainName": "<subdomain>.cloudfront.net",
    "OriginDomainName": "<s3_bucket>.s3.us-east-2.amazonaws.com"
  }
]
```

where **<s3_bucket>** is the private S3 bucket name for your cluster.

- iii. Configure the following environment variable:

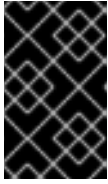
```
AWS_BUCKET=$<s3_bucket>
```

where **<s3_bucket>** is the private S3 bucket name for your cluster.

2. Create a temporary directory to use and assign it an environment variable by running the following command:

```
$ TEMPDIR=$(mktemp -d)
```

3. To cause the Kubernetes API server to create a new bound service account signing key, you delete the next bound service account signing key.



IMPORTANT

After you complete this step, the Kubernetes API server starts to roll out a new key. To reduce the risk of authentication failures, complete the remaining steps as quickly as possible. The remaining steps might be disruptive to workloads.

When you are ready, delete the next bound service account signing key by running the following command:

```
$ oc delete secrets/next-bound-service-account-signing-key \
  -n openshift-kube-apiserver-operator
```

4. Download the public key from the service account signing key secret that the Kubernetes API server created by running the following command:

```
$ oc get secret/next-bound-service-account-signing-key \
  -n openshift-kube-apiserver-operator \
  -ojsonpath='{ .data.service-account\.pub }' | base64 \
  -d > ${TEMPDIR}/serviceaccount-signer.public
```

5. Use the public key to create a **keys.json** file by running the following command:

```
$ ccoctl aws create-identity-provider \
  --dry-run 1 \
  --output-dir ${TEMPDIR} \
  --public-key-file=${TEMPDIR}/serviceaccount-signer.public 2 \
  --name fake 3 \
  --region us-east-1 4
```

- 1** The **--dry-run** option outputs files, including the new **keys.json** file, to the disk without making API calls.
- 2** Specify the path to the public key that you downloaded in the previous step.
- 3** Because the **--dry-run** option does not make any API calls, some parameters do not require real values.
- 4** Specify any valid AWS region, such as **us-east-1**. This value does not need to match the region the cluster is in.

6. Rename the **keys.json** file by running the following command:

```
$ cp ${TEMPDIR}/<number>-keys.json ${TEMPDIR}/jwks.new.json
```

where **<number>** is a two-digit numerical value that varies depending on your environment.

- Download the existing **keys.json** file from the cloud provider by running the following command:

```
$ aws s3api get-object \
  --bucket ${AWS_BUCKET} \
  --key keys.json ${TEMPDIR}/jwks.current.json
```

- Combine the two **keys.json** files by running the following command:

```
$ jq -s '{ keys: map(.keys[])}' ${TEMPDIR}/jwks.current.json ${TEMPDIR}/jwks.new.json >
${TEMPDIR}/jwks.combined.json
```

- To enable authentication for the old and new keys during the rotation, upload the combined **keys.json** file to the cloud provider by running the following command:

```
$ aws s3api put-object \
  --bucket ${AWS_BUCKET} \
  --tagging "openshift.io/cloud-credential-operator/${CLUSTER_NAME}=owned" \
  --key keys.json \
  --body ${TEMPDIR}/jwks.combined.json
```

- Wait for the Kubernetes API server to update and use the new key. You can monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

- To ensure that all pods on the cluster use the new key, you must restart them.



IMPORTANT

This step maintains uptime for services that are configured for high availability across multiple nodes, but might cause downtime for any services that are not.

Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

- Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

- Monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

14. Replace the combined **keys.json** file with the updated **keys.json** file on the cloud provider by running the following command:

```
$ aws s3api put-object \
  --bucket ${AWS_BUCKET} \
  --tagging "openshift.io/cloud-credential-operator/${CLUSTER_NAME}=owned" \
  --key keys.json \
  --body ${TMPDIR}/jwks.new.json
```

10.1.2. Rotating Google Cloud OIDC bound service account signer keys

You can rotate the bound service account signer key for an OpenShift Container Platform cluster on Google Cloud that uses the Cloud Credential Operator (CCO) in manual mode with GCP Workload Identity.

To rotate the key, you delete the existing key on your cluster, which causes the Kubernetes API server to create a new key. To reduce authentication failures during this process, you must immediately add the new public key to the existing issuer file. After the cluster is using the new key for authentication, you can remove any remaining keys.

IMPORTANT

The process to rotate OIDC bound service account signer keys is disruptive and takes a significant amount of time. Some steps are time-sensitive. Before proceeding, observe the following considerations:

- Read the following steps and ensure that you understand and accept the time requirement. The exact time requirement varies depending on the individual cluster, but it is likely to require at least one hour.
- To reduce the risk of authentication failures, ensure that you understand and prepare for the time-sensitive steps.
- During this process, you must refresh all service accounts and restart all pods on the cluster. These actions are disruptive to workloads. To mitigate this impact, you can temporarily halt these services and then redeploy them when the cluster is ready.

Prerequisites

- You have access to the OpenShift CLI (**oc**) as a user with the **cluster-admin** role.
- You have added one of the following authentication options to the Google Cloud account that the **ccctl** utility uses:
 - The **IAM Workload Identity Pool Admin** role

- The following granular permissions:
 - **storage.objects.create**
 - **storage.objects.delete**
- You have configured the **ccoctl** utility.
- Your cluster is in a stable state. You can confirm that the cluster is stable by running the following command:

```
$ oc adm wait-for-stable-cluster --minimum-stable-period=5s
```

Procedure

1. Configure the following environment variables:

```
CURRENT_ISSUER=$(oc get authentication cluster -o
jsonpath='{.spec.serviceAccountIssuer}')
GCP_BUCKET=$(echo ${CURRENT_ISSUER} | cut -d "/" -f4)
CLUSTER_NAME=${GCP_BUCKET%-*}
```



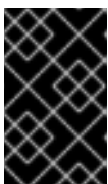
NOTE

Your cluster might differ from this example, and the resource names might not be derived identically from the cluster name. Ensure that you specify the correct corresponding resource names for your cluster.

2. Create a temporary directory to use and assign it an environment variable by running the following command:

```
$ TEMPDIR=$(mktemp -d)
```

3. To cause the Kubernetes API server to create a new bound service account signing key, you delete the next bound service account signing key.



IMPORTANT

After you complete this step, the Kubernetes API server starts to roll out a new key. To reduce the risk of authentication failures, complete the remaining steps as quickly as possible. The remaining steps might be disruptive to workloads.

When you are ready, delete the next bound service account signing key by running the following command:

```
$ oc delete secrets/next-bound-service-account-signing-key \
-n openshift-kube-apiserver-operator
```

4. Download the public key from the service account signing key secret that the Kubernetes API server created by running the following command:

```
$ oc get secret/next-bound-service-account-signing-key \
-n openshift-kube-apiserver-operator \
```



```
-ojsonpath='{ .data.service-account\pub }' | base64 \
-d > ${TEMPDIR}/serviceaccount-signer.public
```

5. Use the public key to create a **keys.json** file by running the following command:

```
$ ccoctl gcp create-workload-identity-provider \
  --dry-run \ ❶
  --output-dir=${TEMPDIR} \
  --public-key-file=${TEMPDIR}/serviceaccount-signer.public \ ❷
  --name fake \ ❸
  --project fake \
  --workload-identity-pool fake
```

- ❶ The **--dry-run** option outputs files, including the new **keys.json** file, to the disk without making API calls.
- ❷ Specify the path to the public key that you downloaded in the previous step.
- ❸ Because the **--dry-run** option does not make any API calls, some parameters do not require real values.

6. Rename the **keys.json** file by running the following command:

```
$ cp ${TEMPDIR}/<number>-keys.json ${TEMPDIR}/jwks.new.json
```

where **<number>** is a two-digit numerical value that varies depending on your environment.

7. Download the existing **keys.json** file from the cloud provider by running the following command:

- For Google Cloud clusters that store OIDC keys in a public bucket, run the following command:

```
$ gcloud storage cp gs://${GCP_BUCKET}/keys.json ${TEMPDIR}/jwks.current.json
```

- For Google Cloud clusters that attach OIDC keys directly to the workload identity pool, run the following command:

```
$ gcloud iam workload-identity-pools providers describe \
  --format json \
  --location global \
  --workload-identity-pool ${CLUSTER_NAME} ${CLUSTER_NAME} \
  | jq -r ".oidc.jwksJson" > ${TEMPDIR}/jwks.current.json
```

8. Combine the two **keys.json** files by running the following command:

```
$ jq -s '{ keys: map(.keys[])}' ${TEMPDIR}/jwks.current.json ${TEMPDIR}/jwks.new.json >
${TEMPDIR}/jwks.combined.json
```

9. To enable authentication for the old and new keys during the rotation, upload the combined **keys.json** file to the cloud provider by running the following command:

- For Google Cloud clusters that store OIDC keys in a public bucket, run the following command:

```
$ gcloud storage cp ${TEMPDIR}/jwks.combined.json gs://${GCP_BUCKET}/keys.json
```

- For Google Cloud clusters that attach OIDC keys directly to the workload identity pool, run the following command:

```
$ gcloud iam workload-identity-pools providers update-oidc ${CLUSTER_NAME} \
  --location=global \
  --workload-identity-pool=${CLUSTER_NAME} \
  --jwk-json-path=${TEMPDIR}/jwks.combined.json
```

10. Wait for the Kubernetes API server to update and use the new key. You can monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

11. To ensure that all pods on the cluster use the new key, you must restart them.



IMPORTANT

This step maintains uptime for services that are configured for high availability across multiple nodes, but might cause downtime for any services that are not.

Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

12. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

13. Monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

14. Replace the combined **keys.json** file with the updated **keys.json** file on the cloud provider by running the following command:

- For Google Cloud clusters that store OIDC keys in a public bucket, run the following command:

```
$ gcloud storage cp ${TEMPDIR}/jwks.new.json gs://${GCP_BUCKET}/keys.json
```

- For Google Cloud clusters that attach OIDC keys directly to the workload identity pool, run the following command:

```
$ gcloud iam workload-identity-pools providers update-oidc ${CLUSTER_NAME} \
  --location=global \
  --workload-identity-pool=${CLUSTER_NAME} \
  --jwk-json-path=${TEMPDIR}/jwks.new.json
```

10.1.3. Rotating Azure OIDC bound service account signer keys

You can rotate the bound service account signer key for an OpenShift Container Platform cluster on Microsoft Azure that uses the Cloud Credential Operator (CCO) in manual mode with Microsoft Entra Workload ID.

To rotate the key, you delete the existing key on your cluster, which causes the Kubernetes API server to create a new key. To reduce authentication failures during this process, you must immediately add the new public key to the existing issuer file. After the cluster is using the new key for authentication, you can remove any remaining keys.

IMPORTANT

The process to rotate OIDC bound service account signer keys is disruptive and takes a significant amount of time. Some steps are time-sensitive. Before proceeding, observe the following considerations:

- Read the following steps and ensure that you understand and accept the time requirement. The exact time requirement varies depending on the individual cluster, but it is likely to require at least one hour.
- To reduce the risk of authentication failures, ensure that you understand and prepare for the time-sensitive steps.
- During this process, you must refresh all service accounts and restart all pods on the cluster. These actions are disruptive to workloads. To mitigate this impact, you can temporarily halt these services and then redeploy them when the cluster is ready.

Prerequisites

- You have access to the OpenShift CLI (**oc**) as a user with the **cluster-admin** role.
- You have created a global Azure account for the **ccoctl** utility to use with the following permissions:
 - **Microsoft.Storage/storageAccounts/listkeys/action**
 - **Microsoft.Storage/storageAccounts/read**

- **Microsoft.Storage/storageAccounts/write**
- **Microsoft.Storage/storageAccounts/blobServices/containers/read**
- **Microsoft.Storage/storageAccounts/blobServices/containers/write**
- You have configured the **ccctl** utility.
- Your cluster is in a stable state. You can confirm that the cluster is stable by running the following command:

```
$ oc adm wait-for-stable-cluster --minimum-stable-period=5s
```

Procedure

1. Configure the following environment variables:

```
CURRENT_ISSUER=$(oc get authentication cluster -o
jsonpath='{.spec.serviceAccountIssuer}')
AZURE_STORAGE_ACCOUNT=$(echo ${CURRENT_ISSUER} | cut -d "/" -f3 | cut -d "." -f1)
AZURE_STORAGE_CONTAINER=$(echo ${CURRENT_ISSUER} | cut -d "/" -f4)
```



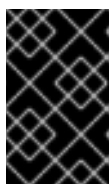
NOTE

Your cluster might differ from this example, and the resource names might not be derived identically from the cluster name. Ensure that you specify the correct corresponding resource names for your cluster.

2. Create a temporary directory to use and assign it an environment variable by running the following command:

```
$ TEMPDIR=$(mktemp -d)
```

3. To cause the Kubernetes API server to create a new bound service account signing key, you delete the next bound service account signing key.



IMPORTANT

After you complete this step, the Kubernetes API server starts to roll out a new key. To reduce the risk of authentication failures, complete the remaining steps as quickly as possible. The remaining steps might be disruptive to workloads.

When you are ready, delete the next bound service account signing key by running the following command:

```
$ oc delete secrets/next-bound-service-account-signing-key \
-n openshift-kube-apiserver-operator
```

4. Download the public key from the service account signing key secret that the Kubernetes API server created by running the following command:

```
$ oc get secret/next-bound-service-account-signing-key \
```

```
-n openshift-kube-apiserver-operator \
-ojsonpath='{ .data.service-account\.pub }' | base64 \
-d > ${TEMPDIR}/serviceaccount-signer.public
```

5. Use the public key to create a **keys.json** file by running the following command:

```
$ ccoctl aws create-identity-provider \ ❶
--dry-run \ ❷
--output-dir ${TEMPDIR} \
--public-key-file=${TEMPDIR}/serviceaccount-signer.public \ ❸
--name fake \ ❹
--region us-east-1 ❺
```

- ❶ The **ccoctl azure** command does not include a **--dry-run** option. To use the **--dry-run** option, you must specify **aws** for an Azure cluster.
- ❷ The **--dry-run** option outputs files, including the new **keys.json** file, to the disk without making API calls.
- ❸ Specify the path to the public key that you downloaded in the previous step.
- ❹ Because the **--dry-run** option does not make any API calls, some parameters do not require real values.
- ❺ Specify any valid AWS region, such as **us-east-1**. This value does not need to match the region the cluster is in.

6. Rename the **keys.json** file by running the following command:

```
$ cp ${TEMPDIR}/<number>-keys.json ${TEMPDIR}/jwks.new.json
```

where **<number>** is a two-digit numerical value that varies depending on your environment.

7. Download the existing **keys.json** file from the cloud provider by running the following command:

```
$ az storage blob download \
--container-name ${AZURE_STORAGE_CONTAINER} \
--account-name ${AZURE_STORAGE_ACCOUNT} \
--name 'openid/v1/jwks' \
-f ${TEMPDIR}/jwks.current.json
```

8. Combine the two **keys.json** files by running the following command:

```
$ jq -s '{ keys: map(.keys[])}' ${TEMPDIR}/jwks.current.json ${TEMPDIR}/jwks.new.json >
${TEMPDIR}/jwks.combined.json
```

9. To enable authentication for the old and new keys during the rotation, upload the combined **keys.json** file to the cloud provider by running the following command:

```
$ az storage blob upload \
--overwrite \
--account-name ${AZURE_STORAGE_ACCOUNT} \
```

```
--container-name ${AZURE_STORAGE_CONTAINER} \
--name 'openid/v1/jwks' \
-f ${TEMPDIR}/jwks.combined.json
```

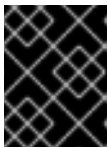
10. Wait for the Kubernetes API server to update and use the new key. You can monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

11. To ensure that all pods on the cluster use the new key, you must restart them.



IMPORTANT

This step maintains uptime for services that are configured for high availability across multiple nodes, but might cause downtime for any services that are not.

Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

12. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

13. Monitor the update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

14. Replace the combined **keys.json** file with the updated **keys.json** file on the cloud provider by running the following command:

```
$ az storage blob upload \
--overwrite \
--account-name ${AZURE_STORAGE_ACCOUNT} \
--container-name ${AZURE_STORAGE_CONTAINER} \
--name 'openid/v1/jwks' \
-f ${TEMPDIR}/jwks.new.json
```

10.1.4. Rotating IBM Cloud credentials

You can rotate API keys for existing IBM Cloud service IDs and update the corresponding cluster secrets to maintain valid cloud provider credentials.

You can rotate API keys for your existing service IDs and update the corresponding secrets.

Prerequisites

- You have configured the **ccctl** utility.
- You have existing service IDs in a live OpenShift Container Platform cluster installed.

Procedure

- Use the **ccctl** utility to rotate your API keys for the service IDs and update the secrets by running the following command:

```
$ ccctl <provider_name> refresh-keys \ ❶
--kubeconfig <openshift_kubeconfig_file> \ ❷
--credentials-requests-dir <path_to_credential_requests_directory> \ ❸
--name <name> ❹
```

- ❶ The name of the provider. For example: **ibmcloud** or **powervs**.
- ❷ The **kubeconfig** file associated with the cluster. For example, **<installation_directory>/auth/kubeconfig**.
- ❸ The directory where the credential requests are stored.
- ❹ The name of the OpenShift Container Platform cluster.



NOTE

If your cluster uses Technology Preview features that are enabled by the **TechPreviewNoUpgrade** feature set, you must include the **--enable-tech-preview** parameter.

10.2. ROTATING CLOUD PROVIDER CREDENTIALS

Some organizations require the rotation of the cloud provider credentials. To allow the cluster to use the new credentials, you must update the secrets that the [Cloud Credential Operator \(CCO\)](#) uses to manage cloud provider credentials.

10.2.1. Rotating cloud provider credentials manually

If your cloud provider credentials are changed for any reason, you must manually update the secret that the Cloud Credential Operator (CCO) uses to manage cloud provider credentials.

The process for rotating cloud credentials depends on the mode that the CCO is configured to use. After you rotate credentials for a cluster that is using mint mode, you must manually remove the component credentials that were created by the removed credential.

Prerequisites

- Your cluster is installed on a platform that supports rotating cloud credentials manually with the CCO mode that you are using:
 - For mint mode, Amazon Web Services (AWS) and Google Cloud are supported.
 - For passthrough mode, Amazon Web Services (AWS), Microsoft Azure, Google Cloud, Red Hat OpenStack Platform (RHOSP), and VMware vSphere are supported.
- You have changed the credentials that are used to interface with your cloud provider.
- The new credentials have sufficient permissions for the mode CCO is configured to use in your cluster.

Procedure

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. In the table on the **Secrets** page, find the root secret for your cloud provider.

Platform	Secret name
AWS	aws-creds
Azure	azure-credentials
Google Cloud	gcp-credentials
RHOSP	openstack-credentials
VMware vSphere	vsphere-creds

3. Click the Options menu  in the same row as the secret and select **Edit Secret**.
4. Record the contents of the **Value** field or fields. You can use this information to verify that the value is different after updating the credentials.
5. Update the text in the **Value** field or fields with the new authentication information for your cloud provider, and then click **Save**.
6. If you are updating the credentials for a vSphere cluster that does not have the vSphere CSI Driver Operator enabled, you must force a rollout of the Kubernetes controller manager to apply the updated credentials.



NOTE

If the vSphere CSI Driver Operator is enabled, this step is not required.

To apply the updated vSphere credentials, log in to the OpenShift Container Platform CLI as a user with the **cluster-admin** role and run the following command:

```
$ oc patch kubecontrollermanager cluster \
  -p='{"spec": {"forceRedeploymentReason": "recovery-"'$( date )'""}}' \
  --type=merge
```

While the credentials are rolling out, the status of the Kubernetes Controller Manager Operator reports **Progressing=true**. To view the status, run the following command:

```
$ oc get co kube-controller-manager
```

7. If the CCO for your cluster is configured to use mint mode, delete each component secret that is referenced by the individual **CredentialsRequest** objects.

- a. Log in to the OpenShift Container Platform CLI as a user with the **cluster-admin** role.
- b. Get the names and namespaces of all referenced component secrets:

```
$ oc -n openshift-cloud-credential-operator get CredentialsRequest \
  -o json | jq -r '.items[] | select (.spec.providerSpec.kind=="<provider_spec>") | \
  .spec.secretRef'
```

where **<provider_spec>** is the corresponding value for your cloud provider:

- AWS: **AWSProviderSpec**
- Google Cloud: **GCPPProviderSpec**

The following example is partial output for the command on an AWS cluster:

```
{
  "name": "ebs-cloud-credentials",
  "namespace": "openshift-cluster-csi-drivers"
}
{
  "name": "cloud-credential-operator-iam-ro-creds",
  "namespace": "openshift-cloud-credential-operator"
}
```

- c. Delete each of the referenced component secrets:

```
$ oc delete secret <secret_name> 1 \
  -n <secret_namespace> 2
```

where:

<secret_name>

Specifies the name of a secret.

<secret_namespace>

Specifies the namespace that contains the secret.

The following example is a command to delete an AWS secret:

```
$ oc delete secret ebs-cloud-credentials -n openshift-cluster-csi-drivers
```

You do not need to manually delete the credentials from your provider console. Deleting the referenced component secrets will cause the CCO to delete the existing credentials from the platform and create new ones.

Verification

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. Verify that the contents of the **Value** field or fields have changed.

Additional resources

- [The Cloud Credential Operator in mint mode](#)
- [The Cloud Credential Operator in passthrough mode](#)
- [vSphere CSI Driver Operator](#)

10.3. REMOVING CLOUD PROVIDER CREDENTIALS

After installing OpenShift Container Platform, some organizations require the removal of the cloud provider credentials that were used during the initial installation. To allow the cluster to use the new credentials, you must update the secrets that the [Cloud Credential Operator \(CCO\)](#) uses to manage cloud provider credentials.

10.3.1. Removing cloud provider credentials

You can remove administrator-level cloud provider credentials from a cluster that uses the Cloud Credential Operator in mint mode to reduce the risk of credential exposure after installation.

For clusters that use the Cloud Credential Operator (CCO) in mint mode, the administrator-level credential is stored in the **kube-system** namespace. The CCO uses the **admin** credential to process the **CredentialsRequest** objects in the cluster and create users for components with limited permissions.

After installing an OpenShift Container Platform cluster with the CCO in mint mode, you can remove the administrator-level credential secret from the **kube-system** namespace in the cluster. The CCO only requires the administrator-level credential during changes that require reconciling new or modified **CredentialsRequest** custom resources, such as minor cluster version updates.



NOTE

Before performing a minor version cluster update (for example, updating from OpenShift Container Platform 4.21 to 4.22), you must reinstate the credential secret with the administrator-level credential. If the credential is not present, the update might be blocked.

Prerequisites

- Your cluster is installed on a platform that supports removing cloud credentials from the CCO. Supported platforms are AWS and Google Cloud.

Procedure

1. In the **Administrator** perspective of the web console, navigate to **Workloads → Secrets**.
2. In the table on the **Secrets** page, find the root secret for your cloud provider.

Platform	Secret name
AWS	aws-creds
Google Cloud	gcp-credentials

3. Click the Options menu  in the same row as the secret and select **Delete Secret**

Additional resources

- [The Cloud Credential Operator in mint mode](#)

10.4. ENABLING TOKEN-BASED AUTHENTICATION

After installing an OpenShift Container Platform cluster on Microsoft Azure or Amazon Web Services (AWS), you can enable Microsoft Entra Workload ID or Security Token Service (STS) to use short-term credentials.

10.4.1. Configuring the Cloud Credential Operator utility

To configure an existing cluster to create and manage cloud credentials from outside of the cluster, extract and prepare the Cloud Credential Operator utility (**ccocctl**) binary.



NOTE

The **ccocctl** utility is a Linux binary that must run in a Linux environment.

Prerequisites

- You have access to an OpenShift Container Platform account with cluster administrator access.
- You have installed the OpenShift CLI (**oc**).

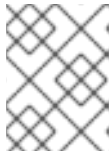
Procedure

1. Set a variable for the OpenShift Container Platform release image by running the following command:

```
$ RELEASE_IMAGE=$(oc get clusterversion -o jsonpath={..desired.image})
```

2. Obtain the CCO container image from the OpenShift Container Platform release image by running the following command:

```
$ CCO_IMAGE=$(oc adm release info --image-for='cloud-credential-operator'
$RELEASE_IMAGE -a ~/.pull-secret)
```

**NOTE**

Ensure that the architecture of the **\$RELEASE_IMAGE** matches the architecture of the environment in which you will use the **ccoctl** tool.

3. Extract the **ccoctl** binary from the CCO container image within the OpenShift Container Platform release image by running the following command:

```
$ oc image extract $CCO_IMAGE \
  --file="/usr/bin/ccoctl.<rhel_version>" \
  -a ~/.pull-secret
```

For **<rhel_version>**, specify the value that corresponds to the version of Red Hat Enterprise Linux (RHEL) that the host uses. If no value is specified, **ccoctl.rhel8** is used by default. The following values are valid:

- **rhel8**: Specify this value for hosts that use RHEL 8.
- **rhel9**: Specify this value for hosts that use RHEL 9.

**NOTE**

The **ccoctl** binary is created in the directory from where you executed the command and not in **/usr/bin/**. You must rename the directory or move the **ccoctl.<rhel_version>** binary to **ccoctl**.

4. Change the permissions to make **ccoctl** executable by running the following command:

```
$ chmod 775 ccoctl
```

Verification

- To verify that **ccoctl** is ready to use, display the help file. Use a relative file name when you run the command, for example:

```
$ ./ccoctl
```

Example output

```
OpenShift credentials provisioning tool
```

```
Usage:
```

```
ccoctl [command]
```

```
Available Commands:
```

```
aws      Manage credentials objects for AWS cloud
azure    Manage credentials objects for Azure
gcp      Manage credentials objects for Google cloud
help     Help about any command
ibmcloud Manage credentials objects for IBM Cloud
nutanix  Manage credentials objects for Nutanix
```

Flags:`-h, --help` help for `ccoctl`Use "`ccoctl [command] --help`" for more information about a command.

10.4.2. Enabling Microsoft Entra Workload ID on an existing cluster

Enable Microsoft Entra Workload ID on an existing Microsoft Azure OpenShift Container Platform cluster. If you did not configure your cluster to use Microsoft Entra Workload ID during installation, you can enable this authentication method post-installation.

IMPORTANT

The process to enable Workload ID on an existing cluster is disruptive and takes a significant amount of time. Before proceeding, observe the following considerations:

- Read the following steps and ensure that you understand and accept the time requirement. The exact time requirement varies depending on the individual cluster, but it is likely to require at least one hour.
- During this process, you must refresh all service accounts and restart all pods on the cluster. These actions are disruptive to workloads. To mitigate this impact, you can temporarily halt these services and then redeploy them when the cluster is ready.
- After starting this process, do not attempt to update the cluster until it is complete. If an update is triggered, the process to enable Workload ID on an existing cluster fails.

Prerequisites

- You have installed an OpenShift Container Platform cluster on Microsoft Azure.
- You have access to the cluster using an account with **cluster-admin** permissions.
- You have installed the OpenShift CLI (**oc**).
- You have extracted and prepared the Cloud Credential Operator utility (**ccoctl**) binary.
- You have access to your Azure account by using the Azure CLI (**az**).

Procedure

1. Create an output directory for the manifests that the **ccoctl** utility generates. This procedure uses `./output_dir` as an example.
2. Extract the service account public signing key for the cluster to the output directory by running the following command:

```
$ oc get secret/next-bound-service-account-signing-key \
  -n openshift-kube-apiserver-operator \
  -ojsonpath='{ .data.service-account\.pub }' | base64 -d \
  > output_dir/serviceaccount-signer.public 1
```

1 This procedure uses a file named **serviceaccount-signer.public** as an example.

3. Use the extracted service account public signing key to create an OpenID Connect (OIDC) issuer and Azure blob storage container with OIDC configuration files by running the following command:

```
$ ./ccoctl azure create-oidc-issuer \
  --name <azure_infra_name> \ 1
  --output-dir ./output_dir \
  --region <azure_region> \ 2
  --subscription-id <azure_subscription_id> \ 3
  --tenant-id <azure_tenant_id> \
  --public-key-file ./output_dir/serviceaccount-signer.public 4
```

- 1** The value of the **name** parameter is used to create an Azure resource group. To use an existing Azure resource group instead of creating a new one, specify the **--oidc-resource-group-name** argument with the existing group name as its value.
- 2** Specify the region of the existing cluster.
- 3** Specify the subscription ID of the existing cluster.
- 4** Specify the file that contains the service account public signing key for the cluster.

4. Verify that the configuration file for the Azure pod identity webhook was created by running the following command:

```
$ ll ./output_dir/manifests
```

Example output

```
total 8
-rw-----. 1 cloud-user cloud-user 193 May 22 02:29 azure-ad-pod-identity-webhook-
config.yaml 1
-rw-----. 1 cloud-user cloud-user 165 May 22 02:29 cluster-authentication-02-config.yaml
```

- 1** The file **azure-ad-pod-identity-webhook-config.yaml** contains the Azure pod identity webhook configuration.

5. Set an **OIDC_ISSUER_URL** variable with the OIDC issuer URL from the generated manifests in the output directory by running the following command:

```
$ OIDC_ISSUER_URL=`awk '/serviceAccountIssuer/ { print $2 }'
./output_dir/manifests/cluster-authentication-02-config.yaml`
```

6. Update the **spec.serviceAccountIssuer** parameter of the cluster **authentication** configuration by running the following command:

```
$ oc patch authentication cluster \
  --type=merge \
  -p '{"spec":{"serviceAccountIssuer":"${OIDC_ISSUER_URL}"}}'
```

- 7. Monitor the configuration update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

- 8. Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

Restarting a pod updates the **serviceAccountIssuer** field and refreshes the service account public signing key.

- 9. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

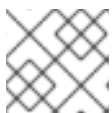
```
All nodes rebooted
```

- 10. Update the Cloud Credential Operator **spec.credentialsMode** parameter to **Manual** by running the following command:

```
$ oc patch cloudcredential cluster \
  --type=merge \
  --patch '{"spec":{"credentialsMode":"Manual"}}'
```

- 11. Extract the list of **CredentialsRequest** objects from the OpenShift Container Platform release image by running the following command:

```
$ oc adm release extract \
  --credentials-requests \
  --included \
  --to <path_to_directory_for_credentials_requests> \
  --registry-config ~/.pull-secret
```



NOTE

This command might take a few moments to run.

- 12. Set an **AZURE_INSTALL_RG** variable with the Azure resource group name by running the following command:

```
$ AZURE_INSTALL_RG=`oc get infrastructure cluster -o jsonpath --template '{.status.platformStatus.azure.resourceGroupName}'`
```

13. Use the **ccctl** utility to create managed identities for all **CredentialsRequest** objects by running the following command:



NOTE

The following command does not show all available options. For a complete list of options, including those that might be necessary for your specific use case, run **\$ ccctl azure create-managed-identities --help**.

```
$ ccctl azure create-managed-identities \
  --name <azure_infra_name> \
  --output-dir ./output_dir \
  --region <azure_region> \
  --subscription-id <azure_subscription_id> \
  --credentials-requests-dir <path_to_directory_for_credentials_requests> \
  --issuer-url "${OIDC_ISSUER_URL}" \
  --dnszone-resource-group-name <azure_dns_zone_resourcegroup_name> \
  --installation-resource-group-name "${AZURE_INSTALL_RG}" \
  --network-resource-group-name <azure_resource_group> \
  --preserve-existing-roles
```

- 1 Specify the name of the resource group that contains the DNS zone.
- 2 Optional: Specify the virtual network resource group if it is different from the cluster resource group.
- 3 Optional: Specify this flag to ensure that any custom role assignments you define on managed identities are not removed during OpenShift Container Platform updates.

14. Apply the Azure pod identity webhook configuration for Workload ID by running the following command:

```
$ oc apply -f ./output_dir/manifests/azure-ad-pod-identity-webhook-config.yaml
```

15. Apply the secrets generated by the **ccctl** utility by running the following command:

```
$ find ./output_dir/manifests -iname "openshift*.yaml" -print0 | xargs -l {} -0 -t oc replace -f {}
```

This process might take several minutes.

16. Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

Restarting a pod updates the **serviceAccountIssuer** field and refreshes the service account public signing key.

17. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```


This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

18. Monitor the configuration update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

19. Optional: Remove the Azure root credentials secret by running the following command:

```
$ oc delete secret -n kube-system azure-credentials
```

10.4.3. Enabling AWS Security Token Service (STS) on an existing cluster

Enable AWS Security Token Service (STS) on an existing OpenShift Container Platform cluster if you did not configure this authentication method during installation.

IMPORTANT

The process to enable STS on an existing cluster is disruptive and takes a significant amount of time. Before proceeding, observe the following considerations:

- Read the following steps and ensure that you understand and accept the time requirement. The exact time requirement varies depending on the individual cluster, but it is likely to require at least one hour.
- During this process, you must refresh all service accounts and restart all pods on the cluster. These actions are disruptive to workloads. To mitigate this impact, you can temporarily halt these services and then redeploy them when the cluster is ready.
- Do not update the cluster until this process is complete.

Prerequisites

- You have installed an OpenShift Container Platform cluster on AWS.
- You have access to the cluster using an account with **cluster-admin** permissions.
- You have installed the OpenShift CLI (**oc**).
- You have extracted and prepared the Cloud Credential Operator utility (**ccocli**) binary.
- You have access to your AWS account by using the AWS CLI (**aws**).

Procedure

1. Create an output directory for **ccoctl** generated manifests.

```
$ mkdir ./output_dir
```

2. Create the AWS Identity and Access Management (IAM) OpenID Connect (OIDC) provider.

- a. Extract the service account public signing key for the cluster by running the following command:

```
$ oc get secret/next-bound-service-account-signing-key \
  -n openshift-kube-apiserver-operator \
  -ojsonpath='{ .data.service-account\.pub }' | base64 -d \
  > output_dir/serviceaccount-signer.public ❶
```

- ❶ This procedure uses a file named **serviceaccount-signer.public** as an example.

- b. Create the AWS IAM identity provider and S3 bucket by running the following command:

```
$ ./ccoctl aws create-identity-provider \
  --output-dir output_dir \ ❶
  --name <name_you_choose> \ ❷
  --region us-east-2 \ ❸
  --public-key-file output_dir/serviceaccount-signer.public ❹
```

- ❶ Specify the output directory you created earlier.

- ❷ Specify a globally unique name. This name functions as a prefix for AWS resources created by this command.

- ❸ Specify the AWS region of the cluster.

- ❹ Specify the relative path to the **serviceaccount-signer.public** file you created earlier.

- c. Save or note the Amazon Resource Name (ARN) for the IAM identity provider. You can find this information in the final line of the output of the previous command.

3. Update the cluster authentication configuration.

- a. Extract the OIDC issuer URL and update the authentication configuration of the cluster by running the following commands:

```
$ OIDC_ISSUER_URL=`awk '/serviceAccountIssuer/ { print $2 }'
  output_dir/manifests/cluster-authentication-02-config.yaml`
$ oc patch authentication cluster --type=merge -p '{"spec":
  {"serviceAccountIssuer":"'${OIDC_ISSUER_URL}'"}}
```

- b. Monitor the configuration update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
-
```

All clusteroperators are stable

4. Restart pods to apply the issuer update.

- a. Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

Restarting a pod updates the **serviceAccountIssuer** field and refreshes the service account public signing key.

- b. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

5. Update the Cloud Credential Operator **spec.credentialsMode** parameter to **Manual** by running the following command:

```
$ oc patch cloudcredential cluster \
  --type=merge \
  --patch '{"spec":{"credentialsMode":"Manual"}}'
```

6. Extract **CredentialsRequests** objects.

- a. Create a **CLUSTER_VERSION** environment variable by running the following command:

```
$ CLUSTER_VERSION=$(oc get clusterversion version -o json | jq -r
'.status.desired.version')
```

- b. Create a **CLUSTER_IMAGE** environment variable by running the following command:

```
$ CLUSTER_IMAGE=$(oc get clusterversion version -o json | jq -r ".status.history[] |
select(.version == \"${CLUSTER_VERSION}\") | .image")
```

- c. Extract **CredentialsRequests** objects from the release image by running the following command:

```
$ oc adm release extract \
  --credentials-requests \
  --cloud=aws \
  --from ${CLUSTER_IMAGE} \
  --to output_dir/cred-reqs
```

7. Create AWS IAM roles and apply secrets.

- a. Create an IAM role for each **CredentialsRequests** object by running the following command:

```
$ ./ccoctl aws create-iam-roles \
  --output-dir ./output_dir/ \ 1
  --name <name_you_choose> \ 2
  --identity-provider-arn <identity_provider_arn> \ 3
  --region us-east-2 \ 4
  --credentials-requests-dir ./output_dir/cred-reqs/ \ 5
  --permissions-boundary-arn=<policy_arn> 6
```

- 1 Specify the output directory you created earlier.
- 2 Specify a globally unique name. This name functions as a prefix for AWS resources created by this command.
- 3 Specify the ARN for the IAM identity provider.
- 4 Specify the AWS region of the cluster.
- 5 Specify the relative path to the folder where you extracted the **CredentialsRequest** files with the **oc adm release extract** command.
- 6 Optional: Specify the Amazon Resource Name (ARN) of the AWS IAM policy to use as the permissions boundary for the IAM roles created by the **ccoctl** utility.

b. Apply the generated secrets by running the following command:

```
$ find ./output_dir/manifests -iname "openshift*.yaml" -print0 | xargs -l {} -0 -t oc replace -f {}
```

8. Finish the configuration process by restarting the cluster.

a. Restart all of the pods in the cluster by running the following command:

```
$ oc adm reboot-machine-config-pool mcp/worker mcp/master
```

b. Monitor the restart and update process by running the following command:

```
$ oc adm wait-for-node-reboot nodes --all
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All nodes rebooted
```

c. Monitor the configuration update progress by running the following command:

```
$ oc adm wait-for-stable-cluster
```

This process might take 15 minutes or longer. The following output indicates that the process is complete:

```
All clusteroperators are stable
```

- Optional: Remove the AWS root credentials secret by running the following command:

```
$ oc delete secret -n kube-system aws-creds
```

Additional resources

- [Microsoft Entra Workload ID](#)
- [Configuring an Azure cluster to use short-term credentials](#)
- [AWS Security Token Service](#)
- [Configuring an AWS cluster to use short-term credentials](#)

10.4.4. Verifying that a cluster uses short-term credentials

You can verify that a cluster uses short-term security credentials for individual components by checking the Cloud Credential Operator (CCO) configuration and other values in the cluster.

Prerequisites

- You deployed an OpenShift Container Platform cluster using the Cloud Credential Operator utility (**ccoctl**) to implement short-term credentials.
- You installed the OpenShift CLI (**oc**).
- You are logged in as a user with **cluster-admin** privileges.

Procedure

- Verify that the CCO is configured to operate in manual mode by running the following command:

```
$ oc get cloudcredentials cluster \
  -o=jsonpath={.spec.credentialsMode}
```

The following output confirms that the CCO is operating in manual mode:

Example output

```
Manual
```

- Verify that the cluster does not have **root** credentials by running the following command:

```
$ oc get secrets \
  -n kube-system <secret_name>
```

where **<secret_name>** is the name of the root secret for your cloud provider.

Platform	Secret name
Amazon Web Services (AWS)	aws-creds

Platform	Secret name
Microsoft Azure	azure-credentials
Google Cloud	gcp-credentials

An error confirms that the root secret is not present on the cluster.

Example output for an AWS cluster

Error from server (NotFound): secrets "aws-creds" not found

- Verify that the components are using short-term security credentials for individual components by running the following command:

```
$ oc get authentication cluster \
  -o jsonpath \
  --template='{ .spec.serviceAccountIssuer }'
```

This command displays the value of the **.spec.serviceAccountIssuer** parameter in the cluster **Authentication** object. An output of a URL that is associated with your cloud provider indicates that the cluster is using manual mode with short-term credentials that are created and managed from outside of the cluster.

- Azure clusters: Verify that the components are assuming the Azure client ID that is specified in the secret manifests by running the following command:

```
$ oc get secrets \
  -n openshift-image-registry installer-cloud-credentials \
  -o jsonpath='{.data}'
```

An output that contains the **azure_client_id** and **azure_federated_token_file** fields confirms that the components are assuming the Azure client ID.

- Azure clusters: Verify that the pod identity webhook is running by running the following command:

```
$ oc get pods \
  -n openshift-cloud-credential-operator
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
cloud-credential-operator-59cf744f78-r8pbq  2/2   Running  2         71m
pod-identity-webhook-548f977b4c-859lz      1/1   Running  1         70m
```

10.5. ADDITIONAL RESOURCES

- [About the Cloud Credential Operator](#)

CHAPTER 11. CONFIGURING ALERT NOTIFICATIONS

You can configure alert notifications to send firing alerts from your cluster to external systems, helping ensure that administrators are notified of conditions that require attention.

In OpenShift Container Platform, an alert is fired when the conditions defined in an alerting rule are true. An alert provides a notification that a set of circumstances are apparent within a cluster. Firing alerts can be viewed in the Alerting UI in the OpenShift Container Platform web console by default. After an installation, you can configure OpenShift Container Platform to send alert notifications to external systems.

11.1. SENDING NOTIFICATIONS TO EXTERNAL SYSTEMS

You can configure alert receivers to notify the appropriate teams or external systems when alerts fire. You can also use the watchdog alert to detect communication failures between Alertmanager and a notification provider.

In OpenShift Container Platform, firing alerts can be viewed in the Alerting UI. Alerts are not configured by default to be sent to any notification systems. You can configure OpenShift Container Platform to send alerts to the following receiver types:

- PagerDuty
- Webhook
- Email
- Slack
- Microsoft Teams

Routing alerts to receivers enables you to send timely notifications to the appropriate teams when failures occur. For example, critical alerts require immediate attention and are typically paged to an individual or a critical response team. Alerts that provide non-critical warning notifications might instead be routed to a ticketing system for non-immediate review.

Checking that alerting is operational by using the watchdog alert

OpenShift Container Platform monitoring includes a watchdog alert that fires continuously. Alertmanager repeatedly sends watchdog alert notifications to configured notification providers. The provider is usually configured to notify an administrator when it stops receiving the watchdog alert. This mechanism helps you quickly identify any communication issues between Alertmanager and the notification provider.

11.2. ADDITIONAL RESOURCES

- [About OpenShift Container Platform monitoring](#)
- [Configuring alerts and notifications for core platform monitoring](#)
- [Configuring alerts and notifications for user workload monitoring](#)

CHAPTER 12. CONVERTING A CONNECTED CLUSTER TO A DISCONNECTED CLUSTER

You can convert a connected OpenShift Container Platform cluster to a disconnected cluster by mirroring required registry content and installation media for use without internet access.

There might be some scenarios where you need to convert your OpenShift Container Platform cluster from a connected cluster to a disconnected cluster.

A disconnected cluster, also known as a restricted cluster, does not have an active connection to the internet. As such, you must mirror the contents of your registries and installation media. You can create this mirror registry on a host that can access both the internet and your closed network, or copy images to a device that you can move across network boundaries.

For information on how to convert your cluster, see the "Converting a connected cluster to a disconnected cluster" procedure in the Disconnected environments section.

12.1. ADDITIONAL RESOURCES

- [Converting a connected cluster to a disconnected cluster](#)

CHAPTER 13. DAY 2 OPERATIONS FOR OPENSIFT CONTAINER PLATFORM CLUSTERS

13.1. DAY 2 OPERATIONS FOR OPENSIFT CONTAINER PLATFORM CLUSTERS

You can use the following Day 2 operations to manage OpenShift Container Platform clusters.

Updating an OpenShift Container Platform cluster

Updating your cluster is a critical task that ensures that bugs and potential security vulnerabilities are patched. For more information, see [Updating an OpenShift Container Platform cluster](#).

Troubleshooting and maintaining OpenShift Container Platform clusters

To maintain and troubleshoot a bare-metal environment where high-bandwidth network throughput is required, see [Troubleshooting and maintaining OpenShift Container Platform clusters](#).

Observability in OpenShift Container Platform clusters

OpenShift Container Platform generates a large amount of data, such as performance metrics and logs from the platform and the workloads running on it. As an administrator, you can use tools to collect and analyze the available data. For more information, see [Observability in OpenShift Container Platform](#).

Security

You can enhance security for high-bandwidth network deployments by following key security considerations. For more information, see [Security basics](#).

13.2. UPGRADING OPENSIFT CONTAINER PLATFORM CLUSTERS

13.2.1. Upgrading an OpenShift Container Platform cluster

OpenShift Container Platform has long term support or extended update support (EUS) on all even releases and update paths between EUS releases. You can update from one EUS version to the next EUS version. It is also possible to update between y-stream and z-stream versions.

13.2.1.1. Cluster updates for OpenShift clusters

Updating your cluster is a critical task that ensures that bugs and potential security vulnerabilities are patched. Often, updates to cloud-native applications require additional functionality from the platform that comes when you update the cluster version. You also must update the cluster periodically to ensure that the cluster platform version is supported.

You can minimize the effort required to stay current with updates by keeping up-to-date with EUS releases and upgrading to select important z-stream releases only.



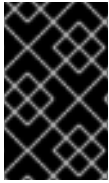
NOTE

The update path for the cluster can vary depending on the size and topology of the cluster.

The following update scenarios are described:

- Control Plane Only updates

- Y-stream updates
- Z-stream updates



IMPORTANT

Control Plane Only updates were previously known as EUS-to-EUS updates. Control Plane Only updates are only viable between even-numbered minor versions of OpenShift Container Platform.

13.2.2. Verifying cluster API versions between update versions

APIs change over time as components are updated. It is important to verify that your application APIs are compatible with the updated cluster version.

13.2.2.1. OpenShift Container Platform API compatibility

When considering what z-stream release to update to as part of a new y-stream update, you must be sure that all the patches that are in the z-stream version you are moving from are in the new z-stream version. If the version you update to does not have all the required patches, the built-in compatibility of Kubernetes is broken.

For example, if the cluster version is 4.15.32, you must update to 4.16 z-stream release that has all of the patches that are applied to 4.15.32.

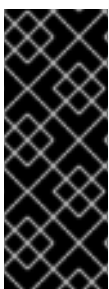
Each cluster Operator supports specific API versions. Kubernetes APIs evolve over time, and newer versions can be deprecated or change existing APIs. This is referred to as "version skew". For every new release, you must review the API changes. The APIs might be compatible across several releases of an Operator, but compatibility is not guaranteed. To mitigate against problems that arise from version skew, follow a well-defined update strategy.

Additional resources

- [Understanding API tiers](#)
- [Kubernetes version skew policy](#)

13.2.2.2. Determining the cluster version update path

Use the [Red Hat OpenShift Container Platform Update Graph](#) tool to determine if the path is valid for the z-stream release you want to update to.

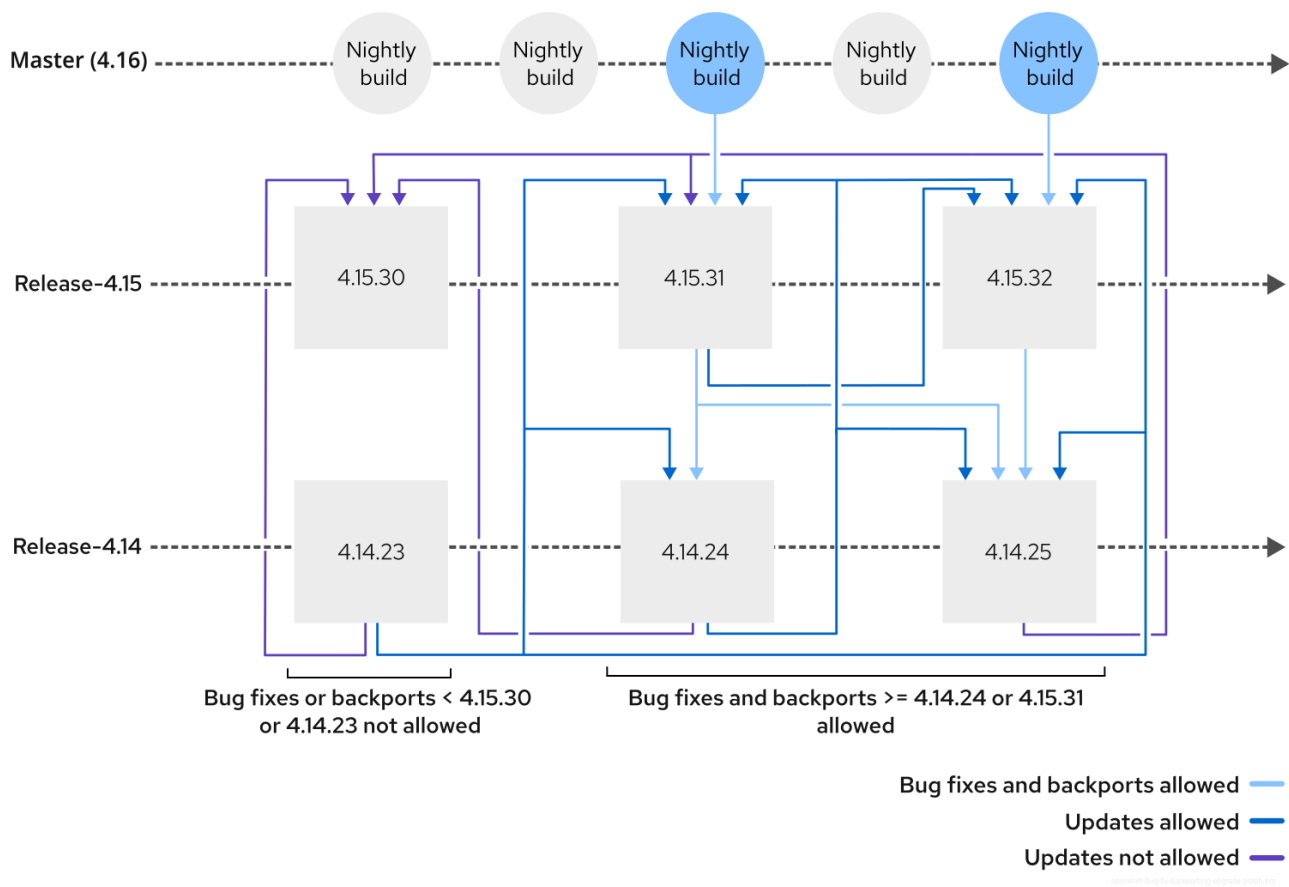


IMPORTANT

The <4.y+1.z> or <4.y+2.z> version that you update to must have the same patch level as the <4.y.z> release you are updating from.

The OpenShift Container Platform update process mandates that if a fix is present in a specific <4.y.z> release, then the that fix must be present in the <4.y+1.z> release that you update to.

Figure 13.1. Bug fix backporting and the update graph



IMPORTANT

OpenShift Container Platform development has a strict backport policy that prevents regressions. For example, a bug must be fixed in 4.16.z before it is fixed in 4.15.z. This means that the update graph does not allow for updates to chronologically older releases even if the minor version is greater, for example, updating from 4.15.24 to 4.16.2.

Additional resources

- [Understanding update channels and releases](#)

13.2.2.3. Selecting the target release

Use the [Red Hat OpenShift Container Platform Update Graph](#) or the [cincinnati-graph-data repository](#) to determine what release to update to.

13.2.2.3.1. Determining what z-stream updates are available

Before you can update to a new z-stream release, you need to know what versions are available.



NOTE

You do not need to change the channel when performing a z-stream update.

1. Determine which z-stream releases are available. Run the following command:

```
$ oc adm upgrade
```

■

Example output

Cluster version is 4.14.34

Upstream is unset, so the cluster will use an appropriate default.

Channel: stable-4.14 (available channels: candidate-4.14, candidate-4.15, eus-4.14, eus-4.16, fast-4.14, fast-4.15, stable-4.14, stable-4.15)

Recommended updates:

VERSION	IMAGE
4.14.37	quay.io/openshift-release-dev/ocp-release@sha256:14e6ba3975e6c73b659fa55af25084b20ab38a543772ca70e184b903db73092b
4.14.36	quay.io/openshift-release-dev/ocp-release@sha256:4bc4925e8028158e3f313aa83e59e181c94d88b4aa82a3b00202d6f354e8dfe
4.14.35	quay.io/openshift-release-dev/ocp-release@sha256:883088e3e6efa7443b0ac28cd7682c2fdbda889b576edad626769bf956ac0858

13.2.2.3.2. Changing the channel for a Control Plane Only update

You must change the channel to the required version for a Control Plane Only update.

**NOTE**

You do not need to change the channel when performing a z-stream update.

1. Determine the currently configured update channel by running the following command:

```
$ oc get clusterversion -o=jsonpath='{.items[*].spec}' | jq
```

Example output

```
{
  "channel": "stable-4.14",
  "clusterID": "01eb9a57-2bfb-4f50-9d37-dc04bd5bac75"
}
```

2. Change the channel to point to the new channel you want to update to by running the following command:

```
$ oc adm upgrade channel eus-4.16
```

3. Confirm the updated channel by running the following command:

```
$ oc get clusterversion -o=jsonpath='{.items[*].spec}' | jq
```

Example output

```
{
  "channel": "eus-4.16",
  "clusterID": "01eb9a57-2bf5-4f50-9d37-dc04bd5bac75"
}
```

13.2.2.3.3. Changing the channel for an early EUS to EUS update

The update path to a brand new release of OpenShift Container Platform is not available in either the EUS channel or the stable channel until 45 to 90 days after the initial GA of a minor release.

To begin testing an update to a new release, you can use the fast channel.

1. Change the channel to **fast- $\langle y+1 \rangle$** . For example, run the following command:

```
$ oc adm upgrade channel fast-4.16
```

2. Check the update path from the new channel. Run the following command:

```
$ oc adm upgrade
```

```
Cluster version is 4.15.33
```

```
Upgradeable=False
```

```
Reason: AdminAckRequired
```

```
Message: Kubernetes 1.28 and therefore {product-title} 4.16 remove several APIs which
require admin consideration. Please see the knowledge article
https://access.redhat.com/articles/6958394 for details and instructions.
```

```
Upstream is unset, so the cluster will use an appropriate default.
```

```
Channel: fast-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.15, eus-4.16,
fast-4.15, fast-4.16, stable-4.15, stable-4.16)
```

```
Recommended updates:
```

```
VERSION  IMAGE
```

```
4.16.14  quay.io/openshift-release-dev/ocp-release@sha256:6618dd3c0f5
```

```
4.16.13  quay.io/openshift-release-dev/ocp-release@sha256:7a72abc3
```

```
4.16.12  quay.io/openshift-release-dev/ocp-release@sha256:1c8359fc2
```

```
4.16.11  quay.io/openshift-release-dev/ocp-release@sha256:bc9006febfe
```

```
4.16.10  quay.io/openshift-release-dev/ocp-release@sha256:dece7b61b1
```

```
4.15.36  quay.io/openshift-release-dev/ocp-release@sha256:c31a56d19
```

```
4.15.35  quay.io/openshift-release-dev/ocp-release@sha256:f21253
```

```
4.15.34  quay.io/openshift-release-dev/ocp-release@sha256:2dd69c5
```

3. Follow the update procedure to get to version 4.16 ($\langle y+1 \rangle$ from version 4.15)



NOTE

You can keep your worker nodes paused between EUS releases even if you are using the fast channel.

4. When you get to the required $\langle y+1 \rangle$ release, change the channel again, this time to **fast- $\langle y+2 \rangle$** .

5. Follow the EUS update procedure to get to the required <y+2> release.

13.2.2.3.4. Changing the channel for a y-stream update

In a y-stream update you change the channel to the next release channel.



NOTE

Use the stable or EUS release channels for production clusters.

1. Change the update channel by running the following command:

```
$ oc adm upgrade channel stable-4.15
```

2. Check the update path from the new channel. Run the following command:

```
$ oc adm upgrade
```

Example output

```
Cluster version is 4.14.34
```

```
Upgradeable=False
```

```
Reason: AdminAckRequired
```

```
Message: Kubernetes 1.27 and therefore {product-title} 4.15 remove several APIs which
require admin consideration. Please see the knowledge article
https://access.redhat.com/articles/6958394 for details and instructions.
```

```
Upstream is unset, so the cluster will use an appropriate default.
```

```
Channel: stable-4.15 (available channels: candidate-4.14, candidate-4.15, eus-4.14, eus-4.15,
fast-4.14, fast-4.15, stable-4.14, stable-4.15)
```

```
Recommended updates:
```

VERSION	IMAGE
4.15.33	quay.io/openshift-release-dev/ocp-release@sha256:7142dd4b560
4.15.32	quay.io/openshift-release-dev/ocp-release@sha256:cda8ea5b13dc9
4.15.31	quay.io/openshift-release-dev/ocp-release@sha256:07cf61e67d3e000
4.15.30	quay.io/openshift-release-dev/ocp-release@sha256:6618dd3c0f5
4.15.29	quay.io/openshift-release-dev/ocp-release@sha256:7a72abc3
4.15.28	quay.io/openshift-release-dev/ocp-release@sha256:1c8359fc2
4.15.27	quay.io/openshift-release-dev/ocp-release@sha256:bc9006febfe
4.15.26	quay.io/openshift-release-dev/ocp-release@sha256:dece7b61b1
4.14.38	quay.io/openshift-release-dev/ocp-release@sha256:c93914c62d7
4.14.37	quay.io/openshift-release-dev/ocp-release@sha256:c31a56d19
4.14.36	quay.io/openshift-release-dev/ocp-release@sha256:f21253
4.14.35	quay.io/openshift-release-dev/ocp-release@sha256:2dd69c5

13.2.3. Preparing a bare-metal cluster for platform update

On bare-metal hardware, you often must update the firmware to take on important security fixes, take on new functionality, or maintain compatibility with the new release of OpenShift Container Platform.

13.2.3.1. Disconnected environment considerations

To update clusters in disconnected environments, you must update your offline image repository.

Additional resources

- [API compatibility guidelines](#)
- [Mirroring images for a disconnected installation by using the oc-mirror plugin v2](#)

13.2.3.2. Ensuring the host firmware is compatible with the update

You are responsible for the firmware versions that you run in your clusters. Updating host firmware is not a part of the OpenShift Container Platform update process. It is not recommended to update firmware in conjunction with the OpenShift Container Platform version.



IMPORTANT

Hardware vendors advise that it is best to apply the latest certified firmware version for the specific hardware that you are running. For each different use case, always verify firmware updates in test environments before applying them in production. For example, workloads with high throughput requirements can be negatively affected outdated host firmware.

You should thoroughly test new firmware updates to ensure that they work as expected with the current version of OpenShift Container Platform. For best results, test the latest firmware version with the target OpenShift Container Platform update version.

13.2.3.3. Ensuring that layered products are compatible with the update

Procedure

Verify that all layered products run on the version of OpenShift Container Platform that you are updating to before you begin the update. This generally includes all Operators.

Procedure

1. Verify the currently installed Operators in the cluster. For example, run the following command:

```
$ oc get csv -A
```

Example output

NAMESPACE	NAME	DISPLAY	VERSION	REPLACES
PHASE				
gitlab-operator-kubernetes.v0.17.2	GitLab		0.17.2	gitlab-operator-
kubernetes.v0.17.1	Succeeded			
openshift-operator-lifecycle-manager	packageserver	Package Server	0.19.0	
Succeeded				

2. Check that Operators that you install with OLM are compatible with the update version. Operators that are installed with the Operator Lifecycle Manager (OLM) are not part of the standard cluster Operators set.

Use the [Operator Update Information Checker](#) to understand if you must update an Operator after each y-stream update or if you can wait until you have fully updated to the next EUS release.

TIP

You can also use the [Operator Update Information Checker](#) to see what versions of OpenShift Container Platform are compatible with specific releases of an Operator.

3. Check that Operators that you install outside of OLM are compatible with the update version. For all OLM-installed Operators that are not directly supported by Red Hat, contact the Operator vendor to ensure release compatibility.
 - Some Operators are compatible with several releases of OpenShift Container Platform. See "Updating the worker nodes" for more information.
 - See "Updating all the OLM Operators" for information about updating an Operator after performing the first y-stream control plane update.

Additional resources

- [Updating the worker nodes](#)
- [Updating all the OLM Operators](#)

13.2.3.4. Applying MachineConfigPool labels to nodes before the update

Prepare **MachineConfigPool** (MCP) node labels to group nodes together in groups of roughly 8 to 10 nodes. With MCP groups, you can reboot groups of nodes independently from the rest of the cluster.

You use the MCP node labels to pause and unpaue the set of nodes during the update process so that you can do the update and reboot at a time of your choosing.

Staggering the cluster update

Sometimes there are problems during the update. Often the problem is related to hardware failure or nodes needing to be reset. Using MCP node labels, you can update nodes in stages by pausing the update at critical moments, tracking paused and unpaused nodes as you proceed. When a problem occurs, you use the nodes that are in an unpaused state to ensure that there are enough nodes running to keep all applications pods running.

Dividing worker nodes into MachineConfigPool groups

How you divide worker nodes into MCPs can vary depending on how many nodes are in the cluster or how many nodes you assign to a node role. By default, the two roles in a cluster are control plane and worker roles.

You can also move nodes between MCP groups if both groups have the same machine config, which is important if you have too many nodes in one large machine config pool. For more information about MCP groups, see *Additional resources*.

**NOTE**

Larger clusters can have as many as 100 worker nodes. No matter how many nodes there are in the cluster, keep each **MachineConfigPool** group to around 10 nodes. This allows you to control how many nodes are taken down at a time. With multiple **MachineConfigPool** groups, you can unpause several groups at a time to accelerate the update, or separate the update over two or more maintenance windows.

Example cluster with 15 worker nodes

Consider a cluster with 15 worker nodes:

- 10 worker nodes are control plane nodes.
- 5 worker nodes are data plane nodes.

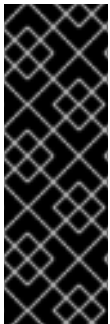
Split the control plane and data plane worker node roles into at least 2 MCP groups each. Having 2 MCP groups per role means that you can have one set of nodes that are not affected by the update.

Example cluster with 6 worker nodes

Consider a cluster with 6 worker nodes:

- Split the worker nodes into 3 MCP groups of 2 nodes each.

Upgrade one of the MCP groups. Allow the updated nodes to sit through a day to allow for verification of application compatibility before completing the update on the other 4 nodes.

**IMPORTANT**

The process and pace at which you unpause the MCP groups is determined by your applications and configuration.

If your pod can handle being scheduled across nodes in a cluster, you can unpause several MCP groups at a time and set the **MaxUnavailable** field in the MCP custom resource (CR) to as high as 50%. This allows up to half of the nodes in an MCP group to restart and get updated.

Additional resources

- [Node configuration management with machine config pools](#)

13.2.3.4.1. Reviewing configured cluster MachineConfigPool roles

Review the currently configured **MachineConfigPool** roles in the cluster.

Procedure

1. Get the currently configured **mcp** groups in the cluster:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED	MACHINECOUNT
------	--------	---------	----------	----------	--------------

	READYMACHINECOUNT	UPDATEDMACHINECOUNT	DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-bere83	True	False	False	3	3	3
0	25d						
worker	rendered-worker-245c4f	True	False	False	2	2	2
0	25d						

2. Compare the list of **mcp** roles to list of nodes in the cluster:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	39d	v1.27.15+6147456
ctrl-plane-1	Ready	control-plane,master	39d	v1.27.15+6147456
ctrl-plane-2	Ready	control-plane,master	39d	v1.27.15+6147456
worker-0	Ready	worker	39d	v1.27.15+6147456
worker-1	Ready	worker	39d	v1.27.15+6147456



NOTE

When you apply an **mcp** group change, the node roles are updated.

Determine how you want to separate the worker nodes into **mcp** groups.

13.2.3.4.2. Creating MachineConfigPool groups for the cluster

Creating **mcp** groups is a 2-step process:

1. Add an **mcp** label to the nodes in the cluster
2. Apply an **mcp** CR to the cluster that organizes the nodes based on their labels

Procedure

1. Label the nodes so that they can be put into **mcp** groups. Run the following commands:

```
$ oc label node worker-0 node-role.kubernetes.io/mcp-1=
```

```
$ oc label node worker-1 node-role.kubernetes.io/mcp-2=
```

The **mcp-1** and **mcp-2** labels are applied to the nodes. For example:

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	39d	v1.27.15+6147456
ctrl-plane-1	Ready	control-plane,master	39d	v1.27.15+6147456
ctrl-plane-2	Ready	control-plane,master	39d	v1.27.15+6147456
worker-0	Ready	mcp-1,worker	39d	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	39d	v1.27.15+6147456

2. Create YAML custom resources (CRs) that apply the labels as **mcp** CRs in the cluster. Save the following YAML in the **mcps.yaml** file:

```
---
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: mcp-2
spec:
  machineConfigSelector:
    matchExpressions:
      - {
        key: machineconfiguration.openshift.io/role,
        operator: In,
        values: [worker,mcp-2]
      }
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/mcp-2: ""
---
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfigPool
metadata:
  name: mcp-1
spec:
  machineConfigSelector:
    matchExpressions:
      - {
        key: machineconfiguration.openshift.io/role,
        operator: In,
        values: [worker,mcp-1]
      }
  nodeSelector:
    matchLabels:
      node-role.kubernetes.io/mcp-1: ""
```

3. Create the **MachineConfigPool** resources:

```
$ oc apply -f mcps.yaml
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/mcp-2 created
```

Verification

Monitor the **MachineConfigPool** resources as they are applied in the cluster. After you apply the **mcp** resources, the nodes are added into the new machine config pools. This takes a few minutes.



NOTE

The nodes do not reboot while being added into the **mcp** groups. The original worker and master **mcp** groups remain unchanged.

- Check the status of the new **mcp** resources:

```
$ oc get mcp
```

Example output

```
NAME      CONFIG          UPDATED  UPDATING  DEGRADED  MACHINECOUNT  READYMACHINECOUNT  UPDATEDMACHINECOUNT  DEGRADEDMACHINECOUNT  AGE
master    rendered-master-be3e83  True     False     False     3               3                   3                       0                       25d
mcp-1     rendered-mcp-1-2f4c4f  False    True      True      1               0                   0                       0                       10s
mcp-2     rendered-mcp-2-2r4s1f  False    True      True      1               0                   0                       0                       10s
worker    rendered-worker-23fc4f  False    True      True      0               0                   0                       0                       25d
```

Eventually, the resources are fully applied:

```
NAME      CONFIG          UPDATED  UPDATING  DEGRADED  MACHINECOUNT  READYMACHINECOUNT  UPDATEDMACHINECOUNT  DEGRADEDMACHINECOUNT  AGE
master    rendered-master-be3e83  True     False     False     3               3                   3                       0                       25d
mcp-1     rendered-mcp-1-2f4c4f  True     False     False     1               1                   1                       0                       7m33s
mcp-2     rendered-mcp-2-2r4s1f  True     False     False     1               1                   1                       0                       51s
worker    rendered-worker-23fc4f  True     False     False     0               0                   0                       0                       25d
```

Additional resources

- [Performing a Control Plane Only update](#)
- [Factors affecting update duration](#)
- [Ensuring that CNF workloads run uninterrupted with pod disruption budgets](#)
- [Ensuring that pods do not run on the same cluster node](#)

13.2.3.5. Preparing the cluster platform for update

Before you update the cluster, perform basic checks and verifications to ensure that the cluster is ready for the update.

Procedure

1. Verify that there are no failed or in progress pods in the cluster by running the following command:

```
$ oc get pods -A | grep -E -vi 'complete|running'
```

**NOTE**

You might have to run this command more than once if there are pods that are in a pending state.

2. Verify that all nodes in the cluster are available:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	32d	v1.27.15+6147456
ctrl-plane-1	Ready	control-plane,master	32d	v1.27.15+6147456
ctrl-plane-2	Ready	control-plane,master	32d	v1.27.15+6147456
worker-0	Ready	mcp-1,worker	32d	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	32d	v1.27.15+6147456

3. Verify that all bare-metal nodes are provisioned and ready.

```
$ oc get bmh -n openshift-machine-api
```

Example output

NAME	STATE	CONSUMER	ONLINE	ERROR	AGE
ctrl-plane-0	unmanaged	cnf-58879-master-0	true	33d	
ctrl-plane-1	unmanaged	cnf-58879-master-1	true	33d	
ctrl-plane-2	unmanaged	cnf-58879-master-2	true	33d	
worker-0	unmanaged	cnf-58879-worker-0-45879	true	33d	
worker-1	progressing	cnf-58879-worker-0-dszsh	false	1d	

An error occurred while provisioning the **worker-1** node.

Verification

- Verify that all cluster Operators are ready:

```
$ oc get co
```

Example output

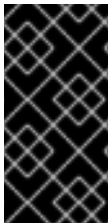
NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
authentication	4.14.34	True	False	False
baremetal	4.14.34	True	False	False
...				
service-ca	4.14.34	True	False	False
storage	4.14.34	True	False	False

Additional resources

- [Investigating pod issues](#)

13.2.4. Configuring application pods before updating your OpenShift Container Platform cluster

Configure application pods to ensure workload availability during OpenShift Container Platform updates. For example, use deployment strategies, pod disruption budgets, anti-affinity rules, and health probes to maintain high availability and prevent service disruption. In the telecommunications industry, most containerized network function (CNF) vendors follow the guidance in Red Hat best practices for Kubernetes to ensure that the cluster can schedule pods properly during an upgrade.



IMPORTANT

Always deploy pods in groups by using **Deployment** resources. **Deployment** resources spread the workload across all of the available pods ensuring there is no single point of failure. When a pod that is managed by a **Deployment** resource is deleted, a new pod takes its place automatically.

Additional resources

- [Red Hat best practices for Kubernetes](#)

13.2.4.1. Ensuring that workloads run uninterrupted with pod disruption budgets

To prevent interruption of upgrading worker nodes, configure the pod disruption budget properly. For more information, see *Additional resources*.

Additional resources

- [Specifying the number of pods that must be up with pod disruption budgets](#)
- [Configuring an OpenShift Container Platform cluster for pods](#)
- [Pod preemption and other scheduler settings](#)

13.2.4.2. Ensuring that pods do not run on the same cluster node

High availability in Kubernetes requires duplicate processes to be running on separate nodes in the cluster. This ensures that the application continues to run even if one node becomes unavailable. In OpenShift Container Platform, processes can be automatically duplicated in separate pods in a deployment. You configure anti-affinity in the **Pod** resource to ensure that the pods in a deployment do not run on the same cluster node.

During an update, setting pod anti-affinity ensures that pods are distributed evenly across nodes in the cluster. This means that node reboots are easier during an update. For example, if there are 4 pods from a single deployment on a node, and the pod disruption budget is set to only allow 1 pod to be deleted at a time, then it will take 4 times as long for that node to reboot. Setting pod anti-affinity spreads pods across the cluster to prevent such occurrences.

Additional resources

- [Configuring a pod affinity rule](#)

13.2.4.3. Application liveness, readiness, and startup probes

You can use liveness, readiness and startup probes to check the health of your live application containers before you schedule an update. These are very useful tools to use with pods that are dependent upon keeping state for their application containers.

Liveness health check

Determines if a container is running. If the liveness probe fails for a container, the pod responds based on the restart policy.

Readiness probe

Determines if a container is ready to accept service requests. If the readiness probe fails for a container, the kubelet removes the container from the list of available service endpoints.

Startup probe

A startup probe indicates whether the application within a container is started. All other probes are disabled until the startup succeeds. If the startup probe does not succeed, the kubelet stops the container, and the container is subject to the pod **restartPolicy** setting.

Additional resources

- [Understanding health checks](#)

13.2.5. Before you update the telco core CNF cluster

Before you start the cluster update, you must pause worker nodes, back up the etcd database, and do a final cluster health check before proceeding.

13.2.5.1. Pausing worker nodes before the update

You must pause the worker nodes before you proceed with the update. In the following example, there are 2 **mcp** groups, **mcp-1** and **mcp-2**. You patch the **spec.paused** field to **true** for each of the **MachineConfigPool** groups.

Procedure

1. Patch the **mcp** CRs to pause the nodes and drain and remove the pods from those nodes by running the following command:

```
$ oc patch mcp/mcp-1 --type merge --patch '{"spec":{"paused":true}}'
```

```
$ oc patch mcp/mcp-2 --type merge --patch '{"spec":{"paused":true}}'
```

2. Get the status of the paused **mcp** groups:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  true
mcp-2  true
```

**NOTE**

The default control plane and worker **mcp** groups are not changed during an update.

13.2.5.2. Backup the etcd database before you proceed with the update

You must backup the etcd database before you proceed with the update.

13.2.5.2.1. Backing up etcd data

Follow these steps to back up etcd data by creating an etcd snapshot and backing up the resources for the static pods. This backup can be saved and used at a later time if you need to restore etcd.

**IMPORTANT**

Only save a backup from a single control plane host. Do not take a backup from each control plane host in the cluster.

For a Two-Node with Fencing (TNF) setup, follow the steps to back up etcd data on only one node in the cluster. The cluster restore process is driven by data from a single node, so you can perform the etcd backup steps on only one node.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have checked whether the cluster-wide proxy is enabled.

TIP

You can check whether the proxy is enabled by reviewing the output of **oc get proxy cluster -o yaml**. The proxy is enabled if the **httpProxy**, **httpsProxy**, and **noProxy** fields have values set.

Procedure

1. Start a debug session as root for a control plane node:

```
$ oc debug --as-root node/<node_name>
```

2. Change your root directory to **/host** in the debug shell:

```
sh-4.4# chroot /host
```

3. If the cluster-wide proxy is enabled, export the **NO_PROXY**, **HTTP_PROXY**, and **HTTPS_PROXY** environment variables by running the following commands:

```
$ export HTTP_PROXY=http://<your_proxy.example.com>:8080
```

```
$ export HTTPS_PROXY=https://<your_proxy.example.com>:8080
```

```
$ export NO_PROXY=<example.com>
```


4. Run the **cluster-backup.sh** script in the debug shell and pass in the location to save the backup to.

TIP

The **cluster-backup.sh** script is maintained as a component of the etcd Cluster Operator and is a wrapper around the **etcdctl snapshot save** command.

```
sh-4.4# /usr/local/bin/cluster-backup.sh /home/core/assets/backup
```

Example script output

```
found latest kube-apiserver: /etc/kubernetes/static-pod-resources/kube-apiserver-pod-6
found latest kube-controller-manager: /etc/kubernetes/static-pod-resources/kube-controller-
manager-pod-7
found latest kube-scheduler: /etc/kubernetes/static-pod-resources/kube-scheduler-pod-6
found latest etcd: /etc/kubernetes/static-pod-resources/etcd-pod-3
ede95fe6b88b87ba86a03c15e669fb4aa5bf0991c180d3c6895ce72eaade54a1
etcdctl version: 3.4.14
API version: 3.4
{"level":"info","ts":1624647639.0188997,"caller":"snapshot/v3_snapshot.go:119","msg":"created
temporary db file","path":"/home/core/assets/backup/snapshot_2021-06-25_190035.db.part"}
{"level":"info","ts":"2021-06-
25T19:00:39.030Z","caller":"clientv3/maintenance.go:200","msg":"opened snapshot stream;
downloading"}
{"level":"info","ts":1624647639.0301006,"caller":"snapshot/v3_snapshot.go:127","msg":"fetching
snapshot","endpoint":"https://10.0.0.5:2379"}
{"level":"info","ts":"2021-06-
25T19:00:40.215Z","caller":"clientv3/maintenance.go:208","msg":"completed snapshot read;
closing"}
{"level":"info","ts":1624647640.6032252,"caller":"snapshot/v3_snapshot.go:142","msg":"fetched
snapshot","endpoint":"https://10.0.0.5:2379","size":"114 MB","took":1.584090459}
{"level":"info","ts":1624647640.6047094,"caller":"snapshot/v3_snapshot.go:152","msg":"saved",
"path":"/home/core/assets/backup/snapshot_2021-06-25_190035.db"}
Snapshot saved at /home/core/assets/backup/snapshot_2021-06-25_190035.db
{"hash":"3866667823","revision":31407,"totalKey":12828,"totalSize":114446336}
snapshot db and kube resources are successfully saved to /home/core/assets/backup
```

In this example, two files are created in the **/home/core/assets/backup/** directory on the control plane host:

- **snapshot_<datetimestamp>.db**: This file is the etcd snapshot. The **cluster-backup.sh** script confirms its validity.
- **static_kuberresources_<datetimestamp>.tar.gz**: This file contains the resources for the static pods. If etcd encryption is enabled, it also contains the encryption keys for the etcd snapshot.

**NOTE**

If etcd encryption is enabled, store this second file separately from the etcd snapshot for security reasons. However, this file is required to restore from the etcd snapshot.

The etcd encryption only encrypts values, not keys. This means that resource types, namespaces, and object names are not encrypted.

13.2.5.2.2. Creating a single automated etcd backup

Follow these steps to create a single etcd backup by creating and applying a custom resource (CR).

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.
- You have access to the OpenShift CLI (**oc**).

Procedure

- If dynamically-provisioned storage is available, complete the following steps to create a single automated etcd backup:
 - a. Create a persistent volume claim (PVC) named **etcd-backup-pvc.yaml** with contents such as the following example:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: etcd-backup-pvc
  namespace: openshift-etcd
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: <storage_amount>
  volumeMode: Filesystem
```

where:

<storage_amount>

Specifies the amount of storage available to the PVC. Adjust this value for your requirements, such as **200Gi**.

- b. Apply the PVC by running the following command:

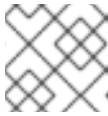
```
$ oc apply -f etcd-backup-pvc.yaml
```

- c. Verify the creation of the PVC by running the following command:

```
$ oc get pvc
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
etcd-backup-pvc	Bound			51s

**NOTE**

Dynamic PVCs stay in the **Pending** state until they are mounted.

- d. Create a CR file named **etcd-single-backup.yaml** with contents such as the following example:

```
apiVersion: operator.openshift.io/v1alpha1
kind: EtcdBackup
metadata:
  name: etcd-single-backup
  namespace: openshift-etcd
spec:
  pvcName: <pvc_name>
```

where:

<pvc_name>

Specifies the name of the PVC to save the backup to. Adjust this value according to your environment, such as **etcd-backup-pvc**.

- e. Apply the CR to start a single backup:

```
$ oc apply -f etcd-single-backup.yaml
```

- If dynamically-provisioned storage is not available, complete the following steps to create a single automated etcd backup:

- a. Create a **StorageClass** CR file named **etcd-backup-local-storage.yaml** with the following contents:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: etcd-backup-local-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: Immediate
```

- b. Apply the **StorageClass** CR by running the following command:

```
$ oc apply -f etcd-backup-local-storage.yaml
```

- c. Create a PV named **etcd-backup-pv-fs.yaml** with contents such as the following example:

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

```

    name: etcd-backup-pv-fs
spec:
  capacity:
    storage: <storage_amount>
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: etcd-backup-local-storage
  local:
    path: /mnt
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - key: kubernetes.io/hostname
          operator: In
        values:
        - <node_name>

```

where:

<storage_amount>

Specifies the amount of storage available to the PV. Adjust this value for your requirements, such as **100Gi**.

<node_name>

Specifies the node to attach this PV to. Replace with the actual node name, such as **master-0**.

- d. Verify the creation of the PV by running the following command:

```
$ oc get pv
```

Example output

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
CLAIM STORAGECLASS	REASON	AGE		
etcd-backup-pv-fs	100Gi	RWO	Retain	Available
local-storage	10s			etcd-backup-

- e. Create a PVC named **etcd-backup-pvc.yaml** with contents such as the following example:

```

kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: etcd-backup-pvc
  namespace: openshift-etcd
spec:
  accessModes:
  - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: <storage_amount>

```

■

where:

<storage_amount>

Specifies the amount of storage available to the PVC. Adjust this value for your requirements, such as **10Gi**.

- f. Apply the PVC by running the following command:

```
$ oc apply -f etcd-backup-pvc.yaml
```

- g. Create a CR file named **etcd-single-backup.yaml** with contents such as the following example:

```
apiVersion: operator.openshift.io/v1alpha1
kind: EtcdBackup
metadata:
  name: etcd-single-backup
  namespace: openshift-etcd
spec:
  pvcName: <pvc_name>
```

where:

<pvc_name>

Specifies the name of the persistent volume claim (PVC) to save the backup to. Adjust this value according to your environment, such as **etcd-backup-pvc**.

- h. Apply the CR to start a single backup:

```
$ oc apply -f etcd-single-backup.yaml
```

Additional resources

- [Backing up etcd](#)

13.2.5.3. Checking the cluster health

You should check the cluster health often during the update. Check for the node status, cluster Operators status and failed pods.

Procedure

1. Check the status of the cluster Operators by running the following command:

```
$ oc get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
authentication	4.14.34	True	False	False
baremetal	4.14.34	True	False	False

cloud-controller-manager	4.14.34	True	False	False	4d23h
cloud-credential	4.14.34	True	False	False	4d23h
cluster-autoscaler	4.14.34	True	False	False	4d22h
config-operator	4.14.34	True	False	False	4d22h
console	4.14.34	True	False	False	4d22h
...					
service-ca	4.14.34	True	False	False	4d22h
storage	4.14.34	True	False	False	4d22h

2. Check the status of the cluster nodes:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	4d22h	v1.27.15+6147456
ctrl-plane-1	Ready	control-plane,master	4d22h	v1.27.15+6147456
ctrl-plane-2	Ready	control-plane,master	4d22h	v1.27.15+6147456
worker-0	Ready	mcp-1,worker	4d22h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	4d22h	v1.27.15+6147456

3. Check that there are no in-progress or failed pods. There should be no pods returned when you run the following command.

```
$ oc get po -A | grep -E -iv 'running|complete'
```

13.2.6. Completing the control plane Only cluster update

Complete the following steps to perform the control plane only cluster update.



IMPORTANT

Control plane only updates were previously known as EUS-to-EUS updates. Control plane only updates are only viable between even-numbered minor versions of OpenShift Container Platform.

13.2.6.1. Acknowledging the control plane only or y-stream update

When you update to all versions from 4.11 and later, you must manually acknowledge that the update can continue.



IMPORTANT

Before you acknowledge the update, verify that you are not using any of the Kubernetes APIs that are removed from the version you are updating to. For example, in OpenShift Container Platform 4.17, there are no API removals. See "Kubernetes API removals" for more information.

Prerequisites

- You have verified that APIs for all of the applications running on your cluster are compatible with the next Y-stream release of OpenShift Container Platform. For more details about compatibility, see "Verifying cluster API versions between update versions".

Procedure

- Check the cluster update status to determine if administrative acknowledgment is required by running the following command:

```
$ oc adm upgrade
```

If the cluster update does not complete successfully, more details about the update failure are provided in the **Reason** and **Message** sections.

Example output:

```
Cluster version is 4.15.45
```

```
Upgradeable=False
```

```
Reason: MultipleReasons
```

```
Message: Cluster should not be upgraded between minor versions for multiple reasons:
AdminAckRequired,ResourceDeletesInProgress
```

```
* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin
consideration. Please see the knowledge article https://access.redhat.com/articles/7031404
for details and instructions.
```

```
* Cluster minor level upgrades are not allowed while resource deletions are in progress;
resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"
```

```
ReleaseAccepted=False
```

```
Reason: PreconditionChecks
```

```
Message: Preconditions failed for payload loaded version="4.16.34"
image="quay.io/openshift-release-dev/ocp-
release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b
8": Precondition "ClusterVersionUpgradeable" failed because of "MultipleReasons": Cluster
should not be upgraded between minor versions for multiple reasons:
AdminAckRequired,ResourceDeletesInProgress
```

```
* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin
consideration. Please see the knowledge article https://access.redhat.com/articles/7031404
for details and instructions.
```

```
* Cluster minor level upgrades are not allowed while resource deletions are in progress;
resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"
```

```
Upstream is unset, so the cluster will use an appropriate default.
```

```
Channel: eus-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.16, fast-4.15,
fast-4.16, stable-4.15, stable-4.16)
```

```
Recommended updates:
```

```
VERSION IMAGE
```

```
4.16.34 quay.io/openshift-release-dev/ocp-
release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b
8
```

**NOTE**

In this example, a linked Red Hat Knowledgebase article ([Preparing to upgrade to OpenShift Container Platform 4.16](#)) provides more detail about verifying API compatibility between releases.

Verification

- Verify the update by running the following command:

```
$ oc get configmap admin-acks -n openshift-config -o json | jq .data
```

If the acknowledgment was successful, the output shows the acknowledged API removals for each update:

```
{
  "ack-4.14-kube-1.28-api-removals-in-4.15": "true",
  "ack-4.15-kube-1.29-api-removals-in-4.16": "true"
}
```

**NOTE**

In this example, the cluster is updated from version 4.14 to 4.15, and then from 4.15 to 4.16 in a Control Plane Only update.

Additional resources

- [Kubernetes API removals](#)

13.2.6.2. Starting the cluster update

When updating from one y-stream release to the next, you must ensure that the intermediate z-stream releases are also compatible.

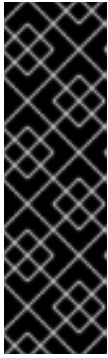
**NOTE**

You can verify that you are updating to a viable release by running the **oc adm upgrade** command. The **oc adm upgrade** command lists the compatible update releases.

Procedure

1. Start the update:

```
$ oc adm upgrade --to=4.15.33
```

IMPORTANT

- **Control plane only update:** Ensure you point to the interim <y+1> release path
- **Y-stream update** - Ensure you use the correct <y.z> release that follows the Kubernetes [version skew policy](#).
- **Z-stream update** - Verify that there are no problems moving to that specific release

Requested update to <version>

+ where:

+ **<version>::** Specifies the version number for your particular update, such as **4.15.33**.

Additional resources

- [Selecting the target release](#)

13.2.6.3. Monitoring the cluster update

You should check the cluster health often during the update. Check for the node status, cluster Operators status and failed pods.

Procedure

- Monitor the cluster update. For example, to monitor the cluster update from version 4.14 to 4.15, run the following command:

```
$ watch "oc get clusterversion; echo; oc get co | head -1; oc get co | grep 4.14; oc get co | grep 4.15; echo; oc get no; echo; oc get po -A | grep -E -iv 'running|complete'"
```

Example output

```
NAME    VERSION AVAILABLE PROGRESSING SINCE STATUS
version 4.14.34 True      True      4m6s Working towards 4.15.33: 111 of 873 done
(12% complete), waiting on kube-apiserver
```

```
NAME          VERSION AVAILABLE PROGRESSING DEGRADED SINCE
MESSAGE
authentication 4.14.34 True      False      False      4d22h
baremetal      4.14.34 True      False      False      4d23h
cloud-controller-manager 4.14.34 True      False      False      4d23h
cloud-credential 4.14.34 True      False      False      4d23h
cluster-autoscaler 4.14.34 True      False      False      4d23h
console        4.14.34 True      False      False      4d22h
...
storage        4.14.34 True      False      False      4d23h
config-operator 4.15.33 True      False      False      4d23h
etcd           4.15.33 True      False      False      4d23h
```

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	4d23h	v1.27.15+6147456
ctrl-plane-1	Ready	control-plane,master	4d23h	v1.27.15+6147456
ctrl-plane-2	Ready	control-plane,master	4d23h	v1.27.15+6147456
worker-0	Ready	mcp-1,worker	4d23h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	4d23h	v1.27.15+6147456

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
openshift-marketplace	redhat-marketplace-rf86t	0/1	ContainerCreating	0	0s

Verification

During the update the **watch** command cycles through one or several of the cluster Operators at a time, providing a status of the Operator update in the **MESSAGE** column.

When the cluster Operators update process is complete, each control plane nodes is rebooted, one at a time.



NOTE

During this part of the update, messages are reported that state cluster Operators are being updated again or are in a degraded state. This is because the control plane node is offline while it reboots nodes.

As soon as the last control plane node reboot is complete, the cluster version is displayed as updated.

When the control plane update is complete a message such as the following is displayed. This example shows an update completed to the intermediate y-stream release.

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE	STATUS
version	4.15.33	True	False	28m	Cluster version is 4.15.33

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED	SINCE	MESSAGE
authentication	4.15.33	True	False	False	5d	
baremetal	4.15.33	True	False	False	5d	
cloud-controller-manager	4.15.33	True	False	False	5d1h	
cloud-credential	4.15.33	True	False	False	5d1h	
cluster-autoscaler	4.15.33	True	False	False	5d	
config-operator	4.15.33	True	False	False	5d	
console	4.15.33	True	False	False	5d	

...

service-ca	4.15.33	True	False	False	5d
storage	4.15.33	True	False	False	5d

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d	v1.28.13+2ca1a23
ctrl-plane-1	Ready	control-plane,master	5d	v1.28.13+2ca1a23
ctrl-plane-2	Ready	control-plane,master	5d	v1.28.13+2ca1a23
worker-0	Ready	mcp-1,worker	5d	v1.28.13+2ca1a23
worker-1	Ready	mcp-2,worker	5d	v1.28.13+2ca1a23

13.2.6.4. Updating the OLM Operators

Software needs to be vetted before it is loaded onto a production cluster. Production clusters are also quite often configured in disconnected network, which means that they are not always directly connected to the internet. Because the clusters are in a disconnected network, the OpenShift Container Platform Operators are configured for manual update during installation so that new versions can be managed on a cluster-by-cluster basis. Complete the following procedure to move the Operators to the newer versions.

Procedure

1. Check to see which Operators need to be updated:

```
$ oc get installplan -A | grep -E 'APPROVED|false'
```

Example output

NAMESPACE	NAME	CSV	APPROVAL
metallb-system	install-nwjnh	metallb-operator.v4.16.0-202409202304	Manual
openshift-nmstate	install-5r7wr	kubernetes-nmstate-operator.4.16.0-202409251605	Manual

2. Patch the **InstallPlan** resources for those Operators:

```
$ oc patch installplan -n metallb-system install-nwjnh --type merge --patch \
'{"spec":{"approved":true}}'
```

Example output

```
installplan.operators.coreos.com/install-nwjnh patched
```

3. Monitor the namespace by running the following command:

```
$ oc get all -n metallb-system
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
pod/metallb-operator-controller-manager-69b5f884c-8bp22	0/1	ContainerCreating	0	4s
pod/metallb-operator-controller-manager-77895bdb46-bqjdx	1/1	Running	0	4m1s
pod/metallb-operator-webhook-server-5d9b968896-vnbhk	0/1	ContainerCreating	0	4s
pod/metallb-operator-webhook-server-d76f9c6c8-57r4w	1/1	Running	0	4m1s
...				

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/metallb-operator-controller-manager-69b5f884c	1	1	0	4s

```
replicaset.apps/metallb-operator-controller-manager-77895bdb46 1      1      1      4m1s
replicaset.apps/metallb-operator-controller-manager-99b76f88 0      0      0      4m40s
replicaset.apps/metallb-operator-webhook-server-5d9b968896 1      1      0      4s
replicaset.apps/metallb-operator-webhook-server-6f7dbfdb88 0      0      0      4m40s
replicaset.apps/metallb-operator-webhook-server-d76f9c6c8 1      1      1      4m1s
```

When the update is complete, the required pods should be in a **Running** state, and the required **ReplicaSet** resources should be ready:

```
NAME                                READY STATUS RESTARTS AGE
pod/metallb-operator-controller-manager-69b5f884c-8bp22 1/1   Running 0       25s
pod/metallb-operator-webhook-server-5d9b968896-vnbhk    1/1   Running 0       25s
...

NAME                                DESIRED CURRENT READY AGE
replicaset.apps/metallb-operator-controller-manager-69b5f884c 1      1      1      25s
replicaset.apps/metallb-operator-controller-manager-77895bdb46 0      0      0      4m22s
replicaset.apps/metallb-operator-webhook-server-5d9b968896 1      1      1      25s
replicaset.apps/metallb-operator-webhook-server-d76f9c6c8 0      0      0      4m22s
```

Verification

- Verify that the Operators do not need to be updated for a second time:

```
$ oc get installplan -A | grep -E 'APPROVED|false'
```

There should be no output returned.



NOTE

Sometimes you have to approve an update twice because some Operators have interim z-stream release versions that need to be installed before the final version.

Additional resources

- [Updating the worker nodes](#)

13.2.6.4.1. Performing the second y-stream update

After completing the first y-stream update, you must update the y-stream control plane version to the new EUS version.

Procedure

1. Verify that the <4.y.z> release that you selected is still listed as a good channel to move to:

```
$ oc adm upgrade
```

Example output

```
Cluster version is 4.15.33
```

Upgradeable=False

Reason: AdminAckRequired

Message: Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin consideration. Please see the knowledge article <https://access.redhat.com/articles/7031404> for details and instructions.

Upstream is unset, so the cluster will use an appropriate default.

Channel: eus-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.16, fast-4.15, fast-4.16, stable-4.15, stable-4.16)

Recommended updates:

VERSION	IMAGE
4.16.14	quay.io/openshift-release-dev/ocp-release@sha256:0521a0f1acd2d1b77f76259cb9bae9c743c60c37d9903806a3372c1414253658
4.16.13	quay.io/openshift-release-dev/ocp-release@sha256:6078cb4ae197b5b0c526910363b8aff540343bfac62ecb1ead9e068d541da27b
4.15.34	quay.io/openshift-release-dev/ocp-release@sha256:f2e0c593f6ed81250c11d0bac94dbaf63656223477b7e8693a652f933056af6e



NOTE

If you update soon after the initial GA of a new Y-stream release, you might not see new y-stream releases available when you run the **oc adm upgrade** command.

- Optional: View the potential update releases that are not recommended. Run the following command:

```
$ oc adm upgrade --include-not-recommended
```

Example output

Cluster version is 4.15.33

Upgradeable=False

Reason: AdminAckRequired

Message: Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin consideration. Please see the knowledge article <https://access.redhat.com/articles/7031404> for details and instructions.

Upstream is unset, so the cluster will use an appropriate default. Channel: eus-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.16, fast-4.15, fast-4.16, stable-4.15, stable-4.16)

Recommended updates:

VERSION	IMAGE
4.16.14	quay.io/openshift-release-dev/ocp-

```

release@sha256:0521a0f1acd2d1b77f76259cb9bae9c743c60c37d9903806a3372c141425365
8
4.16.13 quay.io/openshift-release-dev/ocp-
release@sha256:6078cb4ae197b5b0c526910363b8aff540343bfac62ecb1ead9e068d541da27
b
4.15.34 quay.io/openshift-release-dev/ocp-
release@sha256:f2e0c593f6ed81250c11d0bac94dbaf63656223477b7e8693a652f933056af6e

```

Supported but not recommended updates:

```

Version: 4.16.15
Image: quay.io/openshift-release-dev/ocp-release@sha256:671bc35e
Recommended: Unknown
Reason: EvaluationFailed
Message: Exposure to AzureRegistryImagePreservation is unknown due to an evaluation
failure: invalid PromQL result length must be one, but is 0
In Azure clusters, the in-cluster image registry may fail to preserve images on update.
https://issues.redhat.com/browse/IR-461

```



NOTE

The example shows a potential error that can affect clusters hosted in Microsoft Azure. It does not show risks for bare-metal clusters.

13.2.6.4.2. Acknowledging the y-stream release update

When moving between y-stream releases, you must run a patch command to explicitly acknowledge the update. In the output of the **oc adm upgrade** command, a URL is provided that shows the specific command to run.

Prerequisites

- You have verified that APIs for all of the applications running on your cluster are compatible with the next Y-stream release of OpenShift Container Platform. For more details about compatibility, see "Verifying cluster API versions between update versions".
- Complete the administrative acknowledgment to start the cluster update by running the following command:

```
$ oc adm upgrade
```

If the cluster update does not complete successfully, more details about the update failure are provided in the **Reason** and **Message** sections.

```
Cluster version is 4.15.45
```

```
Upgradeable=False
```

```
Reason: MultipleReasons
```

```
Message: Cluster should not be upgraded between minor versions for multiple reasons:
AdminAckRequired,ResourceDeletesInProgress
```

```
* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin
consideration. Please see the knowledge article https://access.redhat.com/articles/7031404
for details and instructions.
```

* Cluster minor level upgrades are not allowed while resource deletions are in progress;
resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"

ReleaseAccepted=False

Reason: PreconditionChecks

Message: Preconditions failed for payload loaded version="4.16.34"
image="quay.io/openshift-release-dev/ocp-release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b8": Precondition "ClusterVersionUpgradeable" failed because of "MultipleReasons": Cluster should not be upgraded between minor versions for multiple reasons:
AdminAckRequired,ResourceDeletesInProgress

* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin consideration. Please see the knowledge article <https://access.redhat.com/articles/7031404> for details and instructions.

* Cluster minor level upgrades are not allowed while resource deletions are in progress;
resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"

Upstream is unset, so the cluster will use an appropriate default.

Channel: eus-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.16, fast-4.15, fast-4.16, stable-4.15, stable-4.16)

Recommended updates:

VERSION	IMAGE
4.16.34	quay.io/openshift-release-dev/ocp-release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b8



NOTE

In this example, a linked Red Hat Knowledgebase article ([Preparing to upgrade to OpenShift Container Platform 4.16](#)) provides more detail about verifying API compatibility between releases.

- Verify the update by running the following command:

```
$ oc get configmap admin-acks -n openshift-config -o json | jq .data
```

```
{
  "ack-4.14-kube-1.28-api-removals-in-4.15": "true",
  "ack-4.15-kube-1.29-api-removals-in-4.16": "true"
}
```



NOTE

In this example, the cluster is updated from version 4.14 to 4.15, and then from 4.15 to 4.16 in a Control Plane Only update.

Additional resources

- [Preparing to update to OpenShift Container Platform 4.22](#)

13.2.6.5. Starting the y-stream control plane update

After you have determined the full new release that you are moving to, you can run the **oc adm upgrade --to=x.y.z** command.

Procedure

- Start the y-stream control plane update. For example, run the following command:

```
$ oc adm upgrade --to=4.16.14
```

Example output

```
Requested update to 4.16.14
```

You might move to a z-stream release that has potential issues with platforms other than the one you are running on. The following example shows a potential problem for cluster updates on Microsoft Azure:

```
$ oc adm upgrade --to=4.16.15
```

Example output

```
error: the update 4.16.15 is not one of the recommended updates, but is available as a
conditional update. To accept the Recommended=Unknown risk and to proceed with update
use --allow-not-recommended.
Reason: EvaluationFailed
Message: Exposure to AzureRegistryImagePreservation is unknown due to an evaluation
failure: invalid PromQL result length must be one, but is 0
In Azure clusters, the in-cluster image registry may fail to preserve images on update.
https://issues.redhat.com/browse/IR-461
```



NOTE

The example shows a potential error that can affect clusters hosted in Microsoft Azure. It does not show risks for bare-metal clusters.

```
$ oc adm upgrade --to=4.16.15 --allow-not-recommended
```

Example output

```
warning: with --allow-not-recommended you have accepted the risks with 4.14.11 and
bypassed Recommended=Unknown EvaluationFailed: Exposure to
AzureRegistryImagePreservation is unknown due to an evaluation failure: invalid PromQL
result length must be one, but is 0
In Azure clusters, the in-cluster image registry may fail to preserve images on update.
https://issues.redhat.com/browse/IR-461

Requested update to 4.16.15
```

13.2.6.6. Monitoring the second part of a <y+1> cluster update

Monitor the second part of the cluster update to the <y+1> version.

Procedure

- Monitor the progress of the second part of the <y+1> update. For example, to monitor the update from 4.15 to 4.16, run the following command:

```
$ watch "oc get clusterversion; echo; oc get co | head -1; oc get co | grep 4.15; oc get co | grep 4.16; echo; oc get no; echo; oc get po -A | grep -E -iv 'running|complete'"
```

Example output

```
NAME    VERSION AVAILABLE PROGRESSING SINCE STATUS
version 4.15.33 True      True      10m      Working towards 4.16.14: 132 of 903 done
(14% complete), waiting on kube-controller-manager, kube-scheduler
```

```
NAME          VERSION AVAILABLE PROGRESSING DEGRADED SINCE
MESSAGE
authentication 4.15.33 True      False     False    5d3h
baremetal      4.15.33 True      False     False    5d4h
cloud-controller-manager 4.15.33 True      False     False    5d4h
cloud-credential 4.15.33 True      False     False    5d4h
cluster-autoscaler 4.15.33 True      False     False    5d4h
console        4.15.33 True      False     False    5d3h
...
config-operator 4.16.14 True      False     False    5d4h
etcd           4.16.14 True      False     False    5d4h
kube-apiserver 4.16.14 True      True       False    5d4h
NodeInstallerProgressing: 1 node is at revision 15; 2 nodes are at revision 17
```

```
NAME    STATUS ROLES          AGE VERSION
ctrl-plane-0 Ready control-plane,master 5d4h v1.28.13+2ca1a23
ctrl-plane-1 Ready control-plane,master 5d4h v1.28.13+2ca1a23
ctrl-plane-2 Ready control-plane,master 5d4h v1.28.13+2ca1a23
worker-0 Ready mcp-1,worker 5d4h v1.27.15+6147456
worker-1 Ready mcp-2,worker 5d4h v1.27.15+6147456
```

```
NAMESPACE          NAME                                READY
STATUS RESTARTS AGE
openshift-kube-apiserver kube-apiserver-ctrl-plane-0
0/5 Pending 0 <invalid>
```

As soon as the last control plane node is complete, the cluster version is updated to the new EUS release. For example:

```
NAME    VERSION AVAILABLE PROGRESSING SINCE STATUS
version 4.16.14 True      False     123m      Cluster version is 4.16.14
```

```
NAME          VERSION AVAILABLE PROGRESSING DEGRADED
SINCE MESSAGE
authentication 4.16.14 True      False     False    5d6h
baremetal      4.16.14 True      False     False    5d7h
cloud-controller-manager 4.16.14 True      False     False    5d7h
cloud-credential 4.16.14 True      False     False    5d7h
cluster-autoscaler 4.16.14 True      False     False    5d7h
```

```

config-operator          4.16.14 True    False    False 5d7h
console                  4.16.14 True    False    False 5d6h
#...
operator-lifecycle-manager-packageserver 4.16.14 True    False    False 5d7h
service-ca               4.16.14 True    False    False 5d7h
storage                  4.16.14 True    False    False 5d7h

```

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d7h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d7h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d7h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d7h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	5d7h	v1.27.15+6147456

Additional resources

- [Monitoring the cluster update](#)

13.2.6.7. Updating all the OLM Operators

In the second phase of a multi-version upgrade, you must approve all of the Operators and additionally add installations plans for any other Operators that you want to upgrade.

Follow the same procedure as outlined in "Updating the OLM Operators". Ensure that you also update any non-OLM Operators as required.

Procedure

1. Monitor the cluster update. For example, to monitor the cluster update from version 4.14 to 4.15, run the following command:

```
$ watch "oc get clusterversion; echo; oc get co | head -1; oc get co | grep 4.14; oc get co | grep 4.15; echo; oc get no; echo; oc get po -A | grep -E -iv 'running|complete'"
```

2. Check to see which Operators need to be updated:

```
$ oc get installplan -A | grep -E 'APPROVED|false'
```

3. Patch the **InstallPlan** resources for those Operators:

```
$ oc patch installplan -n metallb-system install-nwjnh --type merge --patch \
'{"spec":{"approved":true}}'
```

4. Monitor the namespace by running the following command:

```
$ oc get all -n metallb-system
```

When the update is complete, the required pods should be in a **Running** state, and the required **ReplicaSet** resources should be ready.

Verification

During the update the **watch** command cycles through one or several of the cluster Operators at a time, providing a status of the Operator update in the **MESSAGE** column.

When the cluster Operators update process is complete, each control plane nodes is rebooted, one at a time.



NOTE

During this part of the update, messages are reported that state cluster Operators are being updated again or are in a degraded state. This is because the control plane node is offline while it reboots nodes.

Additional resources

- [Monitoring the cluster update](#)
- [Updating the OLM Operators](#)

13.2.6.8. Updating the worker nodes

You upgrade the worker nodes after you have updated the control plane by unpausing the relevant **mcp** groups you created. Unpausing the **mcp** group starts the upgrade process for the worker nodes in that group. Each of the worker nodes in the cluster reboot to upgrade to the new EUS, y-stream or z-stream version as required.

In the case of control plane only upgrades, note that when a worker node is updated it will only require one reboot and will jump <y+2>-release versions. This is a feature that was added to decrease the amount of time that it takes to upgrade large bare-metal clusters.



IMPORTANT

This is a potential holding point. You can have a cluster version that is fully supported to run in production with the control plane that is updated to a new EUS release while the worker nodes are at a <y-2>-release. This allows large clusters to upgrade in steps across several maintenance windows.

Procedure

1. You can check how many nodes are managed in an **mcp** group. Run the following command to get the list of **mcp** groups:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT		
DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-c9a52144456dbff9c9af9c5a37d1b614	True	False	False
3	3	3	0	36d
mcp-1	rendered-mcp-1-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
mcp-2	rendered-mcp-2-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
worker	rendered-worker-f1ab7b9a768e1b0ac9290a18817f60f0	True	False	False
0	0	0	0	36d

**NOTE**

You decide how many **mcp** groups to upgrade at a time. This depends on how many pods can be taken down at a time and how your pod disruption budget and anti-affinity settings are configured.

2. Get the list of nodes in the cluster:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	5d8h	v1.27.15+6147456

3. Confirm the **MachineConfigPool** groups that are paused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  true
mcp-2  true
```

**NOTE**

Each **MachineConfigPool** can be unpaused independently. Therefore, if a maintenance window runs out of time other MCPs do not need to be unpaused immediately. The cluster is supported to run with some worker nodes still at <y-2>-release version.

4. Unpause the required **mcp** group to begin the upgrade:

```
$ oc patch mcp/mcp-1 --type merge --patch '{"spec":{"paused":false}}'
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/mcp-1 patched
```

5. Confirm that the required **mcp** group is unpaused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  false
mcp-2  true
```

- As each **mcp** group is upgraded, continue to unpause and upgrade the remaining nodes.

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.29.8+f10c92d
worker-1	NotReady,SchedulingDisabled	mcp-2,worker	5d8h	v1.27.15+6147456

13.2.6.9. Verifying the health of the newly updated cluster

After updating the cluster, verify that the cluster is back up and running.

Procedure

- Check the cluster version by running the following command:

```
$ oc get clusterversion
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE	STATUS
version	4.16.14	True	False	4h38m	Cluster version is 4.16.14

This should return the new cluster version and the **PROGRESSING** column should return **False**.

- Check that all nodes are ready:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d9h	v1.29.8+f10c92d
worker-1	Ready	mcp-2,worker	5d9h	v1.29.8+f10c92d

All nodes in the cluster should be in a **Ready** status and running the same version.

- Check that there are no paused **mcp** resources in the cluster:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  false
mcp-2  false
```

- Check that all cluster Operators are available:

```
$ oc get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
authentication	4.16.14	True	False	False 5d9h
baremetal	4.16.14	True	False	False 5d9h
cloud-controller-manager	4.16.14	True	False	False 5d10h
cloud-credential	4.16.14	True	False	False 5d10h
cluster-autoscaler	4.16.14	True	False	False 5d9h
config-operator	4.16.14	True	False	False 5d9h
console	4.16.14	True	False	False 5d9h
control-plane-machine-set	4.16.14	True	False	False 5d9h
csi-snapshot-controller	4.16.14	True	False	False 5d9h
dns	4.16.14	True	False	False 5d9h
etcd	4.16.14	True	False	False 5d9h
image-registry	4.16.14	True	False	False 85m
ingress	4.16.14	True	False	False 5d9h
insights	4.16.14	True	False	False 5d9h
kube-apiserver	4.16.14	True	False	False 5d9h
kube-controller-manager	4.16.14	True	False	False 5d9h
kube-scheduler	4.16.14	True	False	False 5d9h
kube-storage-version-migrator	4.16.14	True	False	False 4h48m
machine-api	4.16.14	True	False	False 5d9h
machine-approver	4.16.14	True	False	False 5d9h
machine-config	4.16.14	True	False	False 5d9h
marketplace	4.16.14	True	False	False 5d9h
monitoring	4.16.14	True	False	False 5d9h
network	4.16.14	True	False	False 5d9h
node-tuning	4.16.14	True	False	False 5d7h
openshift-apiserver	4.16.14	True	False	False 5d9h
openshift-controller-manager	4.16.14	True	False	False 5d9h
openshift-samples	4.16.14	True	False	False 5h24m
operator-lifecycle-manager	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-catalog	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-packageserver	4.16.14	True	False	False 5d9h
service-ca	4.16.14	True	False	False 5d9h
storage	4.16.14	True	False	False 5d9h

All cluster Operators should report **True** in the **AVAILABLE** column.

5. Check that all pods are healthy:

```
$ oc get po -A | grep -E -iv 'complete|running'
```

This should not return any pods.



NOTE

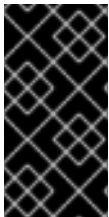
You might see a few pods still moving after the update. Watch this for a while to make sure all pods are cleared.

13.2.7. Completing the y-stream cluster update

Complete the following steps to perform a y-stream cluster update.

13.2.7.1. Acknowledging the control plane only or y-stream update

When you update to all versions from 4.11 and later, you must manually acknowledge that the update can continue.



IMPORTANT

Before you acknowledge the update, verify that you are not using any of the Kubernetes APIs that are removed from the version you are updating to. For example, in OpenShift Container Platform 4.17, there are no API removals. See "Kubernetes API removals" for more information.

Prerequisites

- You have verified that APIs for all of the applications running on your cluster are compatible with the next Y-stream release of OpenShift Container Platform. For more details about compatibility, see "Verifying cluster API versions between update versions".

Procedure

- Check the cluster update status to determine if administrative acknowledgment is required by running the following command:

```
$ oc adm upgrade
```

If the cluster update does not complete successfully, more details about the update failure are provided in the **Reason** and **Message** sections.

Example output:

```
Cluster version is 4.15.45
```

```
Upgradeable=False
```

```
Reason: MultipleReasons
```

```
Message: Cluster should not be upgraded between minor versions for multiple reasons:
AdminAckRequired,ResourceDeletesInProgress
```

* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin consideration. Please see the knowledge article <https://access.redhat.com/articles/7031404> for details and instructions.

* Cluster minor level upgrades are not allowed while resource deletions are in progress; resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"

ReleaseAccepted=False

Reason: PreconditionChecks

Message: Preconditions failed for payload loaded version="4.16.34"

image="quay.io/openshift-release-dev/ocp-release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b8": Precondition "ClusterVersionUpgradeable" failed because of "MultipleReasons": Cluster should not be upgraded between minor versions for multiple reasons: AdminAckRequired,ResourceDeletesInProgress

* Kubernetes 1.29 and therefore OpenShift 4.16 remove several APIs which require admin consideration. Please see the knowledge article <https://access.redhat.com/articles/7031404> for details and instructions.

* Cluster minor level upgrades are not allowed while resource deletions are in progress; resources=PrometheusRule "openshift-kube-apiserver/kube-apiserver-recording-rules"

Upstream is unset, so the cluster will use an appropriate default.

Channel: eus-4.16 (available channels: candidate-4.15, candidate-4.16, eus-4.16, fast-4.15, fast-4.16, stable-4.15, stable-4.16)

Recommended updates:

VERSION IMAGE

4.16.34 quay.io/openshift-release-dev/ocp-release@sha256:41bb08c560f6db5039ccdf242e590e8b23049b5eb31e1c4f6021d1d520b353b8



NOTE

In this example, a linked Red Hat Knowledgebase article ([Preparing to upgrade to OpenShift Container Platform 4.16](#)) provides more detail about verifying API compatibility between releases.

Verification

- Verify the update by running the following command:

```
$ oc get configmap admin-acks -n openshift-config -o json | jq .data
```

If the acknowledgment was successful, the output shows the acknowledged API removals for each update:

```
{
  "ack-4.14-kube-1.28-api-removals-in-4.15": "true",
  "ack-4.15-kube-1.29-api-removals-in-4.16": "true"
}
```


**NOTE**

In this example, the cluster is updated from version 4.14 to 4.15, and then from 4.15 to 4.16 in a Control Plane Only update.

Additional resources

- [Kubernetes API removals](#)

13.2.7.2. Starting the cluster update

When updating from one y-stream release to the next, you must ensure that the intermediate z-stream releases are also compatible.

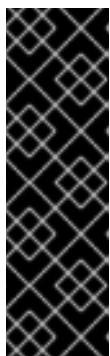
**NOTE**

You can verify that you are updating to a viable release by running the **oc adm upgrade** command. The **oc adm upgrade** command lists the compatible update releases.

Procedure

1. Start the update:

```
$ oc adm upgrade --to=4.15.33
```

**IMPORTANT**

- **Control plane only update:** Ensure you point to the interim <y+1> release path
- **Y-stream update** - Ensure you use the correct <y.z> release that follows the Kubernetes [version skew policy](#).
- **Z-stream update** - Verify that there are no problems moving to that specific release

```
Requested update to <version>
```

+ where:

+ **<version>::** Specifies the version number for your particular update, such as **4.15.33**.

Additional resources

- [Selecting the target release](#)

13.2.7.3. Monitoring the cluster update

You should check the cluster health often during the update. Check for the node status, cluster Operators status and failed pods.

Procedure

- Monitor the cluster update. For example, to monitor the cluster update from version 4.14 to 4.15, run the following command:

```
$ watch "oc get clusterversion; echo; oc get co | head -1; oc get co | grep 4.14; oc get co | grep 4.15; echo; oc get no; echo; oc get po -A | grep -E -iv 'running|complete'"
```

Example output

```
NAME    VERSION AVAILABLE PROGRESSING SINCE STATUS
version 4.14.34 True      True      4m6s Working towards 4.15.33: 111 of 873 done
(12% complete), waiting on kube-apiserver
```

```
NAME          VERSION AVAILABLE PROGRESSING DEGRADED SINCE
MESSAGE
authentication 4.14.34 True      False      False      4d22h
baremetal      4.14.34 True      False      False      4d23h
cloud-controller-manager 4.14.34 True      False      False      4d23h
cloud-credential 4.14.34 True      False      False      4d23h
cluster-autoscaler 4.14.34 True      False      False      4d23h
console        4.14.34 True      False      False      4d22h
```

...

```
storage        4.14.34 True      False      False      4d23h
config-operator 4.15.33 True      False      False      4d23h
etcd           4.15.33 True      False      False      4d23h
```

```
NAME    STATUS ROLES          AGE  VERSION
ctrl-plane-0 Ready control-plane,master 4d23h v1.27.15+6147456
ctrl-plane-1 Ready control-plane,master 4d23h v1.27.15+6147456
ctrl-plane-2 Ready control-plane,master 4d23h v1.27.15+6147456
worker-0 Ready mcp-1,worker 4d23h v1.27.15+6147456
worker-1 Ready mcp-2,worker 4d23h v1.27.15+6147456
```

```
NAMESPACE    NAME                      READY STATUS    RESTARTS AGE
openshift-marketplace redhat-marketplace-rf86t 0/1 ContainerCreating 0 0s
```

Verification

During the update the **watch** command cycles through one or several of the cluster Operators at a time, providing a status of the Operator update in the **MESSAGE** column.

When the cluster Operators update process is complete, each control plane nodes is rebooted, one at a time.



NOTE

During this part of the update, messages are reported that state cluster Operators are being updated again or are in a degraded state. This is because the control plane node is offline while it reboots nodes.

As soon as the last control plane node reboot is complete, the cluster version is displayed as updated.

When the control plane update is complete a message such as the following is displayed. This example shows an update completed to the intermediate y-stream release.

```
NAME    VERSION AVAILABLE PROGRESSING SINCE STATUS
version 4.15.33 True      False      28m Cluster version is 4.15.33
```

```
NAME                VERSION AVAILABLE PROGRESSING DEGRADED SINCE MESSAGE
authentication      4.15.33 True      False      False 5d
baremetal           4.15.33 True      False      False 5d
cloud-controller-manager 4.15.33 True      False      False 5d1h
cloud-credential     4.15.33 True      False      False 5d1h
cluster-autoscaler   4.15.33 True      False      False 5d
config-operator      4.15.33 True      False      False 5d
console             4.15.33 True      False      False 5d
```

```
...
```

```
service-ca          4.15.33 True      False      False 5d
storage             4.15.33 True      False      False 5d
```

```
NAME    STATUS ROLES          AGE VERSION
ctrl-plane-0 Ready control-plane,master 5d v1.28.13+2ca1a23
ctrl-plane-1 Ready control-plane,master 5d v1.28.13+2ca1a23
ctrl-plane-2 Ready control-plane,master 5d v1.28.13+2ca1a23
worker-0 Ready mcp-1,worker 5d v1.28.13+2ca1a23
worker-1 Ready mcp-2,worker 5d v1.28.13+2ca1a23
```

13.2.7.4. Updating the OLM Operators

Software needs to be vetted before it is loaded onto a production cluster. Production clusters are also quite often configured in disconnected network, which means that they are not always directly connected to the internet. Because the clusters are in a disconnected network, the OpenShift Container Platform Operators are configured for manual update during installation so that new versions can be managed on a cluster-by-cluster basis. Complete the following procedure to move the Operators to the newer versions.

Procedure

1. Check to see which Operators need to be updated:

```
$ oc get installplan -A | grep -E 'APPROVED|false'
```

Example output

```
NAMESPACE    NAME    CSV                                APPROVAL
APPROVED
metallb-system install-nwjnh metallb-operator.v4.16.0-202409202304 Manual
false
openshift-nmstate install-5r7wr kubernetes-nmstate-operator.4.16.0-202409251605
Manual false
```

2. Patch the **InstallPlan** resources for those Operators:

```
$ oc patch installplan -n metallb-system install-nwjnh --type merge --patch \
{"spec":{"approved":true}}
```

Example output

```
installplan.operators.coreos.com/install-nwjnh patched
```

3. Monitor the namespace by running the following command:

```
$ oc get all -n metallb-system
```

Example output

```
NAME                                READY STATUS    RESTARTS AGE
pod/metallb-operator-controller-manager-69b5f884c-8bp22  0/1 ContainerCreating 0
4s
pod/metallb-operator-controller-manager-77895bdb46-bqjdx  1/1 Running          0
4m1s
pod/metallb-operator-webhook-server-5d9b968896-vnbhk     0/1 ContainerCreating 0
4s
pod/metallb-operator-webhook-server-d76f9c6c8-57r4w     1/1 Running          0
4m1s
...

NAME                                DESIRED CURRENT READY AGE
replicaset.apps/metallb-operator-controller-manager-69b5f884c  1      1      0      4s
replicaset.apps/metallb-operator-controller-manager-77895bdb46  1      1      1      4m1s
replicaset.apps/metallb-operator-controller-manager-99b76f88    0      0      0      4m40s
replicaset.apps/metallb-operator-webhook-server-5d9b968896     1      1      0      4s
replicaset.apps/metallb-operator-webhook-server-6f7dbfdb88     0      0      0      4m40s
replicaset.apps/metallb-operator-webhook-server-d76f9c6c8      1      1      1      4m1s
```

When the update is complete, the required pods should be in a **Running** state, and the required **ReplicaSet** resources should be ready:

```
NAME                                READY STATUS    RESTARTS AGE
pod/metallb-operator-controller-manager-69b5f884c-8bp22  1/1 Running    0      25s
pod/metallb-operator-webhook-server-5d9b968896-vnbhk     1/1 Running    0      25s
...

NAME                                DESIRED CURRENT READY AGE
replicaset.apps/metallb-operator-controller-manager-69b5f884c  1      1      1      25s
replicaset.apps/metallb-operator-controller-manager-77895bdb46  0      0      0      4m22s
replicaset.apps/metallb-operator-webhook-server-5d9b968896     1      1      1      25s
replicaset.apps/metallb-operator-webhook-server-d76f9c6c8      0      0      0      4m22s
```

Verification

- Verify that the Operators do not need to be updated for a second time:

```
$ oc get installplan -A | grep -E 'APPROVED|false'
```

There should be no output returned.



NOTE

Sometimes you have to approve an update twice because some Operators have interim z-stream release versions that need to be installed before the final version.

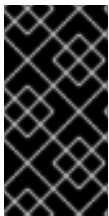
Additional resources

- [Updating the worker nodes](#)

13.2.7.5. Updating the worker nodes

You upgrade the worker nodes after you have updated the control plane by unpausing the relevant **mcp** groups you created. Unpausing the **mcp** group starts the upgrade process for the worker nodes in that group. Each of the worker nodes in the cluster reboot to upgrade to the new EUS, y-stream or z-stream version as required.

In the case of control plane only upgrades, note that when a worker node is updated it will only require one reboot and will jump <y+2>-release versions. This is a feature that was added to decrease the amount of time that it takes to upgrade large bare-metal clusters.



IMPORTANT

This is a potential holding point. You can have a cluster version that is fully supported to run in production with the control plane that is updated to a new EUS release while the worker nodes are at a <y-2>-release. This allows large clusters to upgrade in steps across several maintenance windows.

Procedure

1. You can check how many nodes are managed in an **mcp** group. Run the following command to get the list of **mcp** groups:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT		
DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-c9a52144456dbff9c9af9c5a37d1b614	True	False	False
3	3	3	0	36d
mcp-1	rendered-mcp-1-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
mcp-2	rendered-mcp-2-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
worker	rendered-worker-f1ab7b9a768e1b0ac9290a18817f60f0	True	False	False
0	0	0	0	36d

**NOTE**

You decide how many **mcp** groups to upgrade at a time. This depends on how many pods can be taken down at a time and how your pod disruption budget and anti-affinity settings are configured.

2. Get the list of nodes in the cluster:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	5d8h	v1.27.15+6147456

3. Confirm the **MachineConfigPool** groups that are paused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], ["---","-----"], (.items[] | [(.metadata.name), (.spec.paused)]) | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  true
mcp-2  true
```

**NOTE**

Each **MachineConfigPool** can be unpaused independently. Therefore, if a maintenance window runs out of time other MCPs do not need to be unpaused immediately. The cluster is supported to run with some worker nodes still at <y-2>-release version.

4. Unpause the required **mcp** group to begin the upgrade:

```
$ oc patch mcp/mcp-1 --type merge --patch '{"spec":{"paused":false}}'
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/mcp-1 patched
```

5. Confirm that the required **mcp** group is unpaused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], ["---","-----"], (.items[] | [(.metadata.name), (.spec.paused)]) | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  false
mcp-2  true
```

- As each **mcp** group is upgraded, continue to unpause and upgrade the remaining nodes.

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.29.8+f10c92d
worker-1	NotReady,SchedulingDisabled	mcp-2,worker	5d8h	v1.27.15+6147456

13.2.7.6. Verifying the health of the newly updated cluster

After updating the cluster, verify that the cluster is back up and running.

Procedure

- Check the cluster version by running the following command:

```
$ oc get clusterversion
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE	STATUS
version	4.16.14	True	False	4h38m	Cluster version is 4.16.14

This should return the new cluster version and the **PROGRESSING** column should return **False**.

- Check that all nodes are ready:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d9h	v1.29.8+f10c92d
worker-1	Ready	mcp-2,worker	5d9h	v1.29.8+f10c92d

All nodes in the cluster should be in a **Ready** status and running the same version.

- Check that there are no paused **mcp** resources in the cluster:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  false
mcp-2  false
```

- Check that all cluster Operators are available:

```
$ oc get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
authentication	4.16.14	True	False	False 5d9h
baremetal	4.16.14	True	False	False 5d9h
cloud-controller-manager	4.16.14	True	False	False 5d10h
cloud-credential	4.16.14	True	False	False 5d10h
cluster-autoscaler	4.16.14	True	False	False 5d9h
config-operator	4.16.14	True	False	False 5d9h
console	4.16.14	True	False	False 5d9h
control-plane-machine-set	4.16.14	True	False	False 5d9h
csi-snapshot-controller	4.16.14	True	False	False 5d9h
dns	4.16.14	True	False	False 5d9h
etcd	4.16.14	True	False	False 5d9h
image-registry	4.16.14	True	False	False 85m
ingress	4.16.14	True	False	False 5d9h
insights	4.16.14	True	False	False 5d9h
kube-apiserver	4.16.14	True	False	False 5d9h
kube-controller-manager	4.16.14	True	False	False 5d9h
kube-scheduler	4.16.14	True	False	False 5d9h
kube-storage-version-migrator	4.16.14	True	False	False 4h48m
machine-api	4.16.14	True	False	False 5d9h
machine-approver	4.16.14	True	False	False 5d9h
machine-config	4.16.14	True	False	False 5d9h
marketplace	4.16.14	True	False	False 5d9h
monitoring	4.16.14	True	False	False 5d9h
network	4.16.14	True	False	False 5d9h
node-tuning	4.16.14	True	False	False 5d7h
openshift-apiserver	4.16.14	True	False	False 5d9h
openshift-controller-manager	4.16.14	True	False	False 5d9h
openshift-samples	4.16.14	True	False	False 5h24m
operator-lifecycle-manager	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-catalog	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-packageserver	4.16.14	True	False	False 5d9h
service-ca	4.16.14	True	False	False 5d9h
storage	4.16.14	True	False	False 5d9h

All cluster Operators should report **True** in the **AVAILABLE** column.

5. Check that all pods are healthy:

```
$ oc get po -A | grep -E -iv 'complete|running'
```

This should not return any pods.



NOTE

You might see a few pods still moving after the update. Watch this for a while to make sure all pods are cleared.

13.2.8. Completing the z-stream cluster update

Complete the following steps to perform a z-stream cluster update.

13.2.8.1. Starting the cluster update

When updating from one y-stream release to the next, you must ensure that the intermediate z-stream releases are also compatible.



NOTE

You can verify that you are updating to a viable release by running the **oc adm upgrade** command. The **oc adm upgrade** command lists the compatible update releases.

Procedure

1. Start the update:

```
$ oc adm upgrade --to=4.15.33
```



IMPORTANT

- **Control plane only update:** Ensure you point to the interim <y+1> release path
- **Y-stream update** - Ensure you use the correct <y.z> release that follows the Kubernetes [version skew policy](#).
- **Z-stream update** - Verify that there are no problems moving to that specific release

```
Requested update to <version>
```

+ where:

+ **<version>**:: Specifies the version number for your particular update, such as **4.15.33**.

Additional resources

- [Selecting the target release](#)

13.2.8.2. Updating the worker nodes

You upgrade the worker nodes after you have updated the control plane by unpausing the relevant **mcp** groups you created. Unpausing the **mcp** group starts the upgrade process for the worker nodes in that group. Each of the worker nodes in the cluster reboot to upgrade to the new EUS, y-stream or z-stream version as required.

In the case of control plane only upgrades, note that when a worker node is updated it will only require one reboot and will jump <y+2>-release versions. This is a feature that was added to decrease the amount of time that it takes to upgrade large bare-metal clusters.



IMPORTANT

This is a potential holding point. You can have a cluster version that is fully supported to run in production with the control plane that is updated to a new EUS release while the worker nodes are at a <y-2>-release. This allows large clusters to upgrade in steps across several maintenance windows.

Procedure

1. You can check how many nodes are managed in an **mcp** group. Run the following command to get the list of **mcp** groups:

```
$ oc get mcp
```

Example output

NAME	CONFIG	UPDATED	UPDATING	DEGRADED
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT		
DEGRADEDMACHINECOUNT	AGE			
master	rendered-master-c9a52144456dbff9c9af9c5a37d1b614	True	False	False
3	3	3	0	36d
mcp-1	rendered-mcp-1-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
mcp-2	rendered-mcp-2-07fe50b9ad51fae43ed212e84e1dcc8e	False	False	False
1	0	0	0	47h
worker	rendered-worker-f1ab7b9a768e1b0ac9290a18817f60f0	True	False	False
0	0	0	0	36d



NOTE

You decide how many **mcp** groups to upgrade at a time. This depends on how many pods can be taken down at a time and how your pod disruption budget and anti-affinity settings are configured.

2. Get the list of nodes in the cluster:

```
$ oc get nodes
```

Example output

-

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.27.15+6147456
worker-1	Ready	mcp-2,worker	5d8h	v1.27.15+6147456

- Confirm the **MachineConfigPool** groups that are paused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], ["---","-----"], (.items[] | [(.metadata.name), (.spec.paused)]) | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  true
mcp-2  true
```



NOTE

Each **MachineConfigPool** can be unpaused independently. Therefore, if a maintenance window runs out of time other MCPs do not need to be unpaused immediately. The cluster is supported to run with some worker nodes still at <y-2>-release version.

- Unpause the required **mcp** group to begin the upgrade:

```
$ oc patch mcp/mcp-1 --type merge --patch '{"spec":{"paused":false}}'
```

Example output

```
machineconfigpool.machineconfiguration.openshift.io/mcp-1 patched
```

- Confirm that the required **mcp** group is unpaused:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], ["---","-----"], (.items[] | [(.metadata.name), (.spec.paused)]) | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
mcp-1  false
mcp-2  true
```

- As each **mcp** group is upgraded, continue to unpause and upgrade the remaining nodes.

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d8h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d8h	v1.29.8+f10c92d
worker-1	NotReady,SchedulingDisabled	mcp-2,worker	5d8h	v1.27.15+6147456

13.2.8.3. Verifying the health of the newly updated cluster

After updating the cluster, verify that the cluster is back up and running.

Procedure

1. Check the cluster version by running the following command:

```
$ oc get clusterversion
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	SINCE	STATUS
version	4.16.14	True	False	4h38m	Cluster version is 4.16.14

This should return the new cluster version and the **PROGRESSING** column should return **False**.

2. Check that all nodes are ready:

```
$ oc get nodes
```

Example output

NAME	STATUS	ROLES	AGE	VERSION
ctrl-plane-0	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-1	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
ctrl-plane-2	Ready	control-plane,master	5d9h	v1.29.8+f10c92d
worker-0	Ready	mcp-1,worker	5d9h	v1.29.8+f10c92d
worker-1	Ready	mcp-2,worker	5d9h	v1.29.8+f10c92d

All nodes in the cluster should be in a **Ready** status and running the same version.

3. Check that there are no paused **mcp** resources in the cluster:

```
$ oc get mcp -o json | jq -r '["MCP","Paused"], [{"---","-----"}, (.items[] | [(.metadata.name), (.spec.paused)])] | @tsv' | grep -v worker
```

Example output

```
MCP    Paused
---    -----
master false
```

```
mcp-1 false
mcp-2 false
```

4. Check that all cluster Operators are available:

```
$ oc get co
```

Example output

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
authentication	4.16.14	True	False	False 5d9h
baremetal	4.16.14	True	False	False 5d9h
cloud-controller-manager	4.16.14	True	False	False 5d10h
cloud-credential	4.16.14	True	False	False 5d10h
cluster-autoscaler	4.16.14	True	False	False 5d9h
config-operator	4.16.14	True	False	False 5d9h
console	4.16.14	True	False	False 5d9h
control-plane-machine-set	4.16.14	True	False	False 5d9h
csi-snapshot-controller	4.16.14	True	False	False 5d9h
dns	4.16.14	True	False	False 5d9h
etcd	4.16.14	True	False	False 5d9h
image-registry	4.16.14	True	False	False 85m
ingress	4.16.14	True	False	False 5d9h
insights	4.16.14	True	False	False 5d9h
kube-apiserver	4.16.14	True	False	False 5d9h
kube-controller-manager	4.16.14	True	False	False 5d9h
kube-scheduler	4.16.14	True	False	False 5d9h
kube-storage-version-migrator	4.16.14	True	False	False 4h48m
machine-api	4.16.14	True	False	False 5d9h
machine-approver	4.16.14	True	False	False 5d9h
machine-config	4.16.14	True	False	False 5d9h
marketplace	4.16.14	True	False	False 5d9h
monitoring	4.16.14	True	False	False 5d9h
network	4.16.14	True	False	False 5d9h
node-tuning	4.16.14	True	False	False 5d7h
openshift-apiserver	4.16.14	True	False	False 5d9h
openshift-controller-manager	4.16.14	True	False	False 5d9h
openshift-samples	4.16.14	True	False	False 5h24m
operator-lifecycle-manager	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-catalog	4.16.14	True	False	False 5d9h
operator-lifecycle-manager-packageserver	4.16.14	True	False	False 5d9h
service-ca	4.16.14	True	False	False 5d9h
storage	4.16.14	True	False	False 5d9h

All cluster Operators should report **True** in the **AVAILABLE** column.

5. Check that all pods are healthy:

```
$ oc get po -A | grep -E -iv 'complete|running'
```

This should not return any pods.

**NOTE**

You might see a few pods still moving after the update. Watch this for a while to make sure all pods are cleared.

13.3. UPGRADING OPENSIFT CONTAINER PLATFORM CLUSTERS WITH RHACM AND TALM

13.3.1. About cluster updates with RHACM and TALM

You can use Red Hat Advanced Cluster Management (RHACM) and Topology Aware Lifecycle Manager (TALM) to perform z-stream, y-stream, and EUS-to-EUS updates on spoke clusters managed from a hub cluster.

The policy-based update workflow uses update policies that you define on the RHACM hub while TALM orchestrates their enforcement across target clusters.

If you are managing a single cluster or troubleshooting a specific cluster directly, see "Updating an OpenShift Container Platform cluster" for manual updates of individual clusters.

13.3.1.1. Policy-based cluster updates with RHACM and TALM

You can use Red Hat Advanced Cluster Management (RHACM) and Topology Aware Lifecycle Manager (TALM) to automate cluster updates across spoke clusters managed from a hub cluster.

Optionally, you can manage RHACM policies through a GitOps workflow, storing them in a Git repository for versioning, review, and auditability.

RHACM provides the policy framework for managing cluster configuration across a fleet of spoke clusters from a central hub cluster. You define RHACM policies that declare the target cluster version, and RHACM evaluates compliance across target clusters.

TALM orchestrates the rollout of RHACM policies to target clusters according to your batching strategy. TALM uses **ClusterGroupUpgrade** custom resources (CRs) to coordinate which clusters to update, which policies to apply, and how to manage concurrency. A **ClusterGroupUpgrade** CR progresses through several states during an update:

- **Preparing:** Cluster readiness and policy compliance are being evaluated.
- **InProgress:** Policies are being applied to clusters in batches.
- **Complete:** All clusters are successfully updated and compliant.
- **Failed:** One or more clusters failed to update.

13.3.1.2. Benefits of TALM for large-scale deployments

Topology Aware Lifecycle Manager (TALM) is specifically designed for large-scale deployments with many clusters across many sites.

With TALM, you can do the following:

- Update hundreds or thousands of clusters from a central hub.
- Test updates on a single target cluster before rolling out to the fleet.

- Schedule updates during planned downtime using blocking custom resources.
- Manage update policies through existing GitOps tools, such as Argo CD, and the same workflows and approval processes you already use for cluster configuration.

13.3.1.3. GitOps workflows for RHACM update policies

GitOps provides a declarative approach to managing cluster configuration and lifecycle operations. You define the desired state of your clusters in Git, and automation tools ensure the actual state matches the desired state.

For cluster updates, the GitOps workflow begins with defining the desired cluster version as a policy in a Git repository. After committing and pushing the policy changes through your review process, you apply the policy to the RHACM hub cluster. TALM then orchestrates the rollout to target clusters according to your batching strategy, using **ClusterGroupUpgrade** custom resources (CRs). You can monitor progress and verify successful updates throughout the process.

With a GitOps workflow for cluster updates, you can do the following:

- Track changes through Git commit history for full audit trails.
- Review policy changes through pull requests before application.
- Revert to previous policy versions if needed.
- Maintain consistent update configurations across all clusters.

13.3.1.4. Cluster update scenarios

You can perform z-stream, y-stream, and EUS-to-EUS updates on clusters managed by Red Hat Advanced Cluster Management (RHACM) and Topology Aware Lifecycle Manager (TALM). Each update scenario has different risk profiles, release cadences, and use cases that align with your operational requirements and maintenance windows.

13.3.1.4.1. Z-stream updates

Z-stream updates apply patch releases within a minor version, such as updating from 4.20.0 to 4.20.1. These updates address security vulnerabilities and bug fixes without introducing new features.

Z-stream releases follow a weekly cadence and should be applied as soon as they become available. The risk profile for z-stream updates is low because they do not include API changes or feature modifications.

The following list describes use cases for z-stream updates:

- Applying critical security patches (CVEs)
- Resolving production bugs identified in the current minor version
- Maintaining compliance with security policies that require timely patching
- Performing regular maintenance updates during normal operating windows

When configured with proper pod disruption budgets, z-stream updates cause minimal workload disruption. Control plane nodes update in a rolling fashion, and worker nodes update one at a time or in small batches.

13.3.1.4.2. Y-stream updates

Y-stream updates move between minor versions, such as updating from 4.20.x to 4.21.0. These updates introduce new features, performance improvements, and might include API deprecations.

Typically, Y-stream releases follow a four-month release cadence. You must update through consecutive y-stream versions and cannot skip versions. For example, to update from 4.20 to 4.22, you must update from 4.20 to 4.21, and then from 4.21 to 4.22. If you need to move 2 y-stream versions, consider EUS-to-EUS updates instead.

The risk profile for y-stream updates is medium because they might include deprecated APIs and configuration changes. Review release notes carefully before updating to identify breaking changes that affect your workloads.

The following list describes use cases for y-stream updates:

- Accessing new OpenShift Container Platform features and capabilities
- Maintaining support lifecycle compliance because older versions eventually lose support
- Meeting requirements for updated Operator versions
- Adopting performance improvements and optimizations

Y-stream updates require a planned maintenance window. The control plane update typically completes in 60 to 120 minutes, and worker node updates take an additional 30 to 60 minutes. You can minimize workload disruption by pausing worker nodes and updating the control plane first.

Additional resources

- [EUS-to-EUS updates](#)

13.3.1.4.3. EUS-to-EUS updates

Extended Update Support (EUS) releases receive 18 months of support and include even-numbered minor versions such as 4.18, 4.20, and 4.22. With EUS-to-EUS updates, the worker nodes in your cluster can jump versions, reducing your upgrade timeline.

With control-plane-only updates, you can update the control plane to a new EUS version while leaving worker nodes at the previous version. The control plane and workers can be up to two minor versions apart during this staged approach.

The risk profile for EUS-to-EUS updates is medium to high because they involve larger version jumps. Thorough testing and validation are critical before performing EUS-to-EUS updates in production.

The following list describes use cases for EUS-to-EUS updates:

- Staging control plane updates separately from worker node updates
- Minimizing workload disruption by deferring worker updates to a later maintenance window
- Aligning updates with long-term support cycles that have 18-month support windows
- Managing capacity-constrained environments where full cluster downtime is not feasible

Control-plane-only EUS updates affect only control plane components. Workloads continue running on worker nodes at the previous version. Worker nodes can be updated days or weeks after the control plane, depending on your maintenance schedule.

13.3.1.4.4. Selecting an update scenario

The following decision matrix helps you determine which update scenario to use:

Table 13.1. Decision matrix for update scenarios

Scenario	When to use	Key considerations
Z-stream	Security patches or bug fixes needed	Apply weekly. Minimal risk and minimal downtime.
Y-stream	New features required or approaching end of support	Typically a four-month cadence. Review release notes and plan a maintenance window.
EUS-to-EUS	Long-term planning with staged rollouts	18-month support. Control plane and workers can update separately. Larger version jumps require more validation.

13.3.1.4.5. Application and cluster configuration policy changes

In addition to OpenShift Container Platform and OLM Operator update policies, review and prepare policies for application and cluster configuration changes that the update might require.

This can include changes to user policies, security context constraints (SCCs), or version-specific configuration for your cluster.

For further policy configuration, see the Red Hat Advanced Cluster Management (RHACM) documentation.

13.3.1.5. Additional resources

- [Updating an OpenShift Container Platform cluster](#)
- [Verifying cluster API versions between update versions](#)
- [Using the Topology Aware Lifecycle Manager for cluster updates](#)
- [Bare metal Core reference design specifications](#)
- [How to use the Topology Aware Lifecycle Manager](#)
- [The ultimate guide to OpenShift release and update process for cluster administrators](#)
- [OpenShift Container Platform update documentation](#)
- [OpenShift Container Platform update lifecycle and support policy](#)

13.3.2. Prepare RHACM policies and TALM for cluster updates

Before you can perform policy-based cluster updates, you must configure your hub cluster with the required Red Hat Advanced Cluster Management (RHACM) policies, placement rules, and Topology Aware Lifecycle Manager (TALM) **ClusterGroupUpgrade** custom resources (CRs).

13.3.2.1. Preparing RHACM policies and placement rules for cluster updates

You can organize your Git repository, configure cluster labels, and create placement rules to target clusters for policy-based updates.

Prerequisites

- RHACM hub cluster is deployed and managing target clusters. For more information, see [Red Hat Advanced Cluster Management for Kubernetes documentation](#).
- Topology Aware Lifecycle Manager (TALM) is installed on the RHACM hub cluster. For more information, see "Installing Topology Aware Lifecycle Manager by using the CLI".
- You have a Git repository for storing update policies with appropriate access controls.
- You have cluster-admin privileges on the RHACM hub cluster.
- The **oc** CLI tool is installed and configured.

Procedure

1. Prepare your Git repository structure similar to the following example:

```

upgrade-policies/
├── policies/
│   ├── zstream/
│   │   └── upgrade-4.20.1-policy.yaml
│   ├── ystream/
│   │   └── upgrade-4.20-policy.yaml
│   └── eus/
│       └── upgrade-4.20-eus-policy.yaml
└── cgu/
    ├── core-zstream-upgrade.yaml
    ├── core-ystream-upgrade.yaml
    └── core-eus-upgrade.yaml
  
```



NOTE

If you are using Kustomize for policy management with Argo CD or Flux, you can add a **kustomization.yaml** file to manage policy overlays. This file is optional if you apply policies directly with **oc apply**.

2. Create a **Placement** rule so that update policies can be bound to clusters through labels.

**NOTE**

If you use PolicyGenerator with Argo CD or a ZTP pipeline, **Placement** and **PlacementBinding** resources are generated automatically from your PolicyGenerator configuration. The following manual steps are provided for environments that do not use PolicyGenerator or for reference when troubleshooting placement issues.

The following example uses the label **upgrade-version-to-4-21=""** to bind a policy to a cluster:

```
apiVersion: cluster.open-cluster-management.io/v1beta1
kind: Placement
metadata:
  name: openshift-upgrade-placement
  namespace: openshift-upgrade-policies
  annotations:
    policy.open-cluster-management.io/description: |
      Placement rule for OpenShift upgrade policies. Targets clusters with
      the label "upgrade-version-to-4-21" for 4.20 to 4.21 upgrades.
spec:
  predicates:
    - requiredClusterSelector:
        labelSelector:
          matchLabels:
            upgrade-version-to-4-21: ""
```

3. Create a **PlacementBinding** resource to connect the policies to the placement rule:

```
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: openshift-upgrade-placement-binding
  namespace: openshift-upgrade-policies
  annotations:
    policy.open-cluster-management.io/description: |
      Binding that connects OpenShift upgrade policies to their placement rule.
      This binding ensures that both the main upgrade policy and pre-upgrade
      health check policy are applied to the same set of target clusters.
placementRef:
  name: openshift-upgrade-placement
  kind: Placement
  apiGroup: cluster.open-cluster-management.io
subjects:
  - name: openshift-upgrade-4.21-policy
    kind: Policy
    apiGroup: policy.open-cluster-management.io
  - name: pre-upgrade-health-check-policy
    kind: Policy
    apiGroup: policy.open-cluster-management.io
```

A single policy, placement rule, and placement binding set can be bound to multiple clusters by adding the matching label to each cluster.

4. Add an update label to your target clusters and verify by running the following commands:

```
$ oc label managedcluster <cluster_name> upgrade-version-to-4-21=""
$ oc get managedcluster <cluster_name> -o jsonpath='{.metadata.labels}'
```

The following example shows the output:

```
{
  "app.kubernetes.io/instance": "clusters",
  "cluster.open-cluster-management.io/clusterset": "default",
  "name": "test1",
  "ocp-version": "4.20",
  "openshiftVersion": "4.20.15",
  "upgrade-version-to-4-21": "",
  "upgrade-version-to-4-22": "",
  "unpause-worker-mcp": ""
}
```

- Optional: Label clusters by site location for topology awareness by running the following command:

```
$ oc label managedcluster <cluster_name> site=<site_name>
```

- Create and apply a **ManagedClusterSet** resource for update targets:

```
apiVersion: cluster.open-cluster-management.io/v1beta2
kind: ManagedClusterSet
metadata:
  name: core-bm-clusters
spec:
  clusterSelector:
    selectorType: LabelSelector
    labelSelector:
      matchLabels:
        cluster-type: bare-metal-core
```

Apply the **ManagedClusterSet** resource by running the following command:

```
$ oc apply -f managedclusterset.yaml
```

Verification

- Verify that TALM is active, cluster labels are applied, and the **ManagedClusterSet** resource exists by running the following commands:

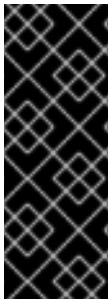
```
$ oc get pods -n openshift-operators | grep cluster-group-upgrades
$ oc get managedclusters --show-labels
$ oc get managedclusterset core-bm-clusters
```

The following example shows the output:

```
cluster-group-upgrades-controller-manager-5d7b9c8f7d-abc12  2/2  Running  0  5m
```

13.3.2.2. ClusterGroupUpgrade custom resource configuration

You can configure **ClusterGroupUpgrade** custom resources (CRs) to orchestrate cluster updates with Topology Aware Lifecycle Manager (TALM). A **ClusterGroupUpgrade** CR defines which clusters to update, which policies to apply, and how to manage batching and concurrency.



IMPORTANT

RHACM policies used for updates must be set to **inform** mode. The **ClusterGroupUpgrade** CR temporarily changes these policies to **enforce** mode at the scheduled update time, applying them to the target clusters. When the **ClusterGroupUpgrade** CR completes or times out, the policies automatically revert to **inform** mode. This ensures that update policies are only enforced at the specific time you intend, rather than immediately when the policy is created.

13.3.2.2.1. Basic ClusterGroupUpgrade CR

The following example shows a basic **ClusterGroupUpgrade** CR:

```
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: <cgu_name>
  namespace: <namespace>
spec:
  clusters:
    - <cluster_name_1>
    - <cluster_name_2>
  managedPolicies:
    - <policy_name>
  enable: false
  remediationStrategy:
    maxConcurrency: <max_concurrency>
    timeout: <timeout_minutes>
```

Table 13.2. ClusterGroupUpgrade CR fields

Field	Description
name	Specifies a name for your ClusterGroupUpgrade CR, for example core-zstream-upgrade-4.20.1 .
namespace	Specifies the namespace for the resource, for example openshift-upgrade-policies .
clusters	Specifies the names of your target clusters.
managedPolicies	Specifies the names of your update policies.
enable	Set to false to create the resource without starting the update. Set to true to start the update immediately.

Field	Description
maxConcurrency	Specifies the number of clusters to update simultaneously. Use 1 for canary testing, 2-5 for production rollouts, or higher for large fleets with proven procedures.
timeout	Specifies the maximum time in minutes to wait for each cluster update. See the timeout guidelines that follow.

13.3.2.2.2. Timeout guidelines

Calculate appropriate timeout values based on your environment:

- Base timeout: 60 minutes for z-stream updates, 120 minutes for y-stream updates.
- Add 30 minutes for each additional worker node beyond 3.
- Add 60 minutes for disconnected environments.
- Add 30 minutes for clusters with large image registries.

13.3.2.2.3. Optional configuration fields

You can add the following optional fields to the **spec** section to customize the update behavior:

- Canary cluster testing
- Pre-update blocking checks
- Post-update label management

13.3.2.2.4. Canary cluster testing

To test the update on a canary cluster before rolling it out to the full fleet, add a **canaries** list under **remediationStrategy** and use **clusterSelector** instead of **clusters** to target clusters by label:

```
clusterSelector:
- <label_selector>
remediationStrategy:
  canaries:
  - <canary_cluster_name>
  maxConcurrency: 1
```

where:

- **<label_selector>**: Specifies a label selector for the target clusters, for example **upgrade-wave=1**.
- **<canary_cluster_name>**: Specifies the name of the canary cluster to update first. TALM updates the canary cluster first, waits for success, and then proceeds with other clusters.

13.3.2.2.5. Pre-update blocking checks

To require pre-update checks to pass before the update begins, add **blockingCRs** and **beforeEnable** fields:

```
beforeEnable:
  addClusterLabels:
    pre-upgrade-check: pending
blockingCRs:
- name: <blocking_cr_name>
  namespace: <blocking_cr_namespace>
  kind: ConfigMap
  conditions:
  - type: <condition_type>
    status: "True"
```

where:

- **<blocking_cr_name>**: Specifies the name of the blocking custom resource, for example **etcd-backup**.
- **<condition_type>**: Specifies the condition that must be met before the update can proceed, for example **BackupComplete**.

13.3.2.2.6. Post-update label management

To update cluster labels after the update completes, add an **afterCompletion** field:

```
afterCompletion:
  addClusterLabels:
    upgraded-to: "<target_version>"
    upgrade-status: complete
  removeClusterLabels:
  - pre-upgrade-check
```

where **<target_version>** is the target version, for example **4.20**.

13.3.2.2.7. Chained ClusterGroupUpgrade CRs for EUS-to-EUS updates

For Extended Update Support (EUS) to EUS updates, you can create chained **ClusterGroupUpgrade** CRs that use **blockingCRs** and **actions** to orchestrate a multi-step update:

```
---
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: upgrade-20to21
  namespace: openshift-upgrade-policies
spec:
  actions:
    afterCompletion:
      deleteClusterLabels:
        upgrade-version-to-4-21: ""
      deleteObjects: true
```

```

      addClusterLabels:
        unpause-workers-4-22: ""
    backup: false
    clusters:
    - <cluster_name>
    enable: true
    managedPolicies:
    - prep-20to22
    - upgrade-ocp-20to21
    - upgrade-olm-20to21
    - upgrade-validate-21
    preCaching: false
    preCachingConfigRef: {}
    remediationStrategy:
      maxConcurrency: 1
      timeout: 480
---
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: upgrade-21to22
  namespace: openshift-upgrade-policies
spec:
  actions:
    afterCompletion:
      deleteClusterLabels:
        upgrade-version-4-22: ""
        unpause-workers-4-22: ""
      deleteObjects: true
    backup: false
    clusters:
    - <cluster_name>
    enable: true
    managedPolicies:
    - upgrade-ocp-21to22
    - upgrade-olm-21to22
    - upgrade-validate-22
    - upgrade-worker-1
    - upgrade-worker-2
    preCaching: false
    preCachingConfigRef: {}
    remediationStrategy:
      maxConcurrency: 1
      timeout: 480
    blockingCRs:
    - name: upgrade-20to21
      namespace: openshift-upgrade-policies

```

where:

- **<cluster_name>**: Specifies the name of the target cluster.
- **actions**: Changes cluster labels as each step completes. This removes temporary update labels and unbinds completed update policies from the cluster.

- **blockingCRs**: Requires the first **ClusterGroupUpgrade** CR to complete before the second **ClusterGroupUpgrade** CR begins enforcing its policies.
- **upgrade-worker-1, upgrade-worker-2**: Separate unpause policies for each **MachineConfigPool** resource. Your cluster might have more machine config pools, requiring additional policies.

13.3.2.2.8. Applying and monitoring a ClusterGroupUpgrade CR

After creating a **ClusterGroupUpgrade** CR, apply and enable it by running the following commands:

```
$ oc apply -f <cgu_cr_filename>.yaml
$ oc patch cgu <cgu_name> \
  -n <namespace> \
  --type merge \
  -p '{"spec":{"enable":true}}'
```

Monitor the **ClusterGroupUpgrade** status by running the following commands:

```
$ oc get cgu <cgu_name> -n <namespace> -o yaml
$ oc get cgu <cgu_name> -n <namespace> -w
```

Check the **status** section for update progress:

The following example shows the output:

+

```
status:
  conditions:
  - message: Upgrade is progressing
    reason: InProgress
    status: "True"
    type: Progressing
  managedPoliciesForUpgrade:
  - name: core-upgrade-4.20.1-policy
    namespace: openshift-upgrade-policies
  remediationPlan:
  - - spoke1
    - spoke2
  status:
    spoke1: complete
    spoke2: inprogress
```

Verify the **ClusterGroupUpgrade** CR entered the expected state by running the following commands:

```
$ oc get cgu -n <namespace>
$ oc get cgu <cgu_name> -n <namespace> -o jsonpath='{.status.clusters}'
```

The following example shows the output:

+

NAME	AGE	STATE	DETAILS
core-zstream-upgrade-4.20.1	5m	InProgress	Remediating non-compliant policies

When the cluster is fully updated and the state of all **ClusterGroupUpgrade** CRs shows **Completed**, verify that all policies show **Compliant** by running the following command:

```
$ oc get policies -n <cluster_name>
```

The following example shows the output:

+

NAME	REMEDIATION ACTION	COMPLIANCE STATE	AGE
upgrade.upgrade-ocp-21to22	inform	Compliant	1h
upgrade.upgrade-olm-21to22	inform	Compliant	10m

Delete the completed **ClusterGroupUpgrade** CRs from the update namespace by running the following command:

```
$ oc delete cgu -n <namespace> <cgu_name>
```

13.3.2.2.9. Troubleshooting

- If the **ClusterGroupUpgrade** CR is stuck in **Preparing** state, verify that all managed policies exist and are bound to target clusters.
- If the **ClusterGroupUpgrade** CR fails with a timeout error, increase the **timeout** value in the **remediationStrategy** section.

13.3.2.3. Additional resources

- [Overview of cluster updates with RHACM and TALM](#)
- [Installing Topology Aware Lifecycle Manager by using the CLI](#)
- [RHACM](#)
- [ClusterGroupUpgrade samples on GitHub](#)

13.3.3. Prepare worker node pools before a cluster update with TALM

You can configure worker node batching to control how many worker nodes update simultaneously and how workloads tolerate disruption during cluster updates by using **MachineConfigPool** and **PodDisruptionBudget** resources.

13.3.3.1. Configuring worker node batching by using machine config pools

You can configure the **MachineConfigPool** resource to control how many worker nodes update simultaneously during a cluster update. Adjusting the **maxUnavailable** setting balances update speed against workload availability.

Prerequisites

- You have access to the target cluster with cluster-admin privileges.
- The **oc** CLI tool is installed and configured.

Procedure

1. Check the current **maxUnavailable** setting for the worker **MachineConfigPool** resource by running the following command:

```
$ oc get mcp worker -o jsonpath='{.spec.maxUnavailable}'
```

2. Set the required **maxUnavailable** value in the worker **MachineConfigPool** resource by running the following command:

```
$ oc patch mcp worker --type merge -p '{"spec":{"maxUnavailable":"<max_unavailable>"}}'
```

where **<max_unavailable>** is one of the following values:

- An integer, for example **1**, to specify the exact number of nodes that can be unavailable during the update.
- A percentage, for example **"50%"**, to specify the proportion of nodes that can be unavailable.



NOTE

The default **maxUnavailable** value is 1 worker node or 33% of worker nodes, whichever is smaller. Setting a higher value accelerates updates but increases the number of nodes unavailable at any given time. Consult your application teams to find the appropriate value for your workloads.

3. Verify the updated setting by running the following command:

```
$ oc get mcp worker -o yaml
```

Verification

- Verify the **MachineConfigPool** resource reflects the updated **maxUnavailable** value by running the following command:

```
$ oc get mcp worker -o jsonpath='{.spec.maxUnavailable}'
```

13.3.3.2. Configuring worker node batching by using pod disruption budgets

You can configure **PodDisruptionBudget** resources to control how your workloads tolerate node draining during cluster updates. Pod disruption budgets ensure that a minimum number of pod replicas remain available during worker node updates.

Prerequisites

- You have access to the target cluster with cluster-admin privileges.

- The **oc** CLI tool is installed and configured.
- You have identified critical workloads that run with replicas.

Procedure

1. Create a **PodDisruptionBudget** resource for each critical workload:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: <pdb_name>
  namespace: <namespace>
spec:
  maxUnavailable: 1
  selector:
    matchLabels:
      app: <app_label>
```

where:

- **<pdb_name>**: Specifies a name for the **PodDisruptionBudget** resource, for example **cnf-workload-pdb**.
 - **<namespace>**: Specifies the namespace where the workload runs, for example **cnf-workload**.
 - **maxUnavailable**: Specifies the maximum number of pods that can be unavailable during a disruption. Set this value to at least **1** to allow node draining to proceed.
 - **<app_label>**: Specifies the label selector that matches the pods for this workload.
2. Apply the **PodDisruptionBudget** resource by running the following command:

```
$ oc apply -f <pdb_filename>.yaml
```

3. Verify that the pod disruption budget allows at least one disruption by running the following command:

```
$ oc get pdb <pdb_name> -n <namespace>
```

The following example shows the output:

NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS	AGE
cnf-workload-pdb	N/A	1	1	30s



IMPORTANT

Ensure the **ALLOWED DISRUPTIONS** column shows a value greater than 0. If this value is 0, the pod disruption budget blocks node draining and updates stall. Do not set **minAvailable** to 100% of replicas, as this prevents any disruption.

4. Repeat steps 1-3 for all critical workloads in your cluster.

5. Verify all pod disruption budgets across the cluster by running the following command:

```
$ oc get pdb -A
```

Verification

- Verify that no pod disruption budgets are blocking disruptions by running the following command:

```
$ oc get pdb -A -o jsonpath='{range .items[?(@.status.disruptionsAllowed==0)]}{.metadata.namespace}{"\t"}{.metadata.name}{"\n"}{end}'
```

This command must produce no output. If the output lists any pod disruption budgets, adjust their configuration before updating.

13.3.3.3. Additional resources

- [Configuring application pods before updating your OpenShift Container Platform cluster](#)
- [Kubernetes PodDisruptionBudget documentation](#)
- [Pod Topology Spread Constraints](#)

13.3.4. Perform health checks before a cluster update with TALM

Running pre-update health checks reduces the risk of update failures and identifies problems early. If any checks fail, resolve the issues before proceeding with the cluster update.

13.3.4.1. Performing health checks before cluster updates

You can perform comprehensive health checks before updating clusters to ensure cluster components are ready for the update. These checks validate cluster Operators, etcd health, storage systems, network connectivity, and custom resources specific to your deployment.

Complete these health checks before starting any cluster update to minimize the risk of update failures.

Prerequisites

- You have access to target clusters with cluster-admin privileges.
- The **oc** CLI tool is installed and configured for the target cluster.

Procedure

1. Verify control plane node readiness by running the following command:

```
$ oc get nodes -l node-role.kubernetes.io/master=
```

The following example shows the output:

NAME	STATUS	ROLES	AGE	VERSION
master-0.example.com	Ready	control-plane,master	30d	v1.33.0
master-1.example.com	Ready	control-plane,master	30d	v1.33.0
master-2.example.com	Ready	control-plane,master	30d	v1.33.0



All control plane nodes must show **Ready** status with no **SchedulingDisabled** notation. If any nodes show **NotReady** or **SchedulingDisabled**, investigate and resolve before updating.

2. Verify worker node readiness by running the following command:

```
$ oc get nodes -l node-role.kubernetes.io/worker=
```

All worker nodes must show **Ready** status. If any nodes show **NotReady** or **SchedulingDisabled**, investigate and resolve before updating.

3. Check cluster Operator status by running the following command:

```
$ oc get co
```

The following example shows the output:

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
authentication	4.20.0	True	False	False 30d
cloud-controller-manager	4.20.0	True	False	False 30d
cluster-autoscaler	4.20.0	True	False	False 30d

All cluster Operators must show the following:

- **AVAILABLE: True**
- **PROGRESSING: False**
- **DEGRADED: False**

If any Operator is degraded or progressing, resolve the issue before proceeding.

4. Verify etcd cluster health by running the following command:

```
$ oc get etcd -o=jsonpath='{range .items[0].status.conditions[?(@.type=="EtcdMembersAvailable")]}{.message}\n'\n'\n'
```

The following example shows the output:

```
3 members are available
```

All etcd members must be available. If the output indicates unhealthy members, investigate etcd pod logs before updating.

5. Check your backup storage location for an etcd backup created within the past 24 hours. If no recent backup exists, create an etcd backup before proceeding. For more information, see "Backing up etcd" in the "Additional resources" section.
6. Verify machine config pool status by running the following command:

```
$ oc get mcp
```

The following example shows the output:

NAME	CONFIG	UPDATED	UPDATING	DEGRADED		
MACHINECOUNT	READYMACHINECOUNT	UPDATEDMACHINECOUNT				
DEGRADEDMACHINECOUNT	AGE					
master	rendered-master-abc123	True	False	False	3	3
3	0					
	30d					
worker	rendered-worker-def456	True	False	False	3	3
3	0					
	30d					

All machine config pools must show the following:

- **UPDATED: True**
- **UPDATING: False**
- **DEGRADED: False**
- **MACHINECOUNT** must equal **READYMACHINECOUNT** and **UPDATEDMACHINECOUNT**

7. Verify persistent volume health by running the following command:

```
$ oc get pv
```

All persistent volumes must show **Bound** or **Available** status. Persistent volumes must not show **Failed** status.

8. Verify that a default storage class exists by running the following command:

```
$ oc get storageclass
```

At least one storage class must be marked as **(default)**.

9. If your cluster uses SR-IOV, verify network node policies by running the following command:

```
$ oc get sriovnetworknodepolicy -n openshift-sriov-network-operator
```

All SR-IOV network node policies must show **Succeeded** or **InProgress** state.

10. Verify OLM Operator health by running the following command:

```
$ oc get csv -A
```

All **ClusterServiceVersions** must show **Succeeded** phase. If any Operators show **Failed** or **Installing** phase, investigate before updating.

11. Verify that no pods are in an unexpected state by running the following command:

```
$ oc get pod -A | grep -E -i -v 'complete|running'
```

The command must produce no output. If any pods are listed, review their status before updating.

12. Verify that pod disruption budgets (PDBs) are configured for all critical cloud-native network function (CNF) workloads by running the following command:

```
$ oc get pdb -A
```

The following example shows the output:

NAME	MIN AVAILABLE	MAX UNAVAILABLE	ALLOWED DISRUPTIONS
AGE			
cnf-workload-pdb	2	N/A	1
			30d

PDBs must allow at least one disruption. Setting **minAvailable** to 100% of replicas blocks node drains during updates.

- Check **PerformanceProfile** status by running the following command:

```
$ oc get performanceprofile
```

The following example shows the output:

NAME	AGE
core-performance	30d

- Verify the **PerformanceProfile** shows **Applied** status by running the following command:

```
$ oc get performanceprofile core-performance -o jsonpath='{.status.conditions[?(@.type=="Applied")].status}'
```

The output must be **True**.

- Verify API server responsiveness by running the following command:

```
$ oc version
```

The command should complete within a few seconds. If the API server is slow to respond, investigate before updating.

- Verify certificate validity by running the following command:

```
$ oc -n openshift-kube-apiserver-operator get secret kube-apiserver-to-kubelet-signer -o jsonpath='{.metadata.annotations.auth\.openshift\.io/certificate-not-after}'
```

Certificates that expire within 30 days must be renewed before updating. If **cert-manager** is installed in the cluster, you can also run **oc get certificates -A** to check additional managed certificates.

- Check CPU and memory utilization on nodes by running the following command:

```
$ oc adm top nodes
```

The following example shows the output:

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
master-0.example.com	500m	25%	8Gi	50%
worker-0.example.com	1000m	50%	16Gi	60%

Nodes should have sufficient headroom for rolling updates. CPU usage should be below 80%, and memory usage should be below 85%.

Troubleshooting

- If cluster Operators are degraded, check Operator logs by running the following command:

```
$ oc logs -n openshift-<operator_namespace> <operator_pod_name>
```

- If etcd members are unhealthy, investigate etcd pod logs by running the following command:

```
$ oc logs -n openshift-etcd <etcd_pod_name>
```

- If PDBs block updates, review PDB configuration to ensure they allow sufficient disruptions.
- If nodes show **NotReady** status, check the node conditions by running the following command:

```
$ oc describe node <node_name> | grep -A 10 Conditions
```

Check kubelet logs by running the following command:

```
$ oc debug node/<node_name> -- chroot /host journalctl -u kubelet
```

Common causes for **NotReady** nodes include disk pressure, memory pressure, and network connectivity issues.

13.3.4.2. Additional resources

- [Checking the cluster health](#)
- [Backing up etcd](#)

13.3.5. Manage worker nodes during a cluster update with TALM

You can pause and unpause worker nodes during cluster updates to stage control plane and worker node updates separately, minimizing workload disruption.

13.3.5.1. Pausing and unpausing worker nodes by using TALM

You can pause worker nodes before updating the control plane by using Topology Aware Lifecycle Manager (TALM) policies. After the control plane update completes, you can unpause worker nodes to apply the update to compute resources during a separate maintenance window.

Pausing worker nodes prevents the **MachineConfigPool** resource from updating worker nodes while the control plane updates, minimizing workload disruption.

Prerequisites

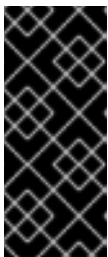
- You have access to clusters with cluster-admin privileges.
- Topology Aware Lifecycle Manager (TALM) is installed on the Red Hat Advanced Cluster Management (RHACM) hub cluster.

Procedure

1. Create a RHACM policy for each **MachineConfigPool** resource that needs to be paused during the update:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: pause-<mcp_name>
  namespace: openshift-upgrade-policies
spec:
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        evaluationInterval:
          compliant: never
        kind: ConfigurationPolicy
        metadata:
          name: pause-<mcp_name>
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: machineconfiguration.openshift.io/v1
                kind: MachineConfigPool
                metadata:
                  name: <mcp_name>
                spec:
                  paused: true
                remediationAction: inform
                severity: low
              remediationAction: inform
```

where **<mcp_name>** is the name of the machine config pool to pause, for example **mcp-1**. This value is used in both the policy name and the **MachineConfigPool** resource name.



IMPORTANT

Create a separate policy for each **MachineConfigPool** resource that you need to pause during the cluster update. If the machine config pool is not paused, any machine configuration change triggers a node reboot. Pausing the machine config pool allows all configuration changes to be applied in a single reboot when the pool is unpaused.

2. Apply the policy to RHACM by running the following command:

```
$ oc apply -f pause-<mcp_name>-policy.yaml
```

3. Include the pause policy in your **ClusterGroupUpgrade** custom resource (CR) before the update policy:

```
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
```

```

metadata:
  name: <cgu_name>
  namespace: <namespace>
spec:
  clusters:
  - <cluster_name>
  managedPolicies:
  - pause-<mcp_name>
  - <upgrade_policy_name>
  enable: false
  remediationStrategy:
    maxConcurrency: 1
    timeout: 120

```

TALM applies the pause policy before applying the update policy, ensuring workers remain paused during the control plane update.

4. After the control plane update completes and you are ready to update worker nodes, create an unpaused policy for each **MachineConfigPool** resource:

```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: unpause-<mcp_name>
  namespace: openshift-upgrade-policies
spec:
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      evaluationInterval:
        compliant: never
      kind: ConfigurationPolicy
      metadata:
        name: unpause-<mcp_name>
      spec:
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: machineconfiguration.openshift.io/v1
            kind: MachineConfigPool
            metadata:
              name: <mcp_name>
            spec:
              paused: false
            remediationAction: inform
            severity: low
          remediationAction: inform

```

where **<mcp_name>** is the name of the machine config pool to unpause, for example **mcp-1**.

The unpause policies for each machine config pool are included as managed policies in the final **ClusterGroupUpgrade** CR. When TALM enforces these policies, the machine config pools unpause and all accumulated configuration changes are applied in a single reboot per node.

5. Apply the unpause policy by using a **ClusterGroupUpgrade** CR:

```

apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: <cgu_name>
  namespace: <namespace>
spec:
  clusters:
  - <cluster_name>
  managedPolicies:
  - unpause-<mcp_name>
  enable: false
  remediationStrategy:
    maxConcurrency: 1

```

6. Apply and enable the **ClusterGroupUpgrade** CR to unpause workers by running the following commands:

```

$ oc apply -f <unpause_cgu_cr_filename>.yaml
$ oc patch cgu <cgu_name> \
  -n <namespace> \
  --type merge \
  -p '{"spec":{"enable":true}}'

```

7. Monitor worker node updates after unpauseing by running the following command:

```
$ oc get mcp <mcp_name> -w
```

The **MachineConfigPool** status shows the rolling update progress. Nodes update in a rolling fashion based on the **maxUnavailable** setting.

Verification

1. Verify that worker nodes are updated and workloads are healthy by running the following commands:

```

$ oc get nodes
$ oc get mcp <mcp_name>
$ oc get pods -A | grep -v Running | grep -v Completed

```

All nodes must show **Ready** status and the expected kubelet version. For the **MachineConfigPool** output, verify that **MACHINECOUNT = READYMACHINECOUNT = UPDATEDMACHINECOUNT**. The **oc get pods** command must return no unexpected pods.

Troubleshooting

- If the pause policy does not take effect, verify the policy is enforced in RHACM by running the following command:

```
$ oc get policy <pause_policy_name> -n <namespace>
```

- If workers update despite being paused, check for manual **MachineConfigPool** edits that override the policy.

- If a worker node update gets stuck, check the node status by running the following command:

```
$ oc describe node <worker_node_name>
```

- Common issues include pod disruption budgets blocking node draining, pods with local storage that cannot be evicted, or node cordoning preventing new pods from scheduling. To identify blocking pod disruption budgets, run the following command:

```
$ oc get pdb -A -o jsonpath='{range .items[?(@.status.disruptionsAllowed==0)]}{.metadata.namespace}{"\t"}{.metadata.name}{"\n"}}{end}'
```

13.3.5.2. Pausing and unpausing worker nodes by using machine config pools

You can pause and unpause worker nodes directly by patching the **MachineConfigPool** resource on the target cluster. Use this method when you are managing a single cluster or when you do not have Topology Aware Lifecycle Manager (TALM) configured.

Prerequisites

- You have access to the target cluster with cluster-admin privileges.
- The **oc** CLI tool is installed and configured.

Procedure

1. Pause the worker **MachineConfigPool** resource by running the following command:

```
$ oc patch mcp worker --type merge -p '{"spec":{"paused":true}}'
```

2. Verify the worker **MachineConfigPool** resource is paused and workloads continue running by running the following commands:

```
$ oc get mcp worker -o jsonpath='{.spec.paused}'
$ oc get mcp worker
$ oc get pods -A -o wide | grep worker
```

3. After the control plane update completes and you are ready to update worker nodes, unpause the worker **MachineConfigPool** resource by running the following command:

```
$ oc patch mcp worker --type merge -p '{"spec":{"paused":false}}'
```

4. Monitor worker node updates after unpausing by running the following commands:

```
$ oc get mcp worker -w
$ oc get nodes -l node-role.kubernetes.io/worker= -w
```

Nodes progress through the following states:

- **Ready,SchedulingDisabled**: Node is being drained
- **NotReady,SchedulingDisabled**: Node is rebooting with new configuration

- **Ready**: Node update complete

Verification

- Verify that worker nodes are updated and workloads are healthy by running the following commands:

```
$ oc get nodes
$ oc get mcp worker
$ oc get pods -A | grep -v Running | grep -v Completed
```

All nodes must show **Ready** status and the expected kubelet version. For the **MachineConfigPool** output, verify that **MACHINECOUNT = READYMACHINECOUNT = UPDATEDMACHINECOUNT**. The **oc get pods** command must return no unexpected pods.

13.3.5.3. Additional resources

- [Complete an EUS-to-EUS cluster update with TALM](#)
- [Prepare worker node pools before a cluster update with TALM](#)
- [Using the Topology Aware Lifecycle Manager for cluster updates](#)

13.3.6. Coordinate OLM-managed Operator updates with TALM

OLM-managed Operators must be compatible with the target OpenShift Container Platform version before you begin a cluster update. Planning Operator updates alongside cluster updates prevents compatibility issues that can block or degrade the update process.

13.3.6.1. Coordinating OLM Operator updates with cluster updates

You can coordinate Operator Lifecycle Manager (OLM) Operator updates with OpenShift Container Platform cluster updates to ensure Operator compatibility. Incompatible Operators can block cluster updates or cause degraded cluster Operators after updates complete.

Prerequisites

- You have an understanding of OLM concepts including subscriptions, channels, and **ClusterServiceVersions** (CSVs).
- You have access to clusters with cluster-admin privileges.
- Topology Aware Lifecycle Manager (TALM) is installed on the Red Hat Advanced Cluster Management (RHACM) hub cluster.
- A cluster update is planned or in progress.

Procedure

1. Check installed Operator versions and subscription channels by running the following commands:

```
$ oc get csv -A
$ oc get subscription -A
```

The following example shows the output:

NAMESPACE	NAME	VERSION	PHASE
openshift-sriov-network-operator	sriov-network-operator.v4.20.0	4.20.0	Succeeded
openshift-ptp	ptp-operator.v4.20.0	4.20.0	Succeeded
openshift-cluster-node-tuning-operator	node-tuning-operator.v4.20.0	4.20.0	Succeeded

The following example shows the output:

NAMESPACE	NAME	PACKAGE	SOURCE
CHANNEL			
openshift-sriov-network-operator	sriov-network-operator-subscription	sriov-network-operator	redhat-operators stable-4.20
openshift-ptp	ptp-operator-subscription	ptp-operator	redhat-operators stable-4.20

- Identify Operators that need channel updates before the cluster update:

Review Operator compatibility in the OpenShift Container Platform release notes and identify which Operator channels support your target OpenShift Container Platform version. For a cluster updating from 4.20 to 4.21, identify Operators still on 4.20 channels. Refer to the [Red Hat Catalog](#) for compatibility details for each Operator.

- Create an OLM Operator update policy.

In environments where Operator updates are set to manual approval, use a TALM-managed policy to coordinate Operator updates. When TALM enforces the policy, it automatically approves the Operator **InstallPlan**:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: upgrade-olm-21
  namespace: openshift-upgrade-policies
  annotations:
    ran.openshift.io/soak-seconds: "60"
spec:
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: upgrade-olm-21
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: operators.coreos.com/v1alpha1
                kind: Subscription
                metadata:
                  name: cluster-logging
                  namespace: openshift-logging
                spec:
                  name: cluster-logging
```

```

channel: stable-6.1
status:
  state: AtLatestKnown

```

where:

- **ran.openshift.io/soak-seconds**: Specifies the soak-seconds annotation that gives the catalog pod time to restart and OLM time to read version data from the new pod before evaluating the **Subscription** status. This avoids a race condition where the policy evaluates too quickly after a catalog image change.
- **channel**: Sets the channel only if it needs to change for the new version. Channels are determined by the Operator development team. Refer to the [Red Hat Catalog](#) for compatibility.
- **status.state: AtLatestKnown**: Ensures the Operator update completes before TALM proceeds to the next step.

You can include multiple Operator subscriptions in the same policy. The following example shows multiple Operators in a single policy:

```

spec:
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: upgrade-olm-20to21
        spec:
          object-templates:
            - complianceType: musthave
              objectDefinition:
                apiVersion: operators.coreos.com/v1alpha1
                kind: Subscription
                metadata:
                  name: sriov-network-operator-subscription
                  namespace: openshift-sriov-network-operator
                spec:
                  name: sriov-network-operator
                  channel: stable
                  installPlanApproval: Manual
                  source: redhat-operators
                  sourceNamespace: openshift-marketplace
                status:
                  state: AtLatestKnown
            - complianceType: musthave
              objectDefinition:
                apiVersion: operators.coreos.com/v1alpha1
                kind: Subscription
                metadata:
                  name: kubernetes-nmstate-operator
                  namespace: openshift-nmstate
                spec:
                  name: kubernetes-nmstate-operator
                  channel: stable

```



```
installPlanApproval: Manual
source: redhat-operators
sourceNamespace: openshift-marketplace
status:
state: AtLatestKnown
```

For an example of creating this policy with zero touch provisioning (ZTP), see [Example OLM update policy in the OpenShift KNI reference repository](#).

4. Monitor Operator updates and verify the new versions by running the following commands:

```
$ oc get csv -n openshift-sriov-network-operator -w
$ oc get csv -n openshift-sriov-network-operator -o jsonpath='{.items[?
(@.spec.version=="4.21.0")].status.phase}'
```

The Operator creates a new **ClusterServiceVersion** and transitions to the new version.

The following example shows the output:

```
Succeeded
```

5. After the control plane update completes, check for pending Operator updates by running the following command:

```
$ oc get csv -A
```

Some Operators might not update until after the cluster update completes.

6. If Operators are configured with manual approval, list and approve pending install plans by running the following commands:

```
$ oc get installplan -A
$ oc patch installplan <install_plan_name> \
  -n <namespace> \
  --type merge \
  -p '{"spec":{"approved":true}}'
```

7. Verify Operator health after cluster update by running the following command:

```
$ oc get csv -A
```

All Operators must show **Succeeded** phase.

8. Verify CNF-specific Operator pods and custom resources by running the following commands:

```
$ oc get pods -n openshift-sriov-network-operator
$ oc get pods -n openshift-ntp
$ oc get performanceprofile -o jsonpath='{range .items[*]}.{metadata.name}{"\t"}
{.status.conditions[?(@.type=="Applied")].status}{"\n"}{end}'
$ oc get ptpconfig -n openshift-ntp
$ oc get sriovnetworknodepolicy -n openshift-sriov-network-operator
$ oc get sriovnetworknodepolicy -n openshift-sriov-network-operator -o jsonpath='{range
.items[*]}.{metadata.name}{"\t"}{.status.syncStatus}{"\n"}{end}'
$ oc get pods -n openshift-ntp -l app=linuxptp-daemon -o wide
```

All custom resources must show expected status.

Review Operator considerations specific to your deployment:

- **Node Tuning Operator:** The Node Tuning Operator provides performance tuning capabilities through **PerformanceProfile** custom resources. Verify that performance profiles are reconciled after the update.
- **SR-IOV Network Operator:** The SR-IOV Network Operator might require node reboots when updating to new versions. Schedule Operator updates during maintenance windows to avoid unexpected node reboots.
- **PTP Operator:** PTP configurations are sensitive to version changes. Validate PTP synchronization after Operator updates.

Verification

1. Verify that all Operators and custom resources are healthy by running the following commands:

```
$ oc get csv -A
$ oc get pods -A | grep -E "openshift-sriov|openshift-ptp|openshift-performance"
$ oc get performanceprofile,ptpconfig,sriovnetworknodepolicy -A
```

All **ClusterServiceVersion** resources must show **Succeeded** phase. All Operator pods must be **Running** and ready. Custom resources must exist and show the expected configuration.

Troubleshooting

- If an Operator blocks the cluster update, update the Operator subscription channel before updating the cluster.
- If an Operator is stuck in **Installing** phase, check catalog source availability by running the following command:

```
$ oc get catalogsource -n openshift-marketplace
```

- If a custom resource is not reconciling, check Operator logs by running the following command:

```
$ oc logs -n <operator_namespace> deployment/<operator_deployment>
```

- If Operator pods are in **CrashLoopBackOff**, check for compatibility issues with the new cluster version.

13.3.6.2. Additional resources

- [Prepare RHACM policies and TALM for cluster updates](#)
- [OpenShift Container Platform update documentation](#)

13.3.7. Complete a z-stream cluster update with TALM

You can update clusters to z-stream patch releases by using RHACM policies and Topology Aware Lifecycle Manager (TALM).

13.3.7.1. Updating clusters with z-stream releases

You can update clusters to z-stream patch releases by using RHACM policies and Topology Aware Lifecycle Manager (TALM). Z-stream updates apply security fixes and bug fixes with minimal risk and should be performed regularly to maintain cluster security.

Apply z-stream updates as soon as they become available to address critical vulnerabilities and maintain security compliance.



NOTE

The following procedure uses **spoke1** and **spoke2** as example cluster names. Replace these with your actual cluster names throughout.

Prerequisites

- You have completed the pre-update health check successfully.
- TALM is installed and configured on the Red Hat Advanced Cluster Management (RHACM) hub cluster.
- A z-stream patch release is available on the Red Hat Customer Portal.
- A recent etcd backup exists that was created within the past 24 hours.

Procedure

1. Verify the current cluster version by running the following command:

```
$ oc get clusterversion
```

The following example shows the output:

```
NAME      VERSION  AVAILABLE  PROGRESSING  SINCE  STATUS
version  4.20.0   True       False        30d    Cluster version is 4.20.0
```

2. Check for available z-stream updates by running the following command:

```
$ oc adm upgrade
```

The following example shows the output:

```
Cluster version is 4.20.0

Upstream is unset, so the cluster will use an appropriate default.
Channel: stable-4.20 (available channels: candidate-4.20, eus-4.20, fast-4.20, stable-4.20)

Recommended updates:

VERSION  IMAGE
4.20.1   quay.io/openshift-release-dev/ocp-release@sha256:abc123...
4.20.2   quay.io/openshift-release-dev/ocp-release@sha256:def456...
```

Review the OpenShift Container Platform release notes for the target version to identify security fixes and bug fixes included in the z-stream release.

3. Create a RHACM policy for the z-stream update:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-upgrade-4.20.1-policy
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: upgrade-cluster-version
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: config.openshift.io/v1
            kind: ClusterVersion
            metadata:
              name: version
            spec:
              channel: stable-4.20
              desiredUpdate:
                version: 4.20.1
```

Replace **4.20.1** with your target z-stream version. The policy is set to **inform** so it does not immediately push the update. The **ClusterGroupUpgrade** custom resource (CR) enforces the policy at the scheduled update time.

4. Apply the policy to the RHACM hub cluster by running the following command:

```
$ oc apply -f core-upgrade-4.20.1-policy.yaml
```

5. Verify the policy and placement resources were created by running the following commands:

```
$ oc get policy core-upgrade-4.20.1-policy -n openshift-upgrade-policies
$ oc get placementbinding -n openshift-upgrade-policies
$ oc get placement -n openshift-upgrade-policies
```

The following example shows the output:

NAME	REMEDIATION ACTION	COMPLIANCE STATE	AGE
core-upgrade-4.20.1-policy	inform	NonCompliant	30s

If no **PlacementBinding** or **Placement** exists, the policy will not apply to any clusters. Create appropriate placement resources for your target clusters.

6. Create a **ClusterGroupUpgrade** CR for the z-stream update:

```
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-zstream-upgrade-4.20.1
  namespace: openshift-upgrade-policies
spec:
  clusters:
    - spoke1
    - spoke2
  managedPolicies:
    - core-upgrade-4.20.1-policy
  enable: false
  remediationStrategy:
    maxConcurrency: 2
    timeout: 60
```

where:

- **clusters:** Specifies your target cluster names.
- **maxConcurrency:** Specifies the number of clusters to update simultaneously.
- **timeout:** Specifies the maximum time in minutes to wait for each cluster update. For example, 60 equals 1 hour per cluster.

7. Apply and enable the **ClusterGroupUpgrade** CR to start the update by running the following commands:

```
$ oc apply -f core-zstream-cgu.yaml
$ oc patch cgu core-zstream-upgrade-4.20.1 \
  -n openshift-upgrade-policies \
  --type merge \
  -p '{"spec":{"enable":true}}'
```

8. Monitor the update progress by running the following commands:

```
$ oc get cgu core-zstream-upgrade-4.20.1 -n openshift-upgrade-policies -w
$ oc --context=spoke1 get clusterversion -w
$ oc --context=spoke1 get co
```

The **ClusterGroupUpgrade** CR progresses through these states:

- **Preparing:** TALM validates cluster readiness
- **InProgress:** Clusters are updating
- **Complete:** All clusters successfully updated

Operators update sequentially during the z-stream update.

Verification

1. Verify the cluster version and Operator health by running the following commands:

```
$ oc --context=spoke1 get clusterversion -o jsonpath='{.items[0].status.desired.version}'  
$ oc --context=spoke1 get co
```

The cluster version must show the target z-stream version. All Operators must show **AVAILABLE=True, PROGRESSING=False, DEGRADED=False**.

2. Check workload health by running the following command:

```
$ oc --context=spoke1 get pods -A | grep -v Running | grep -v Completed
```

Pods must not show unexpected failures or crash loops.

3. Verify CNF-specific resources by running the following commands:

```
$ oc --context=spoke1 get performanceprofile  
$ oc --context=spoke1 get ptpconfig -n openshift-ptp  
$ oc --context=spoke1 get sriovnetworknodepolicy -n openshift-sriov-network-operator
```

All custom resources must show expected status.

4. Run smoke tests for critical workloads to ensure functionality after the update.

Troubleshooting

- If the **ClusterGroupUpgrade** CR is stuck in **Preparing** state, check policy compliance in RHACM by running the following command:

```
$ oc get policy core-upgrade-4.20.1-policy -n openshift-upgrade-policies
```

- If a cluster Operator becomes degraded during the update, check Operator logs by running the following command:

```
$ oc --context=spoke1 logs -n openshift-<operator_namespace>  
deployment/<operator_deployment>
```

- If the update exceeds the timeout, investigate slow nodes or image pull issues by running the following command:

```
$ oc --context=spoke1 get events -A --sort-by='.lastTimestamp' | tail -20
```

13.3.7.2. Additional resources

- [Prepare RHACM policies and TALM for cluster updates](#)
- [Perform health checks before a cluster update with TALM](#)
- [Troubleshoot cluster updates with TALM](#)
- [OpenShift Container Platform update documentation](#)
- [OpenShift Container Platform update lifecycle and support policy](#)

13.3.8. Complete a y-stream cluster update with TALM

Y-stream updates move clusters between minor versions. You can update through a single y-stream release or chain multiple sequential updates to reach a target version that is more than one minor release away.

13.3.8.1. Updating clusters through y-stream releases

You can update clusters through minor version, or y-stream, releases by using RHACM policies and Topology Aware Lifecycle Manager (TALM). Y-stream updates introduce new features and might include API deprecations, requiring careful planning and validation.

You must perform y-stream updates through consecutive versions. You cannot skip versions during y-stream updates. For example, to update from 4.20 to 4.22, you must update from 4.20 to 4.21, and then from 4.21 to 4.22. If you need to move 2 y-stream versions, consider EUS-to-EUS updates instead.



NOTE

The following procedure uses **spoke1** as an example cluster name. Replace it with your actual cluster name throughout.

Prerequisites

- You have completed the pre-update health check successfully.
- TALM is installed and configured on the Red Hat Advanced Cluster Management (RHACM) hub cluster.
- A y-stream release is available, such as 4.21 when updating from 4.20.
- You have reviewed the y-stream release notes for API deprecations and breaking changes.
- You have scheduled a maintenance window. Y-stream updates take 1 to 2 hours.

Procedure

1. Identify deprecated APIs in use on your cluster by running the following command:

```
$ oc get apirequestcounts -o jsonpath='{range .items[?(@.status.removedInRelease!="")]}{.status.removedInRelease}{", "}{.metadata.name}{"\n"}{end}' | sort -k1
```

This command shows APIs that will be removed in future releases. Review the OpenShift Container Platform release notes for your target version and update any resources that use deprecated or removed APIs before updating.

2. Validate OLM Operator compatibility with the target y-stream version by running the following command:

```
$ oc get csv -A
```

All Operators must show **Succeeded** status. Investigate any Operators in a failed state before proceeding.

3. Review Operator subscription channels to ensure they support the target OpenShift Container Platform version by running the following command:

```
$ oc get subscription -A -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.channel}{"\n"}{end}'
```

Update Operator subscriptions if needed before the cluster update.

4. Create a RHACM policy to pause worker nodes:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-pause-worker-nodes
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: pause-worker-mcp
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: machineconfiguration.openshift.io/v1
            kind: MachineConfigPool
            metadata:
              name: worker
            spec:
              paused: true
```

5. Create a RHACM policy for the y-stream update:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-upgrade-4.21-policy
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: upgrade-cluster-version
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
```



```

objectDefinition:
  apiVersion: config.openshift.io/v1
  kind: ClusterVersion
  metadata:
    name: version
  spec:
    channel: stable-4.21
    desiredUpdate:
      version: 4.21.0

```

Replace **4.21.0** with your target y-stream version. Both policies are set to **inform** so they do not immediately push changes. The **ClusterGroupUpgrade** custom resource (CR) enforces these policies at the scheduled update time.

6. Apply the policies to RHACM by running the following commands:

```

$ oc apply -f core-pause-worker-policy.yaml
$ oc apply -f core-upgrade-4.21-policy.yaml

```

7. Verify the policies and placement resources were created by running the following commands:

```

$ oc get policy -n openshift-upgrade-policies
$ oc get placementbinding -n openshift-upgrade-policies
$ oc get placement -n openshift-upgrade-policies

```

If no **PlacementBinding** or **Placement** exists, the policies will not apply to any clusters. Create appropriate placement resources for your target clusters.

8. Create a **ClusterGroupUpgrade** CR for the y-stream update:

```

apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-ystream-upgrade-4.21
  namespace: openshift-upgrade-policies
spec:
  clusters:
    - spoke1
  managedPolicies:
    - core-pause-worker-nodes
    - core-upgrade-4.21-policy
  enable: false
  remediationStrategy:
    maxConcurrency: 1
    timeout: 120
  actions:
    beforeEnable:
      addClusterLabels:
        upgrade-in-progress: "true"
    afterCompletion:
      addClusterLabels:
        upgraded-to: "4.21"
        control-plane-upgraded: "true"

```

For canary deployments, use a single cluster first before updating additional clusters.

9. Apply and enable the **ClusterGroupUpgrade** CR to start the control plane update by running the following commands:

```
$ oc apply -f core-ystream-cgu.yaml
$ oc patch cgu core-ystream-upgrade-4.21 \
  -n openshift-upgrade-policies \
  --type merge \
  -p '{"spec":{"enable":true}}'
```

10. Monitor the control plane update progress by running the following commands:

```
$ oc get cgu core-ystream-upgrade-4.21 -n openshift-upgrade-policies -w
$ oc --context=spoke1 get clusterversion -w
$ oc --context=spoke1 get co
```

Cluster Operators update sequentially during the y-stream update. The control plane update typically takes 60 to 120 minutes.

11. After the control plane update completes, verify control plane health by running the following commands:

```
$ oc --context=spoke1 get nodes -l node-role.kubernetes.io/master=
$ oc --context=spoke1 get co | grep -v "True.*False.*False"
```

All control plane nodes and Operators must be healthy before proceeding.

12. Create an OLM Operator update policy that updates Operator subscription channels to versions compatible with the new y-stream:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-upgrade-olm-4.21
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: upgrade-olm-operators
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: operators.coreos.com/v1alpha1
            kind: Subscription
            metadata:
              name: sriov-network-operator-subscription
              namespace: openshift-sriov-network-operator
            spec:
```

```

channel: stable-4.21
status:
state: AtLatestKnown

```

You can include multiple Operator subscriptions in the same policy. For more details about creating OLM update policies, see "Coordinating OLM Operator updates with cluster updates".

13. Create a **ClusterGroupUpgrade** CR to apply the OLM Operator update policy:

```

apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-olm-upgrade-4.21
  namespace: openshift-upgrade-policies
spec:
  clusters:
  - spoke1
  managedPolicies:
  - core-upgrade-olm-4.21
  enable: true
  remediationStrategy:
    maxConcurrency: 1
    timeout: 60

```

14. Apply the OLM Operator update policy and **ClusterGroupUpgrade** CR by running the following commands:

```

$ oc apply -f core-olm-upgrade-policy.yaml
$ oc apply -f core-olm-cgu.yaml

```

15. Monitor the OLM Operator update and verify that worker nodes remain at the previous version by running the following commands:

```

$ oc get cgu core-olm-upgrade-4.21 -n openshift-upgrade-policies -w
$ oc --context=spoke1 get nodes

```

Worker nodes must still show the previous kubelet version.

16. Create a policy to unpause worker nodes:

```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-unpause-worker-nodes
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: unpause-worker-mcp
      spec:

```

```

remediationAction: inform
severity: high
object-templates:
- complianceType: musthave
  objectDefinition:
    apiVersion: machineconfiguration.openshift.io/v1
    kind: MachineConfigPool
    metadata:
      name: worker
    spec:
      paused: false

```

17. Create a **ClusterGroupUpgrade** CR to unpause workers:

```

apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-unpause-workers-4.21
  namespace: openshift-upgrade-policies
spec:
  clusters:
  - spoke1
  managedPolicies:
  - core-unpause-worker-nodes
  enable: false
  remediationStrategy:
    maxConcurrency: 1

```

18. Apply and enable the unpause **ClusterGroupUpgrade** CR by running the following commands:

```

$ oc apply -f core-unpause-workers-cgu.yaml
$ oc patch cgu core-unpause-workers-4.21 \
  -n openshift-upgrade-policies \
  --type merge \
  -p '{"spec":{"enable":true}}'

```

19. Monitor worker node updates by running the following command:

```
$ oc --context=spoke1 get mcp worker -w
```

Worker nodes update in a rolling fashion. The process typically takes 30 to 60 minutes depending on the number of worker nodes.

Verification

1. Verify all nodes are at the target y-stream version by running the following command:

```
$ oc --context=spoke1 get nodes
```

2. Verify all cluster Operators are healthy by running the following command:

```
$ oc --context=spoke1 get co
```

3. Verify workload health and CNF-specific resources by running the following commands:

```
$ oc --context=spoke1 get pods -A | grep -v Running | grep -v Completed
$ oc --context=spoke1 get performanceprofile
$ oc --context=spoke1 get ptpconfig -n openshift-ptp
$ oc --context=spoke1 get sriovnetworknodepolicy -n openshift-sriov-network-operator
```

4. Run smoke tests for critical workloads.

Troubleshooting

- If API deprecation warnings appear, update manifests to use new API versions before updating.
- If Operator compatibility issues occur, update Operator subscriptions to channels that support the target y-stream.
- If worker node updates get stuck, check **MachineConfigPool** resource status and node logs.

13.3.8.2. Updating through sequential y-stream releases

You can update clusters through multiple sequential y-stream releases when you need to move from an older version to a newer version that is more than one minor release away. You must update through each intermediate y-stream version because skipping versions is not supported.

For example, to update from 4.20 to 4.22, you must first update from 4.20 to 4.21, and then update from 4.21 to 4.22.



NOTE

The following procedure uses **spoke1** as an example cluster name. Replace it with your actual cluster name throughout.



IMPORTANT

When planning to update clusters through multiple sequential y-stream releases, allow at least 24 hours, and up to 48 hours, between y-stream updates to monitor cluster stability.

Prerequisites

- You have completed the pre-update health check successfully.
- TALM is installed and configured on the Red Hat Advanced Cluster Management (RHACM) hub cluster.
- You have access to the target clusters with cluster-admin privileges.
- The **oc** CLI tool is installed and configured for the target clusters.
- All intermediate y-stream releases are available, such as both 4.21 and 4.22 when updating from 4.20 to 4.22.
- You have reviewed the release notes for each intermediate y-stream version for API deprecations and breaking changes.
- You have planned maintenance windows for each y-stream update.

Procedure

1. Determine the complete update path:
Identify your current version and target version, then map out the required intermediate versions.

Table 13.3. Example update paths

Current version	Update path to 4.22
4.20.20	4.20.20 → 4.21.latest → 4.22.0
4.19.10	4.19.10 → 4.20.latest → 4.21.latest → 4.22.0

2. Update to the first intermediate y-stream version:
Follow the procedure in "Updating clusters through y-stream releases" to update to the first intermediate version.

For example, update from 4.20.20 to 4.21.latest.

3. Verify the first y-stream update completed successfully by running the following command:

```
$ oc --context=spoke1 get clusterversion
```

Verify all cluster Operators are healthy by running the following command:

```
$ oc --context=spoke1 get co | grep -v "True.*False.*False"
```

All cluster Operators must show healthy status before proceeding.

4. Wait at least 24 hours, and up to 48 hours, between y-stream updates:
Allow time to monitor cluster stability after the first update. Extend this period for large production deployments or if any anomalies are observed.

During this wait period:

- Monitor workload health and performance.
- Check for unexpected behavior or errors.
- Run smoke tests for critical cloud-native network function (CNF) workloads.
- Verify all custom resources specific to your deployment are functioning correctly.
- Review release notes for the next y-stream version for additional API deprecations or configuration changes.

5. Update OLM Operator subscriptions if needed:
Some Operators might require intermediate channel updates when updating through multiple y-streams. Verify the Operator status by running the following command:

```
$ oc --context=spoke1 get csv -A
```

Verify all Operators are at versions compatible with your current cluster version before proceeding.

6. Create RHACM policies for the second y-stream update:
Update the cluster version policy to target the next y-stream:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-upgrade-4.22-policy
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: upgrade-cluster-version
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: config.openshift.io/v1
            kind: ClusterVersion
            metadata:
              name: version
            spec:
              channel: stable-4.22
              desiredUpdate:
                version: 4.22.0
```

7. Apply the updated policy by running the following command:

```
$ oc apply -f core-upgrade-4.22-policy.yaml
```

8. Create a new **ClusterGroupUpgrade** custom resource (CR) for the second y-stream:

```
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-ystream-upgrade-4.22
  namespace: openshift-upgrade-policies
spec:
  clusters:
  - spoke1
  managedPolicies:
  - core-pause-worker-nodes
  - core-upgrade-4.22-policy
  enable: false
```

```
remediationStrategy:
  maxConcurrency: 1
  timeout: 120
```

9. Apply and enable the **ClusterGroupUpgrade** CR by running the following commands:

```
$ oc apply -f core-ystream-upgrade-4.22-cgu.yaml
$ oc patch cgu core-ystream-upgrade-4.22 \
  -n openshift-upgrade-policies \
  --type merge \
  -p '{"spec":{"enable":true}}'
```

10. Monitor the second y-stream update and verify completion by running the following commands:

```
$ oc get cgu core-ystream-upgrade-4.22 -n openshift-upgrade-policies -w
$ oc --context=spoke1 get clusterversion
$ oc --context=spoke1 get co
```

11. Update Operators for the second y-stream by running the following command:

```
$ oc --context=spoke1 get csv -A | grep -v Succeeded
```

Update Operator channels as needed for the new cluster version.

12. Unpause worker nodes and complete the second y-stream update:
Follow the worker node unpause procedure from "Pausing and unpausing worker nodes by using TALM".
13. Repeat steps 2-12 for any additional intermediate y-stream versions:
If updating through more than two y-streams, repeat the process for each intermediate version.

Verification

1. Verify the final cluster version by running the following command:

```
$ oc --context=spoke1 get clusterversion -o jsonpath='{.items[0].status.desired.version}'
```

2. Run comprehensive health checks:
Follow the procedure in "Performing health checks before cluster updates".
3. Verify workload functionality after all updates:
Run smoke tests and validate that all CNF workloads operate correctly at the final version.

Troubleshooting

- If the cluster becomes degraded after the first y-stream update, resolve all issues before proceeding to the second y-stream.
- If Operator versions are incompatible with an intermediate y-stream, update the Operator subscription before updating the cluster.
- If configuration changes are needed between y-streams, apply those changes during the wait period before proceeding.

13.3.8.3. Additional resources

- [Complete an EUS-to-EUS cluster update with TALM](#)
- [Prepare RHACM policies and TALM for cluster updates](#)
- [Perform health checks before a cluster update with TALM](#)
- [OpenShift Container Platform update documentation](#)
- [OpenShift Container Platform update lifecycle and support policy](#)

13.3.9. Complete an EUS-to-EUS cluster update with TALM

You can partially skip intermediate odd-numbered releases by using control-plane-only EUS-to-EUS updates with RHACM policies and Topology Aware Lifecycle Manager (TALM). With this approach, you can manage control plane and worker version skew during staged rollouts.

13.3.9.1. Updating with EUS-to-EUS control-plane-only updates

You can update clusters from one Extended Update Support (EUS) release to another using control-plane-only updates. This approach updates the control plane while leaving worker nodes at the previous EUS version, enabling staged rollouts that minimize workload disruption.

EUS releases provide 18 months of support and include even-numbered minor versions such as 4.18, 4.20, and 4.22. Control plane and worker nodes can be up to two minor versions apart during EUS-to-EUS updates.



WARNING

Control plane and worker version skew is limited to N-2, or two minor versions. For example, if the control plane is at 4.20, workers must be updated before the control plane reaches 4.22, or they will be in an unsupported configuration. Plan worker updates accordingly to avoid exceeding the version skew policy. For more information about version skew policies, see the Kubernetes version and version skew support policy.

The following procedure demonstrates an EUS-to-EUS update using an example cluster.



NOTE

The following procedure uses **spoke1** as an example cluster name. Replace it with your actual cluster name throughout.

Prerequisites

- You have completed the pre-update health check successfully.
- TALM is installed and configured on the Red Hat Advanced Cluster Management (RHACM) hub cluster.

- The current cluster is at the latest z-stream of a source EUS version, such as 4.20.latest.
- The target EUS version is available, such as 4.22.
- You have verified that the EUS-to-EUS update path is supported in the release notes.
- You have reviewed the release notes for API deprecations and breaking changes between the source and target EUS versions.
- You have scheduled a maintenance window. EUS-to-EUS updates take 2 to 3 hours.

Procedure

1. Verify the current cluster version is at the latest z-stream of the source EUS by running the following command:

```
$ oc --context=spoke1 get clusterversion
```

The following example shows the output:

```
NAME      VERSION AVAILABLE PROGRESSING SINCE STATUS
version  4.20.35  True      False      30d      Cluster version is 4.20.35
```

If not at the latest z-stream, update to the latest patch release before proceeding.

2. Validate that your cloud-native network function (CNF) workloads tolerate control plane and worker version skew:
Verify that your CNF workloads can operate with the control plane at 4.22 while workers remain at 4.20. Consult with application teams if needed.
3. Create a RHACM policy to pause worker nodes:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: core-pause-worker-nodes
  namespace: openshift-upgrade-policies
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: pause-worker-mcp
      spec:
        remediationAction: inform
        severity: high
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: machineconfiguration.openshift.io/v1
            kind: MachineConfigPool
            metadata:
              name: worker
```

```
spec:
  paused: true
```

4. Verify the current cluster channel before creating the update policy by running the following command:

```
$ oc --context=spoke1 get clusterversion -o jsonpath='{.items[0].spec.channel}'
```

The following example shows the output:

```
eus-4.20
```

If the cluster is on a non-EUS channel, such as **stable-4.20**, changing to an EUS channel and setting the required version simultaneously can cause unexpected update behavior.

5. Get the release image reference for the target version.
The image reference is required for the update policy. Retrieve it by running the following command:

```
$ oc adm release info 4.21.5 | head -9
```

The following example shows the output:

```
Name:      4.21.5
Digest:
sha256:93879f84b3165c5b5bd1fdf4563a11155dc61ea35cd93e67dc61c2b66e11c8bb
Created:    2025-03-13T09:17:26Z
OS/Arch:    linux/amd64
Manifests:  758
Metadata files: 2

Pull From: quay.io/openshift-release-dev/ocp-
release@sha256:93879f84b3165c5b5bd1fdf4563a11155dc61ea35cd93e67dc61c2b66e11c8bb
```

Use the **Pull From** value as the **image** field in the update policy.

6. Create a policy for the EUS update.
For an EUS-to-EUS update, you need a separate policy for each y-stream step. The following example shows a policy for the first y-stream update:

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: upgrade-ocp-20to21
  namespace: openshift-upgrade-policies
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/standards: NIST SP 800-53
  labels:
    app.kubernetes.io/instance: policies
spec:
  disabled: false
  policy-templates:
```

```

- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: upgrade-ocp-20to21
  spec:
    object-templates:
      - complianceType: musthave
        objectDefinition:
          apiVersion: config.openshift.io/v1
          kind: ClusterVersion
          metadata:
            name: version
          spec:
            channel: eus-4.22
            desiredUpdate:
              force: true
              image: quay.io/openshift-release-dev/ocp-
release@sha256:df00d64554b9b5d860273f938e0ff3b0c4a7af8356fbd103658ebe6fa2a830c8

              version: 4.21.29
          status:
            history:
              - state: Completed
                version: 4.21.29
            remediationAction: inform
            severity: low
            remediationAction: inform

```

where:

- **channel:** Sets the cluster channel to the target EUS release. Refer to the [update selection tool](#) when setting the channel.
- **image:** Specifies the release image reference obtained from the **oc adm release info** command.
- **version:** Sets the specific y-stream and z-stream release version for the cluster update. The **status.history** section ensures the policy remains noncompliant until the update completes.
- **force:** The **force: true** field bypasses the Cincinnati update graph safety checks. Use this field only when the update path has been verified through the update selection tool or release notes. Removing this field or setting it to **false** causes the update to follow the standard update graph, which might not include a direct path for EUS-to-EUS intermediate hops.



IMPORTANT

For an EUS-to-EUS update, create a policy for each y-stream step. For example, updating from 4.20 to 4.22 requires one policy for 4.20-to-4.21 and another for 4.21-to-4.22. For a complete example of chaining multiple policies across y-stream steps, see "Configuring ClusterGroupUpgrade custom resources for cluster updates".

7. Apply the policies to RHACM by running the following commands:

```
$ oc apply -f upgrade-ocp-20to21-policy.yaml
$ oc apply -f core-pause-worker-nodes-policy.yaml
```

8. Create a **ClusterGroupUpgrade** custom resource (CR) for the EUS update:

```
apiVersion: ran.openshift.io/v1alpha1
kind: ClusterGroupUpgrade
metadata:
  name: core-eus-upgrade-4.20-to-4.22
  namespace: openshift-upgrade-policies
spec:
  clusters:
  - spoke1
  managedPolicies:
  - core-pause-worker-nodes
  - upgrade-ocp-20to21
  enable: false
  remediationStrategy:
    maxConcurrency: 1
    timeout: 180
```

EUS updates might take longer than single y-stream updates, so the timeout is set to 180 minutes, or 3 hours.

9. Apply and enable the **ClusterGroupUpgrade** CR to start the EUS update by running the following commands:

```
$ oc apply -f core-eus-upgrade-cgu.yaml
$ oc patch cgu core-eus-upgrade-4.20-to-4.22 \
  -n openshift-upgrade-policies \
  --type merge \
  -p '{"spec":{"enable":true}}'
```

10. Monitor the EUS control plane update progress by running the following commands:

```
$ oc get cgu core-eus-upgrade-4.20-to-4.22 -n openshift-upgrade-policies -w
$ oc --context=spoke1 get clusterversion -w
$ oc --context=spoke1 get co
```

The cluster version progresses through intermediate versions during the EUS update. For example, updating from 4.20 to 4.22 goes through 4.21 internally. Cluster Operators might take longer to stabilize during EUS updates because of the larger version jump.

11. After the control plane update completes, verify control plane and worker node status by running the following commands:

```
$ oc --context=spoke1 get nodes -l node-role.kubernetes.io/master=
$ oc --context=spoke1 get co | grep -v "True.*False.*False"
$ oc --context=spoke1 get nodes -l node-role.kubernetes.io/worker=
```

The following example shows the output for control plane nodes:

NAME	STATUS	ROLES	AGE	VERSION
master-0.example.com	Ready	control-plane,master	60d	v1.35.0
master-1.example.com	Ready	control-plane,master	60d	v1.35.0
master-2.example.com	Ready	control-plane,master	60d	v1.35.0

Control plane nodes must show the new kubelet version, 4.22. All Operators must be healthy before updating worker nodes.

The following example shows the output for worker nodes:

NAME	STATUS	ROLES	AGE	VERSION
worker-0.example.com	Ready	worker	60d	v1.33.0
worker-1.example.com	Ready	worker	60d	v1.33.0
worker-2.example.com	Ready	worker	60d	v1.33.0

Worker nodes must still show the previous kubelet version, 4.20.

12. Update OLM Operators for the new EUS version:

Operators might need multiple version jumps for EUS-to-EUS updates. Check the status of OLM Operators by running the following command:

```
$ oc --context=spoke1 get csv -A | grep -v Succeeded
```

Update Operator subscriptions to EUS-compatible channels by running the following command:

```
$ oc --context=spoke1 patch subscription sriov-network-operator-subscription \
  -n openshift-sriov-network-operator \
  --type merge \
  -p '{"spec":{"channel":"stable-4.22"}}'
```

13. When ready to update worker nodes, create and apply an unpauses policy:

Determine the timing for worker node updates based on your maintenance schedule. Workers can remain at the previous EUS version for an extended period if needed.

Follow the procedure in "Pausing and unpausing worker nodes by using TALM" to unpause and update workers.

14. Monitor worker node updates to completion by running the following command:

```
$ oc --context=spoke1 get mcp worker -w
```

Verification

1. Verify all nodes are at the target EUS version by running the following command:

```
$ oc --context=spoke1 get nodes
```

2. Verify all cluster Operators are healthy by running the following command:

```
$ oc --context=spoke1 get co
```

3. Validate CNF-specific resources by running the following commands:

```
$ oc --context=spoke1 get performanceprofile
$ oc --context=spoke1 get ptpconfig -n openshift-ptp
$ oc --context=spoke1 get sriovnetworknodepolicy -n openshift-sriov-network-operator
```

4. Run comprehensive smoke tests for all CNF workloads.

Troubleshooting

- If the EUS update path is not available, verify you are at the latest z-stream of the source EUS version.
- If the control plane update gets stuck, check for Operator compatibility issues with the target EUS version.
- If worker nodes fail to update after unpause, check the **MachineConfigPool** resource status and node logs.

13.3.9.2. Additional resources

- [Prepare RHACM policies and TALM for cluster updates](#)
- [Manage worker nodes during a cluster update with TALM](#)
- [Perform health checks before a cluster update with TALM](#)
- [OpenShift Container Platform update documentation](#)
- [OpenShift Container Platform update lifecycle and support policy](#)

13.3.10. Troubleshoot cluster updates with TALM

If a cluster update gets stuck, fails, or results in degraded cluster Operators, use the following diagnostic procedures to identify the root cause and take corrective action.

13.3.10.1. Diagnosing a ClusterGroupUpgrade CR stuck in Preparing state

If a **ClusterGroupUpgrade** custom resource (CR) is stuck in the **Preparing** state, the issue is typically related to policy compliance, cluster readiness, or blocking custom resources.

Prerequisites

- You have a **ClusterGroupUpgrade** CR that is stuck in **Preparing** state.
- You have access to the Red Hat Advanced Cluster Management (RHACM) hub cluster with cluster-admin privileges.

Procedure

1. Check the **ClusterGroupUpgrade** status by running the following command:

```
$ oc get cgu <cgu_name> -n <namespace> -o yaml
```

Look for error messages in the **status.conditions** section. Common causes include managed policies that do not exist or are not bound to target clusters, target clusters that are not in a ready state, and blocking custom resources that are not ready.

2. Check policy compliance in RHACM by running the following command:

```
$ oc get policy <policy_name> -n <namespace>
```

3. Check policy details for noncompliant policies by running the following command:

```
$ oc describe policy <policy_name> -n <namespace>
```

Resolve policy compliance issues before the **ClusterGroupUpgrade** CR can proceed.

13.3.10.2. Diagnosing a ClusterGroupUpgrade CR that failed on some clusters

If a **ClusterGroupUpgrade** custom resource (CR) completes but some clusters show a **failed** status, investigate the failed clusters individually to identify the root cause.

Prerequisites

- You have a **ClusterGroupUpgrade** CR that shows failed clusters.
- You have access to the Red Hat Advanced Cluster Management (RHACM) hub cluster with cluster-admin privileges.

Procedure

1. Check detailed cluster status in the **ClusterGroupUpgrade** CR by running the following command:

```
$ oc get cgu <cgu_name> -n <namespace> -o jsonpath='{.status.clusters}'
```

The following example shows the output:

```
{
  "spoke1": "complete",
  "spoke2": "failed",
  "spoke3": "inprogress"
}
```

2. Check TALM logs for errors related to the failed clusters by running the following command:

```
$ oc logs -n openshift-operators deployment/cluster-group-upgrades-controller-manager
```

3. Check policy violation events on the failed cluster by running the following command:

```
$ oc --context=<failed_cluster> get events -A --sort-by='.lastTimestamp' | tail -20
```

13.3.10.3. Diagnosing degraded cluster Operators during an update

If a cluster Operator becomes degraded during an update, identify the affected Operator and investigate the root cause.

Common Operators that degrade during updates include **authentication**, **console**, **monitoring**, and **network**.

Prerequisites

- You have a cluster update in progress with one or more degraded Operators.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Identify degraded Operators by running the following command:

```
$ oc get co | grep -v "True.*False.*False"
```

The following example shows the output:

NAME	VERSION	AVAILABLE	PROGRESSING	DEGRADED
SINCE MESSAGE				
authentication	4.20.0	True	False	True 5m
APIServerDeploymentDegraded: ...				

2. Check Operator details by running the following command:

```
$ oc describe co authentication
```

Look for error messages in the **status.conditions** section of the output.

3. Check Operator pod logs by running the following command:

```
$ oc logs -n openshift-authentication deployment/oauth-openshift
```

13.3.10.4. Diagnosing an update stuck in Working towards state

If the cluster version shows a "Working towards" message for an extended period, identify which Operators are blocking the update from completing. Updates typically complete within 90 to 180 minutes.

Prerequisites

- You have a cluster update that appears stuck in the "Working towards" state.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Check **ClusterVersion** status by running the following command:

```
$ oc get clusterversion -o yaml
```

Look for the **status.conditions** section to identify blocking resources.

2. Check which Operators are still progressing by running the following command:

```
$ oc get co | grep "True"
```

Any Operator showing **PROGRESSING=True** is still updating.

3. Check **ClusterVersion** history by running the following command:

```
$ oc get clusterversion -o jsonpath='{.status.history}'
```

A missing or stalled history entry indicates the update has timed out.

13.3.10.5. Diagnosing worker nodes stuck in NotReady state

If worker nodes show **NotReady** status during an update, common causes include disk pressure, memory pressure, network connectivity issues, and PID pressure.

Prerequisites

- You have worker nodes showing **NotReady** status during a cluster update.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Identify **NotReady** nodes by running the following command:

```
$ oc get nodes | grep NotReady
```

2. Check node conditions by running the following command:

```
$ oc describe node <node_name> | grep -A 10 Conditions
```

3. Check kubelet logs by running the following command:

```
$ oc debug node/<node_name> -- chroot /host journalctl -u kubelet -n 100
```

13.3.10.6. Diagnosing a stuck worker node update

If worker node updates are stuck, the issue is typically related to pod disruption budgets, machine config daemon failures, or node reboot failures.

Prerequisites

- You have a worker node update that is not progressing.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Check the **MachineConfigPool** resource status by running the following command:

```
$ oc get mcp worker -o yaml
```

Look for degraded machine configs in the status.

2. Check pod disruption budgets by running the following command:

```
$ oc get pdb -A
```

If a pod disruption budget (PDB) shows **ALLOWED DISRUPTIONS** of 0, adjust the PDB configuration.

3. Check machine-config-daemon logs on the stuck node by running the following command:

```
$ oc logs -n openshift-machine-config-operator <machine_config_daemon_pod>
```

13.3.10.7. Diagnosing etcd issues during control plane updates

If etcd members become unhealthy during a control plane update, identify the affected members and investigate the root cause before proceeding.

Prerequisites

- You have a control plane update with suspected etcd issues.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Check etcd cluster health by running the following command:

```
$ oc get etcd -o yaml
```

Look for the **EtcdMembersAvailable** condition in the output.

2. Check etcd pod status by running the following command:

```
$ oc get pods -n openshift-etcd -l app=etcd
```

All etcd pods must be running.

3. View etcd member status by running the following command:

```
$ oc rsh -n openshift-etcd <etcd_pod> etcdctl member list -w table
```

4. Monitor etcd logs for errors by running the following command:

```
$ oc logs -n openshift-etcd <etcd_pod> etcd
```

13.3.10.8. Diagnosing image pull failures during updates

If image pull failures occur during an update, common causes include registry connectivity issues, images not mirrored to a disconnected registry, and image pull authentication failures.

Prerequisites

- You have image pull failures during a cluster update.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Check image pull status on nodes by running the following command:

```
$ oc get events -A --field-selector reason=Failed --sort-by='.lastTimestamp' | grep -i image
```

2. For disconnected environments, verify image mirroring by running the following command:

```
$ oc get imagedigestmirrorset
```



NOTE

In OpenShift Container Platform 4.13 and earlier, image mirroring was configured with **ImageContentSourcePolicy** resources. In OpenShift Container Platform 4.14 and later, **ImageDigestMirrorSet** resources replace **ImageContentSourcePolicy**. If your cluster was originally installed on a version earlier than 4.14, check both resource types.

3. Test image pull from a node by running the following command:

```
$ oc debug node/<node_name> -- chroot /host podman pull <image_url>
```

13.3.10.9. Diagnosing update timeout exceeded

If TALM reports that the update timeout was exceeded, investigate cluster resource utilization, image pull speed, and network latency.

Prerequisites

- You have a **ClusterGroupUpgrade** custom resource (CR) that has timed out.
- You have access to the target cluster with cluster-admin privileges.

Procedure

1. Check cluster resource utilization by running the following command:

```
$ oc adm top nodes
```

High CPU or memory usage can slow updates.

2. Check for slow image pulls by running the following command:

```
$ oc get events -A --sort-by='.lastTimestamp' | grep -i "pull"
```

Slow image pulls can indicate network latency between nodes and registries.

3. Increase the **ClusterGroupUpgrade** CR timeout by running the following command if the cluster is healthy but updating slowly:

```
$ oc patch cgu <cgu_name> -n <namespace> --type merge -p '{"spec": {"remediationStrategy":{"timeout":240}}}'
```

13.3.10.10. Resolving a noncompliant policy hold during an update

During the update, TALM verifies the status of **ClusterVersion** for the OpenShift Container Platform update and **Subscription** for OLM Operators. If errors occur during those updates, the status does not change to the expected value and the policies do not become compliant. TALM holds at that point waiting for compliance.

If you are monitoring the update, you can fix the issue on the target cluster and the update continues automatically when the policy becomes compliant.

ClusterGroupUpgrade custom resource (CR) has timed out

Delete the timed-out CR by running the following command:

```
$ oc delete cgu <timed_out_cgu_name> -n <namespace>
```

All policies revert to **inform** mode, leaving the cluster at the point where the issue occurred.

Fix the issue that caused the timeout, then re-create the **ClusterGroupUpgrade** CR to resume the update by running the following command:

```
$ oc apply -f <cgu_cr_filename>.yaml
```

Policies or TALM are conflicting with changes you need to make

Delete the **ClusterGroupUpgrade** CR to revert all policies to **inform** mode by running the following command:

```
$ oc delete cgu <cgu_name> -n <namespace>
```

After the policies revert to **inform** mode, they do not enforce changes on the cluster.

Fix the issue, then re-create the **ClusterGroupUpgrade** CR to resume the update.

13.3.10.11. Collecting diagnostic information for Red Hat support

If you cannot resolve the update issue, collect diagnostic information and contact Red Hat support for assistance.

Prerequisites

- You have a cluster update issue that you cannot resolve.
- You have access to the target cluster and the Red Hat Advanced Cluster Management (RHACM) hub cluster with cluster-admin privileges.

Procedure

1. Collect must-gather data by running the following command:

```
$ oc adm must-gather
```

2. Collect RHACM must-gather data for TALM-specific issues by running the following command:

```
$ oc adm must-gather --image=registry.redhat.io/rhacm2/must-gather-rhel8
```

3. Collect **ClusterVersion** history by running the following command:

```
$ oc get clusterversion -o jsonpath='{.status.history}' > clusterversion-history.json
```

4. Collect **ClusterGroupUpgrade** status by running the following command:

```
$ oc get cgu <cgu_name> -n <namespace> -o yaml > cgu-status.yaml
```

13.3.10.12. Additional resources

- [Perform health checks before a cluster update with TALM](#)
- [Using the Topology Aware Lifecycle Manager for cluster updates](#)
- [Contact Red Hat support](#)
- [Red Hat Knowledgebase](#)

13.4. TROUBLESHOOTING AND MAINTAINING OPENSIFT CONTAINER PLATFORM CLUSTERS

13.4.1. Troubleshooting and maintaining OpenShift Container Platform clusters

Troubleshooting and maintenance are weekly tasks that can be a challenge if you do not have the tools to reach your goal, whether you want to update a component or investigate an issue. Part of the challenge is knowing where and how to search for tools and answers.

To maintain and troubleshoot a bare-metal environment with high performance requirements, see the following procedures.



IMPORTANT

This troubleshooting information is not a reference for configuring OpenShift Container Platform or developing cloud-native applications.

For information about developing cloud-native applications on OpenShift Container Platform, see [Red Hat Best Practices for Kubernetes](#).

13.4.1.1. Getting Support

If you experience difficulty with a procedure, visit the [Red Hat Customer Portal](#). From the Customer Portal, you can find help in various ways:

- Search or browse through the Red Hat Knowledgebase of articles and solutions about Red Hat products.
- Submit a support case to Red Hat Support.
- Access other product documentation.

To identify issues with your deployment, you can use the debugging tool or check the health endpoint of your deployment. After you have debugged or obtained health information about your deployment, you can search the Red Hat Knowledgebase for a solution or file a support ticket.

13.4.1.1.1. About the Red Hat Knowledgebase

The [Red Hat Knowledgebase](#) provides rich content aimed at helping you make the most of Red Hat's products and technologies. The Red Hat Knowledgebase consists of articles, product documentation, and videos outlining best practices on installing, configuring, and using Red Hat products. In addition, you can search for solutions to known issues, each providing concise root cause descriptions and remedial steps.

13.4.1.1.2. Searching the Red Hat Knowledgebase

In the event of an OpenShift Container Platform issue, you can perform an initial search to determine if a solution already exists within the Red Hat Knowledgebase.

Prerequisites

- You have a Red Hat Customer Portal account.

Procedure

1. Log in to the [Red Hat Customer Portal](#).
2. Click **Search**.
3. In the search field, input keywords and strings relating to the problem, including:
 - OpenShift Container Platform components (such as **etcd**)
 - Related procedure (such as **installation**)
 - Warnings, error messages, and other outputs related to explicit failures
4. Click the **Enter** key.
5. Optional: Select the **OpenShift Container Platform** product filter.
6. Optional: Select the **Documentation** content type filter.

13.4.1.1.3. Submitting a support case

Submit a support case to Red Hat Support to get help with issues you encounter with OpenShift Container Platform.

Prerequisites

- You have access to the cluster as a user with the **cluster-admin** role.

- You have installed the OpenShift CLI (**oc**).
- You have a Red Hat Customer Portal account.
- You have a Red Hat Standard or Premium subscription.

Procedure

1. Log in to [the Customer Support page](#) of the Red Hat Customer Portal.
2. Click **Get support**.
3. On the **Cases** tab of the **Customer Support** page:
 - a. Optional: Change the pre-filled account and owner details if needed.
 - b. Select the appropriate category for your issue, such as **Bug or Defect**, and click **Continue**.
4. Enter the following information:
 - a. In the **Summary** field, enter a concise but descriptive problem summary and further details about the symptoms being experienced, as well as your expectations.
 - b. Select **OpenShift Container Platform** from the **Product** drop-down menu.
 - c. Select **4.22** from the **Version** drop-down.
5. Review the list of suggested Red Hat Knowledgebase solutions for a potential match against the problem that is being reported. If the suggested articles do not address the issue, click **Continue**.
6. Review the updated list of suggested Red Hat Knowledgebase solutions for a potential match against the problem that is being reported. The list is refined as you provide more information during the case creation process. If the suggested articles do not address the issue, click **Continue**.
7. Ensure that the account information presented is as expected, and if not, amend accordingly.
8. Check that the autofilled OpenShift Container Platform Cluster ID is correct. If it is not, manually obtain your cluster ID.
 - To manually obtain your cluster ID using the OpenShift Container Platform web console:
 - a. Navigate to **Home → Overview**.
 - b. Find the value in the **Cluster ID** field of the **Details** section.
 - Alternatively, it is possible to open a new support case through the OpenShift Container Platform web console and have your cluster ID autofilled.
 - a. From the toolbar, navigate to **(?) Help → Open Support Case**.
 - b. The **Cluster ID** value is autofilled.
 - To obtain your cluster ID using the OpenShift CLI (**oc**), run the following command:

```
$ oc get clusterversion -o jsonpath='{.items[].spec.clusterID}'{"\n"}
```


9. Complete the following questions where prompted and then click **Continue**:
 - What are you experiencing? What are you expecting to happen?
 - Define the value or impact to you or the business.
 - Where are you experiencing this behavior? What environment?
 - When does this behavior occur? Frequency? Repeatedly? At certain times?
10. Upload relevant diagnostic data files and click **Continue**. It is recommended to include data gathered using the **oc adm must-gather** command as a starting point, plus any issue specific data that is not collected by that command.
11. Input relevant case management details and click **Continue**.
12. Preview the case details and click **Submit**.

13.4.2. General troubleshooting

When you encounter a problem, the first step is to find the specific area where the issue is happening. To narrow down the potential problematic areas, complete one or more of the following tasks:

- Query your cluster
- Check your pod logs
- Debug a pod
- Review events

13.4.2.1. Querying your cluster

Get information about your cluster so that you can more accurately find potential problems.

Procedure

1. Switch into a project by running the following command:

```
$ oc project <project_name>
```

2. Query your cluster version, cluster Operator, and node within that namespace by running the following command:

```
$ oc get clusterversion,clusteroperator,node
```

```
NAME                                VERSION AVAILABLE PROGRESSING SINCE
STATUS
clusterversion.config.openshift.io/version 4.16.11 True    False    62d    Cluster
version is 4.16.11
```

```
NAME                                VERSION AVAILABLE
PROGRESSING DEGRADED SINCE MESSAGE
clusteroperator.config.openshift.io/authentication 4.16.11 True    False
False    62d
```

clusteroperator.config.openshift.io/baremetal	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/cloud-controller-manager	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/cloud-credential	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/cluster-autoscaler	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/config-operator	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/console	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/control-plane-machine-set	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/csi-snapshot-controller	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/dns	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/etcd	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/image-registry	4.16.11	True	False
False 55d			
clusteroperator.config.openshift.io/ingress	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/insights	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/kube-apiserver	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/kube-controller-manager	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/kube-scheduler	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/kube-storage-version-migrator	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/machine-api	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/machine-approver	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/machine-config	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/marketplace	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/monitoring	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/network	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/node-tuning	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/openshift-apiserver	4.16.11	True	False
False 62d			
clusteroperator.config.openshift.io/openshift-controller-manager	4.16.11	True	
False False 62d			
clusteroperator.config.openshift.io/openshift-samples	4.16.11	True	
False False 35d			
clusteroperator.config.openshift.io/operator-lifecycle-manager	4.16.11	True	
False False 62d			

```

clusteroperator.config.openshift.io/operator-lifecycle-manager-catalog      4.16.11  True
False      False      62d
clusteroperator.config.openshift.io/operator-lifecycle-manager-packageserver 4.16.11  True
False      False      62d
clusteroperator.config.openshift.io/service-ca                            4.16.11  True      False
False      62d
clusteroperator.config.openshift.io/storage                              4.16.11  True      False
False      62d

```

NAME	STATUS	ROLES	AGE	VERSION
node/ctrl-plane-0	Ready	control-plane,master,worker	62d	v1.29.7
node/ctrl-plane-1	Ready	control-plane,master,worker	62d	v1.29.7
node/ctrl-plane-2	Ready	control-plane,master,worker	62d	v1.29.7

For more information, see "oc get" and "Reviewing pod status".

Additional resources

- [oc get](#)
- [Reviewing pod status](#)

13.4.2.2. Checking pod logs

Get logs from the pod so that you can review the logs for issues.

Procedure

1. List the pods by running the following command:

```
$ oc get pod
```

NAME	READY	STATUS	RESTARTS	AGE
busybox-1	1/1	Running	168 (34m ago)	7d
busybox-2	1/1	Running	119 (9m20s ago)	4d23h
busybox-3	1/1	Running	168 (43m ago)	7d
busybox-4	1/1	Running	168 (43m ago)	7d

2. Check pod log files by running the following command:

```
$ oc logs -n <namespace> busybox-1
```

For more information, see "oc logs", "Logging", and "Inspecting pod and container logs".

Additional resources

- [oc logs](#)
- [Logging](#)
- [Inspecting pod and container logs](#)

13.4.2.3. Describing a pod

To troubleshoot pod issues and view detailed information about a pod in OpenShift Container Platform, you can describe a pod using the **oc describe pod** command. The **Events** section in the output provides detailed information about the pod and the containers inside of it.

Procedure

- Describe a pod by running the following command:

```
$ oc describe pod -n <namespace> busybox-1
```

Example output

```
Name:          busybox-1
Namespace:     busy
Priority:       0
Service Account: default
Node:          worker-3/192.168.0.0
Start Time:    Mon, 27 Nov 2023 14:41:25 -0500
Labels:        app=busybox
               pod-template-hash=<hash>
Annotations:   k8s.ovn.org/pod-networks:
...
Events:
  Type    Reason    Age          From    Message
  ----    -
Normal   Pulled    41m (x170 over 7d1h) kubelet Container image
"quay.io/quay/busybox:latest" already present on machine
Normal   Created   41m (x170 over 7d1h) kubelet Created container busybox
Normal   Started   41m (x170 over 7d1h) kubelet Started container busybox
```

Additional resources

- [oc describe](#)

13.4.2.4. Reviewing events

You can review the events in a given namespace to find potential issues.

Procedure

- Check for events in your namespace by running the following command:

```
$ oc get events -n <namespace> --sort-by=".metadata.creationTimestamp"
```

The **--sort-by=".metadata.creationTimestamp"** flag places the most recent events at the end of the output.

- Optional: If the events within your specified namespace do not provide enough information, expand your query to all namespaces by running the following command:

```
$ oc get events -A --sort-by=".metadata.creationTimestamp"
```

The **--sort-by=".metadata.creationTimestamp"** flag places the most recent events at the end of the output.

To filter the results of all events from a cluster, you can use the **grep** command. For example, if you are looking for errors, the errors can appear in two different sections of the output: the **TYPE** or **MESSAGE** sections. With the **grep** command, you can search for keywords, such as **error** or **failed**.

- For example, search for a message that contains **warning** or **error** by running the following command:

```
$ oc get events -A | grep -Ei "warning|error"
```

```

NAMESPACE   LAST SEEN   TYPE      REASON      OBJECT          MESSAGE
openshift   59s        Warning   FailedMount  pod/openshift-1 MountVolume.SetUp failed
for volume "v4-0-config-user-idp-0-file-data" : references non-existent secret key: test

```

- Optional: To clean up the events and see only recurring events, you can delete the events in the relevant namespace by running the following command:

```
$ oc delete events -n <namespace> --all
```

For more information, see "Watching cluster events".

Additional resources

- [Watching cluster events](#)

13.4.2.5. Connecting to a pod

You can directly connect to a currently running pod with the **oc rsh** command, which provides you with a shell on that pod.



WARNING

In pods that run a low-latency application, latency issues can occur when you run the **oc rsh** command. Use the **oc rsh** command only if you cannot connect to the node by using the **oc debug** command.

Procedure

- Connect to your pod by running the following command:

```
$ oc rsh -n <namespace> busybox-1
```

For more information, see "oc rsh" and "Accessing running pods".

Additional resources

- [oc rsh](#)
- [Accessing running pods](#)

13.4.2.6. Debugging a pod

In certain cases, you do not want to directly interact with your pod that is in production.

To avoid interfering with running traffic, you can use a secondary pod that is a copy of your original pod. The secondary pod uses the same components as that of the original pod but does not have running traffic.

Procedure

1. List the pods by running the following command:

```
$ oc get pod
```

NAME	READY	STATUS	RESTARTS	AGE
busybox-1	1/1	Running	168 (34m ago)	7d
busybox-2	1/1	Running	119 (9m20s ago)	4d23h
busybox-3	1/1	Running	168 (43m ago)	7d
busybox-4	1/1	Running	168 (43m ago)	7d

2. Debug a pod by running the following command:

```
$ oc debug -n <namespace> busybox-1
```

```
Starting pod/busybox-1-debug, command was: sleep 3600
Pod IP: 10.133.2.11
```

If you do not see a shell prompt, press Enter.

For more information, see "oc debug" and "Starting debug pods with root access".

Additional resources

- [oc debug](#)
- [Starting debug pods with root access](#)

13.4.2.7. Running a command on a pod

If you want to run a command or set of commands on a pod without directly logging into it, you can use the **oc exec -it** command. You can interact with the pod quickly to get process or output information from the pod. A common use case is to run the **oc exec -it** command inside a script to run the same command on multiple pods in a replica set or deployment.

**WARNING**

In pods that run a low-latency application, the **oc exec** command can cause latency issues.

Procedure

- To run a command on a pod without logging into it, run the following command:

```
$ oc exec -it <pod> -- <command>
```

For more information, see "oc exec" and "Executing remote commands in containers".

Additional resources

- [oc exec](#)
- [Executing remote commands in containers](#)

13.4.3. Cluster maintenance

When deploying OpenShift Container Platform on bare-metal infrastructure, you must pay more attention to certain configurations which can have a significant impact on cluster stability. You can troubleshoot more effectively by completing these tasks:

- Monitor for failed or failing hardware components
- Periodically check the status of the cluster Operators

**NOTE**

For hardware monitoring, contact your hardware vendor to find the appropriate logging tool for your specific hardware.

13.4.3.1. Checking cluster Operators

Periodically check the status of your cluster Operators to find issues early.

Procedure

- Check the status of the cluster Operators by running the following command:

```
$ oc get co
```

13.4.3.2. Watching for failed pods

To reduce troubleshooting time, regularly monitor for failed pods in your cluster.

Procedure

- To watch for failed pods, run the following command:

```
$ oc get po -A | grep -Eiv 'complete|running'
```

13.4.4. Security

Implementing a robust cluster security profile is important for building resilient environments.

13.4.4.1. Authentication

Determine which identity providers are in your cluster. For more information about supported identity providers, see "Supported identity providers" in *Authentication and authorization*.

After you know which providers are configured, you can inspect the **openshift-authentication** namespace to determine if there are potential issues.

Procedure

1. Check the events in the **openshift-authentication** namespace by running the following command:

```
$ oc get events -n openshift-authentication --sort-by='.metadata.creationTimestamp'
```

2. Check the pods in the **openshift-authentication** namespace by running the following command:

```
$ oc get pod -n openshift-authentication
```

3. Optional: If you need more information, check the logs of one of the running pods by running the following command:

```
$ oc logs -n openshift-authentication <pod_name>
```

Additional resources

- [Supported identity providers](#)

13.4.5. Certificate maintenance

Certificate maintenance is required for continuous cluster authentication. As a cluster administrator, you must manually renew certain certificates, while others are automatically renewed by the cluster.

Learn about certificates in OpenShift Container Platform and how to maintain them by using the following resources:

- [Which OpenShift certificates do rotate automatically and which do not in OpenShift 4.x?](#)
- [Checking etcd certificate expiry in OpenShift 4](#)

13.4.5.1. Certificates manually managed by the administrator

The following certificates must be renewed by a cluster administrator:

- Proxy certificates
- User-provisioned certificates for the API server

13.4.5.1.1. Managing proxy certificates

Proxy certificates allow users to specify one or more custom certificate authority (CA) certificates that are used by platform components when making egress connections.



NOTE

Certain CAs set expiration dates and you might need to renew these certificates every two years.

If you did not originally set the requested certificates, you can determine the certificate expiration in several ways. Most Cloud-native Network Functions (CNFs) use certificates that are not specifically designed for browser-based connectivity. Therefore, you need to pull the certificate from the **ConfigMap** object of your deployment.

Procedure

- To get the expiration date, run the following command against the certificate file:

```
$ openssl x509 -enddate -noout -in <cert_file_name>.pem
```

For more information about determining how and when to renew your proxy certificates, see "Proxy certificates" in *Security and compliance*.

Additional resources

- [Proxy certificates](#)

13.4.5.1.2. User-provisioned API server certificates

The API server is accessible by clients that are external to the cluster at **api.<cluster_name>.<base_domain>**. You might want clients to access the API server at a different hostname or without the need to distribute the cluster-managed certificate authority (CA) certificates to the clients. You must set a custom default certificate to be used by the API server when serving content.

For more information, see "User-provided certificates for the API server" in *Security and compliance*

Additional resources

- [User-provisioned certificates for the API server](#)

13.4.5.2. Certificates managed by the cluster

You only need to check cluster-managed certificates if you detect an issue in the logs. The following certificates are automatically managed by the cluster:

- Service CA certificates
- Node certificates

- Bootstrap certificates
- etcd certificates
- OLM certificates
- Machine Config Operator certificates
- Monitoring and cluster logging Operator component certificates
- Control plane certificates
- Ingress certificates

Additional resources

- [Service CA certificates](#)
- [Node certificates](#)
- [Bootstrap certificates](#)
- [etcd certificates](#)
- [OLM certificates](#)
- [Machine Config Operator certificates](#)
- [Monitoring and cluster logging Operator component certificates](#)
- [Control plane certificates](#)
- [Ingress certificates](#)

13.4.5.2.1. Certificates managed by etcd

The etcd certificates are used for encrypted communication between etcd member peers as well as encrypted client traffic. The certificates are renewed automatically within the cluster provided that communication between all nodes and all services is current. Therefore, if your cluster might lose communication between components during a specific period of time, which is close to the end of the etcd certificate lifetime, it is recommended to renew the certificate in advance. For example, communication can be lost during an upgrade due to nodes rebooting at different times.

- You can manually renew etcd certificates by running the following command:

```
$ for each in $(oc get secret -n openshift-etcd | grep "kubernetes.io/tls" | grep -e \
"etcd-peer\|etcd-serving" | awk '{print $1}'); do oc get secret $each -n openshift-etcd -o \
jsonpath="{.data.tls.crt}" | base64 -d | openssl x509 -noout -enddate; done
```

For more information about updating etcd certificates, see [Checking etcd certificate expiry in OpenShift 4](#). For more information about etcd certificates, see "etcd certificates" in *Security and compliance*.

Additional resources

- [etcd certificates](#)

13.4.5.2.2. Node certificates

Node certificates are self-signed certificates, which means that they are signed by the cluster and they originate from an internal certificate authority (CA) that is generated by the bootstrap process.

After the cluster is installed, the cluster automatically renews the node certificates.

For more information, see "Node certificates" in *Security and compliance*.

Additional resources

- [Node certificates](#)

13.4.5.2.3. Service CA certificates

The **service-ca** is an Operator that creates a self-signed certificate authority (CA) when an OpenShift Container Platform cluster is deployed. This allows user to add certificates to their deployments without manually creating them. Service CA certificates are self-signed certificates.

For more information, see "Service CA certificates" in *Security and compliance*.

Additional resources

- [Service CA certificates](#)

13.4.6. Machine Config Operator

The Machine Config Operator provides useful information to cluster administrators and controls what is running directly on the bare-metal host.

The Machine Config Operator differentiates between groups of nodes in the cluster, allowing control plane nodes and worker nodes to run with different configurations. These groups of nodes run worker or application pods, which are called **MachineConfigPool (mcp)** groups. The same machine config is applied to all nodes or only to one MCP in the cluster.

For more information about the Machine Config Operator, see [Machine Config Operator](#).

13.4.6.1. Purpose of the Machine Config Operator

The Machine Config Operator (MCO) manages and applies configuration and updates of Red Hat Enterprise Linux CoreOS (RHCOS) and container runtime, including everything between the kernel and kubelet. Managing RHCOS is important since most telecommunications companies run on bare-metal hardware and use some sort of hardware accelerator or kernel modification. Applying machine configuration to RHCOS manually can cause problems because the MCO monitors each node and what is applied to it.

You must consider these minor components and how the MCO can help you manage your clusters effectively.



IMPORTANT

You must use the MCO to perform all changes on worker or control plane nodes. Do not manually make changes to RHCOS or node files.

13.4.6.2. Applying several machine config files at the same time

When you need to change the machine config for a group of nodes in the cluster, also known as machine config pools (MCPs), sometimes the changes must be applied with several different machine config files. The nodes need to restart for the machine config file to be applied. After each machine config file is applied to the cluster, all nodes restart that are affected by the machine config file.

To prevent the nodes from restarting for each machine config file, you can apply all of the changes at the same time by pausing each MCP that is updated by the new machine config file.

Procedure

1. Pause the affected MCP by running the following command:

```
$ oc patch mcp/<mcp_name> --type merge --patch '{"spec":{"paused":true}}'
```

2. After you apply all machine config changes to the cluster, run the following command:

```
$ oc patch mcp/<mcp_name> --type merge --patch '{"spec":{"paused":false}}'
```

This allows the nodes in your MCP to reboot into the new configurations.

13.4.7. Bare-metal node maintenance

You can connect to a node for general troubleshooting. However, in some cases, you need to perform troubleshooting or maintenance tasks on certain hardware components. This section discusses topics that you need to perform for hardware maintenance.

13.4.7.1. Connecting to a bare-metal node in your cluster

You can connect to bare-metal cluster nodes for general maintenance tasks.

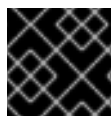


NOTE

Configuring the cluster node from the host operating system is not recommended or supported.

To troubleshoot your nodes, you can do the following tasks:

- Retrieve logs from a node
- Use debugging
- Use SSH to connect to a node



IMPORTANT

Use SSH only if you cannot connect to the node with the **oc debug** command.

Procedure

1. Retrieve the logs from a node by running the following command:

```
$ oc adm node-logs <node_name> -u crio
```

-
- 2. Use debugging by running the following command:

```
$ oc debug node/<node_name>
```

- 3. Set **/host** as the root directory within the debug shell. The debug pod mounts the host's root file system in **/host** within the pod. By changing the root directory to **/host**, you can run binaries contained in the host's executable paths:

```
# chroot /host
```

You are now logged in as root on the node.

- 4. Optional: Use SSH to connect to the node by running the following command:

```
$ ssh core@<node_name>
```

13.4.7.2. Moving applications to pods within the cluster

For scheduled hardware maintenance, you need to consider how to move your application pods to other nodes within the cluster without affecting the pod workload.

Procedure

- Mark the node as unschedulable by running the following command:

```
$ oc adm cordon <node_name>
```

When the node is unschedulable, no pods can be scheduled on the node. For more information, see "Working with nodes".



NOTE

When moving CNF applications, you might need to verify ahead of time that there are enough additional worker nodes in the cluster due to anti-affinity and pod disruption budget.

Additional resources

- [Working with nodes](#)

13.4.7.3. DIMM memory replacement

Dual in-line memory module (DIMM) problems sometimes only appear after a server reboots. You can check the log files for these problems.

When you perform a standard reboot and the server does not start, you can see a message in the console that there is a faulty DIMM memory. In that case, you can acknowledge the faulty DIMM and continue rebooting if the remaining memory is sufficient. Then, you can schedule a maintenance window to replace the faulty DIMM.

Sometimes, a message in the event logs indicates a bad memory module. In these cases, you can schedule the memory replacement before the server is rebooted.

13.4.7.4. Disk replacement

If you do not have disk redundancy configured on your node through hardware or software redundant array of independent disks (RAID), you need to check the following:

- Does the disk contain running pod images?
- Does the disk contain persistent data for pods?

For more information, see "OpenShift Container Platform storage overview" in *Storage*.

Additional resources

- [OpenShift Container Platform storage overview](#)

13.4.7.5. Cluster network card replacement

When you replace a network card, the MAC address changes. The MAC address can be part of the DHCP or SR-IOV Operator configuration, router configuration, firewall rules, or cloud-native application configuration. Before you bring back a node online after replacing a network card, you must verify that these configurations are up-to-date.



IMPORTANT

If you do not have specific procedures for MAC address changes within the network, contact your network administrator or network hardware vendor.

13.5. OBSERVABILITY

13.5.1. Observability in OpenShift Container Platform clusters

OpenShift Container Platform generates a large amount of data, such as performance metrics and logs from both the platform and the workloads running on it. As an administrator, you can use various tools to collect and analyze all the data available. What follows is an outline of best practices for system engineers, architects, and administrators configuring the observability stack.

Unless explicitly stated, the material in this document refers to both Edge and Core deployments.

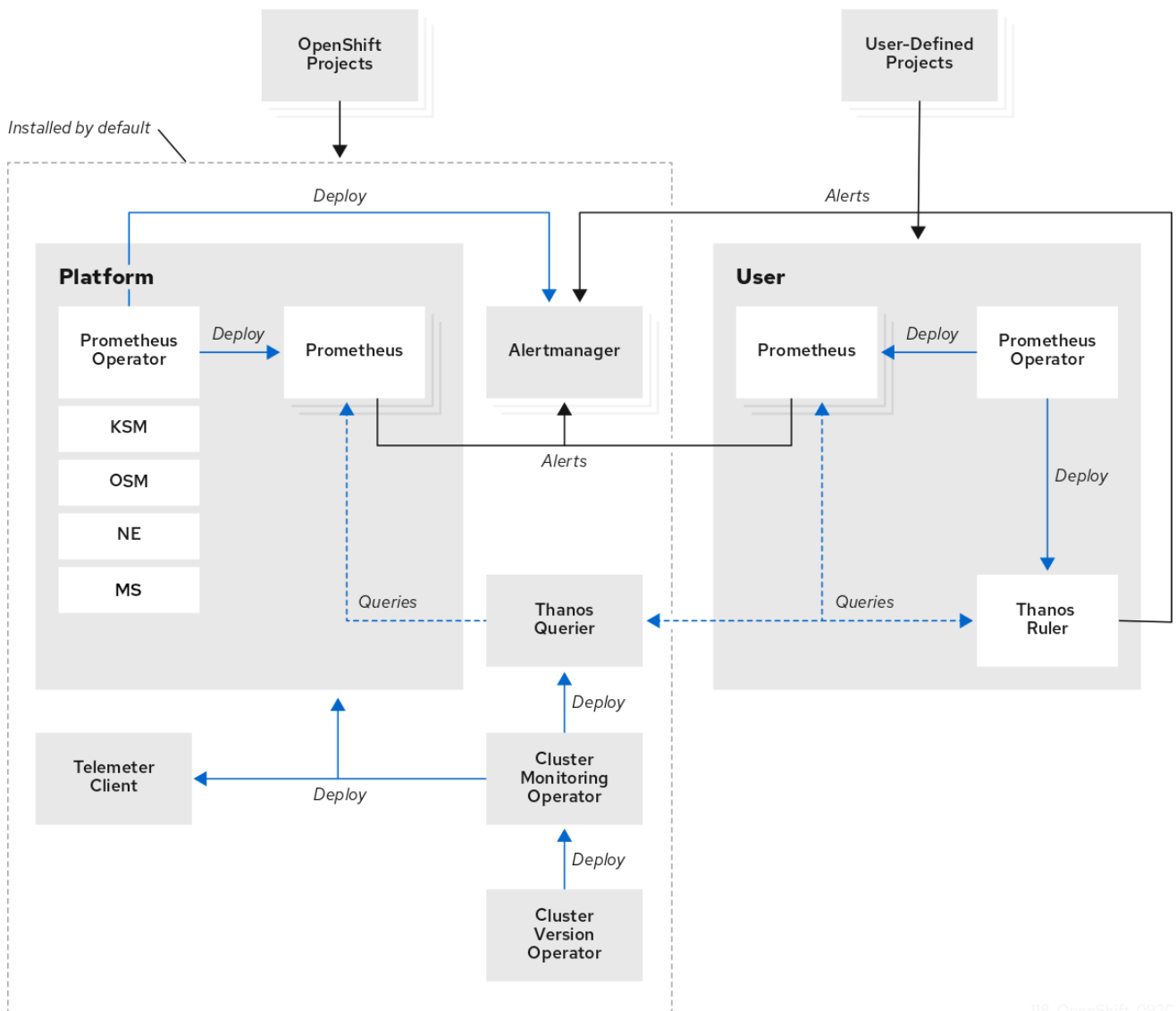
13.5.1.1. Monitoring stack components

The monitoring stack in OpenShift Container Platform consists of several integrated components that collect, analyze, store, and alert on metrics.

The monitoring stack uses the following components:

- Prometheus collects and analyzes metrics from OpenShift Container Platform components and from workloads, if configured to do so.
- Alertmanager is a component of Prometheus that handles routing, grouping, and silencing of alerts.
- Thanos handles long term storage of metrics.

Figure 13.2. OpenShift Container Platform monitoring architecture



118_OpenShift_092C

**NOTE**

For single-node OpenShift clusters, disable Alertmanager and Thanos because the clusters sends all metrics to the hub cluster for analysis and retention.

Additional resources

- [About OpenShift Container Platform monitoring](#)
- [Core platform monitoring first steps](#)

13.5.1.2. Key performance metrics

Depending on your system, you can have hundreds of available measurements.

Consider the following key metrics:

- **etcd** response times
- API response times

- Pod restarts and scheduling
- Resource usage
- OVN health
- Overall cluster operator health

If a metric is important, set up an alert for it.



NOTE

You can check the available metrics by running the following command:

```
$ oc -n openshift-monitoring exec -c prometheus prometheus-k8s-0 -- curl -qsk http://localhost:9090/api/v1/metadata | jq '.data'
```

13.5.1.2.1. Example queries in PromQL

Using the OpenShift Container Platform console, you can explore the following queries in the metrics query browser.



NOTE

The URL for the console is <https://<OpenShift Console FQDN>/monitoring/query-browser>. You can get the Openshift Console FQDN by running the following command:

```
$ oc get routes -n openshift-console console -o jsonpath='{.status.ingress[0].host}'
```

Table 13.4. Node memory & CPU usage

Metric	Query
CPU % requests by node	<code>sum by (node) (sum_over_time(kube_pod_container_resource_requests{resource="cpu"}[60m]))/sum by (node) (sum_over_time(kube_node_status_allocatable{resource="cpu"}[60m])) *100</code>
Overall cluster CPU % utilization	<code>sum by (managed_cluster) (sum_over_time(kube_pod_container_resource_requests{resource="memory"}[60m]))/sum by (managed_cluster) (sum_over_time(kube_node_status_allocatable{resource="cpu"}[60m])) *100</code>

Metric	Query
Memory % requests by node	sum by (node) <code>(sum_over_time(kube_pod_container_resource_requests{resource="memory"}[60m]))/sum by (node) (sum_over_time(kube_node_status_allocatable{resource="memory"}[60m])) *100</code>
Overall cluster memory % utilization	(1-(sum by (managed_cluster) (avg_over_time(node_memory_MemAvailable_bytes[60m]))/sum by (managed_cluster) (avg_over_time(kube_node_status_allocatable{resource="memory"}[60m])))*100

Table 13.5. API latency by verb

Metric	Query
GET	histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver=~"kube-apiserver openshift-apiserver", verb="GET"}[60m])))
PATCH	histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver="kube-apiserver openshift-apiserver", verb="PATCH"}[60m])))
POST	histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver="kube-apiserver openshift-apiserver", verb="POST"}[60m])))
LIST	histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver="kube-apiserver openshift-apiserver", verb="LIST"}[60m])))

Metric	Query
PUT	<code>histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver="kube-apiserver openshift-apiserver", verb="PUT"}[60m])))</code>
DELETE	<code>histogram_quantile (0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver="kube-apiserver openshift-apiserver", verb="DELETE"}[60m])))</code>
Combined	<code>histogram_quantile(0.99, sum by (le,managed_cluster) (sum_over_time(apiserver_request_duration_seconds_bucket{apiserver=~"(openshift-apiserver kube-apiserver)", verb!="WATCH"}[60m])))</code>

Table 13.6. **etcd**

Metric	Query
fsync 99th percentile latency (per instance)	<code>histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[2m]))</code>
fsync 99th percentile latency (per cluster)	<code>sum by (managed_cluster) (histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket[60m])))</code>
Leader elections	<code>sum(rate(etcd_server_leader_changes_seen_total[1440m]))</code>
Network latency	<code>histogram_quantile(0.99, rate(etcd_network_peer_round_trip_time_seconds_bucket[5m]))</code>

Table 13.7. Operator health

Metric	Query
Degraded operators	<code>sum by (managed_cluster, name) (avg_over_time(cluster_operator_conditions{condition="Degraded", name!="version"}[60m]))</code>
Total degraded operators per cluster	<code>sum by (managed_cluster) (avg_over_time(cluster_operator_conditions{condition="Degraded", name!="version"}[60m]))</code>

13.5.1.2.2. Recommendations for storage of metrics

By default, Prometheus does not back up saved metrics with persistent storage. If you restart the Prometheus pods, all metrics data are lost. You must configure the monitoring stack to use the back-end storage that is available on the platform. To meet the high IO demands of Prometheus, use local storage.

For smaller clusters, you can use the Local Storage Operator for persistent storage for Prometheus. Red Hat OpenShift Data Foundation (ODF), which deploys a ceph cluster for block, file, and object storage, is suitable for larger clusters.

To keep system resource requirements low on a single-node OpenShift cluster, do not provision back-end storage for the monitoring stack. Such clusters forward all metrics to the hub cluster where you can provision a third party monitoring platform.

Additional resources

- [Accessing metrics as an administrator](#)
- [Persistent storage using local volumes](#)

13.5.1.3. Monitoring at the far edge network

OpenShift Container Platform clusters at the edge must keep the footprint of the platform components to a minimum. The following procedure is an example of how to configure a single-node OpenShift or a node at the far edge network with a small monitoring footprint.

Prerequisites

- For environments that use Red Hat Advanced Cluster Management (RHACM), you have enabled the Observability service.
- The hub cluster is running Red Hat OpenShift Data Foundation (ODF).

Procedure

1. Create a **ConfigMap** CR, and save it as **monitoringConfigMap.yaml**, as in the following example:

```
apiVersion: v1
```

```

kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    alertmanagerMain:
      enabled: false
    telemetryClient:
      enabled: false
    prometheusK8s:
      retention: 24h

```

2. Apply the **ConfigMap** CR by running the following command on the single-node OpenShift cluster:

```
$ oc apply -f monitoringConfigMap.yaml
```

3. Create a **Namespace** CR, and save it as **monitoringNamespace.yaml**, as in the following example:

```

apiVersion: v1
kind: Namespace
metadata:
  name: open-cluster-management-observability

```

4. Apply the **Namespace** CR by running the following command on the hub cluster :

```
$ oc apply -f monitoringNamespace.yaml
```

5. Create an **ObjectBucketClaim** CR, and save it as **monitoringObjectBucketClaim.yaml**, as in the following example:

```

apiVersion: objectbucket.io/v1alpha1
kind: ObjectBucketClaim
metadata:
  name: multi-cloud-observability
  namespace: open-cluster-management-observability
spec:
  storageClassName: openshift-storage.noobaa.io
  generateBucketName: acm-multi

```

6. Apply the **ObjectBucketClaim** CR by running the following command on the hub cluster:

```
$ oc apply -f monitoringObjectBucketClaim.yaml
```

7. Create a **Secret** CR, and save it as **monitoringSecret.yaml**, as in the following example:

```

apiVersion: v1
kind: Secret
metadata:
  name: multiclusterhub-operator-pull-secret

```

```
namespace: open-cluster-management-observability
stringData:
  .dockerconfigjson: 'PULL_SECRET'
```

8. Apply the **Secret** CR by running the following command in the hub cluster:

```
$ oc apply -f monitoringSecret.yaml
```

9. Get the keys for the NooBaa service and the back-end bucket name from the hub cluster by running the following commands:

```
$ NOOBAA_ACCESS_KEY=$(oc get secret noobaa-admin -n openshift-storage -o json | jq -r '.data.AWS_ACCESS_KEY_ID|@base64d')
```

```
$ NOOBAA_SECRET_KEY=$(oc get secret noobaa-admin -n openshift-storage -o json | jq -r '.data.AWS_SECRET_ACCESS_KEY|@base64d')
```

```
$ OBJECT_BUCKET=$(oc get objectbucketclaim -n open-cluster-management-observability multi-cloud-observability -o json | jq -r .spec.bucketName)
```

10. Create a **Secret** CR for bucket storage and save it as **monitoringBucketSecret.yaml**, as in the following example:

```
apiVersion: v1
kind: Secret
metadata:
  name: thanos-object-storage
  namespace: open-cluster-management-observability
type: Opaque
stringData:
  thanos.yaml: |
    type: s3
    config:
      bucket: ${OBJECT_BUCKET}
      endpoint: s3.openshift-storage.svc
      insecure: true
      access_key: ${NOOBAA_ACCESS_KEY}
      secret_key: ${NOOBAA_SECRET_KEY}
```

11. Apply the **Secret** CR by running the following command on the hub cluster:

```
$ oc apply -f monitoringBucketSecret.yaml
```

12. Create the **MultiClusterObservability** CR and save it as **monitoringMultiClusterObservability.yaml**, as in the following example:

```
apiVersion: observability.open-cluster-management.io/v1beta2
kind: MultiClusterObservability
metadata:
  name: observability
spec:
  advanced:
    retentionConfig:
```

```

    blockDuration: 2h
    deleteDelay: 48h
    retentionInLocal: 24h
    retentionResolutionRaw: 3d
    enableDownsampling: false
    observabilityAddonSpec:
      enableMetrics: true
      interval: 300
    storageConfig:
      alertmanagerStorageSize: 10Gi
      compactStorageSize: 100Gi
      metricObjectStorage:
        key: thanos.yaml
        name: thanos-object-storage
      receiveStorageSize: 25Gi
      ruleStorageSize: 10Gi
      storeStorageSize: 25Gi

```

13. Apply the **MultiClusterObservability** CR by running the following command on the hub cluster:

```
$ oc apply -f monitoringMultiClusterObservability.yaml
```

Verification

1. Check the routes and pods in the namespace to validate that the services have deployed on the hub cluster by running the following command:

```
$ oc get routes,pods -n open-cluster-management-observability
```

Example output

NAME	HOST/PORT	PATH	SERVICES	PORT	TERMINATION	WILDCARD
route.route.openshift.io/alertmanager	observability.cloud.example.com	/api/v2	alertmanager	oauth-proxy		
reencrypt/Redirect	None					
route.route.openshift.io/grafana	observability.cloud.example.com		grafana	oauth-proxy		
reencrypt/Redirect	None					
route.route.openshift.io/observatorium-api	observability.cloud.example.com		observatorium-api	public		
passthrough/None	None					
route.route.openshift.io/rbac-query-proxy	observability.cloud.example.com		rbac-query-proxy	https		
reencrypt/Redirect	None					

NAME	READY	STATUS	RESTARTS	AGE
pod/observability-alertmanager-0	3/3	Running	0	1d
pod/observability-alertmanager-1	3/3	Running	0	1d
pod/observability-alertmanager-2	3/3	Running	0	1d
pod/observability-grafana-685b47bb47-dq4cw	3/3	Running	0	1d
<...snip...>				
pod/observability-thanos-store-shard-0-0	1/1	Running	0	1d
pod/observability-thanos-store-shard-1-0	1/1	Running	0	1d
pod/observability-thanos-store-shard-2-0	1/1	Running	0	1d

**NOTE**

A dashboard is accessible at the grafana route listed. You can use this to view metrics across all managed clusters.

For more information on observability in Red Hat Advanced Cluster Management, see [Observability](#).

13.5.1.4. Alerting

OpenShift Container Platform includes a large number of alert rules, which can change from release to release.

13.5.1.4.1. Viewing default alerts

To review all of the alert rules in a cluster, run the following command:

```
$ oc get cm -n openshift-monitoring prometheus-k8s-rulefiles-0 -o yaml
```

Rules can include a description and provide a link to additional information and mitigation steps. For example, see the rule for **etcdHighFsyncDurations**:

```
- alert: etcdHighFsyncDurations
  annotations:
    description: 'etcd cluster "{{ $labels.job }}"': 99th percentile fsync durations
      are {{ $value }}s on etcd instance {{ $labels.instance }}.'
    runbook_url: https://github.com/openshift/runbooks/blob/master/alerts/cluster-etcd-
operator/etcdHighFsyncDurations.md
    summary: etcd cluster 99th percentile fsync durations are too high.
  expr: |
    histogram_quantile(0.99, rate(etcd_disk_wal_fsync_duration_seconds_bucket{job=~".*etcd.*"}
[5m]))
    > 1
  for: 10m
  labels:
    severity: critical
```

13.5.1.4.2. Alert notifications

You can view alerts in the OpenShift Container Platform console. However, an administrator must configure an external receiver to forward the alerts to. OpenShift Container Platform supports the following receiver types:

PagerDuty

A third-party incident response platform.

Webhook

An arbitrary API endpoint that receives an alert through a **POST** request and can take any necessary action.

Email

Sends an email to a designated address.

Slack

Sends a notification to either a Slack channel or an individual user.

Additional resources

- [Managing alerts](#)

13.5.1.5. Workload monitoring

By default, OpenShift Container Platform does not collect metrics for application workloads. You can configure a cluster to collect workload metrics and create alerts for user workloads.

Prerequisites

- You have defined endpoints to gather workload metrics on the cluster.

Procedure

1. Create a **ConfigMap** CR and save it as **monitoringConfigMap.yaml**, as in the following example:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

Set **enableUserWorkload** to **true** to enable workload monitoring.

2. Apply the **ConfigMap** CR by running the following command:

```
$ oc apply -f monitoringConfigMap.yaml
```

3. Create a **ServiceMonitor** CR, and save it as **monitoringServiceMonitor.yaml**, as in the following example:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  labels:
    app: ui
  name: myapp
  namespace: myns
spec:
  endpoints:
    - interval: 30s
      port: ui-http
      scheme: http
      path: /healthz
  selector:
    matchLabels:
      app: ui
```


- **endpoints** specifies the workload metrics endpoints to scrape.
- **path** specifies a custom scrape path. Prometheus scrapes the **/metrics** path by default.

4. Apply the **ServiceMonitor** CR by running the following command:

```
$ oc apply -f monitoringServiceMonitor.yaml
```

The vendor of the application must decide whether to expose the endpoint for scraping, with metrics that they deem relevant.

5. To enable alerts for user workloads, verify that the **cluster-monitoring-config** ConfigMap has **enableUserWorkload: true** set. If you completed step 1, this is already configured.
6. Create a YAML file for alerting rules and save it as **monitoringAlertRule.yaml**, as in the following example:

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: myapp-alert
  namespace: myns
spec:
  groups:
  - name: example
    rules:
    - alert: InternalErrorsAlert
      expr: flask_http_request_total{status="500"} > 0
  # ...
```

7. Apply the alert rule by running the following command:

```
$ oc apply -f monitoringAlertRule.yaml
```

Additional resources

- [ServiceMonitor\[monitoring.coreos.com/v1\]](#)
- [Enabling monitoring for user-defined projects](#)
- [Managing alerting rules for user-defined projects](#)

13.6. SECURITY

13.6.1. Security basics

Security is a critical component of OpenShift Container Platform deployments, particularly when running cloud-native applications.

You can enhance security for high-bandwidth network deployments by following key security considerations. By implementing these standards and best practices, you can strengthen security in most use cases.

13.6.1.1. RBAC overview

Role-based access control (RBAC) objects determine whether a user is allowed to perform a given action within a project.

Cluster administrators can use the cluster roles and bindings to control who has various access levels to the OpenShift Container Platform platform itself and all projects.

Developers can use local roles and bindings to control who has access to their projects. Authorization is a separate step from authentication, which is more about determining the identity of who is taking the action.

Authorization is managed using the following authorization objects:

Rules

Sets of permitted actions on specific objects. For example, a rule can determine whether a user or service account can create pods. Each rule specifies an API resource, the resource within that API, and the allowed action.

Roles

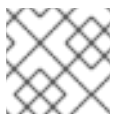
Collections of rules that define what actions users or groups can perform. You can associate or bind rules to multiple users or groups. A role file can contain one or more rules that specify the actions and resources allowed for that role.

Roles are categorized into the following types:

- Cluster roles can be defined at the cluster level. They are not tied to a single namespace. They can apply across all namespaces or specific namespaces when you bind them to users, groups, or service accounts.
- Project roles can be created within a specific namespace, and they only apply to that namespace. You can assign permissions to specific users to create roles and role bindings within their namespace, ensuring they do not affect other namespaces.

Bindings

Associations between users and groups with a role. You can create a role binding to connect the rules in a role to a specific user ID or group. This brings together the role and the user or group, defining what actions they can perform.



NOTE

You can bind more than one role to a user or group.

For more information on RBAC, see "Using RBAC to define and apply permissions".

13.6.1.1.1. Operational RBAC considerations

To reduce operational overhead, manage access through groups rather than handling individual user IDs across multiple clusters. By managing groups at an organizational level, you can streamline access control and simplify administration across your organization.

Additional resources

- [Using RBAC to define and apply permissions](#)

13.6.1.2. Security accounts overview

A service account is an OpenShift Container Platform account that allows a component to directly access the API. Service accounts are API objects that exist within each project. Service accounts provide a flexible way to control API access without sharing a regular user's credentials.

You can use service accounts to apply role-based access control (RBAC) to pods. By assigning service accounts to workloads, such as pods and deployments, you can grant additional permissions, such as pulling from different registries. This also allows you to assign lower privileges to service accounts, reducing the security footprint of the pods that run under them.

For more information about service accounts, see "Understanding and creating service accounts".

Additional resources

- [Understanding and creating service accounts](#)

13.6.1.3. Identity provider configuration

Configuring an identity provider is the first step in setting up users on the cluster. You can manage groups at the organizational level by using an identity provider.

The identity provider can pull in specific user groups that are maintained at the organizational level, rather than the cluster level. This allows you to add and remove users from groups that follow your organization's established practices.



NOTE

You must set up a cron job to run frequently to pull any changes into the cluster.

You can use an identity provider to manage access levels for specific groups within your organization. For example, you can perform the following actions to manage access levels:

- Assign the **cluster-admin** role to teams that require cluster-level privileges.
- Grant application administrators specific privileges to manage only their respective projects.
- Provide operational teams with **view** access across the cluster to enable monitoring without allowing modifications.

For information about configuring an identity provider, see "Understanding identity provider configuration".

Additional resources

- [Understanding identity provider configuration](#)

13.6.1.4. Replacing the kubeadmin user with a cluster-admin user

The **kubeadmin** user with the **cluster-admin** privileges is created on every cluster by default. To enhance the cluster security, you can replace the **kubeadmin** user with a **cluster-admin** user and then disable or remove the **kubeadmin** user.

Prerequisites

- You have created a user with **cluster-admin** privileges.
- You have installed the OpenShift CLI (**oc**).
- You have administrative access to a virtual vault for secure storage.

Procedure

1. Create an emergency **cluster-admin** user by using the **htpasswd** identity provider. For more information, see "About htpasswd authentication".
2. Assign the **cluster-admin** privileges to the new user by running the following command:

```
$ oc adm policy add-cluster-role-to-user cluster-admin <emergency_user>
```

3. Verify the emergency user access:
 - a. Log in to the cluster using the new emergency user.
 - b. Confirm that the user has **cluster-admin** privileges by running the following command:

```
$ oc whoami
```

Ensure the output shows the ID of the emergency user.

4. Store the password or authentication key for the emergency user securely in a virtual vault.



NOTE

Follow the best practices of your organization for securing sensitive credentials.

5. Disable or remove the **kubeadmin** user to reduce security risks by running the following command:

```
$ oc delete secrets kubeadmin -n kube-system
```

Additional resources

- [Configuring an htpasswd identity provider](#)

13.6.1.5. Security considerations

Workloads might handle sensitive data and demand high reliability. A single security vulnerability might lead to broader, cluster-wide compromises. With numerous components running on an OpenShift Container Platform cluster, you must secure each component to prevent any breach from escalating. Ensuring security across the entire infrastructure, including all components, is essential to maintaining the integrity of the network and avoiding vulnerabilities.

The following key security features are essential for all industries that handle sensitive data:

- Security Context Constraints (SCCs): Provide granular control over pod security in the OpenShift Container Platform clusters.
- Pod Security Admission (PSA): Kubernetes-native pod security controls.

- Encryption: Ensures data confidentiality in high-throughput network environments.

13.6.1.6. Advancement of pod security in Kubernetes and OpenShift Container Platform

Kubernetes initially had limited pod security. When OpenShift Container Platform integrated Kubernetes, Red Hat added pod security through Security Context Constraints (SCCs). In Kubernetes version 1.3, **PodSecurityPolicy** (PSP) was introduced as a similar feature. However, Pod Security Admission (PSA) was introduced in Kubernetes version 1.21, which resulted in the deprecation of PSP in Kubernetes version 1.25.

PSA also became available in OpenShift Container Platform version 4.11. While PSA improves pod security, it lacks features provided by SCCs that are still necessary for certain use cases. Therefore, OpenShift Container Platform continues to support both PSA and SCCs.

13.6.1.7. Bare-metal infrastructure

Bare-metal infrastructure for OpenShift Container Platform clusters in telco and finance industries requires specific hardware and network configurations.

Hardware requirements

In several industries, such as telco and finance, clusters are primarily built on bare-metal hardware. This means that the Red Hat Enterprise Linux CoreOS (RHCOS) operating system is installed directly on the physical machines, without using virtual machines. This reduces network connectivity complexity, minimizes latency, and optimizes CPU usage for applications.

Network requirements

Networks in these industries sometimes require much higher bandwidth compared to standard IT networks. For example, Telco networks commonly use dual-port 25 GB connections or 100 GB network interface cards (NICs) to handle massive data throughput. Security is critical, requiring encrypted connections and secure endpoints to protect sensitive personal data.

13.6.1.8. Lifecycle management

Upgrades are critical for security. When a vulnerability is discovered, it is patched in the latest z-stream release. This fix is then rolled back through each lower y-stream release until all supported versions are patched. Releases that are no longer supported do not receive patches. Therefore, it is important to upgrade OpenShift Container Platform clusters regularly to stay within a supported release and ensure they remain protected against vulnerabilities.

For more information about lifecycle management and upgrades, see "Upgrading OpenShift Container Platform clusters".

Additional resources

- [Upgrading an OpenShift cluster](#)

13.6.2. Host security

13.6.2.1. Red Hat Enterprise Linux CoreOS (RHCOS)

Red Hat Enterprise Linux CoreOS (RHCOS) is different from Red Hat Enterprise Linux (RHEL) in key areas. For more information, see "About RHCOS".

A major distinction is the control of **rpm-ostree**, which is updated through the Machine Config Operator.

RHCOS follows the same immutable design used for pods in OpenShift Container Platform. This ensures that the operating system remains consistent across the cluster. For information about RHCOS architecture, see "Red Hat Enterprise Linux CoreOS (RHCOS)".

To manage hosts effectively while maintaining security, avoid direct access whenever possible. Instead, you can use the following methods for host management:

- Debug pod
- Direct SSH
- Console access

Review the following RHCOS security mechanisms that are integral to maintaining host security:

Linux namespaces

Provide isolation for processes and resources. Each container keeps its processes and files within its own namespace. If a user escapes from the container namespace, they could gain access to the host operating system, potentially compromising security.

Security-Enhanced Linux (SELinux)

Enforces mandatory access controls to restrict access to files and directories by processes. SELinux adds an extra layer of security by preventing unauthorized access to files if a process tries to break its confinement.

SELinux follows the security policy of denying everything unless explicitly allowed. If a process attempts to modify or access a file without permission, SELinux denies access. For more information, see [Introduction to SELinux](#).

Linux capabilities

Assign specific privileges to processes at a granular level, minimizing the need for full root permissions. For more information, see "Linux capabilities".

Control groups (cgroups)

Allocate and manage system resources, such as CPU and memory for processes and containers, ensuring efficient usage. As of OpenShift Container Platform 4.16, there are two versions of cgroups. cgroup v2 is now configured by default.

CRI-O

Serves as a lightweight container runtime that enforces security boundaries and manages container workloads.

Additional resources

- [About RHCOS](#)
- [Red Hat Enterprise Linux CoreOS \(RHCOS\)](#)
- [Linux capabilities](#)

13.6.2.2. Command-line host access

Configure an external authenticator to restrict direct access and prevent unauthorized modifications. The Machine Config Operator (MCO) manages these logins and maintains consistency across your cluster.

Examples of external authenticators include lightweight directory access protocol (LDAP) and System

Security Services Daemon (SSSD). If a node reboot leads to timeout issues, create a node disruption policy. With this policy, you can configure an external authenticator on a host without requiring a node reboot. For more information, see "Using node disruption policies to minimize disruption from machine config changes" in the *Additional resources* section.



IMPORTANT

Do not configure direct access to the root ID on any OpenShift Container Platform cluster server.

You can connect to a node in the cluster by using the following methods:

Using debug pod

Red Hat recommends this method to access a node. To debug or connect to a node, run the following command:

```
$ oc debug node/<worker_node_name>
```

After connecting to the node, run the following command to get access to the root file system:

```
# chroot /host
```

This gives you root access within a debug pod on the node. For more information, see "Starting debug pods with root access".

Direct SSH

Avoid using the root user. Instead, use the core user ID (or your own ID). To connect to the node by using SSH, run the following command:

```
$ ssh core@<worker_node_name>
```



IMPORTANT

The core user ID is initially given **sudo** privileges within the cluster.

If you cannot connect to a node by using SSH, add your SSH key to the core user. For more information, see "How to connect to OpenShift Container Platform 4.x Cluster nodes using SSH bastion pod" in the *Additional resources* section.

After connecting to the node using SSH, run the following command to get access to the root shell:

```
$ sudo -i
```

Console Access

Ensure that consoles are secure. Do not allow direct login with the root ID, instead use individual IDs.



NOTE

Follow the best practices of your organization for securing console access.

Additional resources

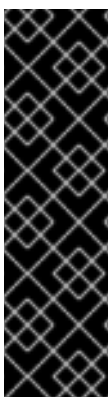
- [Using node disruption policies to minimize disruption from machine config changes](#)
- [Starting debug pods with root access](#)
- [How to connect to OpenShift Container Platform 4.x Cluster nodes using SSH bastion pod](#)

13.6.2.3. Linux capabilities

Linux capabilities define the actions a process can perform on the host system. By default, pods are granted several capabilities unless security measures are applied. These default capabilities are as follows:

- **CHOWN**
- **DAC_OVERRIDE**
- **FSETID**
- **FOWNER**
- **SETGID**
- **SETUID**
- **SETPCAP**
- **NET_BIND_SERVICE**
- **KILL**

You can modify which capabilities that a pod can receive by configuring Security Context Constraints (SCCs).



IMPORTANT

You must not assign the following capabilities to a pod:

- **SYS_ADMIN**: A powerful capability that grants elevated privileges. Allowing this capability can break security boundaries and pose a significant security risk.
- **NET_ADMIN**: Allows control over networking, like SR-IOV ports, but can be replaced with alternative solutions in modern setups.

For more information about Linux capabilities, see the [Linux capabilities](#) man page.

13.6.3. Security context constraints

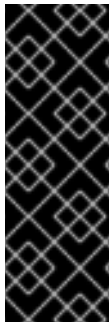
Similar to the way that RBAC resources control user access, administrators can use security context constraints (SCCs) to control permissions for pods. These permissions determine the actions that a pod can perform and what resources it can access. You can use SCCs to define a set of conditions that a pod must run.

Security context constraints allow an administrator to control the following security constraints:

- Whether a pod can run privileged containers with the **allowPrivilegedContainer** flag
- Whether a pod is constrained with the **allowPrivilegeEscalation** flag
- The capabilities that a container can request
- The use of host directories as volumes
- The SELinux context of the container
- The container user ID
- The use of host namespaces and networking
- The allocation of an **FSGroup** that owns the pod volumes
- The configuration of allowable supplemental groups
- Whether a container requires write access to its root file system
- The usage of volume types
- The configuration of allowable **seccomp** profiles

Default SCCs are created during installation and when you install some Operators or other components. As a cluster administrator, you can also create your own SCCs by using the OpenShift CLI (**oc**).

For information about default security context constraints, see [Default security context constraints](#).



IMPORTANT

Do not modify the default SCCs. Customizing the default SCCs can lead to issues when some of the platform pods deploy or OpenShift Container Platform is upgraded. Additionally, the default SCC values are reset to the defaults during some cluster upgrades, which discards all customizations to those SCCs.

Instead of modifying the default SCCs, create and modify your own SCCs as needed. For detailed steps, see [Creating security context constraints](#).

You can use the following basic SCCs:

- **restricted**
- **restricted-v2**

The **restricted-v2** SCC is the most restrictive SCC provided by a new installation and is used by default for authenticated users. It aligns with Pod Security Admission (PSA) restrictions and improves security, as the original **restricted** SCC is less restrictive. It also helps transition from the original SCCs to v2 across multiple releases. Eventually, the original SCCs get deprecated. Therefore, it is recommended to use the **restricted-v2** SCC.

You can examine the **restricted-v2** SCC by running the following command:

```
$ oc describe scc restricted-v2
```

Example output

```

Name:                restricted-v2
Priority:             <none>
Access:
  Users:             <none>
  Groups:            <none>
Settings:
  Allow Privileged:   false
  Allow Privilege Escalation: false
  Default Add Capabilities: <none>
  Required Drop Capabilities: ALL
  Allowed Capabilities: NET_BIND_SERVICE
  Allowed Seccomp Profiles: runtime/default
  Allowed Volume Types:
configMap,downwardAPI,emptyDir,ephemeral,persistentVolumeClaim,projected,secret
  Allowed Flexvolumes: <all>
  Allowed Unsafe Sysctls: <none>
  Forbidden Sysctls: <none>
  Allow Host Network: false
  Allow Host Ports:   false
  Allow Host PID:      false
  Allow Host IPC:      false
  Read Only Root Filesystem: false
  Run As User Strategy: MustRunAsRange
  UID:                 <none>
  UID Range Min:       <none>
  UID Range Max:       <none>
  SELinux Context Strategy: MustRunAs
  User:                <none>
  Role:                <none>
  Type:                <none>
  Level:               <none>
  FSGroup Strategy: MustRunAs
  Ranges:              <none>
  Supplemental Groups Strategy: RunAsAny
  Ranges:              <none>

```

The **restricted-v2** SCC explicitly denies everything except what it explicitly allows. The following settings define the allowed capabilities and security restrictions:

- Default add capabilities: Set to **<none>**. It means that no capabilities are added to a pod by default.
- Required drop capabilities: Set to **ALL**. This drops all the default Linux capabilities of a pod.
- Allowed capabilities: **NET_BIND_SERVICE**. A pod can request this capability, but it is not added by default.
- Allowed **seccomp** profiles: **runtime/default**.

For more information, see [Managing security context constraints](#).

